



SANCHARA

Engineering Handbook

Shiv Vyas · Revision A

Sanchara is an astrology app built on one rule: nothing about the chart is guessed. The Swiss Ephemeris computes a person's real birth chart (planets, houses, nakshatras, dashas, doshas, transits and numerology) from their birth date, time and place, and Claude interprets only those computed facts. It ships as a Next.js web app on Vercel and a native SwiftUI app for iOS, both on one API and one Supabase project. This handbook is the single reference for how it is built, why it is built that way, what keeps it fast and cheap, and what it does when things go wrong. It is written to be studied: every chapter ends with a reading order through the code and questions to check yourself against.

REVISION A 30 SEPTEMBER 2026 VERSION 0.3.0 NEXT.JS 15 · SUPABASE · IOS 17
AUDIENCE: CONTRIBUTORS

Table of contents

Every line is a link as well as a page number. Part two is the features, one chapter per feature, each in the same shape.

PART ONE: FOUNDATIONS

1	Architecture	6
1.1	System context	6
1.2	The one bet	8
1.3	Module map	9
1.4	Two clients, one API	12
1.5	Two auth channels	13
1.6	The privacy boundary	15
1.7	What lives on the phone	16
1.8	Time and the twice-daily job	17
1.9	Study notes	19
2	The chart engine	21
2.1	What a chart is here	21
2.2	Files and the input side	22
2.3	Positions: the ephemeris	23
2.4	Houses: whole-sign and Placidus	25
2.5	Nakshatras and dashas	27
2.6	Derived facts	29
2.7	Doshas	31
2.8	Transits (gochara)	33
2.9	Numerology	35
2.10	Tests	36
2.11	Study notes	38
3	Readings, prompts, and cost	40
3.1	What a reading is	40
3.2	Files	41
3.3	The system prompt in two halves	41
3.4	Models and effort	45
3.5	Caching and the cost of a turn	46
3.6	Guarding the prompt	48
3.7	Tests	49
3.8	Study notes	51
4	Identity and access	53
4.1	Two channels	53
4.2	Files	54
4.3	Session on the web	54
4.4	Session on the phone	56
4.5	Email code	58
4.6	Sign in with Apple and Universal Links	59
4.7	Row Level Security	60
4.8	The admin boundary	63
4.9	Tests	63
4.10	Study notes	65

PART TWO: THE FEATURES

5	Intake and the birth profile	68
5.1	Intent	68
5.2	Files and folders	69
5.3	Model	70
5.4	The server side	72
5.5	The clients	75
5.6	How it works	78
5.7	Performance and cost	79
5.8	Edge cases and tests	80
5.9	Study notes	82
6	Chat	83
6.1	Intent	83
6.2	Files and folders	83
6.3	Model	85
6.4	The server side	86
6.5	The clients	87
6.6	How it works	90
6.7	Performance and cost	91
6.8	Edge cases and tests	92
6.9	Study notes	94
7	Kundli	95
7.1	Intent	95
7.2	Files and folders	96
7.3	Model	97
7.4	The server side	98
7.5	The clients	101
7.6	How it works	105
7.7	Performance and cost	107
7.8	Edge cases and tests	108
7.9	Study notes	110
8	The life timeline	112
8.1	Intent	112
8.2	Files and folders	112
8.3	Model	113
8.4	The server side	115
8.5	The clients	118
8.6	How it works	119
8.7	Performance and cost	121
8.8	Edge cases and tests	122
8.9	Study notes	124

C	Glossary and reading order	258
C.1	Glossary	259
C.2	A reading order through the whole codebase	263
C.3	Where things are	264
C.4	Study notes	267

PART ONE

Foundations

The shape of the system before any screen: how a birth chart is computed and never guessed, how a reading is written from it and what that costs, and how a person and a phone are let in.

- 1. Architecture
- 2. The chart engine
- 3. Readings, prompts, and cost
- 4. Identity and access

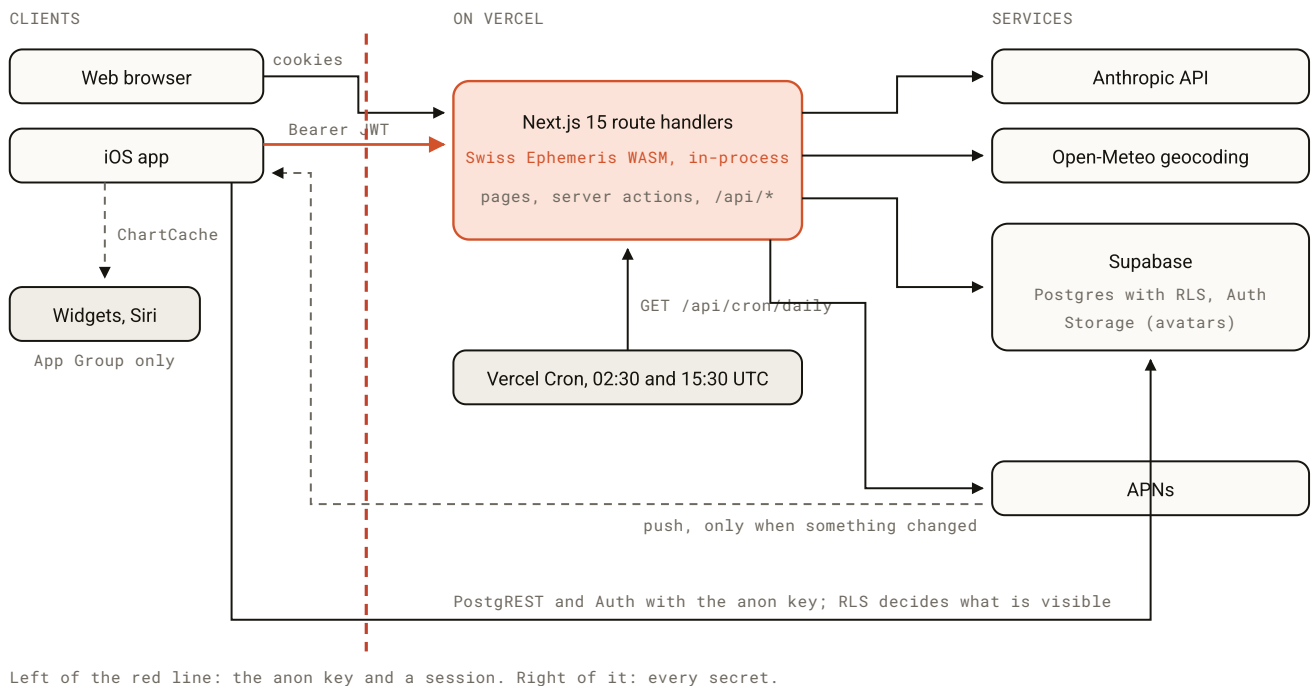
1 Architecture

Sanchara is built on one bet: **the chart is computed, and only the reading is interpreted.** The Swiss Ephemeris runs inside a Next.js route handler and produces every position, house, nakshatra, dasha and dosha as a number. Claude is handed those numbers and asked what they mean. Two clients, a web app and a native iOS app, call the same route handlers, share one Supabase project, and are told apart only by how they prove who they are.

This chapter describes the code on `main` on 30 September 2026, version 0.3.0. It is the map for everything that follows: where the boundaries are, which side of each boundary a piece of code sits on, and why. The product was called Astra during development, and the repository, the bundle identifier `com.shivvyas.astra` and the production host still carry that name. Where the code and this handbook disagree, the code is right and this handbook is a bug.

1.1 System context

There is one server, and it is the Next.js app deployed on Vercel. A web browser reaches it as pages and as `/api/*` route handlers; the iOS app reaches only the route handlers. The server talks to four outside parties: the Anthropic API for readings, Supabase for identity and every table, Apple Push Notification service for alerts, and Open-Meteo for birthplace geocoding. The Swiss Ephemeris is not an outside party. It is a WebAssembly module, loaded once per server process, that runs in the same function as the route handler that called it. Vercel Cron is the fifth caller, and it calls the same server twice a day.



One server, two clients, four services. The server reaches Supabase either as the caller (the cookie or the bearer token is forwarded, so Row Level Security applies) or as the service role (cron, admin reads, and three writes RLS would refuse). The widgets and Siri never cross the network at all; they read what the app cached in the App Group.

Each party, how it is reached, and what proves the request is allowed:

PARTY	REACHED FROM	WITH	WHERE IN THE CODE
Anthropic API	Route handlers and cron only	<code>ANTHROPIC_API_KEY</code>	<code>lib/anthropic.ts</code>
Swiss Ephemeris	In-process, lazily initialised once per server process	Nothing; it is a WASM module in <code>node_modules</code>	<code>lib/astrology/chart.ts</code>
Supabase Postgres	Server as the caller; iOS app directly over PostgREST	Anon key plus the user's JWT, or the service-role key	<code>lib/supabase/</code> , <code>mobile/Sanchara/Sources/Services/Supabase.swift</code>
Supabase Auth	Browser via server actions; iOS app directly	Anon key	<code>app/(auth)/actions.ts</code> , <code>Features/Auth/AuthStore.swift</code>
Supabase Storage	Server, admin client, <code>avatars</code> bucket	Service-role key	<code>app/api/profile/route.ts</code> , <code>app/(app)/app/intake/actions.ts</code>
APNs	Cron, over HTTP/2	A provider token signed with the <code>.p8</code> key	<code>lib/push/apns.ts</code>
Open-Meteo	<code>GET /api/geocode</code> , which is public	Nothing	<code>lib/geo.ts</code>
Vercel Cron	Calls the server	Authorization: Bearer <code>\$CRON_SECRET</code>	<code>vercel.json</code> , <code>app/api/cron/daily/route.ts</code>

1.2 The one bet

The README states it in a sentence: "Nothing about the chart is guessed." Every other decision in the system follows from taking that literally. A language model asked for a birth chart will produce one, and it will be plausible and wrong. So the model is never asked. The chart is computed by `computeChart` in `lib/astrology/chart.ts`, which converts the local birth time to Universal Time with `luxon`, asks the ephemeris for a Julian day, and reads eight bodies with `calc_ut`. Ketu is Rahu plus 180 degrees. For the Vedic chart the ephemeris is put into sidereal mode with the Lahiri ayanamsa and houses are whole-sign from the ascendant; for the Western chart it stays tropical and houses are Placidus. Vimshottari dasha periods come from the Moon's sidereal longitude. All of this is deterministic and unit tested.

```
// lib/astrology/chart.ts
// Lazily initialize the WASM module once per server process.
let swePromise: Promise<InstanceType<typeof SwissEph>> | null = null;
async function getSwe(): Promise<InstanceType<typeof SwissEph>> {
  if (!swePromise) {
    swePromise = (async () => {
      const swe = new SwissEph();
      await swe.initSwissEph();
      return swe;
    })();
  }
  return swePromise;
}
```

The result is stored, not recomputed per request. `saveBirthProfile` in `lib/data/birthProfile.ts` computes both traditions and writes them as one `jsonb` value, `{ vedic, western }`, into `birth_profiles.chart`. Every reader of that column goes through `ensureCurrentChart`, which checks `schemaVersion` against `CHART_SCHEMA_VERSION` (currently 2) and recomputes a stale chart on read. There is no migration job for charts: a chart written under an older house system corrects itself the first time its owner opens the app, and both traditions are rewritten together so a profile is never half upgraded.

Only then does the model appear. `app/api/chat/route.ts` loads the stored chart, derives facts from it (lords, aspects, dignity, combustion, from `lib/astrology/derived.ts`), computes today's transits, and builds a system prompt in `lib/astrology/prompt.ts` that is entirely made of computed values. Claude reads that prompt and writes prose. The models are named in one place:

```
// lib/anthropic.ts
export const READING_MODEL = "claude-sonnet-5";
export const DEEP_READING_MODEL = "claude-opus-4-8";

export const LOW_EFFORT: { readonly output_config: { readonly effort: "low" } } = {
  output_config: { effort: "low" },
};
```

The comment above `LOW_EFFORT` records a measurement on one Vedic reading of the same chart and question: 1,452 output tokens at default effort, 719 at low, 328 with thinking off, and all three cited the same real placements. Routine readings therefore run at low effort; a deep reading keeps the default and upgrades

to the larger model. The point for this chapter is the direction of dependency. The prompt depends on the chart. The chart never depends on the prompt.

WHAT THE BET RULES OUT

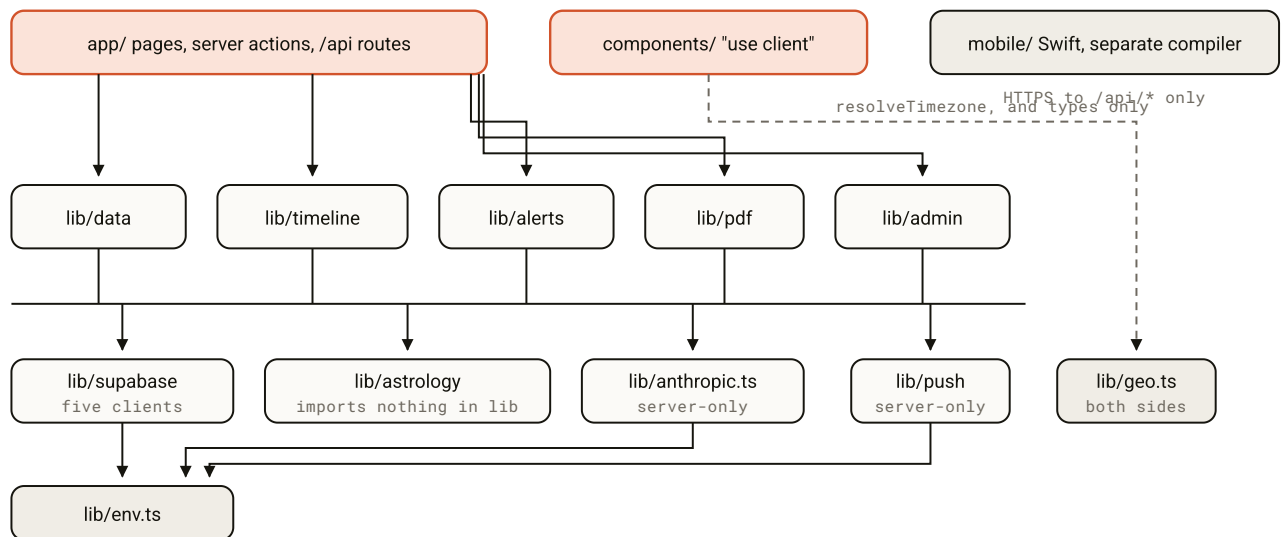
No route ever sends a birth date to the model and asks for positions. No client ever computes a chart; the iOS app decodes the stored `jsonb` into `NatalChart` and draws it. When the phone answers a question without the network, it answers by looking a value up in that decoded chart, never by reasoning about the sky. See 1.7.

1.3 Module map

Three trees hold the code: `lib/` for everything the server does, `app/` for the Next.js routes that expose it, and `mobile/` for the iOS app. `tsconfig.json` excludes `mobile`, so the TypeScript and the Swift never see each other; the phone reaches the server only over HTTPS.

```
lib/
  astrology/      chart, dasha, derived, doshas, transits, numerology, prompt, constants, types
  timeline/       build, describe, explain, extract, load
  alerts/         run, predictions, compose, slots
  data/           birthProfile, chat, history
  supabase/       client, server, middleware, route, admin
  pdf/            kundli
  push/           apns
  admin/          guard, data
  anthropic.ts    model names, effort, the client
  env.ts          server-only environment access
  geo.ts          Open-Meteo geocoding and tz-lookup
app/
  page.tsx        landing page, redirects a signed-in user to /app
  (auth)/         login, signup, check-email, and the server actions
  (app)/app/      home, intake, chat, profile
  admin/          isolated admin area with its own login
  auth/callback/  Supabase code exchange
  api/            twelve route handlers, listed in 1.4
mobile/
  project.yml      XcodeGen spec: three targets
  scripts/make-config.sh writes the generated AppConfig.swift
  Sanchara/Sources/
    SancharaApp.swift, rootView.swift
    Design/        Theme, Components, ReadingCard
    Features/      Auth, Intake, Chat, Kundli, Timeline, Alerts, Profile
    Intents/       App Intents, entities, shortcuts, deep links
    Services/      SancharaAPI, Supabase, Push, ChartCache, WidgetRefresh
  SancharaWidgets/ ReadingWidget, PeriodWidget, ReadingControl
```

Nothing enforces the import rules between `lib/` folders except the imports themselves, so the rules below were read off the code rather than off a manifest. They hold today, and the diagram is the shape they make.



Arrows read "imports". The bus means any module above it may import any module below it; nothing imports sideways within a row.

Four tiers. The feature modules in the second row never import one another; each reaches down to the engine, a Supabase client, the model client or the push client. The leaf row imports only `lib/env.ts`, and `lib/astrology` imports nothing from `lib` at all, which is what makes it testable with no environment and no network.

MODULE	OWNS	IMPORTS FROM LIB	RUNS WHERE
<code>astrology/</code>	The engine: chart, dasha, derived facts, doshas, transits, numerology, the prompt builders, constants and types	Only itself; externally luxon and swisseph-wasm	Server. Pure functions apart from the WASM call in <code>chart.ts</code>
<code>timeline/</code>	The dasha life map, the plain-language meaning per period, extraction of moments from chat history	<code>astrology</code> , <code>anthropic</code> , <code>supabase/route</code>	Server. <code>explain</code> , <code>extract</code> , <code>load</code> are <code>server-only</code> ; <code>build</code> and <code>describe</code> are pure
<code>alerts/</code>	The twice-daily reading, the dosha sweep, the copy the model writes, the local-time slot rule	<code>astrology</code> , <code>anthropic</code> , <code>supabase/admin</code> , <code>push</code>	Server, cron only. <code>slots.ts</code> is pure
<code>data/</code>	Reads and writes of <code>birth_profiles</code> , conversations and messages; the history window a reading sees	<code>astrology</code> , <code>supabase/server</code> , <code>supabase/route</code>	Server
<code>supabase/</code>	Five clients: browser, server (cookies), middleware, route (bearer or cookies), admin (service role)	<code>env</code>	<code>client.ts</code> is "use client"; <code>route.ts</code> and <code>admin.ts</code> are <code>server-only</code>
<code>pdf/</code>	The kundli PDF, drawn with pdf-lib	<code>astrology/constants</code> , <code>astrology/types</code>	Server, <code>server-only</code>
<code>push/</code>	The APNs client: provider token, HTTP/2 sessions, dead-token detection	<code>env</code>	Server, <code>server-only</code>
<code>admin/</code>	<code>requireAdmin</code> and the admin queries	<code>supabase/server</code> , <code>supabase/admin</code>	Server, <code>server-only</code>
<code>anthropic.ts</code>	Model names, the effort setting, one cached client	<code>env</code>	Server, <code>server-only</code>
<code>env.ts</code>	Every environment variable, as a function that throws when a required one is missing	nothing	Server, <code>server-only</code>
<code>geo.ts</code>	Open-Meteo search and <code>resolveTimezone</code>	nothing	Both. <code>IntakeForm</code> calls <code>resolveTimezone</code> in the browser

The `server-only` package is the one mechanical guard. Fourteen files import it, and a client component that pulls any of them in fails the build rather than shipping a secret. Client components are few and shallow: `Chat.tsx`, `AppShell.tsx`, `IntakeForm.tsx`, `PlaceAutocomplete.tsx`, `AdminShell.tsx` and the marketing pieces. Between them they import exactly two things from `lib`: the `ChatMode` and `GeoResult` types, and the `resolveTimezone` function. Everything else a page needs is fetched by a server component or a server action.

THE IOS SIDE

`mobile/project.yml` declares three targets and is the whole rule book for the phone. The Xcode project it generates is ignored by git. The app target depends on the `supabase-swift` package and on the widget target. The widget target is the interesting one, because its source list is not a folder but six named files:

```
# mobile/project.yml
SancharaWidgets:
  type: app-extension
  sources:
    - path: SancharaWidgets
    # Compiled into the extension as well as the app.
    - path: Sanchara/Sources/Design/Theme.swift
    - path: Sanchara/Sources/Design/ReadingCard.swift
    - path: Sanchara/Sources/Services/ChartCache.swift
    - path: Sanchara/Sources/Features/Kundli/ChartData.swift
    - path: Sanchara/Sources/Features/Intake/BirthProfileDetails.swift
    - path: Sanchara/Sources/Intents/OpenKundliIntent.swift
```

The comment on that list says each file is dependency-free on purpose, because pulling in anything that imports Supabase would drag the whole client into a process that has no session to use it with. Those six files import only SwiftUI, Foundation and AppIntents. That is the iOS equivalent of the `server-only` marker: the widget extension is a process with no session, so the only code that may compile into it is code that needs none. Everything under `Features/` and `Services/` that talks to Supabase or to the API stays in the app target. The test target holds fourteen XCTest files and depends on the app.

1.4 Two clients, one API

The web app and the iOS app do not use the server the same way, and the difference is worth stating plainly because it decides where a feature's code lives.

The web app mostly does not call `/api`. Its pages are React server components that import `lib/data` directly and render with the result; its writes are server actions (`authenticate` , `sendMagicLink` , `saveIntake` , `signOut`) that a browser form posts to. The browser itself touches three routes: `POST /api/chat` from `Chat.tsx` , because a streaming reading needs a response body that grows; `GET /api/kundli` as a plain link, because it returns a PDF; and `GET /api/geocode` from `PlaceAutocomplete.tsx` as the user types a birthplace.

The iOS app has no server components and no server actions, so everything it cannot do itself goes through `SancharaAPI` in `mobile/Sanchara/Sources/Services/SancharaAPI.swift` . Its own header comment says why the API exists from the phone's point of view: it "holds everything that cannot ship in a client binary: the Anthropic key, the Swiss Ephemeris, and PDF generation." What the phone can do itself, it does directly against Supabase with the anon key and the user's session: it reads `birth_profiles` , `conversations` , `messages` , `alerts` and `daily_readings` over PostgREST and marks alerts and readings as read, which the RLS policies in migrations 0004 and 0005 allow.

ROUTE	METHOD	CALLED BY	WHAT IT DOES
<code>/api/chat</code>	POST	Browser, iOS	Streams a reading as plain UTF-8 text; the conversation id travels in an <code>x-conversation-id</code> header
<code>/api/kundli</code>	GET	Browser, iOS	The kundli PDF
<code>/api/profile</code>	POST	iOS	Birth-detail intake; the web uses the <code>saveIntake</code> server action for the same work
<code>/api/geocode</code>	GET	Browser, iOS	Birthplace search with the timezone attached; public, no token
<code>/api/timeline</code>	GET	iOS	The dasha life map with pinned moments and per-period meanings
<code>/api/timeline/explain</code>	POST	iOS	Writes the meaning of every period in one model call; <code>maxDuration</code> 90
<code>/api/timeline/scan</code>	POST	iOS	Proposes moments from the user's own messages; writes nothing; <code>maxDuration</code> 120
<code>/api/life-events</code>	POST, DELETE	iOS	Pins and unpins moments, up to 50 in one call
<code>/api/devices</code>	POST, DELETE	iOS	APNs token registration, upserted on the token
<code>/api/account</code>	DELETE	iOS	Deletes the calling user; App Store guideline 5.1.1(v)
<code>/api/cron/daily</code>	GET	Vercel Cron	Readings then the dosha sweep; <code>maxDuration</code> 300
<code>/api/cron/alerts</code>	GET	Not in <code>vercel.json</code>	The dosha sweep alone, behind the same secret; <code>.env.example</code> still names it

Every route except `geocode` and the two cron routes opens the same way: resolve a Supabase client for the caller, ask it for the user, and return 401 when there is none. The user id then comes from that verified session and never from the request body. Every route that runs the ephemeris or the model declares `runtime = "nodejs"`, and `next.config.ts` lists `swisseph-wasm` in `serverExternalPackages`, keeping it external so that Next bundles the `.wasm` binary for the server runtime.

1.5 Two auth channels

Both clients sign in with Supabase Auth, but they carry the session differently, and `lib/supabase/` exists mainly to make that difference invisible to route handlers.

WEB: COOKIES

The browser gets a session through a server action. Password sign-in, sign-up, the magic link and the confirmation resend all live in `app/(auth)/actions.ts`; the email links land on `app/auth/callback/route.ts`, which calls `exchangeCodeForSession` and redirects to `/app`. From then on the session is a set of cookies that `@supabase/ssr` manages, and two pieces of code keep them fresh. `middleware.ts` runs `updateSession` from `lib/supabase/middleware.ts` on every request that is not a static asset, a Next internal or the `.well-known` folder. That function calls `auth.getUser()`, which

refreshes the cookies as a side effect, and then redirects an anonymous request for `/app` to `/login` and for `/admin` (other than `/admin/login`) to `/admin/login`.

`lib/supabase/server.ts` is the client server components and actions use. Its `setAll` is wrapped in a try block whose comment explains the design: a server component cannot set cookies, so the write is ignored there, and the middleware is what actually refreshes them. Reading the two files together is the fastest way to understand the web session.

IOS: A BEARER TOKEN

The phone never sees a cookie. `supabase-swift` signs in directly, with a password, with a six-digit emailed code (`signInWithOTP` then `verifyOTP`), or with Sign in with Apple, where Supabase verifies Apple's identity token through `signInWithIdToken` and the nonce the app generated. The session persists in the Keychain, which `Supa` in `Services/Supabase.swift` notes is the library's default on Apple platforms. When the app calls the API it reads the current access token and puts it in an `Authorization: Bearer` header.

```
// lib/supabase/route.ts
export async function createRouteSupabase(request: Request): Promise<Db> {
  const token = bearerTokenFrom(request);
  if (!token) return createServerSupabase();

  return createClient(env.supabaseUrl(), env.supabaseAnonKey(), {
    global: { headers: { Authorization: `Bearer ${token}` } },
    auth: { persistSession: false, autoRefreshToken: false },
  });
}
```

This is the seam. A route handler calls `createRouteSupabase(request)` and gets back a `Db`, which is just `SupabaseClient`; the comment above the type notes that both the cookie client and the bearer client are instances of it, so every function in `lib/data` and `lib/timeline` accepts either. With a bearer header the client forwards the JWT on every PostgreSQL call and Row Level Security evaluates as that user. Without one it falls through to the cookie client and the web is unchanged. `bearerTokenFrom` is small enough to be fully specified by five tests in `lib/supabase/route.test.ts`: it accepts a lowercase scheme, and returns null for a missing header, a non-bearer scheme, or an empty token.

WHY THE ANON KEY SHIPS IN THE BINARY

`mobile/scripts/make-config.sh` reads exactly two values from `.env.local`, `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`, and writes them into a generated, git-ignored `AppConfig.swift` along with the API base URL. Its header states the rule: these are "designed to ship in clients (RLS is the security boundary, not secrecy)", and the service-role and Anthropic keys "are never read here and must never reach the app binary." The anon key is safe to publish because it grants nothing by itself. Every one of the eleven tables in `supabase/migrations/` has `enable row level security`, and every policy is keyed on `auth.uid()`, so an anon-key request without a valid user JWT can read no rows, and one with a JWT can read only that user's rows. The same key is in every web page's JavaScript through `lib/supabase/client.ts`. Nothing about the phone is special.

```
-- supabase/migrations/0001_init.sql
create policy "birth self" on public.birth_profiles
  for all using (user_id = auth.uid()) with check (user_id = auth.uid());
```

ADMIN: A THIRD DOOR, SAME KEY

The admin area at `/admin` has its own login page but not its own auth system. A user is an admin if a row for them exists in `admins`, a table with no insert policy and therefore no self-service path; `requireAdmin` in `lib/admin/guard.ts` checks it with the service-role client and redirects otherwise. Admin pages then read other users' transcripts with that same client, which bypasses RLS, and this is the only place a page does so.

1.6 The privacy boundary

The boundary in the system-context diagram is a red dashed line between the clients and Vercel. In code it is `lib/env.ts`. The file begins with `import "server-only"`, and every variable is a function, not a constant, so a missing required value throws at the moment it is used rather than at import.

VARIABLE	LEAVES THE SERVER?	READ BY
<code>NEXT_PUBLIC_SUPABASE_URL</code> , <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Yes, by design: browser bundle and iOS binary	<code>lib/supabase/client.ts</code> , <code>make-config.sh</code> , and the server clients
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Never	<code>lib/supabase/admin.ts</code> only
<code>ANTHROPIC_API_KEY</code>	Never	<code>lib/anthropic.ts</code> only
<code>CRON_SECRET</code>	Never; Vercel sends it, the cron routes compare it	<code>app/api/cron/*/route.ts</code>
<code>APNS_KEY_ID</code> , <code>APNS_TEAM_ID</code> , <code>APNS_BUNDLE_ID</code> , <code>APNS_PRIVATE_KEY</code>	Never; optional, and pushes are skipped when absent	<code>lib/push/apns.ts</code>

The service-role client deserves its own rule, and `lib/supabase/admin.ts` states it in one line: "Bypasses RLS. Only ever import from server code behind an admin auth check." In practice it is imported from exactly the places that need to act as nobody in particular: the two cron modules, the admin queries, and three writes that RLS would correctly refuse. Avatar upload writes to the `avatars` bucket with it. Account deletion calls `auth.admin.deleteUser` with it, for the user id taken from the caller's own session. And device registration upserts `device_tokens` with it because, as the route's comment explains, a phone handed from one account to another has to change owner, and the owner-only policy would refuse that update. In each case the decision of *who* was made by the caller's own client first.

What crosses the boundary in the other direction is also bounded. The model receives a first name, the computed chart and derived facts, the numerology numbers, today's transits, the timeline description, and the conversation. The extraction route sends only the user's own messages, never the assistant's, and writes nothing until the user confirms; its comment calls filing an inferred death or divorce onto someone's timeline "a bad thing to be wrong about." `life_events` is the one table holding what a person said about their own

life rather than what the ephemeris computed, and migration 0006 gives it no admin read policy for that reason.

WHAT IS NOT ENFORCED

The `server-only` marker is on fourteen files, but not on `lib/supabase/server.ts` or `lib/data/*`. They are safe today because they import `next/headers` or a server-only module and would fail in a client bundle anyway. A new file in `lib/data` that imported neither would not be caught. Add the marker when you add the file.

1.7 What lives on the phone

Sanchara is not a local-first app; the server holds the truth and the phone reads it. But one piece is deliberately local, and it exists because three processes on the phone need the chart and only one of them can sign in.

`mobile/Sanchara/Sources/Services/ChartCache.swift` opens with that argument: the app, the App Intents extension Siri runs, and the widget timeline provider all need the data, and neither of the last two can hold a Supabase session or wait on a network round trip. So the app writes and the others only read.

```
// mobile/Sanchara/Sources/Services/ChartCache.swift
static let appGroup = "group.com.shivvyas.astra"

private func write<T: Encodable>(_ value: T, to name: String) {
    queue.async {
        guard let data = try? JSONEncoder().encode(value) else { return }
        try? data.write(to: self.url(name), options: [.atomic, .completeFileProtection])
    }
}
```

The cache is four JSON files in the App Group container, `chart`, `details`, `reading` and `period`, plus one short string in the App Group's `UserDefaults` suite, `pendingDestination`, which an intent or a Control Center button writes milliseconds before the app launches and the app drains once. Writes are atomic and use complete file protection; reads and writes go through one serial queue. If the container is missing, which happens in a simulator build without provisioning, the cache falls back to the caches directory and simply stops sharing with the widget.

FILE	WRITTEN BY	READ BY
<code>chart</code>	<code>ProfileStore.loadChart</code> after every successful fetch of <code>birth_profiles.chart</code>	<code>ProfileStore.load</code> on launch, the kundli, <code>OnDeviceReasoner</code> , the intents' entities, <code>PeriodWidget</code> as a fallback for dates
<code>details</code>	<code>ProfileStore.load</code>	<code>IntentData.details()</code> , for the intents
<code>reading</code>	<code>DailyStore</code> and <code>IntentData</code> when a reading is fetched	<code>ReadingWidget</code> , <code>Today'sReadingIntent</code> , while <code>isFresh</code> (today, or under 24 hours old)
<code>period</code>	<code>TimelineStore</code> after each timeline load	<code>PeriodWidget</code>
<code>pendingDestination</code>	<code>OpenKundliIntent</code> , <code>DeepLink</code>	<code>DeepLink.drain</code> on launch and every return to the foreground

Two consequences follow. First, the app launches offline. `ProfileStore.load` reads the cached chart and details before it fetches, and if the fetch fails but both are present it sets the state to `ready` anyway; only a device that has never loaded a chart sees the failure screen. The chart is about 4 KB and changes only when birth details are re-saved, so a stale cache is almost never wrong. Second, the widgets never fetch.

`ReadingWidget` asks to be rebuilt at the next six o'clock, morning or evening, because that is when a new reading can exist; `PeriodWidget` asks for midnight, because a mahadasha changes about once a year but the progress bar should move by a day. `WidgetRefresh` nudges `WidgetKit` when the app itself has just received a new reading or a new period meaning, and only then, because reloads are rate-limited.

The cache holds nothing secret, no token and no session, but a birth chart is personal, so

`AuthStore.signOut` unregisters the push token, signs out, and then calls `ChartCache.shared.clear()`. The comment there says why: the files would otherwise outlive the session, and the widget would go on showing the previous person's chart to whoever signs in next.

ON-DEVICE ANSWERS

The cached chart is also what makes `OnDeviceReasoner` possible. Questions that are lookups, where a planet sits, what is in the seventh house, which dasha is running, are routed to Apple's on-device model with the chart exposed as a tool, and only interpretive questions escalate to `/api/chat`. That is the bet from 1.2 applied on the phone: the model looks a value up in a computed table; it is never asked to produce the table.

1.8 Time and the twice-daily job

`vercel.json` schedules `/api/cron/daily` twice, at 02:30 and 15:30 UTC. Vercel sends `Authorization: Bearer $CRON_SECRET` with each call, and the route refuses everything when the secret is unset rather than expose a job that spends model tokens to anyone who finds the path. It then runs two functions in order, `runDuePredictions` and `runDailyAlerts`, each over up to 500 profiles that have a chart, and returns both summaries as JSON. `maxDuration` is 300 seconds.

The route's own comment carries the design: "Both halves are decided per user rather than per run." The schedule is in UTC, but nothing about a user's reading is. Each user's `timezone` column, captured at intake from their birthplace, sets a local clock, and from that clock `lib/alerts/slots.ts` picks the slot:

```
// lib/alerts/slots.ts
export const NIGHT_STARTS_AT_HOUR = 14;

export function slotForHour(localHour: number): Slot {
  return localHour < NIGHT_STARTS_AT_HOUR ? "morning" : "night";
}

/** Whether this user still needs the reading for the slot they are currently in. */
export function isSlotDue(localHour: number, alreadyWritten: Slot[]): Slot | null {
  const slot = slotForHour(localHour);
  return alreadyWritten.includes(slot) ? null : slot;
}
```

The stored `for_date` is the user's local date, so "my reading for the 26th" means their 26th. And `daily_readings` carries `unique (user_id, for_date, slot)`, which migration 0005 describes as "what makes the job safe to run more often than expected, or twice by accident." `writeDueReading` reads the slots already written for today, returns early when the current one exists, and if two runs race past that check the insert fails with Postgres error `23505`, which the code treats as "already done" rather than as an error. That is the whole idempotency argument for the readings: extra runs find nothing to do, and a race costs one wasted model call, never a duplicate row or a duplicate push.

The dosha sweep is idempotent for a different reason. `runAlertsForUser` recomputes today's sky at the user's birthplace with the same `computeChart` the chat uses, detects the conditions it forms against the natal chart, and diffs them against the open rows in `transit_conditions`, where a row is open while `ended_on` is null. What is in the new set and not the old has started; what is in the old and not the new has ended; what is in both is ignored, which is what stops a standing Sade Sati from becoming a daily nag. A partial unique index, `transit_conditions_open_signature` on `(user_id, signature)` where `ended_on` is null, guarantees one open row per condition however often the sweep runs. Only `caution` and `warning` severities reach a lock screen, at most six per alert, and a natal condition is never announced after a user's first sweep, because a natal condition cannot start or end and a change in one is an artifact of the computation changing. `deriveHadNatalRows` fails closed on a count error for exactly that reason, and its comment names the bug it prevents: pushing "Mangal Dosha has begun" to someone whose chart has not changed since birth.

Pushes go out through `ApnsClient`, which keeps one HTTP/2 session per environment for the life of a run and reuses a provider token for 45 minutes. A token Apple reports as unregistered is deleted from `device_tokens` on the spot, since Apple only says a token is dead when you use it. When APNs is not configured the readings and alerts are still written; the phone reads them from the tables and the widget from the cache, and only the notification is missing.

THE COST SHAPE

Each reading's system prompt has two parts. The chart block is identical for a user on every run and is sent with a one-hour cache marker, so a morning and night reading that land within the hour share it; the today block changes daily and is not cached. The readings run on `READING_MODEL` at low effort. The alert copy in `lib/alerts/compose.ts` runs on `DEEP_READING_MODEL`, also at low effort.

1.9 Study notes

READ IN THIS ORDER

1. `lib/env.ts` , for the boundary. Twenty lines, and every secret in the system is one of its functions.
2. `lib/supabase/route.ts` , then `server.ts` and `middleware.ts` , for the two auth channels meeting in one `Db` type.
3. `app/api/chat/route.ts` , for the bet in practice: watch what is computed before the model is called.
4. `lib/astrology/chart.ts` , for the engine, and `lib/data/birthProfile.ts` , for where its output lives and how it upgrades itself.
5. `mobile/project.yml` , for the three targets and the six files the widget may compile.
6. `mobile/Sanchara/Sources/Services/ChartCache.swift` , then `SancharaAPI.swift` and `Supabase.swift` , for what the phone keeps, what it asks the server for, and what it reads directly.
7. `lib/alerts/slots.ts` , `predictions.ts` and `run.ts` , with migrations 0004 and 0005 open beside them, for the two idempotency arguments.

CHECK YOURSELF

Why is the anon key safe to ship in the iOS binary?

Because every table has Row Level Security with an owner-only policy, so the key grants nothing without a user's JWT and only that user's rows with one. The same key is in every web page.

How does a route handler tell a web caller from an iOS caller?

It does not need to. `createRouteSupabase` builds a bearer client when an `Authorization: Bearer` header is present and falls through to the cookie client otherwise; both are a `SupabaseClient` .

Name the three writes that use the service-role client on behalf of a signed-in user, and what they have in common.

Avatar upload to Storage, account deletion, and device-token registration. In each the user id comes from the caller's verified session first; the admin client only performs a write RLS would refuse.

Which files may the widget extension compile, and what do they have in common?

The six named in `project.yml` : `Theme` , `ReadingCard` , `ChartCache` , `ChartData` , `BirthProfileDetails` and `OpenKundliIntent` . None imports Supabase or the API client, because the extension has no session.

The cron runs at 02:30 and 15:30 UTC. Why does a user in any timezone still get a morning and a night reading?

The slot is chosen from the user's local hour (before 14:00 is morning), the date stored is their local date, and the unique index on `(user_id, for_date, slot)` means a run that finds the slot written does nothing.

What makes the dosha sweep safe to run twice in a day?

It diffs today's conditions against the open rows in `transit_conditions` ; a second run finds the same set open, so nothing has started or ended, and no alert is written. The partial unique index enforces one open row per signature.

Where does a stale chart get fixed, and why is there no migration for it?

In `ensureCurrentChart` , on read. Both traditions are recomputed and rewritten the first time the owner uses the app after a schema change, so no backfill job has to touch every row.

2 The chart engine

Nothing about a chart is guessed. `lib/astrology/` turns a date, a time and a place into planets, houses, nakshatras, dashas, doshas and transits with the Swiss Ephemeris, and every later chapter reads from that output. Claude interprets these numbers; it never produces them.

This chapter describes the engine as it is on `main` on 30 September 2026, version 0.3.0. It is a pure computation layer: no database, no network beyond one geocoding call on the input side, no interpretation. The same nine files serve the web app, the iOS app, the kundli PDF, the timeline, the twice-daily alerts job and the chat. If a number in a reading is wrong, this is where it went wrong, and the tests in section 2.10 are how you find out.

2.1 What a chart is here

A birth chart is the sky at one moment, seen from one place on Earth, projected onto the ecliptic and cut into twelve signs of 30 degrees. The engine computes it in two traditions, and almost everything that follows depends on which one is asked for.

- **Tropical and sidereal.** Western astrology measures longitude from the vernal equinox (tropical). Vedic astrology measures it from the fixed stars (sidereal). The two zodiacs drift apart by about 50 arcseconds a year and the gap today is roughly 24 degrees. That gap is the **ayanamsa**; the engine uses Lahiri's value, the one Indian almanacs use. A Sun at 10 degrees Capricorn tropical is at about 16 degrees Sagittarius sidereal, which is why one birth has a different Sun sign in each tradition.
- **The ascendant** (lagna in Vedic usage) is the point of the ecliptic rising in the east at the birth moment. It passes through all twelve signs every day, which is why the birth time matters.
- **Houses** divide the sky into twelve regions of life, counted from the ascendant. In the **whole-sign** system the ascendant's sign is house 1, the next sign house 2, and so on. In **Placidus** the houses are unequal arcs computed from the latitude and the sidereal time, and one can span parts of two signs. Vedic charts here are whole-sign, Western charts Placidus; section 2.4 explains why the engine once got that wrong and how a stored chart is corrected.
- **Nakshatras** are the 27 lunar mansions of the sidereal zodiac, each 13 degrees 20 minutes wide. The Moon's nakshatra at birth decides the dasha.
- **Dashas** are planetary periods. The Vimshottari system assigns 120 years of life to nine lords in a fixed order; the **mahadasha** is the major period and the **antardasha** the sub-period inside it. Where you are in that sequence is what "which dasha am I in" asks.
- **Doshas** (and the yogas the code files alongside them) are named afflictions read from positions: Mars in certain houses, every planet on one side of the nodes, and so on.
- **Gochara** is transit: today's sky read against the birth chart, by the house each moving planet occupies counted from the ascendant and from the natal Moon. Sade Sati, Saturn's seven and a half years around the natal Moon, is the best known.

The whole output is one JSON value, typed in `lib/astrology/types.ts` :

```
// lib/astrology/types.ts
export type Chart = {
  tradition: Tradition; // "vedic" | "western"
  ascendant: { sign: string; degree: number };
  houses: number[]; // 12 cusp longitudes
  planets: Planet[]; // name, sign, degree, house, retrograde, nakshatra?
  moonSign: string;
  sunSign: string;
  ayanamsa?: number;
  dasha?: DashaInfo; // Vedic only: current mahadasha and antardasha
  /** Bumped when the computation changes in a way that moves stored values. */
  schemaVersion?: number;
  /** Classical readings of the positions above. See lib/astrology/derived.ts. */
  derived?: Derived;
};
```

Three things about this shape decide the rest of the chapter. A planet is stored as a sign and a degree within that sign (0 to 30), not as an absolute longitude; every module that needs the longitude rebuilds it as `signIndex(sign) * 30 + degree`. The `houses` array is always the twelve Placidus cusps, even for a Vedic chart, and nothing renders from it. And `dasha` and `derived` exist only for Vedic, because every technique in them is Vedic doctrine.

2.2 Files and the input side

```
lib/astrology/
├─ types.ts      BirthInput, Planet, Chart, DashaInfo, Tradition, ChatMode
├─ constants.ts  SIGNS, NAKSHATRAS, signOf, degreeInSign, nakshatraOf,
│               houseFrom, ORDINAL, CHART_SCHEMA_VERSION
├─ chart.ts      computeChart: the ephemeris, houses, Ketu, dasha, derived
├─ dasha.ts      Vimshottari: startingDashaLord, computeVimshottari, vimshottariTimeline
├─ derived.ts    lords, dignity, combustion, aspects, deriveFacts, factsFor, isChartStale
├─ doshas.ts     detectNatalDoshas, detectTransitAfflictions, Condition, highestSeverity
├─ transits.ts   transitChart (cached), describeGochara, describeToday
├─ numerology.ts mulank, bhagyank, name number, personal cycle, Lo Shu, relationships
└─ prompt.ts     renders all of the above as text for the model (chapter 3)

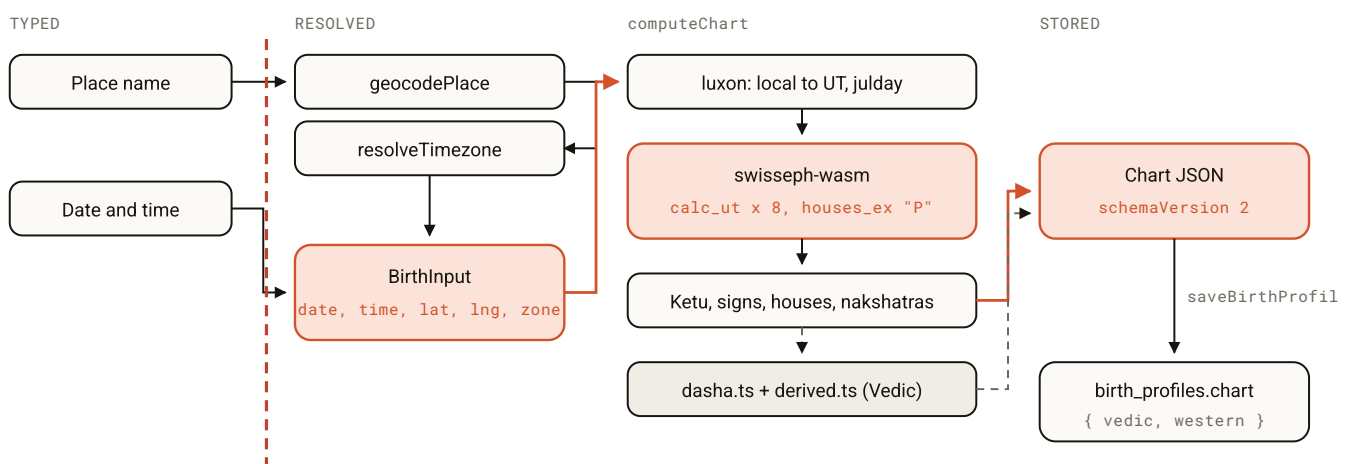
lib/geo.ts       geocodePlace (Open-Meteo), resolveTimezone (tz-lookup)
app/api/geocode/ GET ?q= wraps geocodePlace for the intake forms
lib/data/birthProfile.ts saveBirthProfile computes both charts; ensureCurrentChart upgrades
```

The engine's input is a `BirthInput`: a date as `YYYY-MM-DD`, a time as `HH:mm`, a latitude, a longitude and an IANA time zone name. The place name a person types never reaches the engine. `lib/geo.ts` turns it into those numbers.

```
// lib/geo.ts
export function resolveTimezone(lat: number, lng: number): string {
  return tzlookup(lat, lng);
}

export async function geocodePlace(query: string): Promise<GeoResult[]> {
  const q = query.trim();
  if (q.length < 2) return [];
  const url = `https://geocoding-api.open-meteo.com/v1/search?name=${encodeURIComponent(q)}&count=5`;
  const res = await fetch(url, { headers: { accept: "application/json" } });
  if (!res.ok) return [];
  ...
  return (data.results ?? []).map((r) => ({
    name: [r.name, r.admin1, r.country].filter(Boolean).join(", "),
    lat: r.latitude,
    lng: r.longitude,
    timezone: resolveTimezone(r.latitude, r.longitude),
    country: r.country ?? "",
  }));
}
```

Two choices are worth naming. The geocoder is Open-Meteo's free endpoint, called without a key and capped at five results, and any failure (a short query, a non-2xx response) returns an empty list rather than an error. And the time zone is not taken from the geocoder: `tz-lookup` resolves it offline from the coordinates, so the zone is consistent with the point actually chosen. That is why the only test in `lib/geo.test.ts` is two coordinate pairs, New Delhi to `Asia/Kolkata` and New York to `America/New_York`. The offset in force on the birth date, daylight saving included, is then luxon's job inside `computeChart`.



Left of the line is a form. Everything right of it runs inside a route handler; the WASM ephemeris is in-process, and the only network call is to the geocoder.

From typed birth details to stored JSON. `saveBirthProfile` calls `computeChart` twice, once per tradition, and writes the pair in one row. The dashed edges exist only for a Vedic chart.

2.3 Positions: the ephemeris

The Swiss Ephemeris is the standard high-precision planetary library, here as the `swissep-wasm` package, which compiles it to WebAssembly so that it runs inside a Next.js route handler with no native binary.

`chart.ts` initialises it once per server process and keeps the promise, so concurrent first requests share one initialisation rather than racing to build two:

```
// lib/astrology/chart.ts
let swePromise: Promise<InstanceType<typeof SwissEph>> | null = null;
async function getSwe() {
  if (!swePromise) {
    swePromise = (async () => {
      const swe = new SwissEph();
      await swe.initSwissEph();
      return swe;
    })();
  }
  return swePromise;
}
```

`computeChart(input, tradition)` then runs four numbered steps, which the file itself labels.

1. LOCAL TIME TO UNIVERSAL TIME

luxon parses `"${birthDate}T${birthTime}"` in the input's IANA zone, converts it to UTC, and the engine asks for the Julian day of that instant with `swe.julday(year, month, day, decimalHour)`. A birth at 06:30 in Kolkata and one at 06:30 in New York are different skies, and luxon knows what offset was in force on that date.

2. FLAGS

Every call passes `SEFLG_SWIEPH | SEFLG_SPEED`: use the Swiss Ephemeris data files, and return velocities, because retrograde is read from speed. For a Vedic chart the engine also calls

`set_sid_mode(SE_SIDM_LAHIRI, 0, 0)`, adds `SEFLG_SIDEREAL`, and records `get_ayanamsa_ut(jd)` to three decimals as the chart's `ayanamsa`. A Western chart is tropical and has none.

3. HOUSES AND THE ASCENDANT

`houses_ex(jd, iflag, lat, lng, "P")` returns thirteen cusps (index 1 to 12 used) and an `ascmc` array whose first element is the ascendant longitude. The letter is Placidus, in both traditions. The file's comment records the subtlety that makes this usable for Vedic: "In sidereal mode swisseph applies ayanamsa to cusps too", so the ascendant comes back sidereal and `signOf(ascLon)` is the lagna sign. The cusps become `chart.houses`, rounded to two decimals; for Vedic they are carried but never used to place a planet.

4. PLANETS

Eight bodies are asked for, in the order Sun, Moon, Mercury, Venus, Mars, Jupiter, Saturn and Rahu, where Rahu is `SE_MEAN_NODE`, the mean lunar north node. Each `calc_ut` returns `[lon, lat, dist, lonSpeed, ...]`; the engine keeps the longitude and the longitude speed, and a planet is `retrograde` exactly when that speed is negative. The Moon's normalised absolute longitude is kept aside for the dasha calculation. Each planet is stored as:

```
planets.push({
  name: body.name,
  sign: signOf(lon),
  degree: Number(degreeInSign(lon).toFixed(2)),
  house: tradition === "vedic" ? houseFrom(ascSign, signOf(lon)) : houseOf(lon, cusps),
  retrograde: speed < 0,
  nakshatra: tradition === "vedic" ? nakshatraOf(lon) : undefined,
});
```

Ketu is not an ephemeris body. It is the south node, always exactly opposite Rahu, so the engine rebuilds Rahu's longitude from its stored sign and rounded degree, adds 180, and pushes a ninth planet with `retrograde: true` (the mean nodes always move backwards). Rounding first can move Ketu by up to a hundredth of a degree, well below anything rendered.

The helpers in `constants.ts` are one-line modular arithmetic on a longitude normalised into 0 to 360: `signOf` divides by 30, `degreeInSign` takes the remainder, `nakshatraOf` divides by 360/27. The chart then records `sunSign` and `moonSign`, the dasha for Vedic, the schema version, and, for Vedic only, the derived block (section 2.6).

COST

A chart is eight `calc_ut` calls and one `houses_ex` on an in-process WASM module: milliseconds after the first request, more on the cold start that initialises the module. `chart.test.ts` computes six full charts in about 50 ms. There is no per-chart cost beyond CPU.

2.4 Houses: whole-sign and Placidus

Two functions place a planet in a house, and which one runs is decided by the tradition.

```

// lib/astrology/constants.ts
/** The 1-indexed house of `sign` counted from `from`, the classical "Nth from". */
export function houseFrom(from: string, sign: string): number {
  const a = signIndex(from);
  const b = signIndex(sign);
  if (a < 0 || b < 0) return 0;
  return ((b - a + 12) % 12) + 1;
}

// lib/astrology/chart.ts
function houseOf(longitude: number, cusps: number[]): number {
  const lon = ((longitude % 360) + 360) % 360;
  for (let i = 0; i < 12; i++) {
    const start = cusps[i];
    const end = cusps[(i + 1) % 12];
    if (start <= end) {
      if (lon >= start && lon < end) return i + 1;
    } else {
      // wraps past 360°
      if (lon >= start || lon < end) return i + 1;
    }
  }
  return 1;
}

```

`houseFrom` is whole-sign counting and it is the most-called function in the engine: the chart, the derived facts, every dosha detector and every transit line use it, with the ascendant sign or the natal Moon's sign as `from`. It returns 0 for an unknown name so a caller can tell the failure apart from house 1. `houseOf` walks the Placidus cusps, handling the one pair that wraps through 0 degrees Aries, and is used only for Western charts.

	VEDIC	WESTERN
Zodiac	Sidereal, Lahiri ayanamsa	Tropical
Ascendant	<code>ascmc[0]</code> , sidereal	<code>ascmc[0]</code> , tropical
Planet house	<code>houseFrom(ascSign, sign)</code> : whole-sign	<code>houseOf(lon, cusps)</code> : Placidus
<code>chart.houses</code>	Placidus cusps, carried, unused	Placidus cusps, the truth
Nakshatra	Per planet	Absent
<code>dasha</code> , derived	Present	Absent

THE UPGRADE OF STORED CHARTS

Until the change recorded in `docs/design/2026-09-17-specific-readings.md`, section 0, Vedic planets were placed by `houseOf` too. The kundli renderers and the PDF had always drawn the houses whole-sign (house 1 labelled with the lagna's sign) and then filled them by the stored Placidus number, so a planet could be drawn inside a house labelled with a sign it was not in. Placidus also lets two houses carry the same sign

and one sign appear in none, which leaves "the lord of the 7th" undefined, and every technique in `derived.ts` rests on lordship. So Vedic became whole-sign:

```
// lib/astrology/constants.ts
/**
 * Version 2 moved Vedic charts from Placidus to whole-sign houses. A stored
 * chart below this version has house numbers that disagree with the rashi
 * labels every renderer draws, so readers upgrade it before use.
 */
export const CHART_SCHEMA_VERSION = 2;
```

A chart written before then carries no `schemaVersion` at all, which `isChartStale` reads as 1. The rule in `derived.ts` is that a chart is stale when its version is below 2 or it has no `derived` block. Two mechanisms then cooperate so the app ships without a migration or a backfill job:

- **Readers recompute houses from the sign, never from `planet.house`.** `deriveFacts` builds its own house map with `houseFrom(asc, p.sign)`, and the dosha detectors do the same, so a stale chart still derives correct facts in memory. Both test files pin it with fixtures whose stored house is deliberately 99.
- **The stored row is rewritten on read.** `ensureCurrentChart` in `lib/data/birthProfile.ts` runs on every `getBirthProfile`. If the stored Vedic chart is stale it recomputes both traditions together, so a profile is never half upgraded, writes the pair back and returns the fresh row. A compute or write failure is logged and swallowed: an upgrade that cannot be written is not a reason to fail the request that triggered it. The rewrite is needed because the kundli, the PDF and the iOS app draw from the stored numbers.

A VISIBLE CHANGE

Whole-sign moves planets. One in the lagna's sign at a degree below the lagna's degree moves from house 12 to house 1, and similar shifts happen throughout. The design note records that users' charts change when next computed and that whether to say so in the app is a product decision. It also records the consequence for alerts: a house change alters which natal doshas are detected, and the alerts job suppresses natal conditions for anyone who already has a natal row so the migration cannot push "Mangal Dosha has just begun". See chapter 9.

2.5 Nakshatras and dashas

The 27 nakshatras are listed in `constants.ts` in zodiacal order from Ashwini at 0 degrees Aries. Each is 360/27 degrees, and `nakshatraOf` is just the floor of the longitude over that span. The names, nine per row, are:

INDEX	NAKSHATRAS
0 to 8	Ashwini, Bharani, Krittika, Rohini, Mrigashira, Ardra, Punarvasu, Pushya, Ashlesha
9 to 17	Magha, Purva Phalguni, Uttara Phalguni, Hasta, Chitra, Swati, Vishakha, Anuradha, Jyeshtha
18 to 26	Mula, Purva Ashadha, Uttara Ashadha, Shravana, Dhanishta, Shatabhisha, Purva Bhadrapada, Uttara Bhadrapada, Revati

The rows are not decorative. Vimshottari assigns each nakshatra a lord, and the lords repeat every nine nakshatras, so the lord of nakshatra `n` is `ORDER[n % 9]`. That is the entire content of `startingDashaLord`. The order and the years are:

POSITION	LORD	YEARS	STARTS UNDER IT
0	Ketu	7	Ashwini, Magha, Mula
1	Venus	20	Bharani, Purva Phalguni, Purva Ashadha
2	Sun	6	Krittika, Uttara Phalguni, Uttara Ashadha
3	Moon	10	Rohini, Hasta, Shravana
4	Mars	7	Mrigashira, Chitra, Dhanishta
5	Rahu	18	Ardra, Swati, Shatabhisha
6	Jupiter	16	Punarvasu, Vishakha, Purva Bhadrapada
7	Saturn	19	Pushya, Anuradha, Uttara Bhadrapada
8	Mercury	17	Ashlesha, Jyeshtha, Revati

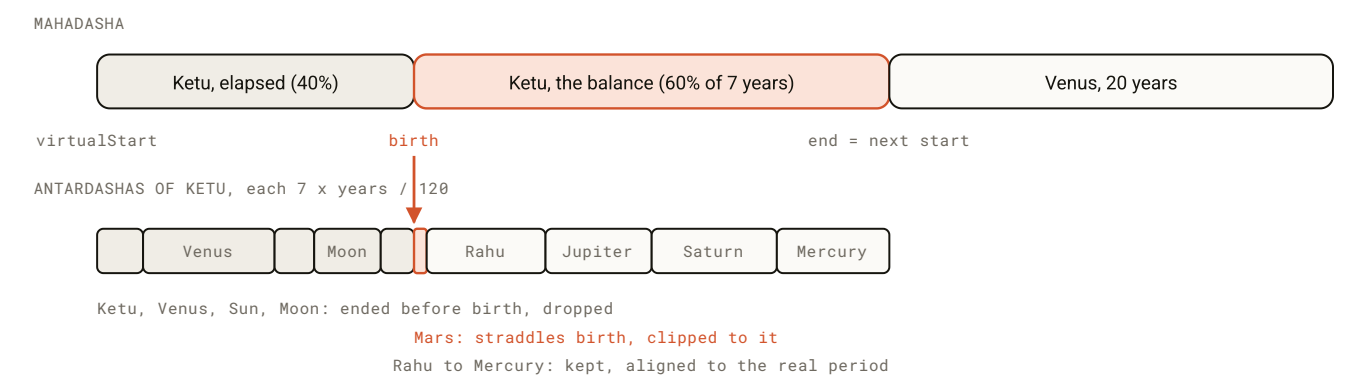
The years sum to 120, the `TOTAL` constant. A year is 365.25 days (`addYears` adds `y * 365.25` days), which the file labels the Vedic year and which the tests pin: seven Ketu years from 1 January 1990 end on 31 December 1996.

THE VIRTUAL START

Nobody is born at the start of a mahadasha. The Moon at birth is some fraction of the way through its nakshatra, and that fraction of the first lord's period has already elapsed. The first span therefore begins at birth and runs for the **balance**, `years * (1 - fractionElapsed)`. But the antardashas inside it must be laid out from where the mahadasha would have begun, not from birth, or every sub-period for the rest of that mahadasha lands in the wrong place. `dasha.ts` carries that point as `virtualStart`:

```
// lib/astrology/dasha.ts
const first = ORDER[startLordIndex];
const spans: Span[] = [{
  lord: first.lord,
  start: birthUt,
  // The first period is the *balance* of the one already running at birth.
  end: addYears(birthUt, first.years * (1 - fractionElapsed)),
  virtualStart: addYears(birthUt, -(first.years * fractionElapsed)),
  fullYears: first.years,
}];
for (let i = 1; i < 12; i++) {
  const o = ORDER[(startLordIndex + i) % 9];
  const start = spans[i - 1].end;
  spans.push({ lord: o.lord, start, end: addYears(start, o.years), virtualStart: start, fullYear
}
```


Twelve spans, not nine: the cycle is 120 years and the engine lays out a third of a second cycle so a long life still has a current period. Each span's nine antardashas run in the same order starting from the span's own lord, each lasting `fullYears * subYears / 120` of the parent, laid out from `virtualStart`. A sub-period that ended before birth is dropped; one straddling birth is clipped to start at birth. As the comment puts it, a chart has nothing to say about time the native was not alive for.



A Moon 40% of the way through Ashwini. The balance of Ketu runs from birth; the sub-periods are measured from the virtual start so that they end exactly where the mahadasha does, which `dasha.test.ts` asserts for every one of the twelve.

Two exported functions share this layout. `computeVimshottari(moonLon, birthUt, now)` answers "where am I now": it finds the span containing `now` (the current UTC instant by default, injectable so tests are deterministic), then the antardasha containing it, and returns both as ISO dates in the `DashaInfo` the chart stores; outside every span it falls back to the first, which keeps the function total. `vimshottariTimeline(moonLon, birthUt)` returns all twelve spans with their antardashas nested, and is what `lib/timeline/build.ts` draws the life timeline from, rebuilding the Moon's longitude from the stored sign and degree. The chart itself carries only the current pair, because that is the only part that ages.

2.6 Derived facts

`derived.ts` exists because a model handed nine planet lines and a lagna writes sign-level prose. What makes an answer specific is which house a planet rules, whether it is strong or weak where it sits, whom it aspects, and where the current dasha lord actually is. The module's header states the scope: "a fixed classical table applied to positions the ephemeris already produced. There is no I/O and no interpretation: the output is the grounding a reading is written from, not the reading."

THE TABLES

SIGN	LORD	SIGN	LORD	SIGN	LORD
Aries	Mars	Leo	Sun	Sagittarius	Jupiter
Taurus	Venus	Virgo	Mercury	Capricorn	Saturn
Gemini	Mercury	Libra	Venus	Aquarius	Saturn
Cancer	Moon	Scorpio	Mars	Pisces	Jupiter

`SIGN_LORD` is declared in zodiacal order and a comment forbids reordering it: both `oppositeSign` and the house loop index into `Object.keys(SIGN_LORD)`. Rahu and Ketu rule nothing, which is the first of three rules the module deliberately leaves out.

PLANET	EXALTED (DEEP DEGREE)	DEBILITATED	MOOLATRIKONA	COMBUSTION ORB, DIRECT / RETROGRADE
Sun	Aries 10	Libra	Leo 0 to 20	never
Moon	Taurus 3	Scorpio	Taurus 4 to 30	12 / 12
Mars	Capricorn 28	Cancer	Aries 0 to 12	17 / 17
Mercury	Virgo 15	Pisces	Virgo 16 to 20	14 / 12
Jupiter	Cancer 5	Capricorn	Sagittarius 0 to 10	11 / 11
Venus	Pisces 27	Virgo	Libra 0 to 15	10 / 8
Saturn	Libra 20	Aries	Aquarius 0 to 20	15 / 15
Rahu, Ketu	none	none	none	never

The debilitation sign is not a table; it is `oppositeSign` of the exaltation sign. The moolatrikona ranges use a half-open interval, `from <= degree < to`.

DIGNITY, AND THE ONE THREE-WAY CASE

`dignityOf(name, sign, degree)` returns one of `exalted`, `debilitated`, `moolatrikona`, `own` or `neutral`, plus `fromDeepPoint`, the distance in degrees from the exact exaltation degree, reported only when the planet is in its exaltation or debilitation sign. Exaltation and debilitation are read by sign, not by degree, so the Sun at 29 Aries is still exalted; the deep point is extra information, not a threshold. The precedence is exalted, then moolatrikona, then own, then debilitated, then neutral.

Two planets break the simple chain: Mercury's exaltation and moolatrikona are both Virgo, and the Moon's are both Taurus. For them a dedicated block owns the whole sign: below the moolatrikona range's `from` the planet is exalted, inside it moolatrikona, above it own sign if the planet rules the sign (Mercury rules Virgo; the Moon's range runs to 30 so the question never arises). The comment above that block warns that it "must return on every path", because falling through would hit the unconditional exaltation check, and asks the reader not to simplify it into a plain if/else chain. So Mercury at 14 Virgo is exalted, at 18 moolatrikona, at 25 own, and at 15.5 exalted, because 15.5 is below `from: 16` even though it is past the deep degree of 15.

COMBUSTION AND ASPECTS

A planet is combust when its shorter arc from the Sun is at or within its orb, and the orb depends on whether it is retrograde: Mercury tightens from 14 to 12 and Venus from 10 to 8, the others do not change. The Sun and the nodes are never combust. `arcFromSun` rebuilds both longitudes from sign and degree and takes the shorter way round, so Venus at 4 Leo is 6 degrees from a Sun at 10 Leo, and Venus at 5 Cancer is 35 degrees away, not 325.

Aspects (graha drishti) are by house, not by degree. Every planet aspects the 7th house from its own; Mars adds the 4th and 8th, Jupiter the 5th and 9th, Saturn the 3rd and 10th. The nodes get the 7th only: the second deliberately absent rule, because the tradition does not agree on a nodal 5th and 9th. `aspectedHouses(name, house)` wraps past 12, so Saturn in the 12th aspects the 2nd, 6th and 9th.

DERIVEFACTS

`deriveFacts(chart)` assembles four lists. For each planet a `PlanetFact`: its whole-sign house (recomputed from the sign, as section 2.4 explained), the houses it `rules` (its signs, each counted from the ascendant, sorted), its dignity and deep-point distance, whether it is combust and how far it is from the Sun, the houses it aspects, the planets in those houses, and the planets it is `conjunct` with (same house, itself excluded). For each of the twelve houses a `HouseFact`: its sign, its lord, where that lord sits and in what dignity, its occupants, and which planets aspect it. For the dasha, two `DashaFact` entries (mahadasha and antardasha) each carrying its lord's own `PlanetFact` as `placement`, which is what lets a reading say the Jupiter period is running from the 8th house rather than just "Jupiter period". And `conditions`, the natal doshas from section 2.7.

The third absent rule is friendship. Dignity never consults planetary friendship, because, in the header's words, friendship is a compound judgement, natural plus temporal, and half of it is misleading rather than partial. (Numerology does use a natural friendship table, for a different purpose; see section 2.9.)

Two small functions decide who gets these facts:

```
// lib/astrology/derived.ts
export function isChartStale(chart: Chart): boolean {
  return (chart.schemaVersion ?? 1) < CHART_SCHEMA_VERSION || !chart.derived;
}

export function factsFor(chart: Chart): Derived | undefined {
  if (chart.tradition !== "vedic") return undefined;
  return chart.derived ?? deriveFacts(chart);
}
```

`factsFor` is what the chat route and the alerts job call. A Western chart gets `undefined`, and the callers fall back to plain chart data rather than resurrecting Vedic technique for a tropical chart. The comment at the end of `computeChart` says the same thing from the other side: "Every technique in derived.ts is Vedic. Western continues to use Placidus houses and gets no derived block at all."

2.7 Doshas

`doshas.ts` reads afflictions from a chart, and from today's chart against a birth chart, into one shape:

```
// lib/astrology/doshas.ts
export type Condition = {
  kind: string;
  signature: string;
  label: string;
  severity: "info" | "caution" | "warning";
  scope: "natal" | "transit";
  /** A plain factual sentence, the grounding the model is allowed to interpret. */
  detail: string;
};
```

The `signature` is what makes the alerts job possible. It folds in the phase and the sign, so Saturn moving to the next sign, or Sade Sati stepping from rising to peak, reads as a new condition rather than the same one continuing. The daily job diffs signatures against the rows it stored last time; one that appears has started, one that disappears has ended, and only those two events send a push. The `detail` is a sentence of fact with the ordinal already in it, and it is the only text the model is given about the condition.

NATAL DOSHAS

`detectNatalDoshas(chart)` runs six checks. They never change for a person, so the job reports them once, when a chart is first read.

KIND	RULE AS CODED	SEVERITY	SIGNATURE
<code>mangal_dosha</code>	Mars in house 1, 2, 4, 7, 8 or 12 from the ascendant, house derived with <code>houseFrom</code> from the sign	caution	<code>mangal_dosha: <house></code>
<code>kaal_sarp</code>	All seven classical planets present, and their arcs measured forward from Rahu are all strictly between 0 and 180, or all above 180	warning	<code>kaal_sarp: <Rahu sign></code>
<code>grahan_sun</code> , <code>grahan_moon</code>	The Sun or the Moon sharing a sign with Rahu or Ketu; one condition per pair	caution	<code>grahan_sun: <node>: <sign></code>
<code>kemadruma</code>	No planet in the 2nd or 12th sign from the Moon, ignoring the Sun and the nodes	caution	<code>kemadruma: <Moon sign></code>
<code>pitru_dosha</code>	Rahu or Ketu in the 9th from the ascendant, from the sign	info	<code>pitru_dosha: <node>: <sign></code>
<code>shrapit_dosha</code>	Saturn and Rahu in the same sign	caution	<code>shrapit_dosha: <sign></code>

Kaal Sarp is the only detector that needs degrees; a planet exactly on the axis breaks the yoga, which the strict inequalities encode. The Mangal and Pitru checks carry a comment on why they compute the house rather than read `mars.house`: a chart stored before the whole-sign change carries a stale Placidus number, and the natal conditions must agree with the whole-sign facts in the same `Derived` object, or, as `derived.test.ts` puts it, one object would carry "two contradictory claims about the same fixed birth fact".

TRANSIT AFFLICTIONS

`detectTransitAfflictions(natal, transit)` is the set that comes and goes, and the set worth a notification when it does. Every rule is a whole-sign count from the natal Moon or the ascendant, or a shared sign.

KIND	RULE AS CODED	SEVERITY
<code>sade_sati</code>	Transit Saturn in the 12th (rising), 1st (peak) or 2nd (setting) from the natal Moon	warning at peak, else caution
<code>shani_dhaiya</code>	Transit Saturn in the 4th or 8th from the natal Moon, only when Sade Sati did not match	caution
<code>kantaka_shani</code>	Transit Saturn in the 4th, 7th or 10th from the ascendant	info
<code>node_over_moon_rah</code> , <code>node_over_moon_ketu</code>	A transiting node in the natal Moon's sign	caution
<code>node_over_lagna_rah</code> , <code>node_over_lagna_ketu</code>	A transiting node in the ascendant sign	caution
<code>angarak_transit</code>	Transit Mars in the natal Moon's sign	caution
<code>ashtama_surya</code>	Transit Sun in the 8th from the natal Moon	info
<code>guru_6_8_12</code>	Transit Jupiter in the 6th, 8th or 12th from the natal Moon	info
<code>guru_chandal_transit</code>	Transit Jupiter and transit Rahu in the same sign	info
<code>retrograde_saturn</code> , <code>retrograde_mars</code> , <code>retrograde_mercury</code>	That planet retrograde and in the ascendant sign or the natal Moon's sign; a retrograde elsewhere is ignored	info

Every transit signature includes the transiting sign, and Sade Sati's includes the phase, so a Saturn ingress or a phase change is a fresh event. `detectConditions` concatenates the natal and transit lists, and `highestSeverity` reduces a list to its worst entry with `info` as the floor, which the alerts job uses to decide whether a push is warranted.

2.8 Transits (gochara)

A transit chart is a birth chart for now: the same `computeChart` , with today's date, and the user's birthplace as the location. `transits.ts` adds a cache and a renderer around it.

```
// lib/astrology/transits.ts
const cache = new Map<string, Promise<Chart>>();
const MAX_ENTRIES = 200;

export async function transitChart(args) {
  const day = args.day ?? utcDayKey();
  // One degree is ~111km; users in the same city share an entry.
  const key = `${day}|${args.tradition}|${args.lat.toFixed(0)}|${args.lng.toFixed(0)}`;
  const hit = cache.get(key);
  if (hit) return hit;
  const pending = computeChart(
    { birthDate: day, birthTime: "12:00", lat: args.lat, lng: args.lng, timezone: "UTC" },
    args.tradition,
  );
  if (cache.size >= MAX_ENTRIES) cache.clear();
  cache.set(key, pending);
  try { return await pending; }
  catch (err) { cache.delete(key); throw err; }
}
```

The sample point is midday UTC, for two reasons the file gives. The obvious one is no ephemeris run per message. The one that pays is that the text built from this chart is identical for the whole UTC day, so it can sit in the cached half of the system prompt (chapter 3) instead of being billed on every turn. Only the Moon moves enough within a day to show at sign resolution, and sampling the middle halves its worst-case drift. The key rounds the coordinates to whole degrees so a city shares one entry; the cache holds promises so concurrent misses share one computation; it clears itself at 200 entries rather than hold stale days; and a failed computation is evicted, never cached.

One caller does not use this cache. The alerts job's condition diff in `lib/alerts/run.ts` calls `computeChart` directly with the actual UTC time of the run, since it is deciding whether an affliction has started at that moment. The reading text the same job writes goes through `transitChart`, for the prompt-cache reason.

DESCRIBEGOCHARA

The old transit text was "Saturn in Pisces, Jupiter in Gemini", which the file notes "was true of everyone alive on that date and related to the reader not at all". What makes a transit mean anything is the house it falls in from the lagna and from the natal Moon. `describeGochara(natal, transit, { includeAfflictions })` writes one line per transiting planet: its sign and retrograde flag; for a Vedic chart its house from the ascendant and its ordinal from the Moon; and any natal planets it sits on by sign, joined so three names read "a, b, and c". For a Vedic chart it then appends a line per transit affliction. The flag exists because the alerts job renders the afflictions itself, filtered by severity, and `false` avoids saying the same thing twice in one prompt; the chat route has no other source and takes the default.

A Western chart gets no house counting and no afflictions, and the comment says why: the house numbers would be whole-sign where Western is Placidus, and Sade Sati has no meaning in that tradition. It keeps the plain sky, where each body is and which natal body it sits on. `describeToday` is the last export: a weekday, a date and one of morning, afternoon, evening or night (thresholds 12, 17 and 21) in the user's zone, coarse enough to stay identical for hours.

2.9 Numerology

`numerology.ts` is the one module with no ephemeris in it. Everything is digit arithmetic on the birth date string, and the core operation is `reduceToDigit`, which sums the digits of a number repeatedly until one digit remains. It never preserves the "master numbers" 11 and 22 some systems keep; every result here is 1 to 9.

FUNCTION	INPUT	RULE	EXAMPLE FROM THE TESTS
<code>computeNumerology : mulank</code>	Day of birth	Reduce the day	23rd gives 5
<code>computeNumerology : bhagyank</code>	Full date	Reduce the sum of all eight digits	1990-01-23 gives 25, then 7
<code>computeNameNumber</code>	A name	Pythagorean: A=1 to I=9, J=1 again, letters only, reduced; 0 when there are no letters	
<code>personalYear</code>	Birth date, today	Reduce birth month + birth day + current year	15 July 1990 in 2026: 7+15+2026 = 2048, then 5
<code>personalMonth</code>	Birth date, today	Reduce personal year + current month	5 + 9 = 14, then 5
<code>personalCycle</code>	Birth date, today	Both of the above as <code>{ year, month }</code>	22 March 1985 on 10 January 2026: year 8, month 9
<code>loShu</code>	Birth date	Count each digit 1 to 9 in the eight digits; zeros are not placed on the grid	1990-07-15: two 1s, two 9s, missing 2, 3, 4, 6, 8

Two comments explain design choices. The personal year changes once a calendar year, which is why the prompt carries it in the half that varies rather than the cached half. `personalCycle` exists because `personalYear` and `personalMonth` take identical arguments, so a caller building the pair from two positional calls could transpose them with every test still passing; one function a test can assert against by name moves that risk out of every call site. The Lo Shu grid is, in its own comment, "the most concrete thing numerology has": three 1s and no 4 is specific in a way a root number never is.

NUMBER RELATIONSHIPS

Numbers 1 to 9 are ruled by planets, and the relationship of two numbers is the natural friendship of their rulers. The rulers are Sun 1, Moon 2, Jupiter 3, Saturn 4, Mercury 5, Venus 6, Mercury 7, Saturn 8, Mars 9. The comment is candid that 4 and 7 belong to Rahu and Ketu, who have no friendships the tradition agrees on, and that giving them Saturn's and Mercury's rows is a convention kept in one place so it can be corrected in one place.

PLANET	FRIENDS	ENEMIES
Sun	Moon, Mars, Jupiter	Venus, Saturn
Moon	Sun, Mercury	none
Mars	Sun, Moon, Jupiter	Mercury
Mercury	Sun, Venus	Moon
Jupiter	Sun, Moon, Mars	Mercury, Venus
Venus	Mercury, Saturn	Sun, Moon
Saturn	Mercury, Venus	Sun, Moon, Mars

The table is genuinely asymmetric: the Moon lists no enemies, but Venus lists the Moon. Read only the first number's row and `numberRelationship(2, 6)` is neutral while `numberRelationship(6, 2)` is enemy, and the prompt renders the result as a reciprocal sentence, so two users would get opposite claims about the same pair. The function resolves it compoundly: the same ruler is a friend; enemy if either direction says enemy; friend only if both directions say friend; neutral otherwise. Sun and Moon are friends, Sun and Venus enemies, Sun and Mercury neutral (the Sun does not list Mercury, so one direction fails).

2.10 Tests

Six test files sit beside the modules, 78 tests, all run by `vitest run` with no network and no database. Only `chart.test.ts` touches the ephemeris; everything else is fixtures. The reference birth for the ephemeris tests is 1 January 1990, 12:00 in New Delhi. `prompt.test.ts` also lives here but tests the rendering for the model, which is chapter 3's subject.

- `chart.test.ts` (9). A Western chart has twelve cusps, all nine bodies, a tropical Sun in Capricorn and an ascendant degree in 0 to 30. The Vedic chart of the same birth has its Sun in Sagittarius, an ayanamsa between 22 and 26, a nakshatra on the Sun and a mahadasha and antardasha present. Then the house rules: every Vedic planet's house equals `houseFrom(ascendant, sign)`; a planet in the lagna's sign is in house 1 whatever its degree; the schema version is stamped; and Western is still Placidus, proven by asserting that at least one planet does not match whole-sign counting. Finally the derived block is stored with a lord for every house, a chart with it stripped is stale and `factsFor` recomputes, and a Western chart has no derived block at all.
- `dasha.test.ts` (9). A Moon at 0 degrees (Ashwini) starts under Ketu; at 120 degrees (Magha) Ketu again, proving the nine-fold repeat. A fixed `now` of 2020 gives Sun for a 1990 birth at 0 degrees (Ketu 7, Venus 20, Sun 6). The timeline has twelve mahadashas with the first ending on 1996-12-31 (7 times 365.25 days), no gap between consecutive periods, nine antardashas per period starting with the period's own lord, the first antardasha starting where its mahadasha starts and the last ending where it ends, a Moon halfway through Ashwini giving a 3.5-year balance ending 1993-07-02 with fewer than nine antardashas, and agreement with `computeVimshottari` about the current pair.
- `derived.test.ts` (23). Dignity by sign not degree; the deep-point distance; Mercury in Virgo at 14, 18, 25 and 15.5; the Moon in Taurus; moolatrikona beating own sign; the nodes neutral. Combustion across a sign boundary, the tighter retrograde orbs for Venus and Mercury, never for the Sun or the nodes. Aspects for everyone, for Mars, Jupiter and Saturn, wrapping past the 12th, and the nodes' 7th only. Then a Scorpio-lagna fixture whose every stored house is 99, proving `deriveFacts` recomputes houses, assigns

lordships (none to the nodes), marks Mars exalted at its deep point, finds conjunctions and aspects by house, marks Mercury combust two degrees from the Sun, gives every house a lord with its placement, carries both dasha lords with their placements, folds in a Mangal dosha from the sign with the stored house still 99, and returns no dasha facts for Western.

- `doshas.test.ts` (13). `houseFrom` counts inclusively and wraps. A spread-out chart raises nothing. Mangal dosha follows the sign and ignores a wrong stored house in both directions, and its ordinal and signature read 7th from a stored 99. Kaal Sarp with everything hemmed between Rahu at 0 Aries and Ketu at 0 Libra, broken by moving Saturn past Ketu. Kemadruma for a Moon with no neighbours, cancelled by a planet in the 12th from it. Grahan for Rahu with the Sun. Pitru from the sign. The three Sade Sati phases with peak at warning; Dhaiya without Sade Sati; a node over the Moon and over the lagna; a retrograde flagged only on a sensitive sign; and `highestSeverity`.
- `numerology.test.ts` (12). Mulank for the 23rd, 9th and 10th; bhagyank for 1990-01-23; the personal year worked example and its stability across the calendar year; the personal month; `personalCycle` against a pair chosen so year and month differ (8 and 9), because the other fixture reduces both to 5 and would hide a transposition; the Lo Shu counts, missing and repeated lists; and relationships, including the asymmetric Moon and Venus pair resolving to enemy in both directions.
- `transits.test.ts` (12). For a Scorpio lagna with a Cancer Moon, transiting Saturn in Pisces gets "house 5 from the ascendant" and "9th from the Moon", Jupiter in Cancer "1st from the Moon", each names the natal planet it sits on, and the text never contains "undefined". Saturn on the natal Moon's sign yields a Sade Sati line by default and none with `includeAfflictions: false`. The same fixture as a Western chart still says where Saturn is and that it is over the natal Moon, but counts no house and names no Sade Sati, while the Vedic version of the same sky gets all three.

2.11 Study notes

READ IN THIS ORDER

1. `lib/astrology/types.ts` , for the shape everything else produces or consumes. Forty-seven lines.
2. `lib/astrology/constants.ts` , for `houseFrom` , which you will then recognise everywhere.
3. `lib/astrology/chart.ts` , top to bottom, with the four numbered steps.
4. `lib/astrology/chart.test.ts` , which states the house rules as assertions.
5. `lib/astrology/dasha.ts` , for the virtual start; then `dasha.test.ts` for the dates it produces.
6. `lib/astrology/derived.ts` , the header comment first, then `dignityOf` , then `deriveFacts` .
7. `lib/astrology/doshas.ts` , reading each detector against the tables in section 2.7.
8. `lib/astrology/transits.ts` , for the cache and the Western guard.
9. `lib/data/birthProfile.ts` , `ensureCurrentChart` , for how a stale chart is corrected.
10. `docs/design/2026-09-17-specific-readings.md` , section 0, for why.

CHECK YOURSELF

Why are Vedic houses whole-sign but Western houses Placidus?

Every Vedic technique in `derived.ts` rests on lordship, and Placidus can put one sign in two houses and another in none, leaving "the lord of the 7th" undefined. Placidus is correct for the Western tradition, which uses none of those techniques.

A stored chart has `schemaVersion` missing and Mars with `house: 12` . What does `detectNatalDoshas` do with that 12?

Nothing. It computes `houseFrom(ascendant.sign, mars.sign)` and ignores the stored number, so the answer agrees with the whole-sign facts. The row is then rewritten by `ensureCurrentChart` the next time its owner is read.

Why does the first mahadasha have a `virtualStart` before birth?

Because the antardashas must be laid out from where the period would have begun, not from birth; otherwise every sub-period for the rest of that mahadasha is misplaced. Sub-periods that ended before birth are dropped and one straddling birth is clipped.

Mercury at 15.5 degrees Virgo: exalted or moolatrikona?

Exalted. The shared-sign block gives exaltation to everything below the moolatrikona range's `from` of 16, even past the deep degree of 15.

Why is the transit chart sampled at midday UTC rather than at request time?

So the gochara text is identical for the whole day and can sit in the cached half of the system prompt. Only the Moon moves enough within a day to matter at sign resolution, and midday halves its worst-case drift.

Which three rules does `derived.ts` refuse to encode, and why?

Nodes ruling signs, nodes aspecting the 5th and 9th, and friendship in dignity. The tradition does not agree on them, and a contested claim repeated in every reading is worse than a missing one.

3 Readings, prompts, and cost

Claude never computes a chart and never sees one it was not handed. A reading is an interpretation of numbers the Swiss Ephemeris produced, delivered through a system prompt built in two halves so that the expensive half is paid for once and read from cache after that.

This chapter describes the reading pipeline as it stands on `main` on 30 September 2026, version 0.3.0. It covers the prompt builders in `lib/astrology/prompt.ts`, the model and effort settings in `lib/anthropic.ts`, the caching and history rules that the chat route applies, the one test file whose name promises more than it delivers, and the measured cost figures in `docs/MODEL_COSTS.md`. The chat feature that sits on top of all this, with its route handler, tables and clients, is chapter 6. The chart engine that produces the facts is chapter 2.

3.1 What a reading is

A reading is one streamed answer from Claude to one question, grounded in facts the server computed. The facts are the user's stored chart (`birth_profiles.chart` , holding a Vedic and a Western chart side by side), the derived facts attached to the Vedic one (lords, aspects, dignity, combustion, natal doshas), the numerology of the birth date and name, today's sky read against the natal chart, and the pinned moments of the user's life timeline. Claude is told, in so many words, that these are the only facts it has.

Three modes exist, and the type system keeps two of them apart from the third. `Tradition` in `lib/astrology/types.ts` is `"vedic" | "western"` : the two ways of computing a chart. `ChatMode` adds `"numerology"` , which has no chart at all and reads from the birth date and the name. The chat route accepts a `ChatMode` ; only the two chart modes ever reach the ephemeris.

A reading is also either standard or deep. Standard runs on Sonnet with brief thinking and a 160 word length rule; deep runs on Opus with the default thinking budget and 320 words. The client decides with one boolean, `deep` , in the request body.

Chat is not the only caller. Every model call in the repository goes through the one client in `lib/anthropic.ts` , and three other places make readings of their own: the daily morning and night readings, the dosha alerts, and the timeline's period meanings and moment extraction. They share the prompt builders and the effort setting, which is why this chapter is a foundations chapter and not a chat chapter.

CALLER	MODEL	EFFORT	PROMPT BUILDERS USED
<code>app/api/chat/route.ts</code> , standard	READING_MOD EL	low	<code>buildChartSystem</code> or <code>buildNumerologySystem</code> , then <code>buildTodaySystem</code>
<code>app/api/chat/route.ts</code> , deep	DEEP_READIN G_MODEL	default	the same
<code>lib/alerts/predictions.ts</code> , the twice-daily readings	READING_MOD EL	low	<code>buildChartSystem</code> then <code>buildTodaySystem</code> with <code>maxWords: 150</code>
<code>lib/alerts/compose.ts</code> , the dosha alert copy	DEEP_READIN G_MODEL	low	its own system string
<code>lib/timeline/explain.ts</code> and <code>lib/timeline/extract.ts</code>	READING_MOD EL	low	their own system strings

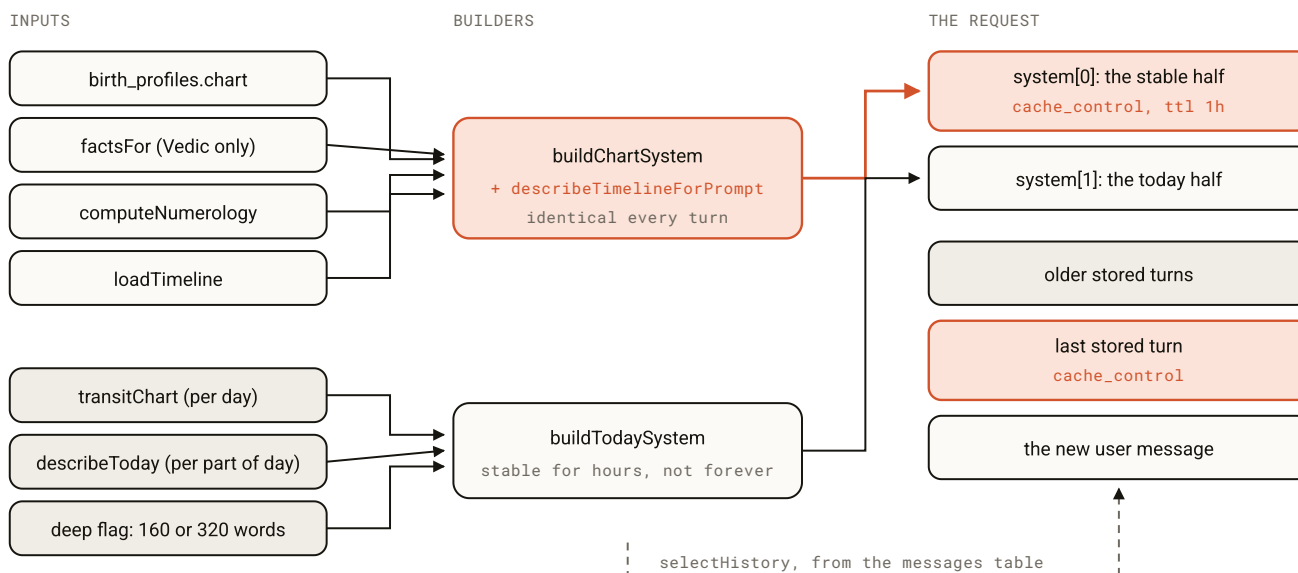
3.2 Files

<code>lib/astrology/prompt.ts</code>	<code>buildChartSystem</code> , <code>buildTodaySystem</code> , <code>buildNumerologySystem</code> , <code>buildSystemPrompt</code> and <code>buildNumerologyPrompt</code> (the joined forms)
<code>lib/astrology/prompt.test.ts</code>	what the prompt must and must not contain
<code>lib/astrology/derived.ts</code>	<code>factsFor</code> : the derived facts, Vedic only
<code>lib/astrology/transits.ts</code>	<code>transitChart</code> (memoised per day), <code>describeGochara</code> , <code>describeToday</code>
<code>lib/astrology/numerology.ts</code>	<code>computeNumerology</code> , <code>computeNameNumber</code> , <code>loShu</code> , <code>personalCycle</code>
<code>lib/timeline/describe.ts</code>	<code>describeTimelineForPrompt</code> , appended to the stable half
<code>lib/anthropic.ts</code>	READING_MODEL, DEEP_READING_MODEL, LOW_EFFORT, the client
<code>lib/data/history.ts</code>	<code>selectHistory</code> and <code>HISTORY_BUDGET_CHARS</code>
<code>lib/data/history.test.ts</code>	the budget rules
<code>lib/data/injection.test.ts</code>	the data helpers accept an injected Supabase client
<code>lib/data/chat.ts</code>	<code>getMessages</code> , <code>appendMessage</code> , <code>getOrCreateConversation</code>
<code>app/api/chat/route.ts</code>	where the halves meet the cache breakpoints
<code>docs/MODEL_COSTS.md</code>	the measured numbers, and why each change was made

`lib/anthropic.ts` begins with `import "server-only"`. Importing it from a client component fails the build, which is the mechanism that keeps `ANTHROPIC_API_KEY` on the server: the key is read through `env.anthropicApiKey()` in `lib/env.ts`, and that module is server-only too. The client is a lazy singleton; `anthropic()` constructs it on first use so that a module import never throws for want of a key.

3.3 The system prompt in two halves

The system prompt is two strings, built by two functions, sent as two blocks in the `system` array of the request. The first block is everything that never changes for a user. The second is everything that does. The split exists for one reason, stated in the comment above `buildChartSystem`: the first half carries a cache breakpoint, so it must be byte-identical on every turn and across conversations, and anything that varies has to live in the second half and come after it.



Four inputs that never change for a user feed the stable half; three that change with the day feed the volatile half. The request sends them as two system blocks, then the trimmed history, then the new message. Hot boxes carry a cache breakpoint.

THE STABLE HALF: BUILDCHARTSYSTEM

`buildChartSystem` takes the first name, the tradition, the chart, optionally the derived facts, and optionally the two numerology numbers. It returns one string in a fixed order: an identity line, the facts block, the numerology line, then three rule lists headed Accuracy, How to answer and Format, and a closing line. The identity line names the system the way the astrologer would: "Vedic (sidereal, Lahiri)" or "Western (tropical)". The facts block is introduced as the user's real chart, computed with the Swiss Ephemeris, and the sentence that follows is the whole grounding contract: these are the only facts you have.

The Accuracy rules forbid stating any placement, lordship, aspect, dasha, condition or number not listed above; require every claim to name the placement it reads from, its house and the technique; ask for tendencies and timing rather than guaranteed outcomes; and tell the model to say what the chart does show when it does not show what was asked, or to ask one short question when one missing detail would change the answer. The How to answer rules ban sign-personality writing outright: a sentence that would be true of a twelfth of the population is not an answer, and the model is told to delete it and read a house lord, an aspect, or the dasha lord's placement instead. The Format rules ask for two or three short sections under bold markdown headings, plain text with no emoji and no bold inside sentences, and a final section titled **In simple words**. The last line is the one the tests check for the spirit of the thing: never invent, and never flatter, and if the honest reading is unremarkable, say so plainly.

The facts block has two renderers, and which one runs is the Vedic versus Western branch of the whole system.

THE VEDIC BRANCH: RENDERCHARTFACTS

When `derived` is present the block is rendered by `renderChartFacts`, which writes five labelled sections from the `Derived` structure that `lib/astrology/derived.ts` computes:

- The ascendant, marked as setting house 1, and the Sun and Moon signs.

- **PLACEMENTS:** one line per planet from `planetLine` . Each line gives sign, degree, house and nakshatra, then the houses the planet rules, its dignity word (exalted, debilitated, in moolatrikona, in its own sign, neutral), "within a degree of exact" when it sits under a degree from its deep point, combustion with the distance from the Sun, retrograde, the houses it aspects, and the planets it is conjunct with.
- **HOUSES:** one line per house giving its sign, its lord, where that lord sits and with what dignity, which planets occupy the house and which aspect it.
- **CURRENT PERIOD:** the mahadasha and antardasha lords with their dates and, for each, the lord's own natal placement, dignity and rulerships, because that placement is what gives the period its character.
- **STANDING CONDITIONS IN THE BIRTH CHART:** the natal doshas, one label and one detail line each.

Two grammar helpers exist because the model copies the prompt's register. `houseSuffix` returns "s" for a plural list of houses and nothing for one, so "aspects house 9" and "aspects houses 4, 7, 8" both read correctly; the test suite pins both forms. `RELATIONSHIP_PHRASE` phrases a number relationship with the pair as the subject ("are friends", "are enemies") to avoid an article agreement error. One sentinel matters more than the grammar: a house whose lord is not a body in the chart comes out of `deriveFacts` with `lordHouse` of 0, and the renderer says "lord Sun (not a body in this chart)" rather than printing house 0 as though it were a placement. Every real chart has all nine bodies, so this is only reachable from a hand-built fixture, but a prompt that stated a placement that does not exist would be teaching the model to do the same.

THE WESTERN BRANCH: RENDERCHART

When `derived` is absent the block is rendered by `renderChart` : the ascendant, the Sun and Moon signs, the dasha line if the chart has one, and one plain line per planet with sign, degree, house, nakshatra and a retrograde flag. No lordship, no aspects, no dignity, no doshas. The comment above the function explains why, and the fix it records is worth knowing because it was a real bug in this release. Every technique `renderChartFacts` adds is Vedic. Handing it to a Western reading would cite Indian technique and Vedic doshas as though they were tropical, and the whole-sign house numbers it derives would disagree with `chart.planets[ ].house` , which for a Western chart is Placidus, and which is what the PDF and the iOS chart view actually render.

The gate is not in the prompt module at all. `factsFor` in `lib/astrology/derived.ts` returns `undefined` for any chart whose `tradition` is not "vedic" , and returns the stored derived block, or a freshly computed one for a chart written before derived facts shipped, otherwise. The chat route calls `factsFor(chart)` and passes whatever comes back, so the Western branch is chosen by the chart's own tradition field and no caller can opt a Western chart into Vedic technique by mistake. The transit text follows the same rule in `describeGochara` : houses from the ascendant and from the Moon, and the transit afflictions, are added only when the natal chart is Vedic; a Western reading gets the plain sky, where each body is and which natal body it sits over.

THE NUMEROLOGY BRANCH: BUILDNUMEROLOGYSYSTEM

Numerology has no chart, so it has its own stable builder. `buildNumerologySystem` introduces Sanchara as a Vedic numerologist and lists the real numbers computed from the birth date and the name: the mulank (root, from the birth day), the bhagyank (destiny, from the full date), the namank (from the full name, which is quoted in the prompt), the digits that repeat in the birth date and the digits that are missing from it (the Lo Shu grid), and whether the namank and the mulank are friends, neutral or enemies. Its Accuracy rule is stricter in

one way: every number cited must match the values above exactly. Its Format rules are the same as the chart prompt's, down to the closing **In simple words** section.

THE VOLATILE HALF: BUILDTODAYSYSTEM

`buildTodaySystem` takes the day, optionally the transit text, optionally a length, and optionally the personal year and month. It writes "Today is ..." with an instruction to use it for anything about today, now, or the current period; the personal cycle line when given (used by numerology); a **SKY TODAY** block when transits are given, followed on its own line by the instruction to name the transiting planet and the natal house or planet it touches; and the length rule, "160 words or fewer unless they ask for more" by default. The chat route sets 160 for a standard reading and 320 for a deep one; the daily readings job sets 150.

The word "today" is deliberately coarse. `describeToday` in `lib/astrology/transits.ts` renders the weekday and long date plus a part of day (morning before noon, afternoon before five, evening before nine, night after) and the zone name, in the user's own timezone. The route computes it with Luxon in `profile.timezone`, never the server's clock. A clock time here would change the prompt every minute, which is the first of the two things that used to break the cache.

WHAT THE ROUTE APPENDS

The route appends one more thing to the stable string before sending it: the life timeline.

`describeTimelineForPrompt` in `lib/timeline/describe.ts` renders the current dasha period (Vedic only, since Western has no dashas) and up to `MAX_MOMENTS`, which is 30, of the user's pinned moments newest first, each with its date at the precision it was pinned and, for Vedic, the period it fell in. It returns an empty string when there is nothing to say, so the route can append it unconditionally. It goes in the stable half on purpose, and the route's comment says why: it only changes when the user pins or removes a moment, so it belongs with the chart and not with the day.

WHY BYTE-IDENTICAL, AND HOW IT IS KEPT SO

A cached prefix is matched by exact bytes. If the stable half differed by one character between two turns, every turn would write a new cache entry and read none. So the rule is that nothing in `buildChartSystem` may depend on the clock, the sky, the length setting, or the order in which a JSON store returns fields. Three tests in `lib/astrology/prompt.test.ts` hold the line: the stable half must not contain "Today is", "SKY TODAY" or "words or fewer"; two calls with the same inputs must return the same string; and a `Derived` that has been through `JSON.parse(JSON.stringify(...))`, which is how a stored chart returns it, must render identically to a freshly computed one.

`buildSystemPrompt` and `buildNumerologyPrompt` join the two halves into one string for callers that do not cache. No route calls them today; the tests do, and they are the simplest way to see the whole prompt at once.

THE BLANK LINE BEFORE THE NUMEROLOGY SENTENCE

`renderChartFacts` ends whichever labelled block comes last with no trailing blank line. The numerology line therefore begins with two newlines of its own, or it would land inside HOUSES, CURRENT PERIOD or STANDING CONDITIONS and read as one more item of that block. The test named for it checks that the line before "Numerology:" is empty.

3.4 Models and effort

```
// lib/anthropic.ts
export const READING_MODEL = "claude-sonnet-5";
export const DEEP_READING_MODEL = "claude-opus-4-8";

export function supportsAdaptiveThinking(model: string): boolean {
  return model === "claude-sonnet-5" || model === "claude-opus-4-8";
}

export const LOW_EFFORT: { readonly output_config: { readonly effort: "low" } } = {
  output_config: { effort: "low" },
};
```

Two models, one switch. Standard readings run on Sonnet 5, which the file's comment describes as intelligent and stable with adaptive thinking for well-reasoned interpretation. A deep reading upgrades to Opus 4.8. Both support adaptive thinking; the comment notes that Haiku 4.5 would not, which is why

`supportsAdaptiveThinking` exists as a function rather than a constant. Every caller spreads `thinking: { type: "adaptive" }` into the request only when that function says yes.

`LOW_EFFORT` is the more consequential export. Thinking tokens are billed as output, and output is the expensive half of a reading. The comment above the constant records a measurement made on a standard Vedic reading of the same chart and question:

SETTING	OUTPUT TOKENS
adaptive thinking, default effort	1,452
adaptive thinking, effort "low"	719
thinking disabled	328

All three cited the same real placements. That is the finding that set the policy: routine readings run at low effort, reasoning intact and spend halved, and deep readings keep the default, because the default is what a person is asking for when they turn Deep on. The chat route applies it with one spread, `...(body.deep ? {} : LOW_EFFORT)`. The daily readings, the alert copy and both timeline jobs apply it unconditionally.

The type of `LOW_EFFORT` is written out longhand because `output_config` is generally available on the API but postdates the types in the installed SDK (`@anthropic-ai/sdk` at `^0.68.0` in `package.json`). The value is cast so the wire format stays exact. The same is true of `"adaptive"`, which each caller casts through `unknown` to `Anthropic.ThinkingConfigParam`.

`max_tokens` is set per caller and is a ceiling, not a target. The route's comment is explicit that the length rule in the prompt is what governs how much is generated. The values are 1,600 for a standard chat reading, 3,000 for a deep one, and 1,600 for both the daily readings and the alert copy.

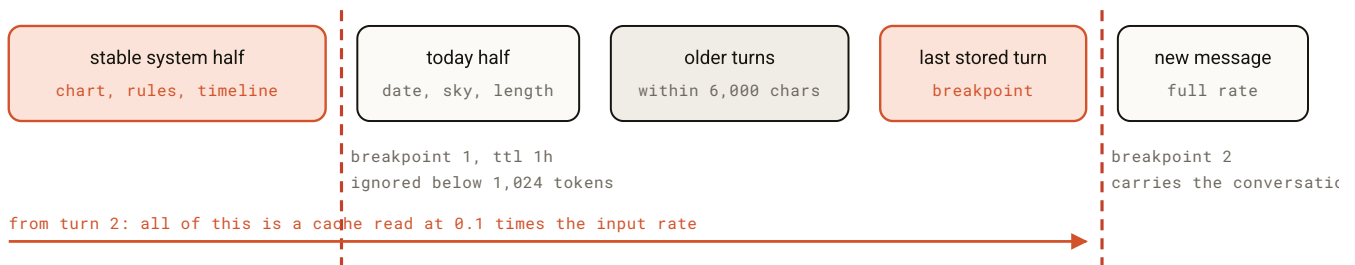
A STALE COMMENT IN THE ROUTE

Above the thinking spread in `app/api/chat/route.ts` a comment still says that only the deep (Opus) model supports adaptive thinking and that Haiku rejects it. That was true of an earlier model pair. `supportsAdaptiveThinking` is the source of record, and it returns true for both current models, so standard readings think too, at low effort.

3.5 Caching and the cost of a turn

The API is stateless. Every turn re-sends the system prompt and the whole retained history and is billed for all of it as input. Prompt caching lets a request declare breakpoints; the prefix up to a breakpoint that matches a recent request byte for byte is billed as a cache read at a tenth of the input rate. The chat route sets two.

ONE REQUEST, LEFT TO RIGHT



Turn 1 of a conversation writes both entries and reads nothing. That is the one turn that costs full price, and it is the shortest turn. Anything that changes the stable half (a pinned moment, a chart recompute, a name change) moves breakpoint 1 and the next turn writes. The today half changes at most a few times a day, so a conversation that spans midnight or a part-of-day boundary pays one write.

Two breakpoints. The first sits after the stable system half with a one hour lifetime; the second sits on the last stored turn and is the one that makes a conversation cheap.

THE TWO BREAKPOINTS

The first breakpoint is on the stable system block: `cache_control: { type: "ephemeral", ttl: "1h" }`. The hour, per the route's comment, is so that a reply half an hour later still reads the chart from cache rather than paying full price for it again. The second is on the last stored turn of the history: the route maps the retained turns into `MessageParam`s and gives the final one a content array with `cache_control: { type: "ephemeral" }`. From the second message of a conversation onward the whole prefix, chart, today and history, is a cache read.

The route's comment and `docs/MODEL_COSTS.md` both record the subtlety. A prefix below the API's minimum cacheable length, 1,024 tokens on Sonnet, is silently not cached. The measured stable half is 942 tokens, so on its own it may fall under the line, and the first breakpoint is then simply ignored. That is why the conversation breakpoint, not the system one, does the real work: by the second turn the prefix up to the last stored turn is well over the minimum, and it includes the system blocks.

MAKING THE PROMPT HOLD STILL

Two things used to change on every request and had to stop before caching could work. The date line carried a clock time, so the prompt differed every minute; it is now the date plus a part of day, as `describeToday`

renders it. And transits were computed for the current instant. `transitChart` in `lib/astrology/transits.ts` now computes them for midday UTC and memoises the promise in a module-level map keyed by day, tradition and the latitude and longitude rounded to whole degrees, so users in the same city share an entry. The map is cleared when it reaches `MAX_ENTRIES`, 200, and a failed computation is deleted from it so an error is never cached. Midday is the sample point because the Moon is the only body that moves enough within a day to matter, and sampling the middle halves the worst-case drift. The memo also stops the ephemeris running on every message, which was the lesser of its two purposes.

THE HISTORY BUDGET

```
// lib/data/history.ts
export const HISTORY_BUDGET_CHARS = 6000;

export function selectHistory(turns: Turn[], budget: number = HISTORY_BUDGET_CHARS): Turn[] {
  const kept: Turn[] = [];
  let used = 0;
  for (let i = turns.length - 1; i >= 0; i--) {
    const turn = turns[i];
    const cost = turn.content.length;
    if (used + cost > budget && kept.length > 0) break;
    kept.unshift(turn);
    used += cost;
  }
  while (kept.length > 0 && kept[0].role === "assistant") kept.shift();
  return kept;
}
```

The route loads every stored message of the conversation with `getMessages` in `lib/data/chat.ts`, ordered by `created_at` ascending, and hands them to `selectHistory`. The budget is in characters, roughly four to a token, so 6,000 characters is about 1,500 tokens. It walks newest first, because the most recent exchange matters most for coherence, and stops when the next turn would overflow, except that the newest turn is always kept even if it alone exceeds the budget. It then drops any leading assistant turns, because the API requires the first message to be from the user, and a reply with no question above it reads as a non sequitur anyway. The old rule was a flat last ten turns; the budget replaced it so that one long reading cannot crowd out the recent exchange, and so a conversation that runs all evening does not keep getting more expensive.

THE MEASURED NUMBERS

Every figure below is reproduced from `docs/MODEL_COSTS.md`, which states that each one was measured against the real API on a real chart asking a real question, not estimated. "Before" is the pipeline before the four changes that document describes: caching, effort, the history budget and the explicit length rule. "After" is the pipeline this chapter describes.

MEASURE	BEFORE	AFTER
System prompt	1,608 tokens	942
System plus a 10-turn history	4,017 tokens	2,868 (8 turns kept)
Input billed from turn 2 on	4,017 at full rate	2,867 as cache read (0.1 times)
Output on a standard reading	1,452 tokens	719
Length of the reading itself	167 words	169

At the Sonnet 5 prices the document quotes, \$2 per million input tokens and \$10 per million output, a mid-conversation turn went from roughly \$0.023 to \$0.008, and the reading did not get shorter. The document also records two figures that did not become defaults: disabling thinking entirely on the standard path would take output to about 328 tokens, a quarter of the original, and was not chosen because chart work benefits from some reasoning; and the alerts job, which runs once per user per day on Opus, had its `max_tokens` cut from 4,096 to 1,600 alongside the move to low effort.

WATCHING IT IN PRODUCTION

After the stream ends the route awaits `finalMessage()` and logs one line per turn:

```
chat usage conv=<id> model=claude-sonnet-5 in=13 cache_write=0 cache_read=2867 out=538
```

The fields are the API's `input_tokens`, `cache_creation_input_tokens`, `cache_read_input_tokens` and `output_tokens`. The diagnostic rule from the cost document: `cache_read` staying at 0 across a conversation means something reintroduced a per-request change into the prefix, and the date line and the transit text are the first two places to look.

THE CHEAPEST TURN NEVER REACHES THE API

`docs/MODEL_COSTS.md` ends with the one saving this chapter does not cover. On iOS 26 with Apple Intelligence, questions that are lookups rather than interpretation ("where is my Saturn", "which dasha am I in") are answered by the on-device model from the cached chart, and never become a paid turn at all. The saving is in the turn count, not the per-turn price. The router, the tool it reads through, and the vocabulary rules it must obey are in chapter 10.

3.6 Guarding the prompt

There is no filter in this repository that inspects a user's message for instructions aimed at the model. The guard is structural, and it is worth being precise about what it consists of.

User text goes in `messages`, never in `system`. The route reads `body.message`, appends it to the messages array and stores it; it is never interpolated into either system string. Two kinds of user-authored text do reach the stable half, and both are the user's own data reaching only the user's own prompt: the first name (and, for numerology, the full name, which is quoted as the source of the namank), and the titles of the moments the user has pinned on their timeline, which `describeTimelineForPrompt` renders verbatim.

The system prompt contains only computed facts and fixed rules. The chart block is rendered from numbers the ephemeris produced. The Accuracy rules tell the model that those numbers are the only facts it has and that a sentence with no placement behind it does not go in the answer. This is what makes an invented placement unlikely: the model has a complete, labelled list, and an instruction to cite from it.

Identity is checked twice. The route calls `supabase.auth.getUser()` and returns 401 without a user. `getOrCreateConversation` in `lib/data/chat.ts`, when handed an existing conversation id, verifies that the row belongs to this `user_id` before reusing it and throws "Conversation not found" otherwise; its comment calls this defence in depth on top of RLS. The tables themselves carry owner-only policies, described in chapter 6. So a user cannot append to, or read history from, a conversation that is not theirs, and the history that becomes part of the prompt is always their own.

WHAT LIB/DATA/INJECTION.TEST.TS ACTUALLY TESTS

Despite its name, this file has nothing to do with prompt injection. It tests **dependency** injection: that the data helpers accept an injected Supabase client and use it, and fall back to the cookie client only when none is given. Its three cases are that `saveBirthProfile` writes through the injected client, that `appendMessage` writes through the injected client with exactly the row shape the `messages` table expects, and that `appendMessage` with no client calls `createServerSupabase` once.

Why that matters to readings is the bearer path. A web request carries Supabase cookies, and `createServerSupabase` in `lib/supabase/server.ts` builds a client from them. An iOS request carries `Authorization: Bearer <jwt>` and no cookies. `createRouteSupabase` in `lib/supabase/route.ts` reads that header and returns a plain `supabase-js` client that forwards the token, so Row Level Security evaluates as that user. Every helper in `lib/data/chat.ts` and `lib/data/birthProfile.ts` takes an optional `db?: Db` for this reason. If a helper ignored it and built its own cookie client, an iOS turn would run as nobody: the conversation insert would fail RLS and the reading would never be stored. The test pins that `createServerSupabase` is not called when a client is injected, which is the whole contract.

NAME VERSUS CONTENTS

If you came to `injection.test.ts` looking for prompt injection guards, the honest answer is that the guards are the four structural facts above, and the tests that hold them are in `prompt.test.ts` (the rules text is present) and `history.test.ts` (the history is well formed). The chat route itself has no test file.

3.7 Tests

All are Vitest, run with `npm test`, and none touches the network. The prompt tests build from a fixture chart (Leo ascendant, Sun in Sagittarius, Moon in Taurus, Venus in Libra, Venus mahadasha, Sun antardasha) and compute its derived facts once so every assertion reads the same numbers the prompt was rendered from.

- `lib/astrology/prompt.test.ts`, **buildSystemPrompt**: the joined prompt carries the name, the tradition, the ascendant, a nakshatra and the dasha lord; it contains "never state a placement", "only this data", "guidance and reflection" and "in simple words"; the stable half carries no "Today is", no "SKY TODAY" and no length rule, while the volatile half carries all three; and two calls with the same inputs return the same string.

- **buildTodaySystem transit framing:** with a multi-line transit block the instruction lands on its own line, not glued to a bullet, and there is no double period.
- **derived facts in the prompt:** pinned to the fixture's computed values rather than the shape of a sentence. The Sun's line says what it rules; the Moon at 3.1° Taurus is exalted; Venus in house 3 "aspects house 9", singular; a Mars in Aries lagna "aspects houses 4, 7, 8"; house 10's line names Venus in house 3 in Libra as its lord; the mahadasha line says where Venus sits; the generic register ban ("true of a twelfth of the population") is present; and a lord that is not a body in the chart is said to be so rather than placed in house 0.
- **a Western chart gets no derived facts:** the C1 regression. With a Leo ascendant and Mars stored in Placidus house 5, whole-sign counting would put it in house 6; the test asserts the prompt reads "Mars: Capricorn 2°, h5", contains no moolatrikona, no "aspects house" and no STANDING CONDITIONS, through both `buildChartSystem` with no derived and `buildSystemPrompt`'s default.
- **numerology stays its own block:** the blank line before "Numerology: Mulank 3, Bhagyank 7."
- **derived facts survive a JSON round trip:** fresh and round-tripped `Derived` render identically.
- **numerology prompt:** for the birth date 1990-07-15 the repeated digits are 1 and 9, the missing digits 2, 3, 4, 6 and 8, and "The namank 1 and the mulank 6 are enemies." appears as one literal sentence, so a swapped argument order or a wrong relationship word fails.
- **buildTodaySystem with personal cycles:** distinct year and month values both render, and neither appears when absent.
- `lib/data/history.test.ts` : everything is kept when it fits; the oldest turns drop first; a window never starts on an assistant turn (and can be empty as a result); the newest turn is kept even when it alone exceeds the budget; an empty conversation yields nothing.
- `lib/data/injection.test.ts` : the three dependency injection cases in section 3.6, with `computeChart` stubbed so the ephemeris binary is not loaded.

3.8 Study notes

READ IN THIS ORDER

1. `lib/astrology/prompt.ts` , top to bottom. Read `renderChart` 's comment before `renderChartFacts` , so the Western branch makes sense as the older, plainer one.
2. `lib/astrology/derived.ts` , just `factsFor` and the comment above it, for the gate that chooses the branch.
3. `lib/astrology/transits.ts` , for the memo key and `describeToday` , which are the two things that made the prompt hold still.
4. `lib/anthropic.ts` , for the three measured effort numbers.
5. `app/api/chat/route.ts` , from the profile read to the `messages.stream` call, watching for the two `cache_control` objects.
6. `lib/data/history.ts` , for the budget.
7. `docs/MODEL_COSTS.md` , for what each change bought.
8. `lib/astrology/prompt.test.ts` , which reads as the list of rules in plain English.

CHECK YOURSELF

Why is the timeline appended to the stable half rather than the today half, when it is user data that can change?

Because it changes only when the user pins or removes a moment, which is rare, while the today half changes several times a day. Putting it in the stable half means a pin costs one cache write; putting it in the today half would gain nothing, since the today half is not behind its own breakpoint.

The stable half measures 942 tokens and Sonnet's minimum cacheable prefix is 1,024. Why does caching work at all?

Because of the second breakpoint on the last stored turn. From turn 2 the prefix up to that turn includes both system blocks plus history, is well over the minimum, and is what is read from cache. The first breakpoint is a bonus when the chart is large enough.

Why does a Western reading get no lordships, aspects or doshas?

`factsFor` returns `undefined` for any non-Vedic chart, so `buildChartSystem` falls back to `renderChart` . The techniques are Vedic, and whole-sign house numbers would contradict the Placidus houses the PDF and the iOS view draw from `chart.planets[ ].house` .

Why is the length rule in the prompt rather than in `max_tokens` ?

Because `max_tokens` truncates; it does not shorten. A model told to write 160 words writes about 160 and stops cleanly. The ceiling of 1,600 exists so a runaway answer is bounded, not to shape a normal one.

What does `selectHistory` do with a stored assistant turn that has no user turn before it in the window?

Drops it. The window is trimmed from the front until the first turn is a user turn, because the API requires that and because an answer with no question reads as a non sequitur. The test named "never starts on an assistant turn" pins it, including the case where the result is empty.

What does `injection.test.ts` guard against?

A data helper that ignores the injected bearer client and builds its own cookie client. On an iOS request that client has no session, RLS denies the write, and the reading is never stored.

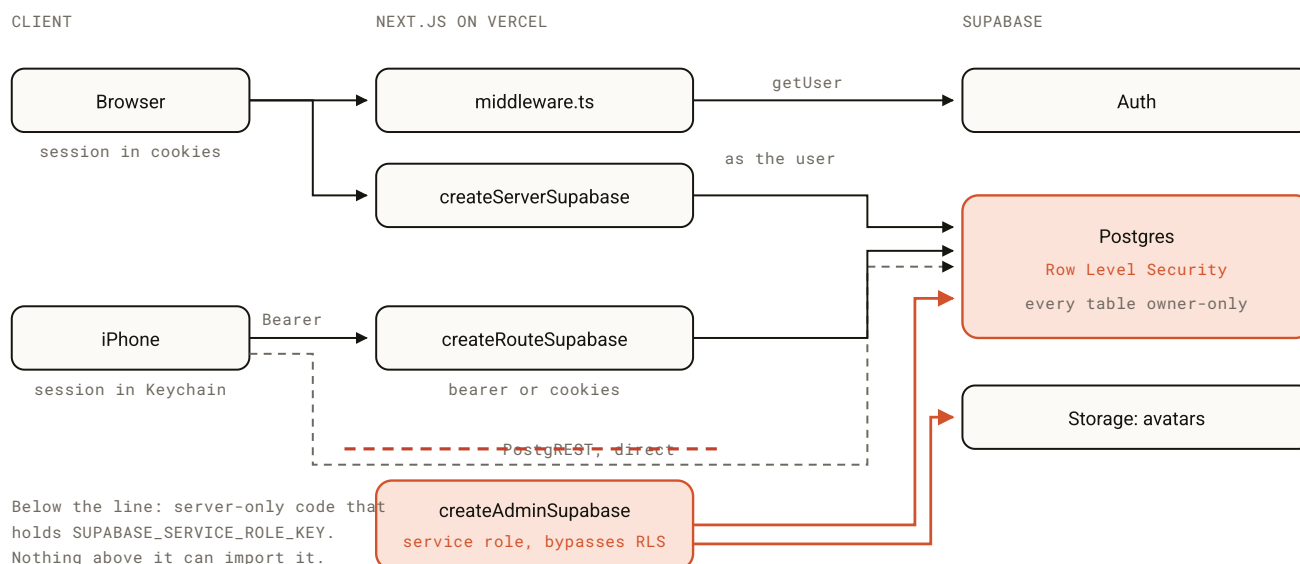
4 Identity and access

One Supabase project, two ways in, and a single rule under both: **the database decides who may read a row, not the code that asked for it**. The web app carries its session in cookies, the iOS app carries a bearer token, and every table refuses anyone but its owner.

This chapter describes identity and access as they are on `main` on 30 September 2026, version 0.3.0. Nothing here is a framework of its own. Supabase Auth issues the sessions, Postgres Row Level Security enforces ownership, and the application code is a thin set of clients that make sure a request reaches Postgres as the person who sent it. The interesting decisions are about where each key is allowed to be, which paths bypass the rule and why, and how the phone gets a session without ever leaving the app.

4.1 Two channels

A browser and a phone reach the same Postgres through different doors. On the web, the session lives in cookies that `@supabase/ssr` reads and refreshes; a server component, a server action, a route handler and the middleware all build a client from those cookies and Postgres sees the request as that user. On iOS, `supabase-swift` holds the session in the Keychain and the app does two things with it: it reads a few tables directly over PostgREST, and it calls the same `/api/*` route handlers the web uses, sending the access token as `Authorization: Bearer`. A route handler builds a client that forwards that header, and again Postgres sees the request as that user.



Cookies on the left top, a bearer token on the left bottom, and both arrive at Postgres as the user so RLS applies. The service-role client is the only path that does not, and it sits behind a wall the bundler enforces.

The design follows from one sentence in the README: every table is protected by Row Level Security, so the anon key is safe to ship in the iOS binary. The anon key and the project URL are public by design. What must never ship is `SUPABASE_SERVICE_ROLE_KEY`, which bypasses RLS, and `ANTHROPIC_API_KEY`. Both are read

only through `lib/env.ts`, which begins with `import "server-only"`, and the script that generates the iOS configuration (`mobile/scripts/make-config.sh`) reads only the two public values and says so in its header.

4.2 Files

```
lib/supabase/
├─ client.ts      createBrowserClient. Present, imported by nothing today
├─ server.ts      createServerSupabase: the cookie client for server code
├─ middleware.ts  updateSession: refresh cookies, redirect the signed-out
├─ route.ts       createRouteSupabase: bearer if present, else cookies
├─ route.test.ts  bearerTokenFrom
├─ admin.ts       createAdminSupabase: the service-role client
middleware.ts     calls updateSession on every path but static assets
lib/env.ts        every secret is read here and nowhere else
lib/admin/
├─ guard.ts       isAdminUser, requireAdmin
├─ data.ts        admin reads (chapter 11)
app/(auth)/
├─ actions.ts     authenticate, sendMagicLink, resendConfirmation, signOut
├─ login/page.tsx the one smart form
├─ signup/page.tsx the same form with a different title
├─ check-email/page.tsx
├─ layout.tsx
app/auth/callback/route.ts exchanges the code in a link for a session
app/admin/login/page.tsx  a separate form, a separate action
app/admin/actions.ts      adminSignIn, adminSignOut
components/AuthForm.tsx   email, password, primary button, magic link button
public/.well-known/apple-app-site-association
docs/EMAIL_OTP.md         the email templates that make codes work
docs/APPLE_SIGN_IN.md     provider and capability setup
supabase/migrations/      every policy, 0001 to 0007
mobile/Sanchara/Sources/
├─ Services/Supabase.swift      Supa.client, Supa.accessToken()
├─ Services/SancharaAPI.swift   puts the bearer on every request
├─ Features/Auth/AuthStore.swift the state machine
├─ Features/Auth/AuthView.swift AuthView and VerifyCodeView
├─ Features/Auth/AppleSignIn.swift nonce, hash, remembered name, the button
├─ RootView.swift              routes on auth state; receives Universal Links
mobile/project.yml             entitlements: applinks, webcredentials, applesignin
mobile/Sanchara/Tests/AuthValidationTests.swift
```

4.3 Session on the web

`createServerSupabase` in `lib/supabase/server.ts` wraps `createServerClient` from `@supabase/ssr` with the Next.js cookie store. Its `setAll` is wrapped in a try block with a comment that explains the whole arrangement: when the client is created from a Server Component, cookies cannot be written, so the write is ignored and the middleware is what refreshes them. That is why the root `middleware.ts` exists at all.

```
// lib/supabase/middleware.ts
const { data: { user } } = await supabase.auth.getUser();

const path = request.nextUrl.pathname;
const isProtectedApp = path.startsWith("/app");
const isProtectedAdmin = path.startsWith("/admin") && path !== "/admin/login";
if ((isProtectedApp || isProtectedAdmin) && !user) {
  const url = request.nextUrl.clone();
  url.pathname = isProtectedAdmin ? "/admin/login" : "/login";
  return NextResponse.redirect(url);
}
return response;
```

`updateSession` does two jobs on every request. It calls `getUser`, which verifies the token with Supabase Auth rather than trusting the cookie's contents, and in doing so refreshes an expiring session and writes the new cookies onto the response. Then it redirects anyone signed out away from `/app` and `/admin`, sending the two areas to their own login pages. The matcher in the root file excludes `_next/static`, `_next/image`, `favicon.ico`, `images/` and `.well-known/`, so static assets and the Apple association file are served without a session lookup.

ONE FORM, TWO OUTCOMES

The login and signup pages render the same `AuthForm` with the same `authenticate` action; only the title differs. The action is the web's "smart" sign-in, and its comment states the intent: an existing account with the right password logs in, a new email creates an account, and an existing email with the wrong password is told so.

1. Try `signInWithPassword`. A session means a returning user; redirect to `/app`.
2. If the error message contains "not confirmed", the account exists but the address was never confirmed; redirect to `/check-email` with the address in the query string.
3. Otherwise call `signUp` with `emailRedirectTo` set to `<origin>/auth/callback`. If Supabase answers that the address is already registered, the first step failed because of the password, and the form shows "Incorrect password for this account. Try again, or use a magic link."
4. A new account with a session (email confirmation switched off in the project) goes to `/app`; without one it goes to `/check-email`.

The order matters. Trying sign-in first makes the common case one round trip, and using the sign-up error to distinguish "wrong password" from "new person" means the form never has to ask which the visitor is. The cost is that the message strings from Supabase are matched by substring ("already registered", "already exists", "already been registered"), which is fragile by nature and is the reason the iOS app chose a different shape (4.4).

`sendMagicLink` calls `signInWithOtp` with the same redirect and bounces back to `/login?sent=1`. `resendConfirmation` calls `auth.resend` with type `signup`. `signOut` signs out and redirects to `/login`; the form that calls it lives in `components/AppShell.tsx` at the bottom of the sidebar.

THE CALLBACK

```
// app/auth/callback/route.ts
const code = searchParams.get("code");
if (code) {
  const supabase = await createServerSupabase();
  await supabase.auth.exchangeCodeForSession(code);
}
return NextResponse.redirect(`${origin}/app`);
```

Every link Supabase sends (confirmation, magic link, the admin's one-time link) resolves to this route with a `code`. The route exchanges it for a session, which writes the cookies, and redirects to `/app` unconditionally. If the code was missing or the exchange failed, the redirect still happens and the middleware bounces the visitor to `/login`. There is no error page for a bad link on the web; the login page is the error page.

`lib/supabase/client.ts` exports a `createBrowserClient` wrapper for client components. No file imports it today. Every read on the web goes through server components and server actions, so the browser never holds a Supabase client. Keep that in mind before reaching for it: adding a browser client would mean the anon key and the session start being used from JavaScript, which is safe under RLS but is a second channel to reason about.

4.4 Session on the phone

`Supa.client` in `mobile/Sanchara/Sources/Services/Supabase.swift` is the app's single `SupabaseClient`, built from the generated `AppConfig`. The file's comment records the requirement that made `supabase-swift` the right choice: it persists the session in the Keychain by default, never in `UserDefaults`. `Supa.accessToken()` returns the current access token or nil, and that is the whole bridge to the API.

```
// mobile/Sanchara/Sources/Services/SancharaAPI.swift
private static func request(_ path: String, method: String) async throws -> URLRequest {
  var req = URLRequest(url: AppConfig.apiBaseURL.appendingPathComponent(path))
  req.httpMethod = method
  guard let token = await Supa.accessToken() else { throw APIError.unauthorized }
  req.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
  return req
}
```

Every call in `SancharaAPI` goes through this function except `geocode`, whose comment says why: the route is public, so it carries no token. A 401 from any route maps to `APIError.unauthorized`, whose description is "Your session expired. Please sign in again."

THE BEARER CHANNEL ON THE SERVER

```
// lib/supabase/route.ts
export async function createRouteSupabase(request: Request): Promise<Db> {
  const token = bearerTokenFrom(request);
  if (!token) return createServerSupabase();

  return createClient(env.supabaseUrl(), env.supabaseAnonKey(), {
    global: { headers: { Authorization: `Bearer ${token}` } },
    auth: { persistSession: false, autoRefreshToken: false },
  });
}
```

This is the file that lets one set of route handlers serve both clients. `bearerTokenFrom` reads the `Authorization` header, accepts the scheme case-insensitively, and returns null for a missing header, a different scheme, or an empty token. With a token, the handler gets a plain `supabase-js` client that forwards the header on every request, so RLS evaluates as that user; the client neither persists nor refreshes anything, because the phone owns the session. Without a token, the handler falls through to the cookie client and the web behaves as it always did. The exported `Db` type is `SupabaseClient`, and the comment notes that both clients are instances of it, which is what lets `lib/data/birthProfile.ts` accept either.

Nine route handlers use `createRouteSupabase`: `account`, `chat`, `devices`, `kundli`, `life-events`, `profile`, `timeline`, `timeline/explain` and `timeline/scan`. Each begins the same way, `getUser` then a 401 when there is no user, and the user id used afterwards always comes from that verified session, never from the body.

DIRECT READS

Not everything on the phone goes through Vercel. `ProfileStore` reads `birth_profiles` straight from Supabase with `Supa.client.from(...)`, and the alerts and daily-reading stores do the same for their tables. There is no user id in those queries. The `birth self` policy already restricts the result to the caller's own row, so the query is simply "select from `birth_profiles` where `chart` is not null, limit 1". That is the practical meaning of RLS as the boundary: the client does not filter by owner because it cannot see anyone else's rows to begin with.

THE STATE MACHINE

`AuthStore` is an `@Observable` class on the main actor with four states: `loading`, `signedOut`, `signedIn`, and `awaitingCode(email:purpose:)`. `start()` sets the initial state from whether a session exists, then listens to `authStateChanges: signedIn` and `tokenRefreshed` with a session mean signed in, `signedOut` means signed out. `RootView` switches on the state and shows `LoadingScreen`, `AuthView`, `VerifyCodeView` or `SignedInView`.

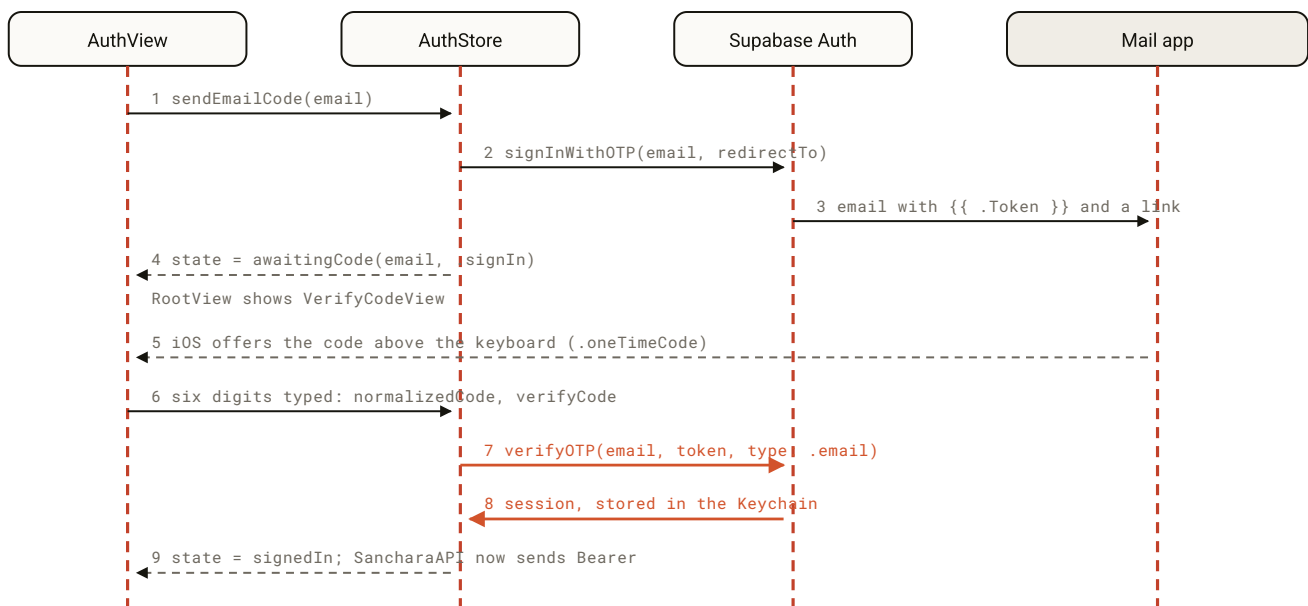
The store's header comment explains the one deliberate divergence from the web. Sign-in and sign-up are separate paths: "a person who mistypes a password should be told that, not silently pushed into creating a second account." `AuthView` has a two-segment picker, and a device that has never signed in opens on "Create account" (`AuthStore.hasSignedInBefore`, a `UserDefaults` flag under a key that keeps the pre-rename prefix so that returning users are not shown the first-run screen). `signIn` maps "not confirmed" to `awaitingCode` with purpose `confirmSignUp`; `signUp` maps "already registered" to an error and sets `shouldSwitchToSignIn`, which the view watches to flip the picker with the email still filled in. Both validate

the address and the password length (`minimumPasswordLength = 6` , Supabase's default) before spending a round trip.

`signOut` does three things in a fixed order: unregister the push token while the bearer token is still valid, sign out, then `ChartCache.shared.clear()` . The comment says why the third step exists: the chart, the birth details and the last reading live on disk in the App Group, and without clearing them the widget would keep showing them to whoever signs in next.

4.5 Email code

On the phone, the passwordless path sends a six-digit code rather than a link. `docs/EMAIL_OTP.md` gives the reason and the one setting it depends on. The reason: a code is typed into the app that is already open, with no hop out to a mail client and no Universal Link that a browser might swallow. The setting: Supabase decides link-versus-code by what the email template contains. A template with only `{{ .ConfirmationURL }}` sends a link and no code, and the app's code screen would have nothing to accept. Both the "Magic Link" and "Confirm signup" templates therefore include `{{ .Token }}` and keep the link as well, so the same email serves the phone (type the code) and the web (tap the link).



The code never touches Vercel. Supabase Auth issues it, the phone verifies it with Supabase Auth, and the session that comes back is what every later API call carries as a bearer token.

Three details in `AuthStore` decide how this feels in practice.

- **Which code was sent decides how it is verified.** `CodePurpose` has two cases. `signIn` verifies with `EmailOTPTType.email`; `confirmSignUp`, reached when a password sign-up needs its address confirmed, verifies with `.signup`. `resendCode` re-sends through the matching path: `signInWithOTP` for the first, `auth.resend(type: .signup)` for the second.
- **Digits only.** `normalizedCode` keeps the numeric characters and takes the first six, because people paste codes with spaces and mail clients wrap them in punctuation. `VerifyCodeView` applies it on every change, and once six digits are present it submits on its own; the comment calls a separate button

"ceremony". The field is marked `.textContentType(.oneTimeCode)`, so iOS offers the code from the email above the keyboard.

- **The link still works.** `sendEmailCode` passes `redirectTo` pointing at `auth/callback` on the API host, so anyone who taps the link instead of typing the code lands back in the app through a Universal Link (4.6).

A failed verification whose message contains "expired" or "invalid" is reported as "That code didn't work. It may have expired, send a new one." Codes expire after an hour by default. The screen also offers "Use a different email", which simply calls `signOut` to return to `signedOut`.

The web has no code screen. Its magic-link button calls the same `signInWith0tp` and so receives the same email, but the only thing it can do with it is follow the link to `/auth/callback`. The code in that email is for the phone.

4.6 Sign in with Apple and Universal Links

APPLE

Sign in with Apple is fully native. Apple returns an identity token and Supabase verifies it directly through `signInWithIdToken`; there is no web sheet, no redirect, no password and no confirmation email. The header of `AppleSignIn.swift` calls it the fastest way onto Sanchara, and `docs/APPLE_SIGN_IN.md` records the two one-time steps it needs: the Apple provider enabled in the Supabase project with the bundle id `com.shivvyas.astra` as a client id, and the Sign In with Apple capability on the App ID. The entitlement `com.apple.developer.applesignin` is already in `mobile/project.yml`. For the native flow no Services ID, secret key or key id is needed; those exist only for Apple's web OAuth, which the site does not offer.

```
// mobile/Sanchara/Sources/Features/Auth/AppleSignIn.swift
SignInWithAppleButton(.continue) { request in
    let nonce = AppleSignIn.randomNonce()
    rawNonce = nonce
    request.requestedScopes = [.fullName, .email]
    request.nonce = AppleSignIn.sha256(nonce)
} onCompletion: { result in
    let nonce = rawNonce
    Task { await auth.completeAppleSignIn(result: result, rawNonce: nonce) }
}
```

The nonce is the part worth understanding. `randomNonce` draws 32 characters from a 64-character alphabet using `SecRandomCopyBytes`, rejecting any byte of 252 or above so the modulo does not bias the distribution. Its SHA-256 goes to Apple, which signs it into the identity token; the raw value goes to Supabase, which hashes it and compares. The comment states what this buys: a captured token cannot be replayed against the project, because the replayer does not hold the raw nonce that produced its hash.

`completeAppleSignIn` treats a cancelled sheet (`ASAuthorizationError.canceled`) as a normal action and reports nothing. On success it extracts the identity token, calls `signInWithIdToken(credentials: .init(provider: .apple, idToken:, nonce:))`, and then keeps `credential.fullName` if Apple sent one. Apple sends the name only on the very first authorization for this App ID, ever, so `AppleSignIn.remember`

stores it in `UserDefaults` and the intake screen (chapter 5) pre-fills the first and last name from it. The keys keep their pre-rename prefix on purpose: the bundle id did not change, existing installs upgrade in place, and a renamed key would lose a name Apple will never send again.

KNOWN GAP

Apple's guideline 5.1.1(v) asks apps that offer Sign in with Apple to revoke the Apple token when a person deletes their account. `DELETE /api/account` deletes the Supabase user and everything that cascades from it, but it does not call Apple's revoke endpoint. `docs/APPLE_SIGN_IN.md` lists the three steps to close this (a Services ID and key to mint a client secret, capturing the authorization code at sign-in, and posting the refresh token to the revoke endpoint before the Supabase user is removed) and says it is worth doing before the first App Store submission that includes Apple sign-in. As of 0.3.0 it is open.

UNIVERSAL LINKS

Every email Supabase sends carries a link that resolves to `/auth/callback` on the production host. On a phone with the app installed, that URL should open the app, not Safari. Two files make that happen. The entitlements in `mobile/project.yml` declare `applinks:astra.shivvyas.com` and `webcredentials:astra.shivvyas.com`. And the site serves `public/.well-known/apple-app-site-association`, which names the app as the team id joined to `com.shivvyas.astra`, and claims exactly two paths: `/auth/callback` and `/auth/callback?*`. The `webcredentials` entry is what lets the iOS password autofill offer credentials saved for the site to the app.

When iOS hands the URL to the app, `RootView.onOpenURL` receives it. Two kinds of URL arrive there, and the comment is explicit: Supabase's auth callback and the app's own `sanchara://` scheme from a widget tap. `DeepLink.isDeepLink` checks the scheme; a deep link goes to `DeepLink.shared.handle`, anything else goes to `AuthStore.handle(url:)`, which calls `Supa.client.auth.session(from: url)` and moves to `signedIn`. Handing a widget URL to the auth store would produce "That link has expired", which is the failure the check prevents. Without the app installed, the same URL reaches the web route in 4.3 and the person is signed in on the web instead.

4.7 Row Level Security

RLS is the boundary. The clients above exist only to make sure a request reaches Postgres with the right identity; what that identity may then see is decided entirely by the policies in `supabase/migrations/`. Every table has `enable row level security`, and every policy is written in terms of `auth.uid()`. Here is all of them.

TABLE	POLICY	COMMANDS	RULE	MIGRATION
profiles	profiles self	all	id = auth.uid()	0001
birth_profiles	birth self	all	user_id = auth.uid()	0001
conversations	conv self	all	user_id = auth.uid()	0001
messages	msg self	all	a conversations row exists with this conversation_id and user_id = auth.uid()	0001
admins	admins read self-or-admin	select only	user_id = auth.uid() or public.is_admin()	0001
device_tokens	device tokens self	all	user_id = auth.uid()	0004
transit_conditions	conditions self read	select only	user_id = auth.uid()	0004
alerts	alerts self read; alerts self update	select; update	user_id = auth.uid()	0004
daily_readings	daily readings self read; daily readings self update	select; update	user_id = auth.uid()	0005
life_events	life events self	all	user_id = auth.uid()	0006
life_event_scans	life event scans self	all	user_id = auth.uid()	0006
period_readings	period readings self	all	user_id = auth.uid()	0007

Every `for all` policy carries both `using` and `with check` with the same predicate, so a user can neither read another person's row nor insert a row that claims another owner. The three tables with select-only or select-and-update policies are the ones the scheduled jobs write: `transit_conditions`, `alerts` and `daily_readings`. The comments in 0004 and 0005 say it directly: the daily job writes them with the service-role client, and the policies exist so the app can read its own rows and mark them read. A user cannot insert an alert. `admins` has no insert, update or delete policy at all; the comment on the table reads "presence in this table = admin. No self-service path". The only way in is the SQL statement in `docs/SUPABASE_SETUP.md`, run by hand.

Two functions complete the picture. `public.is_admin()` is `security definer` and checks for the caller's row in `admins`; it is used by the `admins` policy so an admin can see the whole list.

`public.handle_new_user()` runs on `after insert` on `auth.users` and creates the `profiles` row, taking a display name from the sign-up metadata or the part of the email before the `@`. Every table with a `user_id` references `auth.users(id)` on `delete cascade`, which is what makes account deletion one call (chapter 11).

WHERE THE SERVICE-ROLE KEY IS ALLOWED

```
// lib/supabase/admin.ts
import "server-only";
// Bypasses RLS. Only ever import from server code behind an admin auth check.
export function createAdminSupabase() {
  return createClient(env.supabaseUrl(), env.supabaseServiceRoleKey(), {
    auth: { persistSession: false, autoRefreshToken: false },
  });
}
```

`import "server-only"` makes the bundler fail if a client component ever imports this file, so the key cannot leak into a browser bundle by accident. Beyond that the rule is convention, and the convention is narrow. These are the only production files that call `createAdminSupabase`, and each has a reason RLS cannot serve.

FILE	USES THE SERVICE ROLE TO	WHY RLS CANNOT DO IT
<code>lib/admin/guard.ts</code>	look up the caller in <code>admins</code>	The guard runs before any admin session is proven; reading through the caller's own client would work for their own row but the guard is also the place that must never be wrong
<code>lib/admin/data.ts</code>	list users, read birth profiles and transcripts	Admin reads cross owners by definition; the 0001 comment says admin reads happen via the service-role client
<code>app/admin/actions.ts</code>	<code>auth.admin.generateLink</code> for a one-time login link	Minting a link for another user is an Auth admin API
<code>app/(app)/app/intake/actions.ts</code> , <code>app/api/profile/route.ts</code>	upload the avatar to the <code>avatars</code> bucket	No storage policy is defined for users; the upload path is <code><user_id>/avatar.<ext></code> and the user id comes from the verified session
<code>app/api/account/route.ts</code>	<code>auth.admin.deleteUser</code>	Deleting an auth user is an Auth admin API; the id is the caller's own
<code>app/api/devices/route.ts</code>	upsert and delete <code>device_tokens</code>	The comment explains: a device handed from one account to another must change owner, and the owner-only policy would correctly refuse that update
<code>lib/alerts/run.ts</code> , <code>lib/alerts/predictions.ts</code>	write conditions, alerts and daily readings for every user	The cron job has no user session; it is authenticated by <code>CRON_SECRET</code> instead

Note what every entry has in common: the row being written is still scoped to a user id that came from `getUser` on the caller's own client, or the code is a scheduled job with no caller at all. The admin client is never used to act on a user id taken from a request body.

4.8 The admin boundary

The admin area at `/admin` has its own login page, its own sign-in action, and a guard on every page. Being an admin is not a role or a claim; it is a row in `admins`. Three layers check it.

1. **The middleware** redirects any signed-out request under `/admin`, except `/admin/login` itself, to `/admin/login`. This proves a session exists, not that it is an admin's.
2. `adminSignIn` in `app/admin/actions.ts` calls `signInWithPassword` with the cookie client, then `isAdminUser`. A valid account that is not in `admins` is signed out again immediately and sent back with "Not authorized". There is no sign-up on this form, no magic link, and no way to become an admin from it.
3. `requireAdmin` in `lib/admin/guard.ts` is the first line of every admin page and every admin action. It reads the user from the cookie client, redirects to `/admin/login` if there is none, and redirects with "Not authorized" if `isAdminUser` is false. The layering means a person removed from `admins` while signed in loses access on their next request, not at their next login.

```
// lib/admin/guard.ts
export async function requireAdmin() {
  const supabase = await createServerSupabase();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) redirect("/admin/login");
  if (!(await isAdminUser(user.id))) redirect("/admin/login?error=Not+authorized");
  return user;
}
```

The admin session is an ordinary Supabase session in the same cookies as the app's. What separates the two areas is the guard, the separate login page, and the layout: `app/admin/layout.tsx` is a passthrough with `force-dynamic`, and each authenticated page wraps itself in `AdminShell` so the sidebar never appears on the login screen. What an admin can read and do once inside is chapter 11's subject; the short version is that the admin reads users, birth details and transcripts, can send a password reset and mint a one-time login link, and writes nothing to a user's data.

TWO THINGS TO KEEP STRAIGHT

The web's user login accepts a magic link; the admin login does not. And `is_admin()` in SQL and `isAdminUser` in TypeScript answer the same question by different routes: the SQL function runs inside a policy as security definer, the TypeScript function runs on the server with the service-role client. Change one without the other and the admins list becomes visible to the wrong people or the guard starts lying.

4.9 Tests

Identity is the part of a system where a test is cheapest to write and most expensive to lack, and the coverage here is narrow but exact. Nothing in it needs a network or a Supabase project.

- `lib/supabase/route.test.ts` pins `bearerTokenFrom` with five cases: a token is extracted from a Bearer header; a lowercase scheme is accepted; a missing header returns null; a `Basic` scheme returns

null; a Bearer scheme with an empty token returns null. The last two are what keep a malformed header from falling through to the cookie client with a garbage token attached.

- `mobile/Sanchara/Tests/AuthValidationTests.swift` covers the pure functions on `AuthStore` and `AppleSignIn`. `testAcceptsOrdinaryAddresses` and `testRejectsMalformedAddresses` pin `emailLooksValid` (no leading `@`, a dot in the domain, no leading or trailing dot, no spaces). `testCodeNormalisationKeepsOnlyDigits` pins `normalizedCode` against "123 456", "Code: 123-456.", a ten-digit paste that must be cut to six, and letters. `testAppleNonceIsRandomAndHashed` pins the 32-character nonce, that two nonces differ, and that the SHA-256 hex is 64 characters and stable.
- The route handler tests for account, devices and profile each begin with a case that a request with no user gets a 401 and touches nothing, which is the bearer channel's contract exercised from the other side. Chapter 11 lists them.

What is not tested: the middleware redirect, the smart sign-in's error-string matching, the callback route, and `requireAdmin`. All four depend on a live Auth server or on Next.js request plumbing, and none has a test today. The error-string matching in `app/(auth)/actions.ts` is the one most likely to break silently if Supabase rewords a message, and would be the first to cover.

4.10 Study notes

READ IN THIS ORDER

1. `supabase/migrations/0001_init.sql` , for the tables, the trigger, `is_admin()` and the first five policies. Everything else in this chapter exists to satisfy these.
2. `lib/supabase/route.ts` , forty lines, and its test. This is the whole bearer channel.
3. `lib/supabase/middleware.ts` and the root `middleware.ts` , for cookie refresh and the two redirects.
4. `app/(auth)/actions.ts` , for the smart form, then `app/auth/callback/route.ts` .
5. `mobile/Sanchara/Sources/Features/Auth/AuthStore.swift` , top to bottom. The comments carry the reasoning.
6. `mobile/Sanchara/Sources/Features/Auth/AppleSignIn.swift` , for the nonce and the remembered name.
7. `docs/EMAIL_OTP.md` , for the template setting that decides link or code.
8. `lib/supabase/admin.ts` and `lib/admin/guard.ts` , then grep for `createAdminSupabase` and check each caller against the table in 4.7.

CHECK YOURSELF

Why can the anon key ship inside the iOS binary?

Because it grants nothing by itself. Every table has RLS enabled and every policy is written against `auth.uid()` , so a request with the anon key and no session sees no rows, and a request with a session sees only its owner's rows.

What does `createRouteSupabase` do when the header is `Authorization: Bearer` with nothing after it?

`bearerTokenFrom` returns null for an empty token, so the handler falls through to the cookie client. On a phone that means `getUser` finds no user and the route answers 401.

Why does the iOS app separate sign-in from sign-up when the web merges them?

The web's merged form tells a wrong password from a new address by matching Supabase's error strings after a sign-up attempt. On the phone the store's comment says a person who mistypes a password should be told that, not pushed toward a second account, so the screen picks the path and the store never guesses.

Which tables can a user read but never insert into, and who inserts?

`transit_conditions` , `alerts` and `daily_readings` . The scheduled jobs in `lib/alerts/` write them with the service-role client; the user can only select, and update `read_at` on the last two.

Why does `randomNonce` reject bytes of 252 and above?

The alphabet has 64 characters and 256 is a multiple of 64, but the loop discards bytes at or above 252 so that a future change to the alphabet size cannot bias the modulo; the comment says every character must be equally likely.

Why does `RootView.onOpenURL` check the URL scheme before handing it to the auth store?

A widget tap opens the app with a `sanchara://` URL. Passing that to `session(from:)` would fail and show "That link has expired" for something that was never a link.

How does someone become an admin?

Only by a row inserted into `public.admins` by hand in the SQL editor. The table has a select policy and no write policy, and no code path inserts into it.

PART TWO

The features

One chapter per feature. Each follows the same shape: intent, files, model, the server side, the web and iOS clients, how it works end to end, what keeps it fast and cheap, what breaks and how it is handled, and how to study it.

- 5. Intake and the birth profile
- 6. Chat
- 7. Kundli
- 8. The life timeline
- 9. Daily readings and dosha alerts
- 10. On-device answers, Siri, and widgets
- 11. Profile, account, and admin
- 12. The landing page and the web shell

5 Intake and the birth profile

Everything Sanchara does starts from one row: a name, a date, a time, a place and the two charts computed from them. Intake is the form that fills it, on the web and on the phone, and the birth profile is the row that every other feature reads. The one idea that shaped it is that the chart is computed once, at save time, and stored beside the details that produced it.

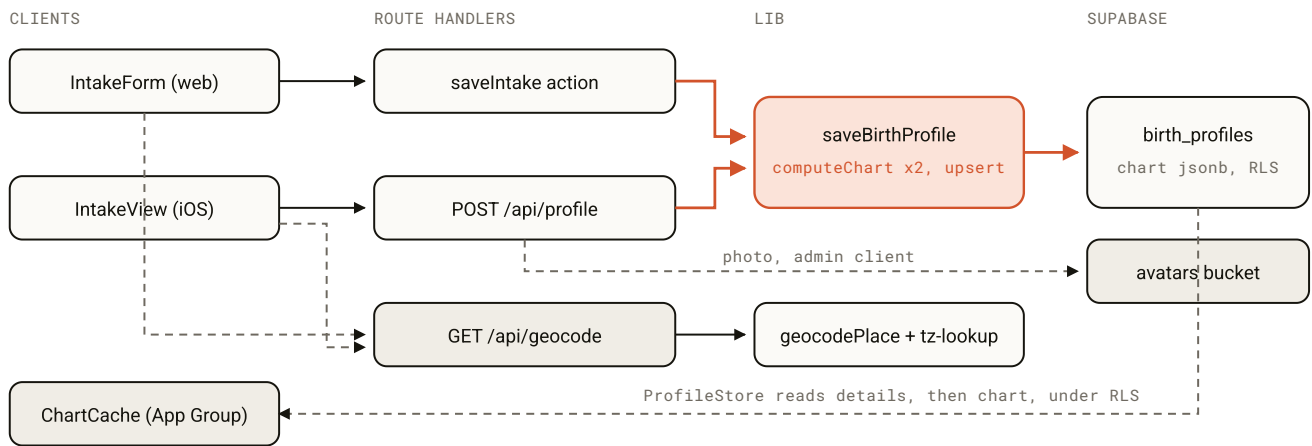
This chapter describes the code on `main` on 30 September 2026, version 0.3.0. A person who signs in has, at first, nothing but an auth user. The web app and the iOS app both notice that and send them to intake. When they submit, the server geocodes nothing and guesses nothing: the form has already resolved a place to coordinates and an IANA timezone, and the route hands those to the Swiss Ephemeris, computes a Vedic and a Western chart, and writes all of it into `birth_profiles` in one upsert. From then on the chat, the kundli, the timeline, the daily readings and the widget all read that stored chart. Nothing recomputes it unless the details change, or the chart's schema falls behind.

5.1 Intent

A birth chart is only as good as the moment and place it is cast for. The intake form therefore refuses to submit until it holds a real latitude, longitude and timezone, and it gets them from a geocoder rather than from the person. Typing "Mumbai" produces a list of candidates with their timezones already attached, and picking one fills four hidden fields. Someone whose village the geocoder does not know can type coordinates by hand, and the timezone is still derived for them, offline, from the coordinates.

The second intent is that the computed chart belongs with the details. `birth_profiles` carries a `chart` jsonb column holding both traditions. Readers never call the ephemeris; they read the column. This is what lets the iOS app cache the chart in the App Group and answer Siri, App Intents and the widget without a network, and it is what makes a chart upgrade a read-time concern rather than a migration.

The third intent is that the two clients do the same work through the same function. The web form submits to a server action, the iOS form posts multipart form data to a route handler, and both end in `saveBirthProfile`. The route exists only because a native client cannot invoke a Next.js server action; its own comment says so.



Two forms, one save. Both clients resolve a place through the public geocode route, then submit through their own door to the same `saveBirthProfile`. The iOS app reads the row back directly from Supabase and caches it on disk for the extension processes.

5.2 Files and folders

<code>app/api/profile/route.ts</code>	POST: the native intake door, returns JSON
<code>app/api/profile/route.test.ts</code>	401, 200, missing field, bad number
<code>app/api/geocode/route.ts</code>	GET ?q=: birthplace search, no auth
<code>app/(app)/app/intake/page.tsx</code>	the web intake screen
<code>app/(app)/app/intake/actions.ts</code>	saveIntake, the server action
<code>app/(app)/app/profile/page.tsx</code>	the same form, prefilled, for editing
<code>app/(app)/app/page.tsx</code>	home: "Enter birth details" when there is no row
<code>components/IntakeForm.tsx</code>	the form, TimePicker, ManualCoords
<code>components/PlaceAutocomplete.tsx</code>	typeahead over /api/geocode
<code>lib/geo.ts</code>	geocodePlace, resolveTimezone
<code>lib/geo.test.ts</code>	two coordinates, two timezones
<code>lib/data/birthProfile.ts</code>	getBirthProfile, saveBirthProfile, ensureCurrentChart
<code>lib/data/birthProfile.test.ts</code>	the upgrade path and the read failure
<code>supabase/migrations/0001_init.sql</code>	birth_profiles and its RLS policy
<code>supabase/migrations/0002_avatar.sql</code>	avatar_url
<code>supabase/migrations/0003_numerology_mode.sql</code>	numerology as a conversation mode
<code>mobile/Sanchara/Sources/RootView.swift</code>	auth state, then profile state
<code>mobile/Sanchara/Sources/Features/Intake/IntakeView.swift</code>	the form and IntakeStore
<code>mobile/Sanchara/Sources/Features/Intake/ProfileStore.swift</code>	has this account a chart yet
<code>mobile/Sanchara/Sources/Features/Intake/BirthProfileDetails.swift</code>	the row, decoded
<code>mobile/Sanchara/Sources/Features/Intake/PhotoField.swift</code>	PhotosPicker and avatarJPEG
<code>mobile/Sanchara/Sources/Services/SancharaAPI.swift</code>	saveProfile, geocode, BirthProfileI
<code>mobile/Sanchara/Sources/Services/ChartCache.swift</code>	the App Group cache

The decisions live in `lib/data/birthProfile.ts` and `lib/geo.ts`. Both route handlers and the server action are thin: they authenticate, validate the shape, upload a photo if one came, and call the lib. That is why the tests for the route mock `saveBirthProfile` entirely and the tests for the lib mock only the database.

5.3 Model

THE TABLE

```
-- supabase/migrations/0001_init.sql
create table public.birth_profiles (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  first_name text not null,
  last_name text not null,
  birth_date date not null,
  birth_time time not null,
  place_name text not null,
  lat double precision not null,
  lng double precision not null,
  timezone text not null,
  chart jsonb,
  updated_at timestamptz not null default now(),
  unique (user_id)
);
alter table public.birth_profiles enable row level security;

create policy "birth self" on public.birth_profiles
  for all using (user_id = auth.uid()) with check (user_id = auth.uid());
```

One row per user, enforced by `unique (user_id)`, which is also the conflict target of the upsert. `chart` is nullable so that a row can in principle exist without one, though nothing in the current code writes a row that way: `saveBirthProfile` always writes the details and the chart together. The policy is owner-only for every verb. The iOS app reads this table directly through supabase-swift with the user's own session, and that policy is the whole reason it can.

Migration 0002 adds `avatar_url text`, nullable. Migration 0003 is not a column on this table at all: it drops and recreates the check constraint on `conversations.tradition` so that a chat can run in `numerology` mode alongside `vedic` and `western`. It belongs to this chapter only in that numerology is computed from the birth date and name stored here; the chat mode itself is in chapter 6. The full schema is catalogued in chapter 13.

THE ROW, IN TYPESCRIPT AND IN SWIFT

```
// lib/data/birthProfile.ts
export type BirthProfileRow = {
  id: string;
  user_id: string;
  first_name: string;
  last_name: string;
  birth_date: string;
  birth_time: string;
  place_name: string;
  lat: number;
  lng: number;
  timezone: string;
  avatar_url: string | null;
  chart: { vedic: unknown; western: unknown } | null;
};
```

Postgres hands `birth_time` back as `HH:mm:ss`, while every form and the ephemeris input use `HH:mm`. That is why `ensureCurrentChart` and the kundli route both `slice(0, 5)` before use, and why the Swift struct carries a `birthTimeShort` computed property doing the same `prefix(5)`. The Swift side, `BirthProfileDetails`, decodes the same columns under snake-case coding keys and adds `fullName`. It is deliberately dependency-free, because `project.yml` compiles it into the widget extension as well as the app.

WHAT GOES INTO THE CHART

```
// lib/astrology/types.ts
export type BirthInput = {
  birthDate: string; // YYYY-MM-DD
  birthTime: string; // HH:mm
  lat: number;
  lng: number;
  timezone: string; // IANA
};
```

Five values, all of which the form must resolve before its submit button enables. The result of `computeChart(birth, tradition)` is a `Chart` carrying the ascendant, twelve house cusps, the planets with sign, degree, house, retrograde flag and (Vedic only) nakshatra, the Moon and Sun signs, the ayanamsa, the running dasha, a `schemaVersion`, and the `derived` facts. How those are computed is chapter 2; this chapter cares only that the column holds `{ vedic, western }` and that `schemaVersion` is what decides whether the stored copy is still current.

5.4 The server side

GEOCODING

```
// lib/geo.ts
export async function geocodePlace(query: string): Promise<GeoResult[]> {
  const q = query.trim();
  if (q.length < 2) return [];
  const url = `https://geocoding-api.open-meteo.com/v1/search?name=${encodeURIComponent(q)}&count=5`;
  const res = await fetch(url, { headers: { accept: "application/json" } });
  if (!res.ok) return [];
  ...
  return (data.results ?? []).map((r) => ({
    name: [r.name, r.admin1, r.country].filter(Boolean).join(", "),
    lat: r.latitude,
    lng: r.longitude,
    timezone: resolveTimezone(r.latitude, r.longitude),
    country: r.country ?? "",
  }));
}
```

The geocoder is Open-Meteo's, which needs no key and returns up to five candidates. The timezone does not come from the geocoder. `resolveTimezone` wraps `tz-lookup`, a table of timezone polygons that runs in process, so every candidate arrives with the IANA name the ephemeris needs. The same function runs in the browser for manually typed coordinates. The package ships no types, hence the `@ts-expect-error` on its import. A query under two characters and any non-2xx from Open-Meteo both return an empty list rather than an error, so the typeahead never shows a failure state; it simply shows nothing.

`app/api/geocode/route.ts` is three lines: read `q`, call `geocodePlace`, return the array. It does not check for a session. The iOS client's comment says why that matters: "The route is public, so this call carries no token." A birthplace search happens before a chart exists and reveals nothing about the account.

THE NATIVE DOOR: POST /API/PROFILE

```
// app/api/profile/route.ts
export const runtime = "nodejs";

const REQUIRED = [
  "first_name", "last_name", "birth_date", "birth_time",
  "place_name", "lat", "lng", "timezone",
] as const;
```

The route runs on the Node runtime because `swissep-wasm` is loaded inside `saveBirthProfile`; the comment above the export says exactly that. The handler resolves the caller with `createRouteSupabase(request)`, which returns a bearer-token client for a native caller and falls back to the cookie client for the web, so RLS evaluates as the user either way. No user is a 401. Then it reads `formData()` and checks each required field is a non-blank string, answering `{ error: "Missing required`

field: <name>" } with a 400 for the first one that is not. lat and lng are parsed with Number and must be finite, or the 400 reads "lat and lng must be numbers".

A photo part is optional. It is used only if it is a File, has a size, and has an image/ content type. uploadAvatar picks an extension from the type (png, webp, otherwise jpg), writes the bytes to the avatars bucket at <userId>/avatar.<ext> with upsert: true, and returns the public URL with a ?v=<timestamp> query so a replaced photo is not served from cache. The upload uses the service-role client, which bypasses RLS; the storage bucket has no per-user policy in the migrations, and the path is fixed to the authenticated user's id by the route, not by the caller. An upload error returns null and the save proceeds without a photo rather than failing.

Finally the route calls saveBirthProfile inside a try. A thrown error is logged and answered with a 500 and the text "Could not compute your chart. Please try again." Success is { ok: true }. The iOS client shows the error string from any non-2xx body verbatim, which is why the messages are written for a person.

THE WEB DOOR: SAVEINTAKE

app/(app)/app/intake/actions.ts is the same procedure as a server action. It gets the user from the cookie client, redirects to /login if there is none, runs an identical uploadAvatar, calls saveBirthProfile with the eight fields coerced by String and Number, and redirects to /app. It does not validate presence: the browser form marks the visible inputs required and its submit button is disabled until a place is resolved, so the action trusts the form. The two uploadAvatar functions are copies, not a shared import.

SAVEBIRTHPROFILE

```
// lib/data/birthProfile.ts
const [vedic, western] = await Promise.all([
  computeChart(birth, "vedic"),
  computeChart(birth, "western"),
]);
const row: Record<string, unknown> = {
  user_id: input.userId,
  ...
  chart: { vedic, western },
  updated_at: new Date().toISOString(),
};
// Only overwrite the avatar when a new one was uploaded.
if (input.avatarUrl) row.avatar_url = input.avatarUrl;

const { error } = await supabase.from("birth_profiles").upsert(row, { onConflict: "user_id" });
if (error) throw new Error(error.message);
```

Both traditions are computed concurrently and written in one upsert keyed on user_id. Because it is an upsert, a first save and an edit are the same call. The avatar is the one column that is conditionally set: an edit without a new photo leaves the stored URL alone, which is the behaviour the iOS BirthProfileInput documents from its side ("leaving this nil keeps the existing photo"). A rejected write resolves with error rather than throwing, so the function turns it into a throw; the route's catch and the action's error boundary depend on that.

GETBIRTHPROFILE AND THE WHOLE-SIGN UPGRADE

`getBirthProfile` selects the caller's row with `maybeSingle()`, throws on a read error, returns `null` when there is no row, and otherwise passes the row through `ensureCurrentChart`. Three server-component pages read through it: home, chat and profile. The chat page redirects to `/app/intake` when `!profile?.chart`; the profile page redirects when there is no row; the home page renders an "Enter birth details" call to action instead of redirecting.

```
// lib/astrology/derived.ts
export function isChartStale(chart: Chart): boolean {
  return (chart.schemaVersion ?? 1) < CHART_SCHEMA_VERSION || !chart.derived;
}
```

`CHART_SCHEMA_VERSION` is 2. The constant's comment explains the bump: a chart stored below this version "has house numbers that disagree with the rashi labels every renderer draws", because house assignment for Vedic charts moved to whole-sign counting from the ascendant (`houseFrom(ascSign, signOf(lon))` in `computeChart`) while Western charts keep Placidus cusps. A chart with no `derived` block is also stale, since the readings in 0.3.0 cite lords, aspects and dignity from it.

`ensureCurrentChart` is the upgrade path, and its comment is the design in four sentences. Readers could always derive correct facts in process, but the kundli, the PDF and the iOS app draw from the stored chart, so it has to be rewritten for them to agree. Doing it on read means no migration and no backfill: a chart corrects the first time its owner uses the app. Both traditions are recomputed together, so a profile is never half upgraded. And a failure is swallowed, because an upgrade that cannot be written is not a reason to fail the request that triggered it.

```
// lib/data/birthProfile.ts
let stage: "compute" | "write" = "compute";
try {
  const birth: BirthInput = { ..., birthTime: String(profile.birth_time).slice(0, 5), ... };
  const [vedic, western] = await Promise.all([
    computeChart(birth, "vedic"),
    computeChart(birth, "western"),
  ]);
  stage = "write";
  const { error } = await supabase
    .from("birth_profiles")
    .update({ chart: { vedic, western } })
    .eq("user_id", profile.user_id);
  if (error) throw new Error(error.message);
  return { ...profile, chart: { vedic, western } };
} catch (err) {
  const reason = stage === "compute" ? "recompute failed" : "write failed";
  console.error(`chart upgrade skipped (${reason})`, err);
  return profile;
}
```

The `stage` variable exists so the log says which half failed. The check on the resolved `error` is the subtle line: without it, a write rejected by RLS would fall through to the return and the function would claim an

upgrade it never wrote. The test that pins this is named for exactly that failure mode.

WHERE THE UPGRADE RUNS

Only where the stored chart is read as a whole. `getBirthProfile` runs it for the three web pages, and `app/api/kundli/route.ts` calls `ensureCurrentChart` itself before rendering the PDF. The iOS app reads the column directly from Supabase and does not run it; a stale chart on a phone corrects the next time its owner opens the web app or downloads the PDF, and the next `ProfileStore.load()` picks up the rewritten row.

5.5 The clients

WEB: INTAKEFORM

`components/IntakeForm.tsx` is a client component wrapping a plain HTML form whose `action` is the server action. It takes an optional `initial`, and `editing = !!initial` changes the button text from "Save & build my chart" to "Update my details". The place is held in state as a `GeoResult` and mirrored into four hidden inputs, `place_name`, `lat`, `lng` and `timezone`. The submit button is `disabled={!geo}`, so the action can only ever receive a resolved place.

Three parts of the form are worth reading.

- **PlaceAutocomplete** fetches `/api/geocode?q=` on every change once the query has two non-blank characters, with no debounce, and lists each result as "name · timezone". Picking one calls `onPick`, sets the input text to the picked name and clears the list. Typing again clears the pick, which sets `geo` back only when a new result is chosen.
- **TimePicker** is three `select` s (hour 1 to 12, minute 0 to 59, AM or PM) that compute a 24-hour `HH:mm` into a hidden `birth_time` input. The comment says why: it "avoids the flaky native `<input type="time">`". `parseTime` reverses the mapping for editing, and defaults to 12:00 PM when there is nothing to parse, matching the hint "noon is fine if unknown".
- **ManualCoords** replaces the autocomplete when the person clicks "Can't find your birthplace?". It takes an optional place name and two decimal inputs, and on every change calls `resolveTimezone(lat, lng)` in the browser, so even a hand-typed place gets a real timezone. A non-finite pair clears `geo` and disables submit again. The name falls back to `"lat, lng"` when none was typed.

The photo control is a hidden `file` input behind a circular label, accepting png, jpeg and webp, with a preview from `URL.createObjectURL`. The date input is native, with a "Selected: Monday, February 9, 1998" echo under it, because a bare date field on a dark theme is easy to misread.

`app/(app)/app/intake/page.tsx` renders the form with no initial value under the heading "Your birth details". `app/(app)/app/profile/page.tsx` renders the same component with `initial` built from the row (slicing `birth_time` to five characters) and the same `saveIntake` action, so editing is intake with defaults. The profile chapter, chapter 11, covers the rest of that page.

IOS: ROOTVIEW DECIDES WHO SEES INTAKE

```
// mobile/Sanchara/Sources/RootView.swift
struct SignedInView: View {
    @State private var profile = ProfileStore()

    var body: some View {
        Group {
            switch profile.state {
            case .loading:      LoadingScreen()
            case .needsIntake:  IntakeView { await profile.load() }
            case .ready:       ChatView(profile: profile)
            case .failed(let message): ... "Try again" and "Sign out"
            }
        }
        .task { await profile.load() }
    }
}
```

`RootView` switches on `AuthStore.state`; a signed-in user lands in `SignedInView`, which owns a `ProfileStore` and switches on its state. `needsIntake` shows `IntakeView` with an `onSaved` closure that simply reloads the store, and a successful reload flips the state to `ready`, which swaps the whole screen for `ChatView`. There is no navigation push and no flag to clear: the store's state is the router.

IOS: PROFILESTORE

`load()` first fills `chart` and `details` from `ChartCache` if they are nil, so a cold launch with a cached chart draws immediately. It then selects the nine detail columns from `birth_profiles` with `.not("chart", operator: .is, value: "null")` and `limit(1)`: a row without a chart counts as needing intake. If there is a row it saves the details to the cache and calls `loadChart()`, a second select of only the `chart` column, decoded as `ChartBundle`, of which the `vedic` half is kept and cached. The comment on the class gives the reason for two queries: the jsonb is roughly 4 KB and changes only when the details are re-saved, and the details are needed to decide the route before the chart is needed to draw anything.

The catch is where the offline story lives. If the network call throws and both a cached chart and cached details exist, the state is `ready`; only a device that has never loaded a profile gets `failed` with "Could not load your chart. Check your connection and try again." A failure inside `loadChart` is silent: "Keep whatever the cache gave us."

IOS: INTAKEVIEW AND INTAKESTORE

`IntakeView` is the native counterpart of `IntakeForm` and its doc comment says so, posting to the same route. It takes an optional `initial: BirthProfileDetails`, an optional `onCancel` (present only when shown as a sheet from the profile) and the `onSaved` closure. The form state is an `@Observable IntakeStore`:

- Names default to empty, or to `AppleSignIn.rememberedName` on a first run, so a person who signed in with Apple does not retype what Apple already gave.

- `birthDate` defaults to 25 years ago and is bounded by `earliestBirthDate` (1 January 1900) and today; `birthTime` defaults to 12:00. Both use wheel pickers, and the time picker carries the hint "A wrong hour moves the ascendant."
- `placeQuery` has a `didSet` that calls `scheduleSearch()`, which cancels the previous task, ignores queries under two characters, sleeps 300 ms, then calls `SancharaAPI.geocode`. The comment: "Debounced so a typed city name is one request, not one per keystroke." A picked `GeoResult` replaces the search field with a card showing name and timezone and a "Change" button.
- `isComplete` requires non-blank first and last names and a picked place. The save button is disabled and faded otherwise.
- `prefill(from:)` fills the form from saved details, parsing the date and the five-character time with `en_US_POSIX` formatters, and builds the place from the stored coordinates and timezone rather than searching again, because "the birthplace has not moved".
- `save()` formats the date as `yyyy-MM-dd` and the time as `HH:mm` in the device's own calendar, since the picked values are wall-clock at the birthplace exactly as the web form sends them, builds a `BirthProfileInput`, and calls `SancharaAPI.saveProfile`. On failure, `errorMessage` is the API error's description, shown above the button.

`SancharaAPI.saveProfile` attaches the Supabase access token as a bearer header and sends a hand-built multipart body: the eight text parts, then, if `photoJPEG` is non-empty, a `photo` part named `avatar.jpg` with type `image/jpeg`. A 401 becomes `APIError.unauthorized` ("Your session expired. Please sign in again."); any other non-2xx becomes `APIError.server` carrying the response body as the message.

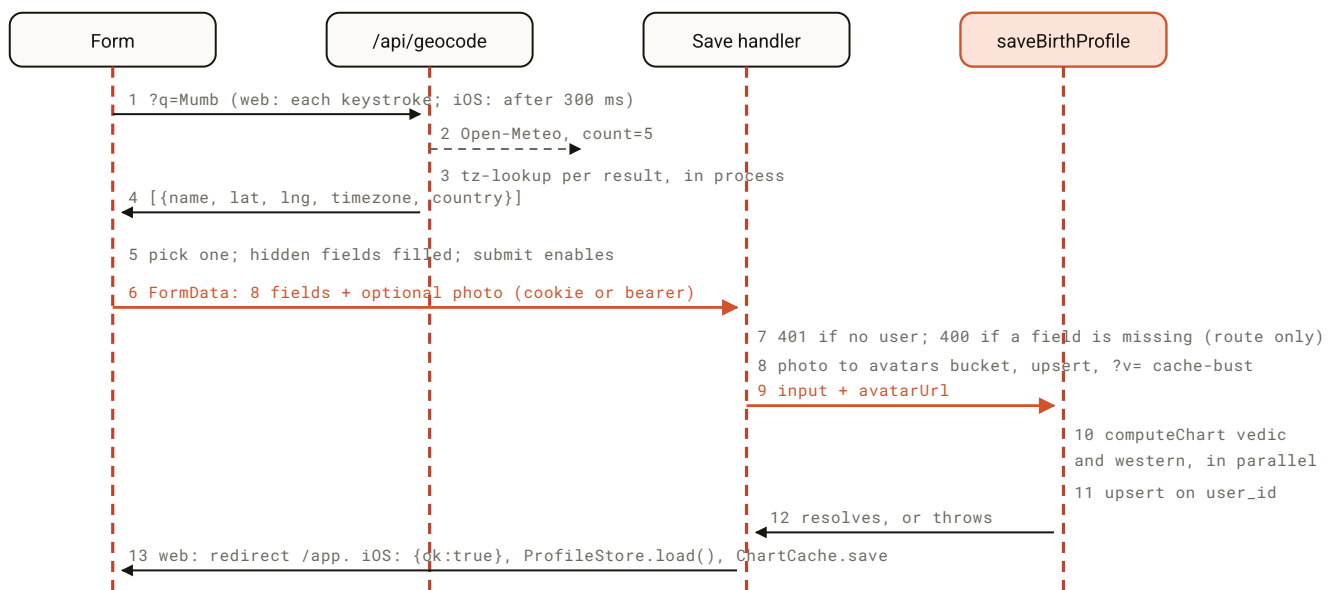
IOS: THE PHOTO

`SancharaPhotoField` wraps `PhotosPicker`, which runs out of process, so the app never requests photo library permission; it receives one image. The store's `photoItem` setter loads the item as `Data`, decodes a `UIImage` and runs `avatarJPEG()`, which scales the longest side down to 1024 points and encodes at quality 0.8. The comment gives the reason: a phone photo is several megabytes and is shown at 72 points. `photoJPEG` stays nil unless a new image was chosen, which is what lets an edit keep the existing photo. The field shows the new image if there is one, else the existing avatar URL through `AsyncImage`, else "Add photo".

IOS: EDITING FROM THE PROFILE

`ProfileView` presents `IntakeView(initial: profile.details, onCancel: ...)` as a sheet, with an `onSaved` that reloads the store and dismisses. Because the store's `load()` refetches both the details and the chart and rewrites the cache, an edited birth time reaches the kundli, Siri and the widget through the same path as a first save.

5.6 How it works



From a typed city to a stored chart. The place is resolved before submit, so the save handler never geocodes. The Swiss Ephemeris runs twice, once per tradition, inside one request.

FIRST INTAKE

1. A signed-in user with no row is routed to intake. On the web, `/app` shows the "Enter birth details" link, and `/app/chat` and `/app/profile` redirect. On iOS, `ProfileStore.load()` finds no row with a chart and `SignedInView` renders `IntakeView`.
2. They type a city. The web fetches on every change; iOS waits 300 ms after the last keystroke. Both hit `GET /api/geocode`, which calls Open-Meteo and stamps a timezone on each candidate with `tz-lookup`.
3. They pick a candidate. The web fills four hidden inputs; iOS sets `place`. Submit becomes enabled.
4. They submit. The web posts the form to `saveIntake` with the session cookie; iOS posts multipart to `/api/profile` with a bearer token. Either handler uploads a photo if there is one and calls `saveBirthProfile`.
5. `saveBirthProfile` converts local birth time to UT in the given zone, runs the ephemeris for both traditions, and upserts the row.
6. The web redirects to `/app`, which now finds a profile and greets the person by first name. iOS receives `{ ok: true }`, calls `profile.load()`, which fetches the details and the chart, writes both to `ChartCache`, and sets `ready`; the root switches to `ChatView`.

EDITING

The same path with defaults. The web profile page passes `initial`; iOS passes the cached `BirthProfileDetails`. A submit without a new photo carries no photo part, so `avatarUrl` is null and the stored URL survives the upsert. A submit with changed birth data recomputes both charts, which both pages tell the person in advance: "Changing your birth data recomputes your chart."

THE UPGRADE ON READ

A row written before version 2 of the chart schema is read by `getBirthProfile` or the kundli route, found stale by `isChartStale`, recomputed from its own stored details, and updated in place. The caller gets the new chart in the same request. If the recompute or the write fails, the caller gets the old row and a log line naming the stage. No cron job, no migration, and no request ever fails because of it.

5.7 Performance and cost

- **Two ephemeris runs per save, in parallel.** `Promise.all` over the two `computeChart` calls. The WebAssembly module is loaded once per function instance through `getSwe()`; the save is the only place in intake that touches it.
- **The Node runtime, not Edge.** The route declares `runtime = "nodejs"` because the WASM ephemeris needs it. The geocode route makes no such declaration, since it only fetches and runs a lookup table.
- **Geocoding is free and keyless.** Open-Meteo's search endpoint takes no API key, and `tz-lookup` resolves timezones in process, so a search costs one outbound HTTP call and no third-party account.
- **Request volume differs by client.** The web autocomplete fetches per keystroke after two characters; a six-letter city is up to five requests. iOS debounces to one. Neither caches results.
- **The photo is shrunk on the phone.** 1024 points on the long side at JPEG 0.8 before upload. The web sends whatever the file input received; the bucket stores it as-is.
- **One read, one small column.** The iOS store fetches the details and the chart in two selects but caches both, so a warm launch reads from disk and the network refresh is additive. The chart is about 4 KB.
- **Upgrades are amortised.** A stale chart costs one extra pair of ephemeris runs and one update, once, on the first read after the schema bump, and only for the rows that are actually read.

5.8 Edge cases and tests

CONDITION	BEHAVIOUR
No session on the route	401 before the body is read; <code>saveBirthProfile</code> is never called
No session on the server action	Redirect to <code>/login</code>
A required field missing or blank (route)	400 naming the field; nothing saved
<code>lat</code> or <code>lng</code> not a finite number (route)	400 "lat and lng must be numbers"
Birthplace not found by the geocoder	Empty list; the web offers manual coordinates, with the timezone still derived in the browser
Query under two characters, or Open-Meteo returns non-2xx	<code>geocodePlace</code> returns <code>[]</code> , never throws
Birth time unknown	Both forms default to 12:00 and say noon is fine; the ascendant will be approximate and the iOS hint says so
Photo upload fails	<code>uploadAvatar</code> returns null; the profile is saved without a photo
Edit without a new photo	<code>avatar_url</code> is left out of the upsert row, so the stored URL stays
Replaced photo served from cache	The public URL carries <code>?v=<timestamp></code>
Ephemeris throws during save	Route: 500 "Could not compute your chart. Please try again." Action: the error propagates to Next.js
Upsert rejected by RLS or a constraint	The resolved <code>error</code> is turned into a throw, so it is not reported as success
Stored chart is schema 1 or lacks <code>derived</code>	Recomputed and rewritten on read; both traditions together
Upgrade recompute or write fails	Logged as "recompute failed" or "write failed"; the original row is returned; the request succeeds
Read of <code>birth_profiles</code> fails	<code>getBirthProfile</code> throws rather than returning null, so an outage is not shown as "no profile"
Row exists but <code>chart</code> is null	Web chat redirects to intake; iOS <code>ProfileStore</code> filters such rows out and shows intake
iOS offline at launch, cache present	State is <code>ready</code> from the cache; the refresh failure is swallowed
iOS offline at launch, no cache	<code>failed</code> with "Try again" and "Sign out"
Photo the phone cannot decode	"That image could not be read. Try another." and <code>photoJPEG</code> stays nil
Simulator build without the App Group entitlement	<code>ChartCache</code> falls back to the caches directory; the app works, the widget does not share

TESTS

All Vitest, all passing without a network, a database or the ephemeris on the route side:

- `lib/geo.test.ts` : `resolveTimezone` "maps New Delhi coordinates to Asia/Kolkata" and "maps New York coordinates to America/New_York". The lookup table is real; only the geocoder is untested, since it is an HTTP call.
- `app/api/profile/route.test.ts` , with `createRouteSupabase` and `saveBirthProfile` mocked: "returns 401 when there is no user", "saves the profile and returns 200 for a valid payload" (asserting the lib received `lat` as the number 19.076 and the timezone string), "returns 400 when a required field is missing" (the timezone is omitted), and "returns 400 when lat is not a number".
- `lib/data/birthProfile.test.ts` , with a fake `Db` and the real `computeChart` . For `ensureCurrentChart` : "returns the profile unchanged when there is no stored vedic chart", "returns the profile unchanged when the stored chart is already current", "recomputes and writes both traditions together for a stale chart" (asserting the written `schemaVersion` equals `CHART_SCHEMA_VERSION`), "swallows a thrown compute/write exception and returns the original profile", and "swallows a resolved write error (RLS/constraint) and returns the original profile unchanged", which asserts the log says "write failed". For `getBirthProfile` : "returns an upgraded profile when the stored chart is stale", "returns the original profile unchanged when the stored chart is already current", "returns null when there is no stored profile", and "throws on a genuine read failure instead of reporting it as no profile".

The iOS intake has no unit tests of its own. `IntakeStore` 's formatting and debounce are exercised only by hand; the Swift test target covers the kundli (chapter 7) and the chart facts, not this form.

5.9 Study notes

READ IN THIS ORDER

1. `supabase/migrations/0001_init.sql` , the `birth_profiles` table and its policy.
2. `lib/data/birthProfile.ts` , the save, the read, and the upgrade on read.
3. `lib/data/birthProfile.test.ts` , especially the resolved-error test and its comment.
4. `lib/geo.ts` , how a place becomes coordinates and a timezone.
5. `app/api/profile/route.ts` , then `app/(app)/app/intake/actions.ts` , the two doors.
6. `components/IntakeForm.tsx` , the hidden fields, the time picker and manual coordinates.
7. `mobile/Sanchara/Sources/Features/Intake/ProfileStore.swift` , cache first, then network.
8. `mobile/Sanchara/Sources/Features/Intake/IntakeView.swift` , the debounce and the save.
9. `mobile/Sanchara/Sources/RootView.swift` , how the store's state routes the screen.

CHECK YOURSELF

Why does the form, not the server, resolve the timezone?

Because the person must see and confirm it. The candidate list shows the timezone beside each place, and the manual path derives one in the browser, so what reaches the ephemeris is what the person saw. The server trusts the five values it is given and never geocodes.

Why are both charts recomputed when only the Vedic house rule changed?

So a profile is never half upgraded. `ensureCurrentChart` writes `{ vedic, western }` as one value; a partial rewrite would leave two charts from two different computations under one `schemaVersion` check.

What would go wrong if `ensureCurrentChart` ignored the resolved error ?

`supabase-js` resolves rather than throws on a rejected write. The function would return the recomputed chart as if it were stored, the PDF would draw from it, and the row would still hold the stale one; the next read would repeat the whole recompute.

Why does `ProfileStore` fetch the details and the chart separately?

The details decide the route (intake or chat) and are small; the chart is about 4 KB and only needed to draw. Fetching them apart lets the screen appear before the chart lands, and lets a chart fetch fail quietly while the cached copy stands in.

Why does `getBirthProfile` throw on a read error instead of returning null?

Three pages treat null as "never completed intake" and send the person to the form. A database outage rendered as null would ask an existing user to re-enter their birth details.

Why is the avatar column only written when a new photo arrived?

The save is an upsert of the whole row. Writing `avatar_url: null` on every edit would erase the photo whenever someone corrected a birth time.

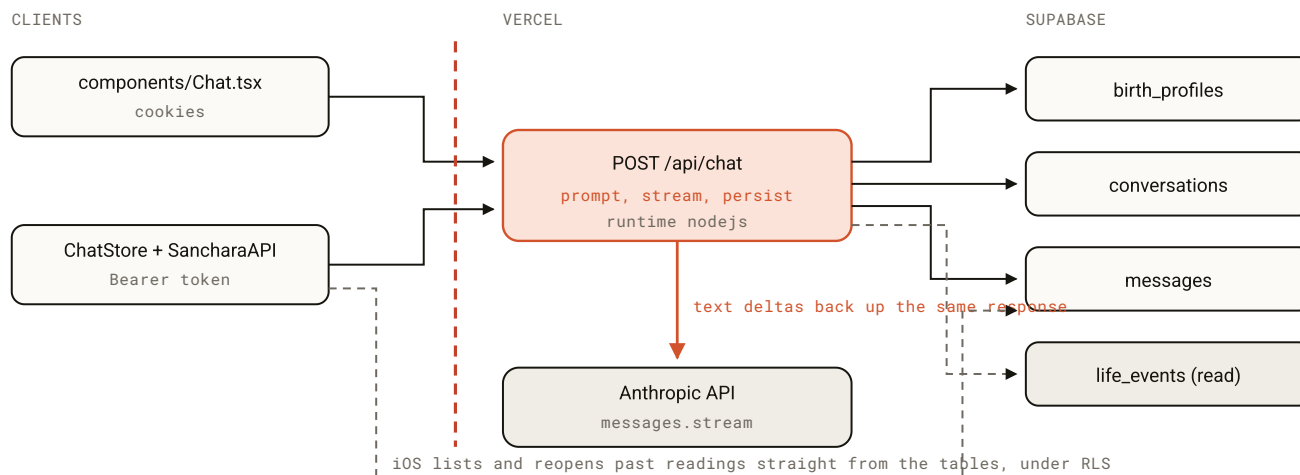
6 Chat

One question in, one reading out, streamed as plain text while it is written. The server owns the prompt, the model and the transcript; the two clients own nothing but the optimistic bubble and the conversation id they were handed in a response header.

6.1 Intent

Chat is the product. Every other feature either feeds it (intake computes the chart it reads from, the timeline gives it the user's life) or is a scheduled version of it (the morning and night readings). A conversation is a sequence of turns in one mode, Vedic, Western or numerology, and the mode is fixed per conversation because it decides which chart, and which system prompt, the whole transcript was read from. A user can hold many conversations; the web lists them in a sidebar, iOS in a sheet, and either client can reopen one and continue it.

This chapter describes the code on `main` on 30 September 2026, version 0.3.0. The prompt builders, the model settings, the cache breakpoints and the history budget that the route applies are chapter 3; this chapter takes them as given and follows one turn from a tap to a stored row. The iOS files that answer some questions on the phone without a server call, `OnDeviceReasoner`, `OnDeviceSuggestions` and `ChartFacts`, live in the same folder as the chat feature but belong to chapter 10; here they appear only where `ChatStore` hands a question to them.



One route, two callers, three tables. Both clients send a turn to the same handler and read the same byte stream back. Only iOS reads the conversation tables directly, and only to list and reopen; every write goes through the route.

6.2 Files and folders

```
app/api/chat/route.ts
lib/data/chat.ts

lib/data/history.ts
```

```
POST: auth, mode, prompt, stream, persist
getOrCreateConversation, appendMessage,
getMessages, listConversations
selectHistory, the resend budget (chapter 3)
```

lib/supabase/route.ts	createRouteSupabase: bearer or cookie client
supabase/migrations/0001_init.sql	conversations and messages, RLS, indexes
supabase/migrations/0003_numerology_mode.sql	tradition check gains 'numerology'
app/(app)/layout.tsx	lists conversations for the sidebar
app/(app)/app/chat/page.tsx	new chat, or reopen ?c=<id>
components/Chat.tsx	the client component: composer, stream, transcript
components/AppShell.tsx	the sidebar of past readings
mobile/Sanchara/Sources/Features/Chat/	
└─ ChatModels.swift	ChatMode, ChatMessage, Conversation, StoredMessage
└─ ChatStore.swift	one transcript: send, stream, open, new
└─ ChatView.swift	the screen, its toolbar and sheets
└─ HistoryView.swift	past readings as a sheet
└─ MarkdownText.swift	the reading's markdown, block by block
└─ OnDeviceReasoner.swift	chapter 10
└─ OnDeviceSuggestions.swift	chapter 10
└─ ChartFacts.swift	chapter 10
mobile/Sanchara/Sources/Services/SancharaAPI.swift	chatStream and the bearer request
lib/astrology/prompt.test.ts	the prompt's contents
lib/data/history.test.ts	the budget
lib/data/injection.test.ts	helpers write through the injected client
app/smoke.test.ts	the app layout module loads

Everything that decides something is on the server. The two clients share no code and are written in two languages, but they are the same program: the iOS store's doc comment says it mirrors `components/Chat.tsx`, the intro prompt is one string copied verbatim, and the mode dot colours in `ChatModels.swift` are the hex values from the web component. When the two disagree, the web is the reference and the iOS file says so.

6.3 Model

TWO TABLES

```
-- supabase/migrations/0001_init.sql
create table public.conversations (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  tradition text not null check (tradition in ('vedic','western')),
  title text,
  created_at timestamptz not null default now()
);
create index on public.conversations (user_id, created_at desc);

create table public.messages (
  id uuid primary key default gen_random_uuid(),
  conversation_id uuid not null references public.conversations(id) on delete cascade,
  role text not null check (role in ('user','assistant')),
  content text not null,
  created_at timestamptz not null default now()
);
create index on public.messages (conversation_id, created_at);
```

A conversation is an owner, a mode and a title; a message is a role and a body. There is no `updated_at`, no token count, no model name and no deep flag on either row. The transcript is the only thing stored, which is what lets any client, and the timeline's moment extraction, read it back without knowing how it was produced. Migration `0003_numerology_mode.sql` drops and re-adds the tradition check so it accepts `'numerology'`; the column is still called `tradition` although the TypeScript type that fills it is `ChatMode`. The two indexes are exactly the two queries: a user's conversations newest first, and a conversation's messages oldest first.

Row Level Security is owner-only on both. `"conv self"` checks `user_id = auth.uid()` for every operation. `"msg self"` has no owner column to check, so it joins: a message is visible or writable only if a conversation row with that id and the caller's `user_id` exists. Deleting a user cascades to their conversations and from there to their messages, which is what the account deletion route relies on.

THE WIRE SHAPE

The request body is four fields, typed inline in the route: `conversationId` (optional; absent starts a new conversation), `tradition` (a `ChatMode`), `message` and `deep` (optional). The response is `text/plain; charset=utf-8`, the model's text deltas concatenated with no framing, plus two headers: `x-conversation-id`, which is the id to send back on the next turn, and `cache-control: no-store`. There is no SSE, no JSON envelope and no final event; the stream closing is the end of the reading. Both clients read the header first and the bytes second.

THE CLIENT-SIDE TYPES

On the web the transcript is `Msg[]`, a role and a content string. On iOS it is `[ChatMessage]`, which adds two things: a `UUID` identity assigned at creation, so SwiftUI keeps a row in place while its content grows during streaming, and a `Source` of `.server` or `.onDevice`. The doc comment on `Source` is worth reading in full:

the source is shown in the transcript, not merely tracked, because the two behave differently in ways a person can feel, and an on-device answer is not saved with the rest of the reading. `Conversation` and `StoredMessage` are `Decodable` views of the two tables, read straight from Supabase; `Conversation.mode` maps an unknown tradition string to `.vedic` rather than failing to decode.

6.4 The server side

THE ROUTE, IN ORDER

`app/api/chat/route.ts` exports one `POST` and declares `runtime = "nodejs"`, because the ephemeris is a WebAssembly module loaded from disk and the streaming SDK wants a full Node runtime. The handler runs in two phases: a preparation phase inside one `try`, where any failure returns a status and a sentence, and a streaming phase after the response has already begun, where a failure can only be written into the stream.

1. **Auth.** `createRouteSupabase(request)` returns a bearer client if the request carries `Authorization: Bearer`, otherwise the cookie client. `auth.getUser()` with no user is a 401 with the body `Unauthorized`.
2. **Validation.** An empty or whitespace message is a 400 `Empty message`; a tradition outside `vedic`, `western`, `numerology` is a 400 `Invalid mode`.
3. **The profile.** One select on `birth_profiles` (RLS returns only the caller's row, so `maybeSingle()` with no filter is correct). A database error is a 500; a row with no chart is a 400 `No chart`. Complete `intake first.`; and `ensureCurrentChart` recomputes and rewrites a chart whose schema version is behind or which lacks derived facts, so an old profile gets the current prompt without the user doing anything.
4. **Today.** Luxon renders the date in `profile.timezone`, and `describeToday` coarsens it to a part of day.
5. **The prompt.** Numerology builds `buildNumerologySystem` plus a today block carrying the personal year and month. The two chart modes pick `profile.chart[tradition]`, compute `factsFor` (undefined for Western), fetch the day's transit chart and describe it against the natal chart inside a `try` that degrades to no transit text, build `buildChartSystem`, append the timeline inside another `try` that logs and moves on, and build the today block with the deep-dependent word limit.
6. **The conversation.** `getOrCreateConversation` either verifies the given id belongs to this user or inserts a row with the message's first 80 characters as its title.
7. **History.** `getMessages` then `selectHistory`; the last retained turn gets the cache breakpoint; the new message is pushed on the end; and the new user message is written to `messages` before the model is called.

Anything thrown in those seven steps is caught once, logged as `chat pre-stream error`, and answered with a 500 whose body is a sentence a client can show. The "Conversation not found" thrown for a foreign id surfaces this way, as a 500 rather than a 403 or 404; the clients treat any non-2xx the same, so the distinction is not lost on anyone.

THE STREAM

```
// app/api/chat/route.ts
const stream = new ReadableStream({
  async start(controller) {
    let full = "";
    try {
      const s = anthropic().messages.stream({ /* model, thinking, effort, system, messages */ })
      for await (const event of s) {
        if (event.type === "content_block_delta" && event.delta.type === "text_delta") {
          full += event.delta.text;
          controller.enqueue(encoder.encode(event.delta.text));
        }
      }
      const usage = (await s.finalMessage()).usage;
      console.log(`chat usage conv=${conversationId} model=${model} ...`);
    } catch (err) {
      console.error("chat stream error", err);
      controller.enqueue(encoder.encode("\n\n[The stars are momentarily clouded. Please try again]"));
    } finally {
      if (full.trim()) await appendMessage(conversationId, "assistant", full, supabase);
      controller.close();
    }
  },
});
```

Only `text_delta` events are forwarded. Thinking blocks, which both models produce under adaptive thinking, are consumed and never sent, so the client sees the reading and not the reasoning. The accumulator `full` holds exactly what was streamed, and the `finally` stores it as the assistant turn only if it is non-blank. Two consequences follow. If the model fails before producing anything, the user turn is stored, the apology line goes down the wire, and no assistant row is written, so the next turn's history ends on a user message and `selectHistory` passes it through. If the model fails part-way, the partial text is stored and the apology is not, because the apology is enqueued to the client but never appended to `full`. A conversation therefore never contains the bracketed error text.

THE DATA HELPERS

`lib/data/chat.ts` is four functions, each taking an optional `db?: Db` and building the cookie client when none is given. `getOrCreateConversation` is the only one with logic: given an id it selects it with both `id` and `user_id` filters and throws if nothing comes back, defence in depth on top of RLS, per its comment; without one it inserts and returns the new id. `appendMessage` is one insert. `getMessages` selects role, content and time for one conversation, ascending. `listConversations` selects id, tradition, title and time for the caller, descending, with no user filter because RLS supplies it.

6.5 The clients

WEB: THE PAGE

`app/(app)/app/chat/page.tsx` is a server component. It loads the birth profile and redirects to `/app/intake` when there is no chart, so the composer can never be shown to someone the route would

reject. It reads `?c=` from the search params. With an id it loads that conversation's messages and renders `Chat` with `key={c}`, the stored messages, and the conversation's own tradition as the initial mode (falling back to Vedic if the row is not in the list). Without one it renders `Chat` with `key="new"` and `autostart` set only when the user has no conversations at all. The `key` is what makes switching between readings in the sidebar remount the component rather than mutate it. The surrounding `app/(app)/layout.tsx` calls `listConversations` and hands the result to `AppShell`, whose sidebar links each one as `/app/chat?c=<id>`.

WEB: THE COMPONENT

`components/Chat.tsx` is a client component holding the mode, the deep flag, the messages, the input and a `busy` flag in state, and the conversation id in a ref, because it changes during a fetch and must not trigger a render. `send(textArg, silent)` is the whole feature: append the user bubble (unless silent) and an empty assistant bubble, post the body, take the id from the header, then loop on `res.body.getReader()` with a `TextDecoder` in streaming mode and append each chunk to the last message. A non-OK response, a missing body or a thrown fetch all replace the last bubble with "Something went wrong. Please try again." In `finally`, `busy` clears and, if this was the first turn of a new conversation, `router.refresh()` re-runs the layout so the sidebar shows the new reading.

The composer is one input and three buttons: a mode pill that cycles Vedic, Western, Numerology; a Deep toggle, hidden below the `sm` breakpoint; and send, disabled while busy or while the input is blank. Enter sends. Under it is the one line of copy that appears in the same words on iOS: "For guidance and reflection. Not a substitute for professional advice." The empty state shows a greeting with the first name and the composer centred; once there is a message, the transcript scrolls above and the composer pins to the bottom with a safe-area inset. An assistant message renders through `ReactMarkdown` with `remark-gfm` inside the `prose-reading` styles; while it is still empty it shows "Reading your chart..." with a shimmer, and while it is still growing it shows a caret. An effect scrolls to the bottom on every change to `messages`.

IOS: THE STORE

`ChatStore` is `@Observable` and `@MainActor`. Its state matches the web component's field for field (messages, input, mode, deep, a streaming flag, an error string, the conversation id) and adds the list of past conversations, a `streamTask` so a turn can be cancelled, and a `chart` whose `didSet` builds an `OnDeviceReasoner`. `send(text, silent:, forceServer:)` appends the bubbles, then in a task does one thing the web does not: if the turn is not silent, not forced to the server and the mode is Vedic, it asks the reasoner to route the question, and if the answer comes back `.answered`, writes it into the assistant bubble with source `.onDevice` and stops. The comment says why only Vedic: the router reads a Vedic chart, so a Western or numerology question has nothing local to answer from. Everything else goes to `SancharaAPI.chatStream`, and the loop over its events either records the conversation id or appends text.

The other entry points are short. `start()` loads the conversation list and, for a brand-new account with nothing to show, sends the intro prompt silently; the web does the same through `autostart`. `newReading()` cancels any stream and clears the transcript without touching the server, since the previous reading is already stored. `open(conversation)` cancels, adopts the conversation's id and mode, and reads its messages straight from the `messages` table through `Supa.client`. `loadConversations()` reads the `conversations` table the same way and keeps the last known list on failure rather than raising an error. `expandLastAnswer()` removes an on-device answer and resends the same question with `forceServer`, so a misrouted lookup costs one tap.

IOS: THE VIEWS

`ChatView` is mounted by `RootView` with the `ProfileStore`, and owns the store plus the alerts, daily readings and suggestion engines that share its toolbar. Its body is the web layout in SwiftUI: an opening screen with the greeting and the composer when the transcript is empty, otherwise a `ScrollViewReader` transcript, the suggestion chips, and the composer. Three `.task` modifiers run on appear: one hands `profile.chart` to the store and calls `start()`; one loads the inbox and, only now that the user has a chart, asks for push permission; one warms the timeline cache for the widget. The composer is a vertical-axis `TextField` of one to four lines, the mode pill, the Deep pill and an arrow button disabled while streaming or blank, followed by the same disclaimer line. An assistant row is `ShimmerText` while empty, then `MarkdownText` with a `StreamingCaret` on the last row while streaming, and, when the source is on-device, a footer that says so and offers "Ask for a full reading". All three motion views take an `active` flag that is false under Reduce Motion.

The toolbar carries history (a clock), the wordmark, new reading, the inbox bell with an unread dot, and an avatar menu. `HistoryView` is a sheet listing `chat.conversations` with title, mode dot and time; tapping one calls `open` and dismisses; New calls `newReading`. Deep links and push taps are handled here too: an `.ask(question)` destination, which is how a question Siri declined arrives, starts a new reading and sends with `forceServer`.

`MarkdownText` exists because `AttributedString(markdown:)` handles inline syntax but collapses block structure. It splits the text into headings, paragraphs, bullets (with the ordinal kept for numbered items) and rules, gives each the spacing the web's CSS gives it, and parses only the inline runs with Foundation. A parse failure falls back to the plain text.

IOS: THE API CALL

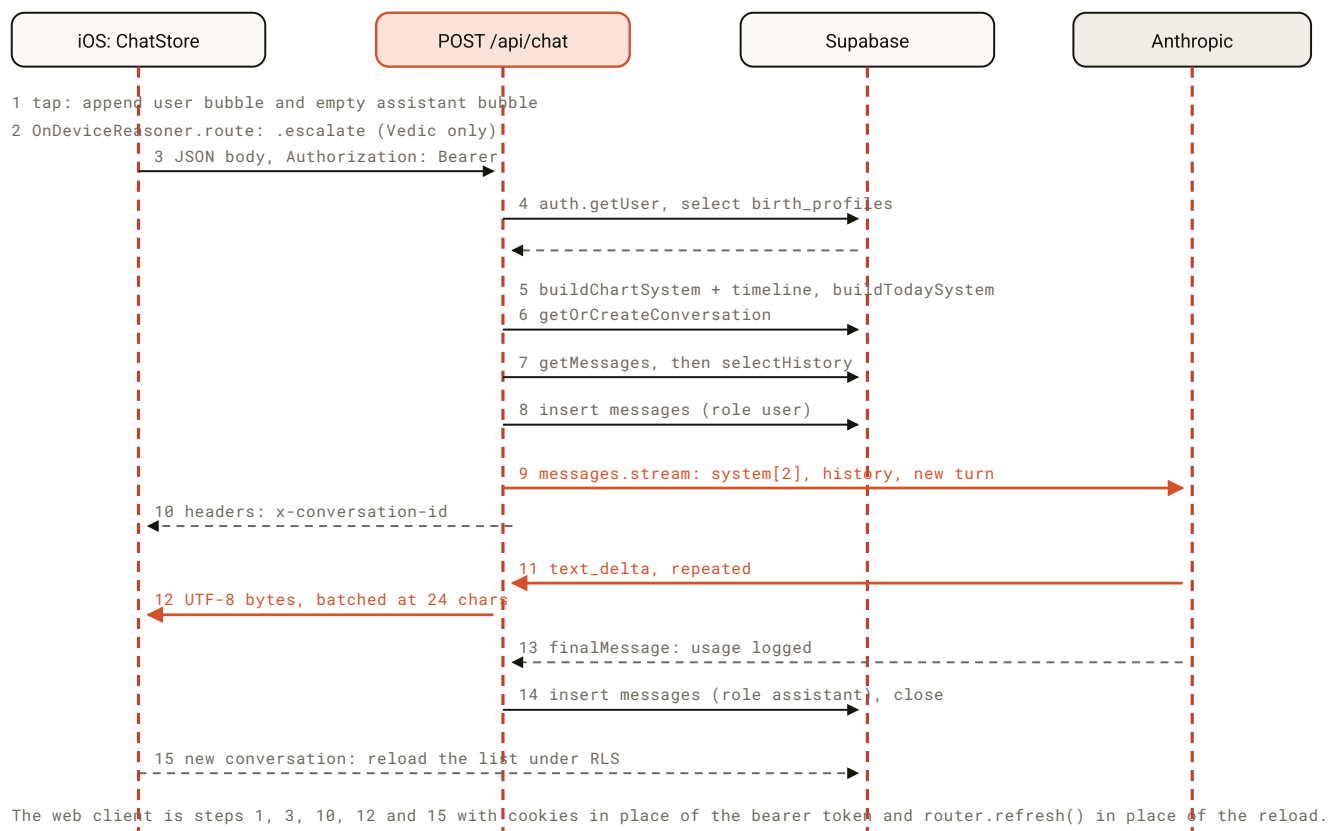
`SancharaAPI.chatStream` returns an `AsyncThrowingStream<ChatEvent, Error>`. The request is built by the shared `request(_:method:)`, which asks `Supa.accessToken()` for the current session token and throws `.unauthorized` without one, so a signed-out client never reaches the network. The body is a `ChatRequest` with the same four fields as the web, and the timeout is 120 seconds because, as the comment says, readings can think for a while before the first byte. It uses `URLSession.shared.bytes(for:)`. A non-2xx status reads the whole body as the error sentence (401 becomes `.unauthorized`); a 2xx yields `.conversationId` from the header, then bytes.

The byte loop handles the two problems a raw text stream has. A chunk boundary can split a multi-byte character, so bytes accumulate in `undecoded` until they decode as UTF-8. And a redraw per byte would repaint the transcript thousands of times for one reading, so decoded text is batched and yielded at 24 characters, with the remainder flushed at the end. `onTermination` cancels the task, which is how `newReading()` stops a reading in flight.

WHAT IS CACHED ON THE DEVICE

For chat, nothing durable. The transcript and the conversation list live in the store's memory and are re-read from Supabase on open. The chart the store hands to the reasoner comes from `ProfileStore`, which is the same chart `ChartCache` keeps in the App Group for Siri and the widget; that cache is described in chapter 10. An on-device answer is never written anywhere.

6.6 How it works



One streamed turn. The user message is stored before the model is called and the assistant message after the stream closes; between them the client sees the id first and then bytes.

A NEW CONVERSATION

The client sends no `conversationId`. The route inserts a conversation titled with the first 80 characters of the message, stores the user turn, and starts a stream whose history is that one turn. The response header carries the new id; the web stores it in the ref and iOS in `conversationId`, and both send it on every later turn. When the stream ends the web calls `router.refresh()` and iOS calls `loadConversations()`, so the new reading appears in the list. On this first turn the whole prefix is a cache write; the route's comment notes that a first turn is short enough that missing the cache costs little.

A LATER TURN

The client sends the id. The route verifies ownership, loads every stored message, trims to the budget, marks the last retained turn as the cache breakpoint, appends and stores the new message, and streams. The mode is whatever the client sends; the row's `tradition` is not consulted and not updated. Both clients keep the mode with the conversation (the web page passes the row's tradition as the initial mode; iOS's `open` sets `mode = conversation.mode`), but nothing on the server prevents a client from continuing a Vedic conversation in numerology, and the stored transcript would then be a mix.

REOPENING A PAST READING

The web reopens through the URL: the sidebar link sets `?c=`, the page loads the messages on the server and renders them as `initialMessages`, and the new `key` remounts the component. iOS reopens through `ChatStore.open`, which reads the `messages` table directly with the Supabase client under RLS, no route involved. Both then continue with the later-turn flow above.

THE INTRO READING

A brand-new account opens on a reading rather than an empty room. When the user has no conversations, the web mounts with `autostart` and iOS's `start()` finds an empty list, and each sends the same string silently: "Welcome me with a short reading for today based on my chart. Two or three short sections." Silent means the prompt itself is not shown as a user bubble, because, as the iOS comment puts it, the question is scaffolding the user never asked. It is stored, though: the route sees an ordinary message and writes it to `messages`, so reopening that conversation later shows the scaffolding line. On iOS a silent turn also bypasses the on-device router.

DEEP

The toggle sends `deep: true`. The route switches to `DEEP_READING_MODEL`, keeps the default thinking effort, raises the length rule to 320 words and the ceiling to 3,000 tokens. Nothing about the conversation records that a turn was deep, and the next turn can be standard.

6.7 Performance and cost

- **Every reading is billed on the server**, and the mechanics are chapter 3: two cache breakpoints, low effort on standard readings, a 6,000 character history budget and a length rule. The measured mid-conversation turn is about \$0.008 on Sonnet 5.
- **Two profile reads per turn, not more.** The route selects `birth_profiles` once and, only when the timeline is loaded, `loadTimeline` reads what it needs. The transit chart is memoised per day and neighbourhood, so the ephemeris runs once per city per day rather than once per message.
- **Whole history loaded, then trimmed in memory.** `getMessages` has no limit clause; every turn loads every message of the conversation and `selectHistory` discards most of it. On a conversation of hundreds of turns this is a larger read than it needs to be, though the index on `(conversation_id, created_at)` makes it cheap. A limit is the obvious improvement and has not been needed.
- **Time to first byte is thinking time.** With adaptive thinking on, the model reasons before the first `text_delta`. Both clients show a shimmer rather than nothing, and iOS sets a 120 second request timeout for it.
- **Redraw cost is bounded on iOS.** The 24 character batch in `chatStream` is the difference between a few hundred transcript updates per reading and several thousand. The web appends per network chunk, which the browser already coalesces.
- **The cheapest turn is one the phone answers.** A Vedic lookup on a device with Apple Intelligence never posts at all. See chapter 10 for what qualifies.
- **The intro reading costs a turn per new account**, and it is a first turn, so it is also the one that writes the cache rather than reading it.

6.8 Edge cases and tests

CONDITION	BEHAVIOUR
No session	401 <code>Unauthorized</code> . iOS never sends: <code>request</code> throws <code>.unauthorized</code> when there is no access token, and the bubble shows "Your session expired. Please sign in again."
Blank message	400 <code>Empty message</code> . Both composers disable send on blank input, so this is a guard against a crafted request, not a UI state
Unknown mode	400 <code>Invalid mode</code>
No chart on the profile	400 <code>No chart. Complete intake first.</code> The web page redirects to intake before the composer renders
Chart written by an older release	<code>ensureCurrentChart</code> recomputes both charts and rewrites the row before the prompt is built
Transit computation fails	Caught; the today block simply has no SKY TODAY section and the reading proceeds
Timeline load fails	Logged as <code>chat timeline context skipped</code> ; the stable half is sent without it
Conversation id belongs to someone else	<code>getOrCreateConversation</code> throws; the route answers 500 with a sentence, and nothing is written. RLS would refuse the insert regardless
Model fails before the first byte	The user turn is stored, the client receives the bracketed apology, and no assistant row is written
Model fails mid-stream	The partial text is stored as the assistant turn; the apology is shown but not stored
Chunk boundary splits a multi-byte character	iOS holds bytes in <code>undecoded</code> until they decode; the web's <code>TextDecoder</code> in streaming mode does the same
User starts a new reading mid-stream (iOS)	<code>newReading</code> cancels the task; <code>onTermination</code> cancels the request; the server still finishes and stores what it produced
Conversation list fails to load (iOS)	The last known list stays; no banner, because the transcript is unaffected
A stored conversation has an unknown tradition	iOS decodes it as Vedic; the web falls back to Vedic when the row is missing from the list
Reduce Motion is on	Glow, shimmer and caret render static
On-device answer was wrong for the question	"Ask for a full reading" removes it and resends with <code>forceServer</code>

TESTS

The chat feature is covered by the tests of the modules it is built from, not by a test of the route or the views. There is no `app/api/chat/route.test.ts` , and no Swift test constructs a `ChatStore` or calls `chatStream` . What does exist:

- `lib/astrology/prompt.test.ts` : everything the route puts in the system blocks, listed in section 3.7. For chat the load-bearing cases are that the stable half carries no date, sky or length rule, and that a Western chart gets Placidus houses and no Vedic technique.

- `lib/data/history.test.ts` : the five budget rules, including that a window never begins on an assistant turn, which is what keeps a conversation whose last model call failed valid on the next turn.
- `lib/data/injection.test.ts` : `appendMessage` writes `{ conversation_id, role, content }` through the injected client and never builds the cookie client when one is given. This is the test that stands between an iOS turn and a silent RLS failure.
- `app/smoke.test.ts` : the app layout module imports and exports a function. It proves the Next.js scaffold loads under Vitest and nothing more.
- Swift: `SuggestionTests` , `OnDeviceRoutingTests` and `ChartFactsTests` exercise the three on-device files in the Chat folder. They belong to chapter 10; none touches the transcript, the store or the stream.

WHAT IS NOT PINNED

The two behaviours in the stream's `finally` , that a failed first turn stores no assistant row and that a partial reading is stored without the apology, are read from the code and not from a test. The same is true of the 24 character batch and the UTF-8 hold-back in `chatStream` . They are the first things to cover if the route or the client ever changes shape.

6.9 Study notes

READ IN THIS ORDER

1. `app/api/chat/route.ts` , once straight through. It is the feature; everything else is a client of it.
2. `lib/data/chat.ts` , for the ownership check and the four queries.
3. `supabase/migrations/0001_init.sql` , the two tables and the two policies, then `0003_numerology_mode.sql` .
4. `components/Chat.tsx` , `send` first, then the composer.
5. `app/(app)/app/chat/page.tsx` , for `key` and `autostart` .
6. `mobile/Sanchara/Sources/Features/Chat/ChatStore.swift` , `send` first, and the comment on `chart` .
7. `mobile/Sanchara/Sources/Services/SancharaAPI.swift` , the `chatStream` section at the end.
8. `mobile/Sanchara/Sources/Features/Chat/ChatView.swift` , the three `.task` modifiers and `onDeviceFooter` .

CHECK YOURSELF

Why is the user message stored before the model is called rather than after, alongside the reply?

So that a failed model call still leaves the question in the transcript. The assistant row is written in `finally` only if text arrived, and `selectHistory` tolerates a history that ends on a user turn.

Why does the response carry the conversation id in a header rather than in the body?

The body is the reading, streamed as bare text with no framing. A header arrives before the first byte and does not need parsing out of the stream; both clients read it first and then loop on bytes.

The iOS app reads `conversations` and `messages` directly. Why is that safe?

Both tables are owner-only under RLS, and the Supabase client carries the user's session, so the read can only ever return that user's rows. Writes still go through the route, which is the only code that builds a prompt.

What happens to the intro reading's prompt text?

It is hidden in the client (silent) but stored on the server as an ordinary user message, so a reopened first conversation shows it.

Why does `Chat` take a `key` from the page?

Switching readings must remount the component so its state (messages, mode, the id in the ref) starts from the new conversation rather than carrying the old one's ref forward.

Why does the route forward only `text_delta` events?

Adaptive thinking produces thinking blocks first. The client should see the reading, not the reasoning, and the stored assistant turn should be what the user saw.

7 Kundli

The kundli is the chart as a thing to look at: a North Indian diagram of twelve fixed houses with the rashis rotating through them, drawn from the stored chart and never interpreted. The same JSON becomes a two-page PDF on the server and a tappable SwiftUI drawing on the phone, and the one idea that shaped both is that the lagna is the only input the layout needs.

This chapter describes the code on `main` on 30 September 2026, version 0.3.0. Nothing here is generated by a model. Every number on the page or the screen comes from the `vedic` half of `birth_profiles.chart`, which chapter 5 showed being computed and stored at intake. The server renders that JSON with pdf-lib into a document meant to be printed and kept. The iOS app decodes the same JSON into `NatalChart`, resolves it into twelve `KundliHouse` values, and draws them with a geometry table that is, polygon for polygon, the one the PDF uses. On the web there is no on-screen diagram: the home and profile pages link to the PDF.

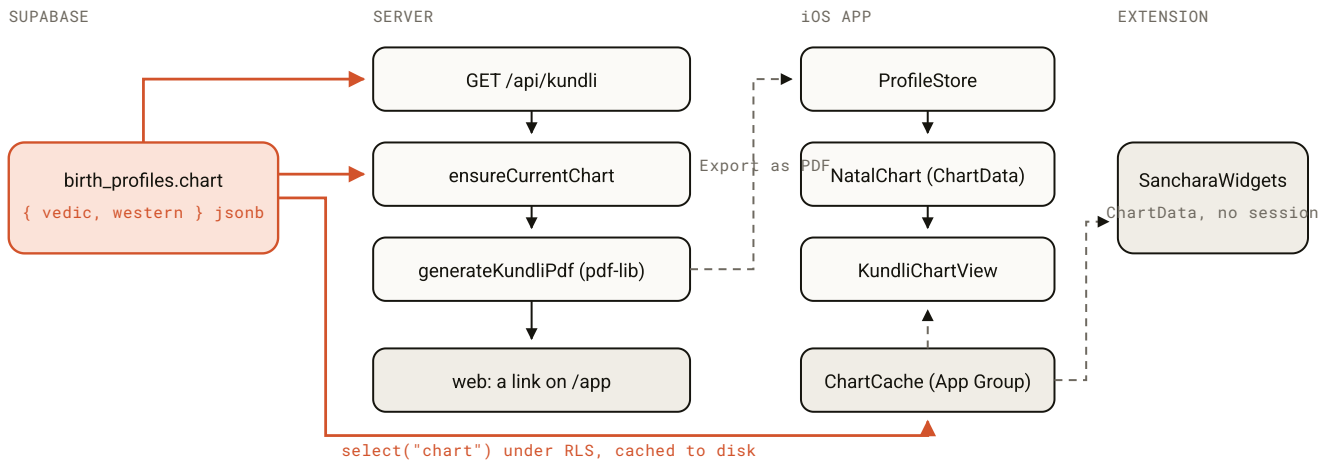
7.1 Intent

The PDF renderer's own comment states the intent for the document: "A birth chart is not a report; in most families it is a piece of paper that gets kept, and this is the version people print, mail to a relative, or hand to an astrologer." So it is set like a document, a serif for the name and the data, a sans for labels, one accent used sparingly, in two pages ordered by the questions people ask. Page one is the diagram and the three values a chart is identified by. Page two is the full table, read both ways: by graha and by house.

The screen's intent is narrower and `KundliView` states it: "help someone find where a planet sits and what a house holds." The diagram shows glyphs and rashi numerals only; degrees, nakshatras and retrogrades live in the table under it and in the sheet that opens when a house is tapped. The table repeats everything the diagram encodes in plain rows that take unbounded Dynamic Type, so nobody has to read the diagram to get at the numbers.

A third intent runs under both: the diagram must be correct in the way that does not look wrong.

`KundliTests` puts it plainly: "a chart with the signs rotated one house off is still a perfectly handsome diagram." The tests therefore pin the rotation rule and the geometry, and leave the pixels to a snapshot that only checks something was drawn.



One JSON, three consumers. The route reads the row itself and upgrades a stale chart before rendering. The app never calls the route for the chart; it reads the column directly and keeps a copy on disk that the widget extension can decode with the same `ChartData` types.

7.2 Files and folders

app/api/kundli/route.ts	GET: the PDF, as an attachment
lib/pdf/kundli.ts	generateKundliPdf, HOUSE_POLY, prettyDate, shortDate
lib/pdf/kundli.test.ts	pages, WinAnsi, missing dasha, UTC dates
lib/astrology/numerology.ts	computeNumerology, computeNameNumber (read by the route)
lib/data/birthProfile.ts	ensureCurrentChart (called by the route)
app/(app)/app/page.tsx	"Download your kundli (PDF)"
app/(app)/app/profile/page.tsx	"Download kundli (PDF)"
mobile/Sanchara/Sources/Features/Kundli/	
├ ChartData.swift	ChartBundle, NatalChart, ChartPlanet, Rashi, KundliHouse, P
├ KundliChartView.swift	KundliGeometry, KundliFrame, the drawing
└ KundliView.swift	the screen, HouseDetailSheet, Export as PDF
mobile/Sanchara/Sources/Intents/OpenKundliIntent.swift	"Open my kundli", app and extension
mobile/Sanchara/Sources/Intents/DeepLink.swift	drains the pending destination
mobile/Sanchara/Sources/Services/ChartCache.swift	the App Group cache
mobile/Sanchara/Sources/Services/SancharaAPI.swift	kundliPDF()
mobile/Sanchara/Tests/KundliTests.swift	rotation, geometry, spoken text
mobile/Sanchara/Tests/KundliSnapshotTests.swift	renders a PNG to look at
mobile/project.yml	which of these the widget target compiles

The route reads the row and delegates everything else. The PDF module and `ChartData.swift` are the two places where the diagram's geometry is written down, once for each coordinate system, and each carries a comment pointing at the other.

7.3 Model

THE INPUT TO THE PDF

```
// lib/pdf/kundli.ts
export type KundliInput = {
  name: string;
  birthDate: string;
  birthTime: string;
  place: string;
  timezone: string;
  chart: Chart; // the vedic chart
  numerology: { mulank: number; bhagyank: number; namank: number };
};
```

`Chart` is the type from `lib/astrology/types.ts`: ascendant sign and degree, the planets with sign, degree, house, retrograde flag and nakshatra, Moon and Sun signs, the optional ayanamsa and the optional `dasha`. The renderer uses the `house` field on each planet as stored; it does not recompute houses from longitudes, which is why the route runs `ensureCurrentChart` first. The three numerology values are computed by the route from the birth date and the full name.

THE SAME JSON IN SWIFT

```
// mobile/Sanchara/Sources/Features/Kundli/ChartData.swift
struct ChartBundle: Decodable, Equatable, Sendable {
  let vedic: NatalChart
  let western: NatalChart?
}

struct NatalChart: Codable, Equatable, Sendable {
  let tradition: String
  let ascendant: Ascendant
  let planets: [ChartPlanet]
  let moonSign: String
  let sunSign: String
  let ayanamsa: Double?
  let dasha: DashaInfo?
  let derived: DerivedFacts?
  ...
}
```

The file's comment says it "mirrors `lib/astrology/types.ts` field for field", with one deliberate omission: the twelve cusp longitudes in the TypeScript `houses` array are not decoded, because the drawing needs house numbers, not cusps, and `houses` on the Swift side is a computed property that resolves them. `derived` is optional at the top level so a chart cached before the derived facts shipped still decodes; the comment's rule is that "a widget or a Siri intent must never fail to decode a chart it has already stored."

`ChartPlanet` adds three computed views of a row. `glyph` maps each graha to an astronomical symbol, with Rahu and Ketu borrowing the ascending and descending node signs, and a long comment on why the

Moon is U+263D rather than U+263E: on iOS the latter resolves to the emoji font and draws a solid crescent heavier than the eight line-art glyphs beside it, and the text-presentation selector does not fix it. `degreeText` prints `14°32'`, the form every printed kundli uses. `spokenDescription` is one sentence with no abbreviations, used for VoiceOver and handed to the on-device model as a fact.

RASHIS AND HOUSES

```
// mobile/Sanchara/Sources/Features/Kundli/ChartData.swift
extension NatalChart {
    var ascendantRashi: Rashi { Rashi(english: ascendant.sign) ?? .mesha }

    var houses: [KundliHouse] {
        let ascIndex = ascendantRashi.rawValue
        return (1...12).map { number in
            let rashi = Rashi(rawValue: (ascIndex + number - 1) % 12) ?? .mesha
            return KundliHouse(
                number: number,
                rashi: rashi,
                planets: planets.filter { $0.house == number }
            )
        }
    }
}
```

This is the whole rule of a North Indian chart, and the comment above it says so: houses are fixed on the page and the rashis rotate, so house 1 always holds the lagna's rashi and each house after it takes the next sign.

`Rashi` is an enum from `mesha` (0) to `meena` (11) carrying both the English name the ephemeris returns and the Sanskrit name a kundli is labelled with, plus `number`, the 1 to 12 numeral printed in each house. The PDF has the same two tables as constants, `SIGNS` and `RASHI`, and the same modular arithmetic in `drawChart` and the bhava table.

`KundliHouse` carries the number, its rashi, the planets filed under it, `isLagna`, a one-line `domain` ("Self, body, how you begin" for house 1, and so on), and an `accessibilityLabel` that ends in "Empty." for an unoccupied house. The twelve domain strings are duplicated in the PDF as `BHAVA_MEANING`, and the PDF's comment gives the reason: "so the paper and the screen say the same thing."

7.4 The server side

GET /API/KUNDLI

The route runs on the Node runtime. It resolves the caller through `createRouteSupabase`, so a bearer token from the phone and a cookie from the browser both evaluate as the user under RLS, and answers 401 with no user. It then selects the profile row with `maybeSingle()`. A read error is logged and answered 500 with "Something went wrong loading your profile. Please try again." A row without a chart, or no row, is a 400: "Complete your birth details first." Then `ensureCurrentChart` runs, so a chart stored before the whole-sign change is rewritten before it is drawn.

```
// app/api/kundli/route.ts
const chart = (p.chart as { vedic: Chart }).vedic;
const fullName = `${p.first_name} ${p.last_name}`.trim();
const num = computeNumerology(String(p.birth_date));

const pdf = await generateKundliPdf({
  name: fullName,
  birthDate: String(p.birth_date),
  birthTime: String(p.birth_time).slice(0, 5),
  place: String(p.place_name),
  timezone: String(p.timezone),
  chart,
  numerology: { mulank: num.mulank, bhagyank: num.bhagyank, namank: computeNameNumber(fullName)
});

return new Response(Buffer.from(pdf), {
  headers: {
    "content-type": "application/pdf",
    "content-disposition": 'attachment; filename="sanchara-kundli.pdf"',
    "cache-control": "no-store",
  },
});
```

Only the Vedic chart is rendered; there is no Western kundli. The response is an attachment named `sanchara-kundli.pdf` with `no-store`, since the document is personal and the chart may have been upgraded moments before. The README describes this folder as "Chart JSON and PDF", but on 30 September 2026 the handler has a single `GET` that returns the PDF. The chart JSON reaches the iOS app by a direct Supabase read, described in 7.5, not through this route.

THE PAGE

`generateKundliPdf` creates a document with four standard fonts, Times Roman and Helvetica in regular and bold, and sets the title to the name followed by "Vedic kundli", the author to "Sanchara" and the subject to "Vedic birth chart". Pages are 595 by 842 points (A4) with a 56-point margin; those three numbers are the constants `W`, `H` and `M`. The palette is written with its contrast ratios: the brand orange (`ACCENT`) is 3.3:1 on white and is used only for rules and fills; text in orange uses `ACCENT_TEXT`, a darkened variant at 5.3:1; `MUTED` grey is 5.4:1. The inner chart lines are a mid grey, `GRID`, because full ink at 1.2 points "drew a heavy black cage around type set at 9pt".

pdf-lib has no character-spacing control, so the small-capital labels are drawn by `tracked()`, one character at a time, advancing by each glyph's width plus a spacing. `trackedWidth()` computes the same sum so a label can be centred or right-aligned. Every page gets a `header` (a 4-point accent band along the top, "SANCHARA" tracked at the left, a right-hand label, a rule) and a `footer` (a rule and one line of 7.5-point text).

PAGE ONE

1. The name at 27 points serif bold, then "Born 12 September 1994 at 04:35" and a provenance line: place, timezone and, when the chart has an ayanamsa, "Lahiri ayanamsa 24.17°".

2. `identityBand` : a 62-point tinted band with a 3-point accent bar at its left edge, split into three columns headed LAGNA, CHANDRA and SURYA. Each shows the Sanskrit rashi at 15 points serif bold with the English name beneath; the lagna's caption also carries its degree.
3. `drawChart` , a 414-point square centred horizontally with its bottom edge 168 points from the foot of the page.
4. Two captions under the square explaining that houses hold fixed positions with the top diamond always first, that the numeral in each house is its rashi from 1 Mesha to 12 Meena, and that an asterisk marks a retrograde graha.
5. The footer: "Computed with the Swiss Ephemeris by Sanchara. For guidance and reflection, not a substitute for professional advice."

DRAWING THE SQUARE

```
// lib/pdf/kundli.ts
const HOUSE_POLY: [number, number][][] = [
  [[0.25, 0.75], [0.5, 1], [0.75, 0.75], [0.5, 0.5]], // 1 top kite
  [[0, 1], [0.5, 1], [0.25, 0.75]], // 2
  [[0, 1], [0.25, 0.75], [0, 0.5]], // 3
  [[0, 0.5], [0.25, 0.75], [0.5, 0.5], [0.25, 0.25]], // 4 left kite
  ...
  [[1, 1], [0.75, 0.75], [0.5, 1]], // 12
];
```

The twelve houses are unit-square polygons with y running up, pdf-lib's convention. The comment says they are "the same twelve polygons the iOS app draws (`KundliGeometry`), expressed for the other coordinate system." `anchor(house)` gives the point a house's contents are centred on: the centroid for the four kites, and for the eight triangles the centroid pulled 14 percent of the way towards the middle of the square, because a triangle's centroid sits two-thirds of the way out to its long outer edge and contents centred there overhang the border. The comment calls this "the same trap the app's chart fell into on its first render."

`drawChart` first washes the lagna house with a pale tint using `drawSvgPath` (flipping y with `H - p.y` because the SVG path convention runs down), so the orientation is legible without a caption. It strokes the border in ink at 1 point, then the two diagonals and the four diamond edges in `GRID` at 0.6. For each house it computes the rashi number as $((\text{ascIndex} + \text{house} - 1) \% 12) + 1$, prints it at 9 points serif 7 points above the anchor (accent text for the lagna, muted otherwise), and then lays the occupants out two to a row, 11 points per row, as 7.5-point bold sans labels such as `Sa* 11° Ju 17°`. Two per row is so "a four-graha house has to fit inside the narrowest triangle without the abbreviations shrinking below legibility." The abbreviations come from `ABBR` (Su, Mo, Me, Ve, Ma, Ju, Sa, Ra, Ke).

PLAIN ASCII ON PURPOSE

The standard PDF fonts are WinAnsi-encoded, and pdf-lib throws on a character outside that set rather than substituting one. The retrograde symbol and a superscript r are both outside it, so the chart marks retrograde with an asterisk and the table with "(R)". A single stray character would not degrade the page; it would fail the whole route with a 500. The test named for this feeds the renderer every retrograde, every house occupied and a name with accented letters.

PAGE TWO

The header's right-hand label is the name in capitals. Four sections follow, each opened by `section()`, a tracked accent-text label with a rule running from it to the margin.

- **GRAHAS AND NAKSHATRAS.** Five columns at fixed offsets from the margin (graha at 0, rashi at 104, degree at 214, house at 274, nakshatra at 324), 19-point rows, alternate rows tinted because "a tint on alternate rows does the work five vertical rules would". A row reads `Sa Saturn (R)`, the Sanskrit rashi, `11°36'` from `degreeText`, the house number, and the nakshatra in muted sans, or a dash when absent.
- **BHAVAS.** The same data the other way round: twelve 17-point rows, one per house, giving the rashi by the rotation rule, the grahas in it joined by commas (or a dash), and the `BHAVA_MEANING` line. House 1 is printed "1 · Lagna" in accent text. The comment says this table answers "what's in my seventh", the question people bring to a printed chart, and fills the lower half of the page that the first draft left empty.
- **VIMSHOTTARI DASHA,** only when the chart has a `dasha`: two rows, Mahadasha and Antardasha, with the lord at 12 points serif bold and the span right-aligned as "Apr 2012" to "Apr 2032" via `shortDate`.
- **NUMEROLOGY.** Three columns, MULANK, BHAGYANK and NAMANK, each a 22-point serif bold number in accent text with a note beside it: "Root, from the day of birth", "Destiny, from the whole date", "Name, from its letters".

The footer repeats the identity line: "Sanchara · name · born date at time, place."

DATES

```
// lib/pdf/kundli.ts
export function prettyDate(iso: string) {
  const [y, m, d] = iso.slice(0, 10).split("-").map(Number);
  if (!y || !m || !d) return iso;
  // Constructed in UTC and read back in UTC: a bare date formatted in a local
  // zone west of Greenwich comes back as the day before.
  return new Date(Date.UTC(y, m - 1, d)).toLocaleDateString("en-GB", {
    day: "numeric", month: "long", year: "numeric", timeZone: "UTC",
  });
}
```

`prettyDate` and `shortDate` are exported "for the test, not for callers": the UTC pinning is the kind of thing that silently regresses, and it cannot be asserted through the finished PDF because pdf-lib compresses its content streams. A malformed date is passed through unchanged rather than printed as "Invalid Date".

7.5 The clients

WEB

The web app has no chart drawing. `app/(app)/app/page.tsx` shows "Download your kundli (PDF)" under the "Start a reading" button once a profile exists, and `app/(app)/app/profile/page.tsx` shows "Download kundli (PDF)" above the edit form. Both are plain anchors to `/api/kundli`; the browser sends the session cookie, the route answers with an attachment, and the browser saves it.

IOS: WHERE THE CHART COMES FROM

`ProfileStore.loadChart()` selects the `chart` column from `birth_profiles` for rows where it is not null, decodes it as `ChartBundle`, keeps the `vedic` half as `NatalChart` and writes it to `ChartCache.shared.save(chart:)`. The cache is a JSON file named `chart.json` in the App Group container, written atomically with complete file protection, and read back on the next launch before any network call. `ChatView` and `ProfileView` both present `KundliView(details:chart:)` as a sheet, guarded by `if let details = profile.details, let chart = profile.chart`, so the sheet cannot open on a device that has never loaded a chart.

IOS: KUNDLIVIEW

The screen is a `NavigationStack` titled "Your kundli" with a Done button, holding five blocks in a scroll view capped at 480 points wide.

- **Summary.** The full name in a serif title, a birth line built from the details ("12 September 1994 at 04:35" using the device's long date style) and the place, then three stat columns, Lagna, Chandra and Surya, each with the Sanskrit rashi and an English caption; the lagna's caption is its degree. Lagna leads "because the whole diagram is oriented from it."
- **Rashi chart.** `KundliChartView` bound to `selectedHouse`, under a caption that teaches the diagram: houses sit in fixed places, the numeral in each is its rashi counted from Mesha, tap a house to see what it holds.
- **Vimshottari dasha**, when present: Mahadasha and Antardasha with "until Apr 2032", formatted by `ChartFacts.calendarDay` in UTC.
- **Grahas.** One row per planet: glyph, name with a retrograde mark, nakshatra, then "Vrishchika 2°42'" and "House 1" on the right. Each row is a button that opens the house the planet sits in. This table is "the accessible path through the chart and the one that survives the largest text sizes."
- **Footer.** "Export as PDF", an error line, and when the chart has an ayanamsa, "Sidereal, Lahiri ayanamsa 24.17°. Computed with the Swiss Ephemeris."

Tapping a house, on the diagram or through a planet row, sets `selectedHouse`, and a `sheet(item:)` presents `HouseDetailSheet` at medium or large detents: the house number (or "House 1 · Lagna"), the Sanskrit and English rashi, the domain line, and either the planets with degree and nakshatra or the sentence "No graha sits here. An empty house is read through its lord and the aspects reaching it", pointing the person back to the reading.

```
// mobile/Sanchara/Sources/Features/Kundli/KundliChartView.swift
enum KundliGeometry {
    static func polygon(house: Int) -> [CGPoint] {
        let tl = CGPoint(x: 0, y: 0), tr = CGPoint(x: 1, y: 0)
        let bl = CGPoint(x: 0, y: 1), br = CGPoint(x: 1, y: 1)
        let t = CGPoint(x: 0.5, y: 0), r = CGPoint(x: 1, y: 0.5)
        let b = CGPoint(x: 0.5, y: 1), l = CGPoint(x: 0, y: 0.5)
        let c = CGPoint(x: 0.5, y: 0.5)
        let tlq = CGPoint(x: 0.25, y: 0.25), trq = CGPoint(x: 0.75, y: 0.25)
        let blq = CGPoint(x: 0.25, y: 0.75), brq = CGPoint(x: 0.75, y: 0.75)

        switch house {
        case 1: return [tlq, t, trq, c]
        case 2: return [tl, t, tlq]
        case 3: return [tl, tlq, l]
        case 4: return [l, tlq, c, blq]
        ...
        default: return [tr, trq, t]
        }
    }
}
```

The geometry is a unit square with y running down, SwiftUI's convention, named at its nine significant points so "the table below reads like the drawing". `labelAnchor` is the PDF's `anchor` with a pull-in of 0.12 instead of 0.14, and its comment tells the story the PDF's comment refers to: the first render clipped the retrograde marks off Mercury and half of Jupiter. `path(house:in:)` scales a polygon into a rect. `KundliFrame` is a `Shape` that strokes every line of the chart as one path: the rect, the two diagonals, and the diamond through the four midpoints.

The view is a `GeometryReader` that takes the smaller of width and height as the side and stacks three layers. First, twelve `HouseRegion` shapes, each filled (accent at 0.16 when selected, 0.05 for the lagna, a 2 percent white wash on even houses so the regions read as distinct without twelve borders), outlined only for the lagna (accent at half strength) and the selected house (full accent, 1.25 points), each a tappable accessibility button carrying the house's label. Second, the frame stroked in `Theme.chartLine` at 1 point, hit-testing off. Third, the contents positioned at each anchor: the rashi numeral in a serif at a caption-relative `ScaledMetric` of 11, and the glyphs two per row at a footnote-relative 15, with a small accent "R" overlaid top-right of a retrograde planet. Both metrics scale with Dynamic Type but are relative to small styles, since "the diagram has fixed geometry, so the scaling is capped."

The comment on `stroke(for:)` records a visual decision worth keeping: the first version gave the lagna a full-strength outline, and "the result was a large glowing diamond that read as 'this house is selected'", which is the meaning selection needs for itself. The lagna is now marked, not spotlighted.

IOS: THE INTENT AND THE WIDGET TARGET

```
// mobile/Sanchara/Sources/Intents/OpenKundliIntent.swift
struct OpenKundliIntent: AppIntent {
    static var title: LocalizedStringResource = "Open my kundli"
    static var openAppWhenRun = true

    func perform() async throws -> some IntentResult {
        ChartCache.shared.setPendingDestination("kundli")
        return .result()
    }
}
```

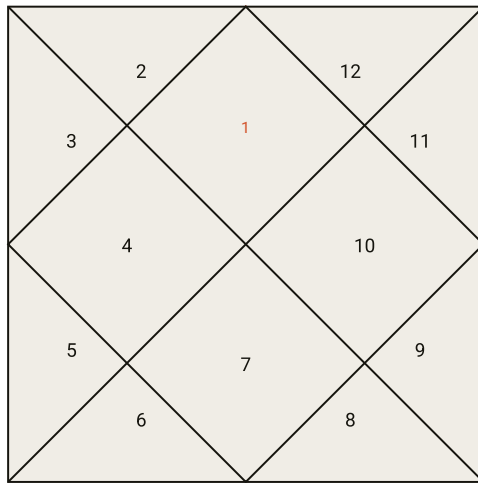
This is the one intent that opens the app, because "a diagram is the answer, and Siri has nowhere to put one." It is kept in a file of its own with no dependency beyond `ChartCache`, because it is compiled into the widget extension as well as the app: a Control Center button runs it from a process with no Supabase session. It writes the destination to the App Group's `UserDefaults` suite rather than memory, since the control that ran it may be in a different process from the app it is about to launch.

```
# mobile/project.yml, target SancharaWidgets
sources:
  - path: SancharaWidgets
  - path: Sanchara/Sources/Design/Theme.swift
  - path: Sanchara/Sources/Design/ReadingCard.swift
  - path: Sanchara/Sources/Services/ChartCache.swift
  - path: Sanchara/Sources/Features/Kundli/ChartData.swift
  - path: Sanchara/Sources/Features/Intake/BirthProfileDetails.swift
  - path: Sanchara/Sources/Intents/OpenKundliIntent.swift
```

The widget target lists six app files by path, and the comment in `project.yml` gives the rule: "Each of these is dependency-free on purpose; pulling in anything that imports Supabase would drag the whole client into a process that has no session to use it with." The widget target declares no package dependency at all. That is why `ChartData.swift` imports only Foundation, why `PeriodProgress` lives in it ("the period widget needs it too, and this file is one of the few compiled into both targets"), and why `KundliChartView`, which imports SwiftUI and `Theme`, is not in the list: the widget decodes the chart and reads its facts but does not draw the diagram. The widgets themselves are in chapter 10; the project layout is in chapter 14.

7.6 How it works

THE DIAGRAM'S GEOMETRY



unit square, side 300 here; (0,0) top left in SwiftUI, bottom left in pdf-lib

HOUSE POLYGONS

Kites 1, 4, 7, 10: a midpoint, the two nearest quarter points, the centre. Area 1/8.

Triangles: a corner, a midpoint, a quarter point. Area 1/16. Twelve tile the square.

LABEL ANCHOR

Kite: centroid. Triangle: centroid moved towards (0.5, 0.5) by 0.12 (app), 0.14 (PDF).

ROTATION RULE

$\text{rashi}(\text{house}) = (\text{asc} + \text{house} - 1) \bmod 12$
asc = index of the lagna sign, Aries = 0

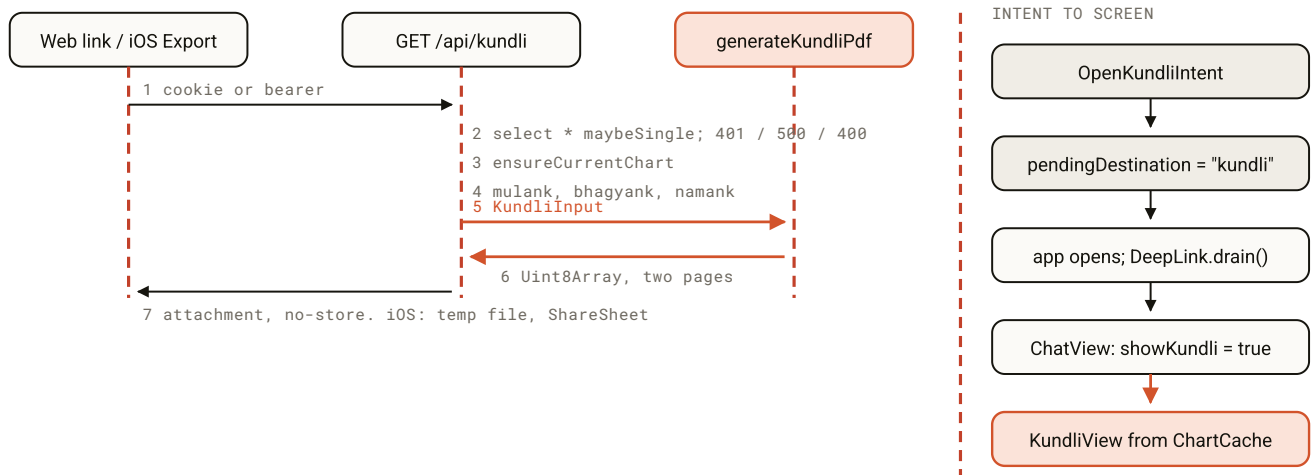
Scorpio lagna (asc 7):

1 Vrishchika, 2 Dhanu, 3 Makara, 4 Kumbha,
5 Meena, 6 Mesha ... 12 Tula

The twelve regions as `KundliGeometry.polygon` and `HOUSE_POLY` define them, with the numerals at their label anchors. House 1 is the top kite and the rest run anticlockwise. The rashi printed in a house is the only thing that changes from one chart to the next.

Read the picture against the tests. "The four kites are an eighth of the square each, the eight triangles a sixteenth. Together that is the whole square and no more" is `testTheTwelveHousesTileTheSquareExactly`, computed by the shoelace formula. `testHouseOneIsAtTheTopAndSevenAtTheBottom` asserts house 1's anchor has y below 0.35 and house 7's above 0.65, with 4 on the left and 10 on the right, which the comment calls "the one orientation rule a North Indian chart cannot get wrong". And `testEveryLabelAnchorSitsInsideItsOwnHouse` runs a winding test on every anchor, since the kites are concave and a centroid test would not do.

DOWNLOADING THE PDF



Left, the PDF: one authenticated GET, the upgrade check, then pdf-lib. Right, the intent: a destination written to the App Group from whichever process ran it, drained by the app when it comes to the front, and answered from the cached chart with no network.

1. On the web the anchor navigates to `/api/kundli` with the session cookie. On iOS, "Export as PDF" calls `SancharaAPI.kundliPDF()`, which attaches the Supabase access token as a bearer header; `isPreparingPDF` relabels the button "Preparing..." and blocks a second tap.
2. The route reads the row, refuses without a user or a chart, and upgrades a stale chart.
3. It computes the three numerology numbers from the birth date and full name.
4. `generateKundliPdf` lays out both pages and returns the bytes.
5. The browser saves `sanchara-kundli.pdf`. iOS writes the data to `<first-last>-kundli.pdf` in the temporary directory and presents a `ShareSheet` with that URL; a thrown error becomes the footer's error line.

OPENING THE KUNDLI FROM SIRI OR A CONTROL

1. `OpenKundliIntent.perform()` writes "kundli" to the `pendingDestination` key in the App Group `UserDefaults` suite and returns; `openAppWhenRun` brings the app forward.
2. `ChatView` calls `DeepLink.shared.drain()` whenever the scene phase becomes active. `drain()` reads and clears the key in one step through `takePendingDestination()`, "so a destination is never handled twice", and sets `pending = .kundli`.
3. `ChatView` observes `DeepLink.shared.pending`, sets `showKundli = true` and clears `pending`. The sheet reads `profile.chart`, which on a warm launch came from `ChartCache` before any network call.

A widget tap reaches the same place by URL: `DeepLink.handle(url:)` maps the host `kundli` of a `sanchara://` URL to the same destination, with no App Group hop, and `RootView` routes such URLs to `DeepLink` rather than to the auth store so they are not mistaken for an expired sign-in link.

7.7 Performance and cost

- **No ephemeris on the hot path.** Rendering reads stored JSON. The only time the route computes anything astronomical is the one-off upgrade of a pre-version-2 chart, which chapter 5 describes.
- **Standard fonts, no embedding.** The four `StandardFonts` ship with every PDF reader, so the document carries no font bytes and the test fixture's two pages are small. The price is WinAnsi, which is why the renderer is ASCII-only.
- **No caching, by design.** `cache-control: no-store`. The document is personal, cheap to regenerate, and may reflect an upgrade that happened during this request.
- **The screen draws from disk.** `ProfileStore.load()` reads `chart.json` from the App Group before its network refresh, so the kundli sheet opens from cache on a warm launch and works offline. The chart is about 4 KB.
- **Houses resolved once per read.** `NatalChart.houses` is a computed property that filters the planet list twelve times; with nine planets that is trivial, and the view calls it inside a `ForEach` rather than storing it.
- **The widget stays small.** Compiling six dependency-free files into the extension keeps the Supabase client, SwiftUI drawing code and the network out of a process that must render in a tight memory budget and cannot sign in.
- **No model call, no cost.** Nothing in this chapter calls Claude. The kundli is the one feature that is free at any volume beyond Vercel function time and a few kilobytes of Postgres reads.

7.8 Edge cases and tests

CONDITION	BEHAVIOUR
No session	401 from the route; on iOS <code>APIError.unauthorized</code> ("Your session expired. Please sign in again.")
Profile read fails	500 "Something went wrong loading your profile. Please try again.", logged
No row, or a row with a null chart	400 "Complete your birth details first."
Chart stored at schema version 1	Rewritten by <code>ensureCurrentChart</code> before rendering; on failure the old chart is rendered
Chart with no dasha	The Vimshottari section is skipped on paper and the dasha block is omitted on screen
Chart with no ayanamsa	The provenance line drops the ayanamsa; the iOS footer line is omitted
Ascendant sign not in <code>SIGNS</code>	PDF: <code>rashiOf</code> returns the raw sign with number 0 and the numerals count from index 0. iOS: <code>ascendantRashi</code> falls back to Mesha
Planet name not in <code>ABBR</code>	The first two letters are printed; on iOS <code>glyph</code> does the same
Four grahas in one triangular house	Two per row, both renderers; the anchor pull-in keeps the stack inside the border
Retrograde planet	Asterisk in the PDF chart, "(R)" in its table, an accent "R" on the glyph on screen, a retrograde symbol in the on-screen table
A character outside WinAnsi in any string	pdf-lib throws and the route answers 500; the renderer avoids emitting any, and the crowded test proves it for names with accents
Server running west of Greenwich	Dates are built and formatted in UTC, so the birthday does not shift by a day
Malformed date string	Passed through unchanged, never "Invalid Date"
iOS offline	The diagram and table draw from the cached chart; only Export as PDF fails, with the error shown in the footer
Export tapped twice	<code>isPreparingPDF</code> guards re-entry
Intent runs before the app has ever loaded a chart	The destination is drained but the sheet's <code>if let</code> guard has nothing to show; the person lands on chat
Cached chart predates <code>derived</code>	Decodes with <code>derived</code> nil; the kundli does not use it
Very large Dynamic Type	Numerals and glyphs scale relative to caption and footnote styles, so the diagram stays legible; the table under it grows without bound
VoiceOver	Each house is a button labelled with its rashi and every planet in a sentence, or "Empty."; the frame and contents are hidden from the tree

TESTS

- `lib/pdf/kundli.test.ts` (Vitest, five tests, real pdf-lib): "produces a two-page document" reloads the bytes and checks the page count and title; "never emits a character the standard fonts cannot encode" renders the fixture and then a crowded variant with every planet retrograde, every house occupied and an

accented name; "survives a chart with no dasha and no ayanamsa"; "renders dates in UTC wherever the server is" sets `process.env.TZ` to America/Los_Angeles, Asia/Kolkata, Pacific/Kiritimati and UTC in turn and asserts the same strings, including the two dates either side of a UTC day boundary; and "passes a malformed date through rather than printing Invalid Date". The file's comment says what is deliberately not tested: "Nothing here checks how it looks; that is what eyes are for."

- `mobile/Sanchara/Tests/KundliTests.swift` (XCTest, twelve tests, no UI). Rotation:
`testHouseOneHoldsTheLagnaRashi`, `testRashisRunForwardFromTheLagnaAndWrapAtPisces` (Scorpio lagna: houses 2, 5, 6 and 12 hold Dhanu, Meena, Mesha and Tula),
`testAnAriesLagnaPutsEachRashiInItsOwnNumberedHouse`,
`testPlanetsAreFiledUnderTheirOwnHouse`. Formatting:
`testDegreesArePrintedInDegreesAndMinutes` (14.5 is `14°30'`, 29.983 is `29°58'`),
`testEveryGrahaHasItsOwnGlyph` (nine distinct single-character glyphs across `ChartFacts.bodyNames`),
`testSpokenDescriptionNamesEverythingVoiceOverNeeds`,
`testAnEmptyHouseSaysSoRatherThanReadingAsBlank`. Geometry:
`testTheTwelveHousesTileTheSquareExactly`, `testKitesAndTrianglesHaveTheAreasTheyShould`,
`testHouseOneIsAtTheTopAndSevenAtTheBottom`, `testEveryLabelAnchorSitsInsideItsOwnHouse`.
- `mobile/Sanchara/Tests/KundliSnapshotTests.swift`: one test, `testRendersTheChart`, renders `KundliChartView` at 340 points through `ImageRenderer` at scale 3, asserts the PNG is over 2,000 bytes, writes it to `kundli-chart.png` in the temporary directory and prints its path as `KUNDLI_SNAPSHOT_PATH`. The comment explains the restraint: "a snapshot test that fails whenever anyone touches a colour is a test that gets deleted."

The PDF's `HOUSE_POLY` and `anchor` have no tests of their own; the Swift geometry tests cover the same polygons in the other coordinate system, and the PDF test covers what could take the route down.

7.9 Study notes

READ IN THIS ORDER

1. `mobile/Sanchara/Sources/Features/Kundli/ChartData.swift` , the rotation rule in `NatalChart.houses` and the glyph comment.
2. `mobile/Sanchara/Sources/Features/Kundli/KundliChartView.swift` , `KundliGeometry` first, then the three layers of the view.
3. `mobile/Sanchara/Tests/KundliTests.swift` , to see which properties of the drawing are pinned and why the pixels are not.
4. `lib/pdf/kundli.ts` , `HOUSE_POLY` , `anchor` and `drawChart` against the Swift versions, then the two pages top to bottom.
5. `lib/pdf/kundli.test.ts` , the WinAnsi test and the timezone test.
6. `app/api/kundli/route.ts` , the three refusals and the upgrade call.
7. `mobile/Sanchara/Sources/Intents/OpenKundliIntent.swift` and `DeepLink.swift` , then the `SancharaWidgets` target in `mobile/project.yml` .

CHECK YOURSELF

Why is the lagna the only input the layout needs?

Because the houses never move. House 1 is always the top kite; the rashi in each house is `(asc + house - 1) mod 12` ; and each planet already carries its house number from the stored chart. Everything else on the diagram is a lookup.

Why does the route call `ensureCurrentChart` when the drawing code never computes a house?

That is exactly why. The renderers trust `planet.house` as stored, so a chart written under the old house rule would print numerals and planets that disagree. The upgrade rewrites the row before either renderer sees it.

Why is the triangle's anchor pulled towards the centre, and why by two different amounts?

A triangle's centroid sits two-thirds of the way out to its long edge, so a numeral over a row of glyphs centred there crosses the border. Each renderer chose its own pull-in for its own type sizes: 0.12 on screen, 0.14 on paper.

Why does the PDF mark retrograde with an asterisk when the app uses a symbol?

The PDF uses the standard WinAnsi fonts and pdf-lib throws on any character outside that encoding. The app renders with system fonts and can use the symbol. Neither is wrong; the encoding decided it.

Why can the widget extension compile `ChartData.swift` but not `KundliChartView.swift` ?

`ChartData` imports only Foundation and is listed by path in the widget target. The view imports SwiftUI and `Theme` and is not listed. The extension decodes the cached chart to answer questions; it does not draw the diagram.

Why does the intent write to UserDefaults instead of presenting the kundli itself?

It may be running in the widget extension, a process that can present nothing. The app drains the key when it comes to the front, and reads-and-clears in one step so the destination is never handled

twice.

8 The life timeline

A person's whole life as Vimshottari divides it, drawn from the stored chart, with the things that actually happened to them pinned into the periods they fell in. The server computes and explains; the phone renders. Nothing reaches the map that the user did not type or confirm.

This chapter describes the code on `main` on 30 September 2026, version 0.3.0. The timeline is the largest feature in the app and the only one that stores what the user says about their own life rather than what the ephemeris computes. That fact shapes nearly every decision below: why the tables are owner-only in both directions, why a scan of the chat history proposes and never writes, why an extracted moment carries the sentence it was read from, and why a date the user gave to the year is never rendered to the day.

8.1 Intent

A dasha is abstract until something real sits inside it. The chart already knows that a person's Saturn period runs from 2019 to 2038; what makes that a life map rather than a table is that they moved house in 2021 and changed jobs in 2024, and both fell in the Mercury sub-period. The timeline answers "why was that year like that" against their year, not a generic one. It does four things:

- Lays out the twelve mahadashas from birth and places every pinned moment in the mahadasha and antardasha it happened in.
- Writes a plain-language theme and meaning for each period, for someone who has never heard the word dasha, and caches them until the moments inside a period change.
- Fills itself in from the user's own chat messages, proposing dated moments with the quote that gave them away, each one confirmed or discarded by hand.
- Tells the chat, so a reading about 2021 knows what 2021 held.

The idea that shaped it is in the comment at the top of `app/api/timeline/scan/route.ts`: "this is inferred content about someone's own life, and silently filing an AI's guess about a death or a divorce onto their timeline would be a bad thing to be wrong about." Every path onto the map goes through a confirmation the user taps.

8.2 Files and folders

```
lib/timeline/
├─ build.ts           buildTimeline, the types, eventsHashFor, NOW_LORD
├─ load.ts            loadBirthRow, loadTimeline, loadScanState (server-only)
├─ explain.ts         the meaning prompt, parseExplanations, explainTimeline
├─ extract.ts         the scan prompt, parseExtractedEvents, selectTranscript
├─ describe.ts        describeTimelineForPrompt, the block the chat sees
└─ *.test.ts          build, describe, explain, extract (47 tests)

app/api/
├─ timeline/route.ts  GET: the whole map plus scan state
```

```

└─ timeline/explain/route.ts POST: write or rewrite every meaning
└─ timeline/scan/route.ts   POST: propose moments from chat history
└─ life-events/route.ts     POST: pin one or a batch; DELETE: unpin one

supabase/migrations/
└─ 0006_life_events.sql     life_events, life_event_scans
└─ 0007_period_readings.sql period_readings

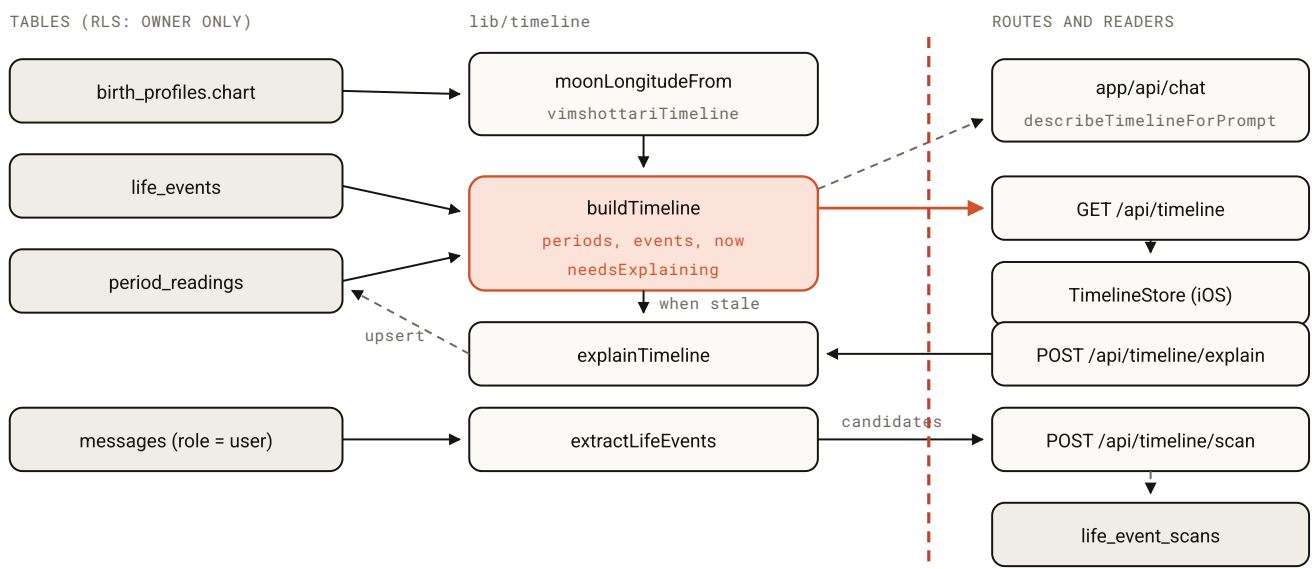
mobile/Sanchara/Sources/Features/Timeline/
└─ TimelineStore.swift      DashaBand, LifeEvent, CandidateEvent, CandidateResolver,
                             TimelinePayload, TimelineDate, TimelineStore, NewLifeEvent
└─ TimelineView.swift       the sheet: NowCard, BandRow, EventRow, the scan offer
└─ AddMomentView.swift      pin one moment by hand
└─ CandidatesView.swift     confirm what the scan found
└─ PeriodExplainerView.swift what a dasha is, in five static paragraphs

mobile/Sanchara/Tests/
└─ TimelineTests.swift
└─ CandidateResolverTests.swift
└─ PeriodProgressTests.swift

```

`build.ts`, `describe.ts` and the pure halves of `explain.ts` and `extract.ts` have no Node-only imports, which is why they are tested without a database or a model. `load.ts`, `explainTimeline` and `extractLifeEvents` are `server-only` and are the only places that touch Supabase or Claude. The dasha arithmetic lives in `lib/astrology/dasha.ts`; this chapter only calls its `vimshottariTimeline`. There is no web page for the timeline as of this date: the four route handlers are the whole web surface, and the iOS app is the only client that draws it.

8.3 Model



The stored chart, the pinned moments and the cached meanings join on the server into one `Timeline` object. The iOS app and the chat route both read it; only the explain route writes meanings back, and the scan route writes nothing but its own record.

THE THREE TABLES

`life_events` holds what the user pinned. The comment in `supabase/migrations/0006_life_events.sql` calls it "the one table in the schema holding what the user tells us about their own life rather than what the ephemeris computes, so it is owner-only in both directions": one policy for all operations, with `using` and `with check` both on `user_id = auth.uid()`, and no admin read policy.

```
-- supabase/migrations/0006_life_events.sql
create table public.life_events (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  occurred_on date not null,
  precision text not null default 'day' check (precision in ('day', 'month', 'year')),
  title text not null check (char_length(title) between 1 and 120),
  note text check (char_length(note) <= 2000),
  source text not null default 'manual' check (source in ('manual', 'extracted')),
  created_at timestamptz not null default now()
);
create index on public.life_events (user_id, occurred_on);
```

`occurred_on` is a date, not a timestamp, because "people remember 'March 2021', not an hour, and a false hour would imply the chart says more than it can." `precision` records how much of the date the user knew; a month-precision event is stored on the 1st, a year-precision one on 1 January, and the precision travels with it. `source` separates a typed moment from a confirmed extraction, "so a bad extraction run can be undone without touching hand-entered events."

`life_event_scans` is one row per user (`user_id` as primary key, `scanned_at`, `found_count`), so the offer to mine the chat history appears once, and again only when the user has said enough new things since.

`period_readings`, from `0007_period_readings.sql`, holds the written meanings: one row per mahadasha plus one whose `lord` is the literal `now`, keyed on (`user_id`, `lord`, `period_start`). The `now` row's `period_start` is the current antardasha's start, "so it goes stale by itself when the sub-period changes." `events_hash` records which pinned moments the text was written against. The column checks allow 80 and 800 characters; the parser trims to 60 and 600, so the database limit is headroom, never the thing that fails.

THE TYPES

```
// lib/timeline/build.ts
export type Timeline = {
  birthDate: string;
  today: string;
  periods: TimelinePeriod[];
  events: TimelineEvent[];
  now: TimelineNow | null;
  /** Any period, or the now summary, needs (re)writing. */
  needsExplaining: boolean;
};
```


`TimelinePeriod` is a `DashaPeriod` (lord, ISO start and end, nested antardashas) plus `isCurrent`, `isPast`, `eventCount`, `theme`, `meaning` and `stale`. `TimelineEvent` carries the stored columns plus the two fields that are "the whole point of the feature": `mahadasha` and `antardasha`, the lords running when it happened, or null when the event sits outside the computed span. `TimelineNow` names the current pair and the antardasha's span. The GET route spreads a `ScanState` (`scanned`, `messagesSinceScan`) over the top.

On iOS the same shape is `TimelinePayload`, decoding into `DashaBand`, `LifeEvent` and `NowPeriod`. Every field that arrived after the first release decodes with `decodeIfPresent` and a default, "so an app update and an API deploy need not land in the same minute." `CandidateEvent` is the scan's proposal: an optional `occurredOn`, a precision that may be `unknown`, a title, and the `evidence` quote.

8.4 The server side

BUILDING THE MAP WITHOUT THE EPHEMERIS

Vimshottari needs one number: the Moon's sidereal longitude at birth. The chart stores each planet as a sign plus a degree within it, so `moonLongitudeFrom` reassembles `signIndex * 30 + degree`: "This is what lets the timeline load without re-running the Swiss Ephemeris, which costs a WASM init per process." It returns null for a chart with no Moon or an unrecognised sign, and the caller treats null as "cannot draw."

`loadTimeline` in `load.ts` reads the birth row, reassembles the Moon, fetches `life_events` and `period_readings` in parallel, and hands everything to `buildTimeline`. `today` is the UTC date: "A dasha band is years wide, so a few hours of skew cannot move an event into the wrong period, unlike the daily readings, where the local date is the whole point."

`buildTimeline` calls `vimshottariTimeline` for the raw twelve periods. Each stored event is placed in the period whose `[start, end)` contains `occurred_on`, then in the antardasha inside it, and the list is sorted oldest first. Each period gets its event count, its current and past flags, and the stored reading for its `(lord, start)`; the now summary is looked up under `NOW_LORD = "now"` and the antardasha start. Then:

```
// lib/timeline/build.ts
const fresh = !!row && row.events_hash === eventsHashFor(p, events);
...
const needsExplaining = periods.some((p) => p.stale) || (now !== null && now.meaning === null);
```

A period is stale when it has no reading or when the moments inside it changed since the reading was written. The test is the hash:

```
// lib/timeline/build.ts
export function eventsHashFor(period, events): string {
  const keys = events
    .filter((e) => within(e.occurredOn, period.start, period.end))
    .map((e) => `${e.occurredOn}:${e.title.toLowerCase()}`)
    .sort();
  if (keys.length === 0) return "0";
  let h = 2166136261;
  const text = keys.join("\n");
  for (let i = 0; i < text.length; i++) {
    h ^= text.charCodeAt(i);
    h = Math.imul(h, 16777619) >>> 0;
  }
  return h.toString(16).padStart(8, "0");
}
```

A 32-bit FNV-1a over the sorted `date:title` keys. Sorting first means the order rows come back from Postgres "can never make a fresh explanation look stale." A small non-cryptographic hash is enough because "this detects change, it does not protect anything," and keeping it pure "keeps this module free of Node-only imports."

WRITING THE MEANINGS

`explain.ts` turns the map into one model call. `buildExplainSystem` addresses the user by first name, tells the model to "write for someone who has never heard the word dasha and will not be told it," and sets the rules that matter on a life map: past chapters in the past tense, the current one in the present, a future one "as what it tends to bring, never as fate, and never as a warning." When the user has pinned a moment inside a chapter, "name it and say in one clause how it fits the chapter. Never invent a moment they did not give you." Jargon is banned outright: no malefic, benefic, exalted, lord, house, transit, dasha, antardasha.

`buildExplainTask` lists every period as `LORD | START | years (state)`, the current period's sub-periods, and each pinned moment under its period, ending with a `now` line. The reply is one line per chapter, `LORD | START | THEME | MEANING`, plus a final line whose LORD is the word `now`.

`parseExplanations` is the safety property. It builds a map of every legitimate `lord|start` key from the computed periods and the now row, and keeps a line only when its key is in that map: "a hallucinated period can never appear on the map." The first line for a key wins, the theme is cut at `MAX_THEME = 60`, and everything after the third pipe is the meaning, "pipes and all," cut at `MAX_MEANING = 600`.

`explainTimeline` calls `READING_MODEL` with `max_tokens: 3000`, the shared `LOW_EFFORT` output config, and the system block marked `cache_control: { type: "ephemeral", ttl: "1h" }`; any error returns an empty array. `POST /api/timeline/explain (maxDuration = 90)` asks the model only if `needsExplaining`, then upserts every returned row on `user_id, lord, period_start`, stamping each period's `events_hash` and `"0"` for the now row, whose freshness "is the antardasha start in its key." One stale period rewrites all of them: "the extra output costs less than a second call would." The route reloads and returns the GET shape, so the client replaces its state and never merges.

MINING THE HISTORY

`selectTranscript` decides what the model sees: the user's messages oldest first, each prefixed with the date it was written, whitespace collapsed so one message stays one line, anything under 12 characters dropped, and a hard stop at `maxChars = 24_000` that ends before a message rather than through it. Oldest first is deliberate: the early "here's my situation" messages are "where most life history gets told."

`buildExtractionSystem` says what counts (a job, a move, a marriage or separation, a birth, a death, an illness, a business begun or closed) and what does not: never an event inferred from a question, a hypothetical, or "something you read in an astrological reading." Relative phrases resolve from the message date ("last March" in a 2025-06 message is 2024-03), ages from the birth date ("when I was 25" is the birth year plus 25), always at the precision given. An event that cannot be dated even to a year is kept with `?`. The title is in the person's own framing, "Left the job in Chicago", not "Career upheaval under Saturn", and the model must quote the words that told it as EVIDENCE. The reply is `DATE | PRECISION | TITLE | EVIDENCE`, one per line, or NONE.

`parseExtractedEvents` trusts none of that. Pipe-delimited lines rather than JSON, "for the reason recorded in `lib/alerts/compose.ts`: the model reliably breaks JSON string escaping and the whole reply is lost, where a malformed line here costs one event." The date is resolved by shape (`YYYY-MM-DD`, `YYYY-MM`, `YYYY`) and checked to be a real calendar day; the PRECISION field is read and discarded, because "a model that writes '2019 | day | ...' does not know the day, whatever it claims in the second field." Anything before birth or after today is dropped, duplicates are removed on lower-cased `date:title` (title alone for undated events), titles are cut at 120 characters and evidence at 160. Dated candidates return oldest first, then the undated ones.

`POST /api/timeline/scan (maxDuration = 120)` reads only the user's own messages, at most `MAX_MESSAGES = 400`: "Assistant turns are full of astrological narrative that reads like biography and would extract as fiction." Section 8.6 walks through the rest.

PINNING

`POST /api/life-events` takes `{ events: [...] }`, at most `MAX_BATCH = 50`, "one confirmation sheet's worth"; a manual add is the same call with one element. The date must match `YYYY-MM-DD`, must not be before the birth date ("That date is before you were born.") or after today ("The timeline only holds what has happened."), and the title must be non-empty, cut at 120; the note is cut at 2000. Precision falls back to `day` and source to `manual`. Dates are checked here as well as in the extractor "because this route is reachable directly, and an event before someone was born would render as a period that does not exist." `DELETE` filters on both the id and the caller's `user_id`, belt and braces over RLS.

THE BLOCK THE CHAT SEES

`describeTimelineForPrompt` renders the timeline as text, or returns `""` so the caller can append it unconditionally. In Vedic mode it opens with a `Now:` line naming the current period and sub-period, then lists up to `MAX_MOMENTS = 30` moments, most recent first, each with the pair it fell in. Western readings have no dashas, so they get the moments alone, and nothing at all when there are none. `app/api/chat/route.ts` appends it to `stableSystem` in the chart branch (Vedic and Western) inside a try/catch that logs "chat timeline context skipped" and carries on. It lives in the cached block because "it only changes when they pin or remove a moment."

8.5 The clients

The web app has no timeline screen. Everything below is the iOS feature, reached from the chat toolbar as a sheet (`ChatView` presents `TimelineView()`) and from a tap on the period widget, which opens `sanchara://timeline` and lands as `DeepLink.Destination.timeline` .

THE STORE

`TimelineStore` is an `@Observable` , main-actor class with four states: `loading` , `ready` , `needsProfile` , `failed(String)` . `load()` calls `SancharaAPI.timeline()` , applies the payload, and then, if `needsExplaining` , spawns `explain()` as a separate task, so the bands are on screen before the meanings exist. A failure there is silent, because "the timeline without meanings is still the timeline." A server reply containing "birth details" maps to `needsProfile` .

```
// mobile/Sanchara/Sources/Features/Timeline/TimelineStore.swift
static let rescanThreshold = 10
var canOfferScan: Bool { !scanned || messagesSinceScan >= Self.rescanThreshold }
var isRescan: Bool { scanned }
```

Never scanned: offer. Scanned, and at least ten user messages since: offer again with different copy. `scan()` sets `scanned = true` and `messagesSinceScan = 0` locally on success, so the offer disappears even when nothing was found. `confirm` resolves the chosen candidates through `CandidateResolver` , posts them as one batch, clears the list and reloads. `add` is the same with one event. `remove` is optimistic: the row disappears under the finger, the delete is sent, and the reload restores it if the delete failed.

WHAT IS CACHED

Every applied payload also writes `ChartCache.CachedPeriod` : the current lord, the running antardasha and its span, the period's `theme` and the now row's `meaning` , then calls `WidgetRefresh.reloadPeriod()` . `TimelineStore.warmCache()` does the same fetch without a screen, and `ChatView` runs it once per launch so "the widget fills in without the timeline ever being opened." The widget, the shared `PeriodProgress` helper and the App Group file are chapter 10's; the timeline is the one writer of that file's `period` entry.

THE VIEWS

`TimelineView` is titled "Your life." Bands are laid out "in information proportion, not time proportion: a period is as tall as what happened in it. A 20-year mahadasha with nothing pinned should not dwarf the year the user got married." Time is carried by the year range and the "N years" caption. From the top: the scan offer when `canOfferScan` ; the `NowCard` when there is a `now` ; one `BandRow` per period; a footer naming the technique and offering "What are these periods?"

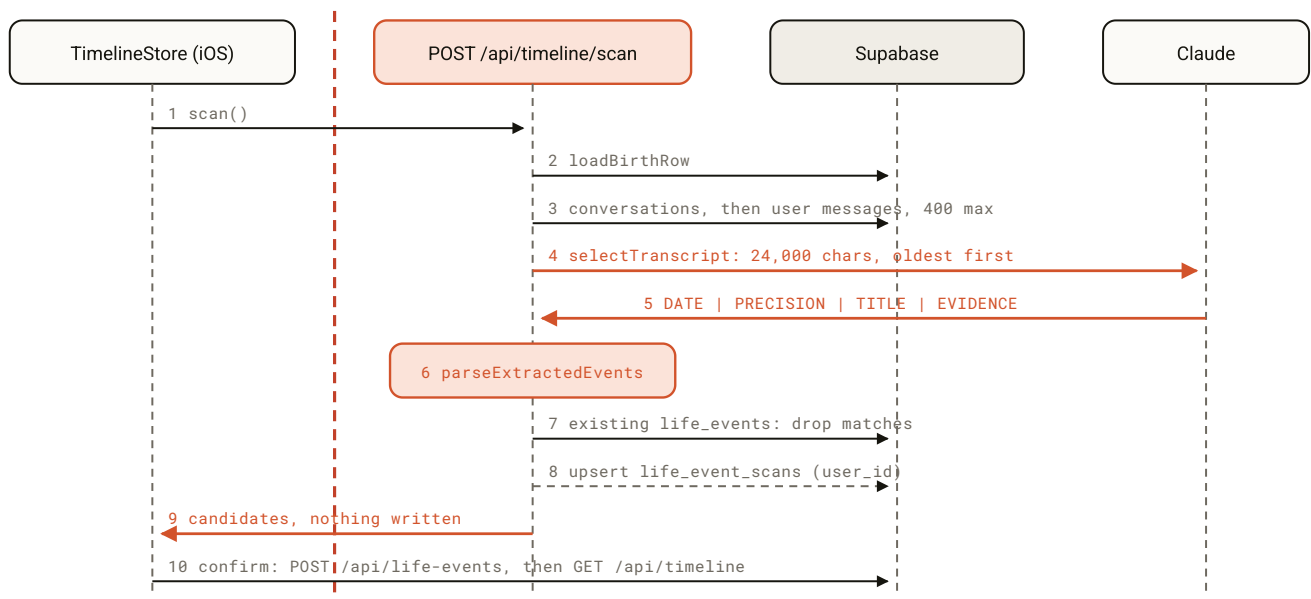
`NowCard` is "the one place the meaning of now is spelled out": the pair, the years, a progress capsule, and the now meaning, or "Writing what this means for you..." while `isExplaining` . `BandRow` shows the lord, a NOW pill on the current period, the years, the theme, and the moment count; expanding reveals the meaning and the sub-periods. A `stale` meaning is shown at 60 percent opacity, "dimmed, and about to be replaced." Future periods sit at 45 percent because they "recede rather than disappear." Each `EventRow` shows the title, the date at the precision given, the "during" label naming the pair, and a Remove item behind a confirmation dialog.

`AddMomentView` asks for a title, a date bounded from birth to today, a segmented precision ("That day", "That month", "That year") and an optional note, and on save snaps the date to the precision so the server "stores a real date either way; this keeps it from implying a day the user never gave." `PeriodExplainerView` is five static paragraphs, "static on purpose," so the page explaining the idea can "be trusted the way a caption is trusted."

`CandidatesView` is where proposals become moments, and its doc comment is the policy: "Nothing reaches the timeline without passing through here. These are inferred claims about someone's own life, a wrong one is not a wrong prediction, it is the app telling a person something untrue about their father's death, so every candidate is opt-in rather than opt-out." Each row shows the title, its date or "When was this?", and the evidence quote in italics. Dated candidates arrive pre-ticked ("the work of confirming a good list should be one tap, not twelve"); undated ones show a year menu from the birth year to this year instead. The add button is live only when `CandidateResolver.canAdd` says so: something is chosen and every chosen undated candidate has a year.

`TimelineDate` pins every formatter to UTC, because "parsing in the device's zone would slide a morning event onto the previous day west of Greenwich," and `LifeEvent.dateLabel` picks a day, month or year formatter by precision "so a year-precision memory never claims a day it does not have."

8.6 How it works



A scan reads, proposes and records that it ran. The only write to `life_events` is the user's own confirmation, one batch, through a different route.

OPENING THE TIMELINE

1. `TimelineView.task` calls `store.load()`, which calls `GET /api/timeline`. The route loads the timeline and the scan state in parallel; a missing chart or an unreadable Moon answers 400 with "Complete your birth details first."

2. The store applies the payload, writes the period cache, and shows the bands. If `needsExplaining` it starts `explain()` in the background.
3. On appear, if there is no now card the scroll jumps to the current band: "where am I now' is the question people arrive with."

WRITING MEANINGS

1. `explain()` calls `POST /api/timeline/explain`. The route checks `needsExplaining` itself, so a needless call costs two reads and no model call.
2. `explainTimeline` runs; `parseExplanations` keeps only lines naming a real period or the now row; the rows are upserted with their hashes.
3. The route returns the GET shape, the store applies it, and the card that said "Writing what this means for you..." shows the now meaning.

Because the now row is keyed on the antardasha start, the day a sub-period turns over the lookup finds no row, `now.meaning` is null, `needsExplaining` is true, and the next open rewrites the map with a fresh now line. No cron and no expiry column; the key is the expiry.

PINNING A MOMENT BY HAND

1. The plus button opens `AddMomentView`, bounded to the birth date and today the store already holds.
2. Save snaps the date to the chosen precision and posts one `NewLifeEvent` with `source: "manual"`; the route validates against the birth row again and inserts.
3. The store reloads. The new moment sits in its band; that band's hash no longer matches its reading, so it is `stale`, `needsExplaining` is true, and the reload's `explain()` rewrites every meaning, this time naming the moment.

THE SCAN

Following the numbered arrows in the figure:

1. The offer's button calls `store.scan()`, with a 120-second timeout on the phone and `maxDuration = 120` on the server.
2. The route loads the birth row; without one it answers 400.
3. It selects the user's conversation ids, then their own messages, oldest first, at most 400. No conversations short-circuits: the scan is recorded with `found_count = 0` and an empty list returns.
4. `selectTranscript` trims to 24,000 characters; `extractLifeEvents` sends them at `max_tokens: 2000`. An empty transcript returns before any call.
5. The model replies with pipe-delimited lines or NONE.
6. `parseExtractedEvents` validates, deduplicates, trims and orders.
7. The route drops any candidate whose lower-cased `occurred_on:title` is already pinned. An undated candidate "has no date to match on, so a pinned moment with the same name is taken to be the same moment."
8. `recordScan` upserts `life_event_scans` on `user_id`; an error here is logged, not raised.

9. The candidates come back with nothing written to `life_events` ; the store's non-empty list presents `CandidatesView` .
10. The user unticks what is wrong, picks a year for anything undated, and taps add. `confirm` posts the batch with `source: "extracted"` to `/api/life-events` and reloads.

WHAT MAKES THE SCAN IDEMPOTENT

Pressing the button twice, or a week later, cannot damage the map, for four reasons that stack. The scan route never inserts into `life_events` ; its only write is the one-row-per-user upsert into `life_event_scans` , which overwrites rather than accumulates. The parser deduplicates within a reply. The route filters against what is already pinned, so "a re-scan after more chatting proposes only what is genuinely new." And the store clears its candidate list after a confirmation or a cancel, so a stale sheet cannot be re-submitted. The one thing not guarded is a user who confirms the same candidate in two separate sheets: `life_events` has no unique constraint, and the next scan would suppress the candidate, not the duplicate row. That is the user's own action on their own data, and Remove is one long press away.

THE CHAT SEES THE TIMELINE

On every chat turn in a chart mode the route calls `loadTimeline` and appends `describeTimelineForPrompt` to `stableSystem` . That block is part of the cached prefix, so "why was 2021 so hard" is answered against a prompt that lists what the user pinned in 2021 and which pair it fell in. When a moment is pinned or removed the block changes and the cache misses once; until then it is free. A failure to load is logged and the turn proceeds without it.

8.7 Performance and cost

- **No ephemeris on the read path.** `moonLongitudeFrom` reassembles the Moon from the stored chart, so `GET /api/timeline` is two parallel Postgres reads plus arithmetic. The WASM engine is never initialised for the timeline.
- **One model call per rewrite, not per period.** Twelve periods and the now line come from a single `max_tokens: 3000` call at low effort, with the system prompt cached for an hour. Rewrites happen only when the hash or the now key changes; opening the timeline ten times a day with nothing new makes zero model calls.
- **The scan is bounded three ways.** 400 messages from the database, 24,000 characters to the model, 2,000 output tokens back, and only user turns, which are the shorter half of any transcript.
- **The chat pays two reads per turn.** `loadTimeline` runs on every chart turn to build the block, which costs input tokens only when it changes.
- **The phone caches one thing.** The period entry for the widget. The map itself is a few kilobytes, fetched on every open, always current.
- **Long timeouts were earned.** Explain and scan carry a 120-second client timeout and 90- and 120-second `maxDuration` on the server; both run longer than a chat turn.

8.8 Edge cases and tests

CONDITION	BEHAVIOUR
No chart yet, or a chart with no Moon or an unknown sign	<code>loadTimeline</code> returns null; every route answers 400; the store shows <code>needsProfile</code>
Event dated beyond the computed span	<code>mahadasha</code> and <code>antardasha</code> are null; no "during" label
Stored precision or source with an unexpected value	Precision falls back to <code>day</code> , source to <code>manual</code>
Events returned in a different order	Sorted before rendering and hashing; a fresh reading never looks stale
Model names a period that does not exist, repeats one, or puts a pipe in a meaning	Line dropped; first line wins; the pipe is kept
Theme over 60 or meaning over 600 characters	Trimmed before the column checks at 80 and 800
Model call fails during explain	No upsert, existing rows untouched, <code>needsExplaining</code> stays true; the store tries again on the next open
Antardasha turns over	The now row's key no longer matches; the next open rewrites all meanings
Extracted date before birth or after today, or impossible (2019-02-31)	Dropped by the parser; the same bounds in <code>/api/life-events</code> answer 400
Model claims day precision for a year-only date	The date's shape wins: year precision
Event the model cannot date	Kept with a null date, listed after the dated ones; the sheet asks for a year and refuses to add without one
Older three-field reply	Title runs to the end, evidence is empty
Message under 12 characters, or the budget reached mid-message	Left out whole; nothing is cut in half
User has no conversations	Scan recorded with <code>found_count = 0</code> , empty candidates, no model call
Delete fails on the server	Row was removed optimistically; the reload restores it
App talks to a server without the explain fields	Decoders default; no theme, no now card
Device west of Greenwich, or clock earlier than the birth date	All dates are UTC; <code>yearRange</code> clamps to a single year rather than an invalid range
More than 50 events in one batch	400 "At most 50 events at a time."

TESTS

Vitest, in `lib/timeline/`, 47 tests, none needing a database or a model:

- `build.test.ts`: twelve periods for a full life; exactly one current period; an event placed in its mahadasha and antardasha and counted into its period; sorted oldest first regardless of insertion order; null periods beyond the span; the precision and source fallbacks; `moonLongitudeFrom` reassembling Taurus 15.5 as 45.5 and returning null for a missing Moon or "Ophiuchus".

- `describe.test.ts` : the Vedic block names the current period and each moment's pair; the Western block has moments but no dasha language, and is empty with none; exactly thirty most recent moments; dates at the precision given.
- `explain.test.ts` : the hash is stable in any order, is `0` for an empty period, and ignores moments outside; the parser reads a period line and the now line, drops lines that do not name a real period, keeps pipes in the meaning, trims to 60 and 600, and skips a line missing either; the prompts list every lord and the pinned moment.
- `extract.test.ts` : NONE; the date shape trusted over a wrong PRECISION field; events outside the person's life and impossible dates dropped; malformed lines skipped without losing good ones; a fourth field read as evidence; an undated event kept after the dated ones; `selectTranscript` stamping the date, dropping short messages and stopping at the budget; the prompt containing "birth year plus 25" and "write ? for the date".

XCTest, in `mobile/Sanchara/Tests/` :

- `TimelineTests` : a band's length in whole years; progress clamped outside the period and zero for a zero-length band; the "during" label for a pair, a mahadasha alone, and neither; a date rendered only to the precision given ("a year label must not leak a day or month number"); malformed dates parsing to nil rather than today.
- `CandidateResolverTests` : a dated candidate passes through unchanged; an undated one needs a year and becomes 1 January at year precision; add is blocked until every chosen undated candidate has a year; the year range clamps a clock set before the birth date; the payload decodes with and without the explain fields.
- `PeriodProgressTests` : the shared fraction at start, midpoint and end, clamped; years remaining never negative; `DashaBand.progress` agreeing with the helper; the cached period surviving a round trip through `ChartCache` and removed by `clear()` .

8.9 Study notes

READ IN THIS ORDER

1. `lib/astrology/dasha.ts` , for `vimshottariTimeline` and why the first period is a balance.
2. `lib/timeline/build.ts` , the join, the now key and `eventsHashFor` .
3. `lib/timeline/load.ts` , how the Moon is recovered and why today is UTC.
4. `lib/timeline/explain.ts` and `app/api/timeline/explain/route.ts` , the prompt rules, the parser and the upsert.
5. `lib/timeline/extract.ts` and `app/api/timeline/scan/route.ts` , the transcript, the parser that trusts nothing, and the duplicate filter.
6. `app/api/life-events/route.ts` , the second validation of every date.
7. `lib/timeline/describe.ts` and the block in `app/api/chat/route.ts` .
8. `mobile/Sanchara/Sources/Features/Timeline/TimelineStore.swift` , the tolerant decoders, `CandidateResolver` and the store.
9. `mobile/Sanchara/Sources/Features/Timeline/CandidatesView.swift` , the confirmation policy.

CHECK YOURSELF

Why can the timeline load without the Swiss Ephemeris when every other chart feature needs it?

Vimshottari needs only the Moon's sidereal longitude at birth, and the stored chart carries the Moon's sign and degree. `moonLongitudeFrom` reassembles `signIndex * 30 + degree` , which saves a WASM init per process.

How does the "where you are now" meaning expire when the sub-period changes, with no expiry column?

Its row is keyed on the antardasha start. When the sub-period turns over the lookup uses a new start, finds no row, and `needsExplaining` becomes true.

Why are the events sorted before hashing?

Postgres does not promise an order, and a different order would give a different hash for the same moments, making a fresh reading look stale and costing a needless model call.

Why does the scan route never write to `life_events` ?

The candidates are an inference about someone's own life, and a wrong one about a death or a divorce would be filed as fact. Every candidate passes through `CandidatesView` , and only the user's confirmation calls `/api/life-events` .

Why is the PRECISION field read and then discarded?

The date's own shape is the evidence. A model that writes a year and claims day precision does not know the day; trusting the shape keeps the map from drawing a marker it cannot justify.

9 Daily readings and dosha alerts

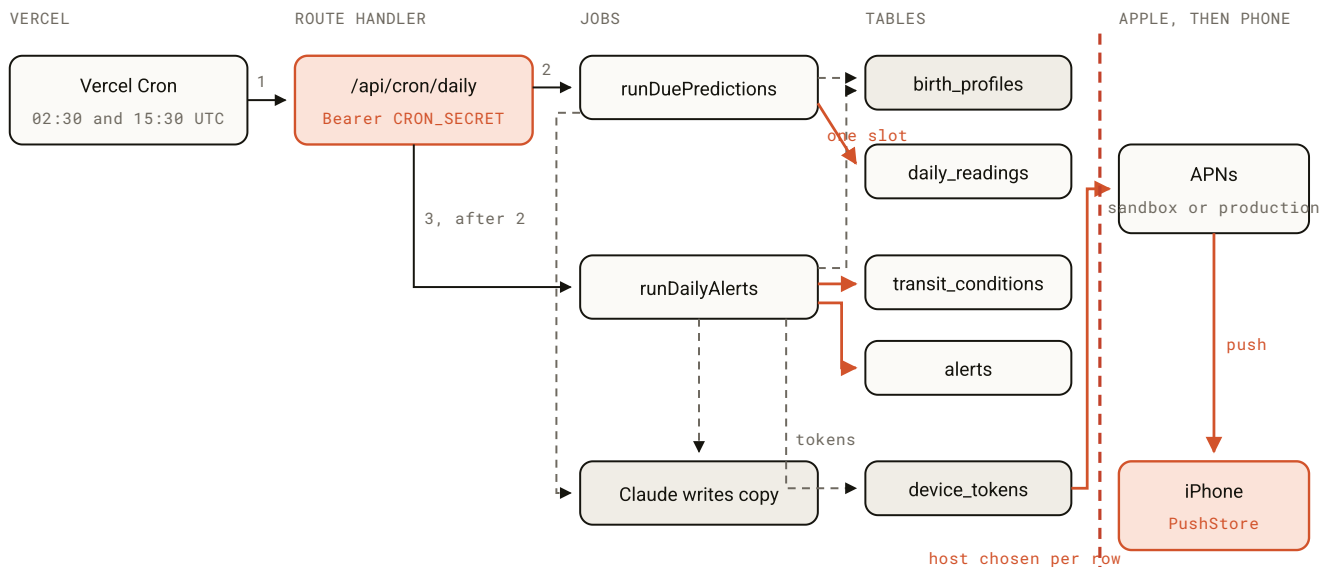
Twice a day a scheduled job writes each person a reading for the half of the day they are in, and sends a push only when something in their sky actually changed. The clock is the user's, not the server's; the astrology is decided in code, not by the model; and a condition that merely continues is never announced twice.

This chapter describes the code on `main` on 30 September 2026, version 0.3.0. It is the one feature in Sanchara that runs with nobody looking. A chat route that misbehaves is seen and retried by the person in front of it; a cron job that misbehaves wakes people at night, bills a model call nobody asked for, or says "Mangal Dosha has begun" to someone whose chart has not changed since the day they were born. Most of the design below exists to make each of those impossible rather than unlikely.

9.1 Intent

Two things are pushed to a person, both written from their own chart against today's sky. A reading, morning and night, kept by date so the app shows an archive rather than a notification that vanishes. And a dosha alert, sent only when an affliction starts or ends: a Sade Sati stepping from rising to peak, Saturn leaving the eighth from the Moon, a node arriving over the ascendant. A standing condition is recorded and shown, but it does not nag.

Three decisions shape everything else. The slot a reading belongs to is decided by the user's local hour, so one fixed UTC schedule serves every timezone. The dosha engine (`lib/astrology/doshas.ts`) decides deterministically which conditions hold, and Claude only writes the words. And natal conditions, fixed at birth, never alert after a user's first read, because a change in one is an artifact of the computation moving under them and never an event.



Dashed arrows are reads and model calls. Solid hot arrows are the writes and the push. Nothing left of the wall keeps a token in mer

One route, two jobs in sequence, five tables. The predictions job writes one reading per user per slot; the alerts job writes an alert only on a change. Both send each device token to the APNs host its own row names.

9.2 Files and folders

app/api/cron/ ├ daily/route.ts └ alerts/route.ts	the scheduled job: readings, then the dosha sweep the older entry point; runs only the dosha sweep
app/api/devices/ ├ route.ts └ route.test.ts	POST registers a device token, DELETE removes it session user wins over the body, environment defaults
lib/alerts/ ├ slots.ts ├ predictions.ts ├ run.ts ├ compose.ts └ *.test.ts	morning or night from the local hour; is the slot due the twice-daily reading, one per user per slot the dosha sweep: diff, record, partition, alert, push the alert copy, the reply parser, the fallback slots, partitioning, parsing, fallbacks
lib/push/apns.ts lib/astrology/doshas.ts vercel.json	token-based APNs client over HTTP/2 detectConditions, the signatures the sweep diffs the two cron schedules
supabase/migrations/ ├ 0004_alerts_push.sql └ 0005_daily_readings.sql	device_tokens, transit_conditions, alerts daily_readings
mobile/Sanchara/Sources/ ├ Services/Push.swift ├ Features/Alerts/ │ ├── DailyStore.swift │ ├── AlertsStore.swift │ ├── InboxView.swift │ └ AlertsView.swift └ Features/Chat/ChatView.swift	PushStore, the app delegate, ApsEnvironment DailyReading, grouping by date, the widget cache SancharaAlert, open, markRead the bell sheet: Daily and Alerts tabs, DailyReadingView AlertDetailView, severityColor where a tapped push becomes an open sheet

```
mobile/Sanchara/Tests/
├─ ApsEnvironmentTests.swift    the entitlement parser
├─ DailyReadingTests.swift      grouping and labels

docs/PUSH_ALERTS.md            the one-time setup: APNs key, env vars, entitlement
```

The split inside `lib/alerts` is by what can be tested without a network. `slots.ts` is two pure functions; `compose.ts` exports its parser and fallback separately from the model call; `run.ts` pulls its two hardest decisions out into `partitionForAlerts` and `deriveHadNatalRows`. Everything that touches Supabase or APNs stays in the orchestrating functions.

9.3 Model

A CONDITION AND ITS SIGNATURE

```
// lib/astrology/doshas.ts
export type Severity = "info" | "caution" | "warning";

export type Condition = {
  kind: string;
  signature: string;
  label: string;
  severity: Severity;
  scope: "natal" | "transit";
  /** A plain factual sentence, the grounding the model is allowed to interpret. */
  detail: string;
};
```

The `signature` is the whole mechanism. It folds in the phase and the sign involved, so that Saturn moving on, or Sade Sati stepping from rising to peak, reads as a new condition rather than the same one continuing. Natal detectors produce signatures such as `mangal_dosha:7` or `kaal_sarp:<sign>`; transit detectors produce `sade_sati:<phase>:<sign>`, `kantaka_shani`, `node_over_moon<node>`, `ashtama_surya`, `guru_6_8_12`, `retrograde<planet>` and the rest, each suffixed with the sign that makes it specific. The sweep never compares labels or kinds. It compares signatures against the set of open rows, and the difference is the news.

THE COPY

```
// lib/alerts/compose.ts
export type AlertCopy = {
  title: string;
  body: string;
  detail: string;
};

// lib/alerts/slots.ts
export type Slot = "morning" | "night";
export const NIGHT_STARTS_AT_HOUR = 14;
```

One copy type serves both a reading and an alert: a title, one plain sentence for the lock screen, and a markdown detail ending in a section titled "In simple words". The prompt asks for under 38 characters of title and under 140 of body; the parser cuts at 60 and 200, so a reply a few characters over is trimmed rather than discarded.

FOUR TABLES

TABLE	COLUMNS THAT MATTER	UNIQUENESS	APP MAY
device_tokens	token (primary key), user_id, platform (only ios), environment (sandbox or production), updated_at	the token itself	read, write and delete its own rows, though the route uses the admin client (see 9.4)
transit_conditions	kind, signature, label, severity, scope, detail, started_on, ended_on	partial unique index on (user_id, signature) where ended_on is null	read its own rows
alerts	severity, kinds text[], title, body, detail, created_at, read_at	none beyond the id	read, and update (to mark read)
daily_readings	for_date, slot, tradition (default vedic), title, body, detail, created_at, read_at	unique (user_id, for_date, slot)	read, and update (to mark read)

Every table has Row Level Security on, and every policy is owner-only. The jobs write with the service-role client, which bypasses RLS; the policies let the app read its own inbox and mark things read through PostgREST with no route handler in between. `transit_conditions` is the state the feature hangs on: a row is open while `ended_on` is null, and the partial unique index allows one open row per signature per user, so two runs racing cannot open a condition twice. `daily_readings.for_date` is the user's own local date, with the migration's reason: "my reading for the 26th" has to mean their 26th.

WHY ENDED_ON AND NOT A DELETE

A closed row is history. The unique index is partial precisely so a condition can end, stay on record with its dates, and open again under the same signature when Saturn returns. The table is a ledger of every affliction the engine has seen, not a set of current ones.

9.4 The server side

TWO ROUTES, ONE CURRENT

```
// vercel.json
"crons": [
  { "path": "/api/cron/daily", "schedule": "30 2 * * *" },
  { "path": "/api/cron/daily", "schedule": "30 15 * * *" }
]
```

Both schedules hit `/api/cron/daily`, at 02:30 and 15:30 UTC, which is 08:00 and 21:00 in India. The handler runs `runDuePredictions()`, then `runDailyAlerts()`, and answers with both summaries.

`/api/cron/alerts` is the older route: it runs only the dosha sweep, is not named in `vercel.json`, and survives for running that half by hand. Its docstring still says the cron fires it, and the comment in `0004_alerts_push.sql` names it too; both are out of date. Read `vercel.json` as the truth about what is scheduled.

```
// app/api/cron/daily/route.ts
export const runtime = "nodejs";
export const maxDuration = 300;

export async function GET(request: Request) {
  const secret = env.cronSecret();
  if (!secret || request.headers.get("authorization") !== `Bearer ${secret}`) {
    return new Response("Unauthorized", { status: 401 });
  }
  ...
}
```

Vercel sends `Authorization: Bearer $CRON_SECRET` on scheduled invocations, and the same header runs the job by hand with `curl`. A missing secret refuses everything; "no secret means open" would expose a job that spends model tokens for every user to anyone who finds the path. The runtime is Node because the Swiss Ephemeris runs as WebAssembly, and the five-minute `maxDuration` is the budget for up to 500 users, each of whom may cost a chart computation and a model call.

SLOTS

```
// lib/alerts/slots.ts
export function slotForHour(localHour: number): Slot {
  return localHour < NIGHT_STARTS_AT_HOUR ? "morning" : "night";
}

export function isSlotDue(localHour: number, alreadyWritten: Slot[]): Slot | null {
  const slot = slotForHour(localHour);
  return alreadyWritten.includes(slot) ? null : slot;
}
```

Anything before 14:00 local is that day's morning reading; anything after is that night's. `isSlotDue` asks one question: does this user still need the reading for the slot they are in right now? It does not backfill. A user in the evening with no morning reading gets tonight's, not a stale "good morning" hours late, which the test `does not backfill a slot the user has moved past pins`.

THE READING

`runDuePredictions` selects every `birth_profiles` row with a computed chart, up to 500, and calls `writeDueReading` for each inside its own `try`; one user failing is counted and logged, and the loop continues. Per user:

1. The Vedic chart is required; a profile without one counts as skipped.
2. The zone is `profile.timezone`, or UTC when null. `nowLocal` and `forDate` come from Luxon in that zone.
3. The slots already written for that user and date are read from `daily_readings`, and `isSlotDue(nowLocal.hour, slots)` decides. Null means nothing to do.
4. `transitChart` computes today's sky at the user's birthplace, and `detectConditions(chart, transits)` finds the same afflictions the alert job tracks, so the reading and the alerts never contradict each other.
5. `composePrediction` writes the copy and the row is inserted with `tradition: "vedic"`. A unique violation (Postgres code `23505`) is returned as null, "already done", not thrown: the index is the safety net for two runs racing.
6. The reading is pushed to every device with `kind: "daily"` and a thread id of `sanchara-daily-morning` or `sanchara-daily-night`.

```
// lib/alerts/predictions.ts
const flagged = args.conditions
  .filter((c) => c.severity !== "info" && c.scope !== "natal")
  .slice(0, 4)
  .map((c) => `- ${c.label}: ${c.detail}`)
  .join("\n");

const stable = buildChartSystem({ firstName, tradition: "vedic", chart, derived: factsFor(chart)
const todayBlock = buildTodaySystem({ today, transits, maxWords: 150 });
...
system: [
  // Identical for this user on every run, so it is a cache read whenever
  // two of their readings land within the hour.
  { type: "text", text: stable, cache_control: { type: "ephemeral", ttl: "1h" } },
  { type: "text", text: todayBlock },
],
```

The prompt has three parts. The stable block is the same chart system prompt the chat uses, with the derived facts and numerology, marked for a one-hour prompt cache. The today block is the gochara description, called with `includeAfflictions: false` because the job lists the flagged conditions itself; asking for them twice was the double-listing fixed in 0.3.0. The task is one of two briefs, morning ("look forward, not back") or night ("reflect on the day, then say what tomorrow morning opens with"), followed by at most four flagged conditions, to be worked in only where they bear on the day and without alarm. Natal conditions are left out of that list for the same reason they never alert: they are not about today. The model is `READING_MODEL` (`claude-sonnet-5`) at `LOW_EFFORT` with 1,600 output tokens. If the call fails or the reply cannot be parsed, `fallbackPrediction` stores a plain reading that greets the user by first name and points at the app, so the slot is never empty.

THE DOSHA SWEEP

`runDailyAlerts` has the same outer shape: the same profile query, a per-user `try`, and a summary of `processed`, `alerted`, `pushesSent`, `failures` and `pushConfigured`. Per user, `runAlertsForUser` works in a fixed order, and the order is the correctness.

1. Compute today's sky with `computeChart` at the user's latitude and longitude, the same way the chat route does. Then `detectConditions(natal, transit)`, indexed by signature.
2. Count the user's natal rows in `transit_conditions`. This is the only fact that decides whether a natal condition may alert, and it fails closed (below).
3. Load the open rows (`ended_on is null`). `started` is every current condition whose signature is not open; `ended` is every open row whose signature is not current.
4. Insert every started row with `started_on` as the user's local date, and close every ended row the same way. The code warns at each step that these are the raw, unfiltered lists: a natal filter inlined here would stop natal rows being written, and the same condition would re-detect as "starting" forever.
5. Only now partition. `partitionForAlerts` keeps everything when the user had no natal rows (a first read, where the whole natal set is genuinely news) and otherwise routes natal conditions to `silent`. Then keep only `caution` and `warning` (`NOTIFIABLE`), and at most six (`MAX_CONDITIONS_PER_ALERT`), because past that an alert becomes a list.

6. If nothing is left, return without an alert. Otherwise the severity is the highest among the started conditions, or the ended ones if nothing started; `composeAlert` writes the copy; one row goes into `alerts` with the `kinds` involved; and the push goes out.

```
// lib/alerts/run.ts
export function deriveHadNatalRows(result: { count: number | null; error: { message: string } } |
  if (result.error) return true;
  return (result.count ?? 0) > 0;
}
```

Supabase returns `count: null` alongside an error, so the naive `(count ?? 0) > 0` would read a failed count as "no natal rows", treat an existing user as a first-time reader, and let their natal conditions through unfiltered. That is the bug the 0.3.0 move to whole-sign houses would have caused: the detectors shifted, some natal signatures changed, and "Mangal Dosha has begun" would have gone to people whose chart has not changed since birth. On error the function assumes the opposite of the naive read. It does not throw, because suppressing a natal alert costs nothing real (the condition still shows in the app and the next run reconciles), while aborting the user would drop their legitimate transit alerts for the day.

COMPOSING THE ALERT

`composeAlert` uses `DEEP_READING_MODEL` (`claude-opus-4-8`), again at low effort and 1,600 tokens; an alert is a title, a sentence and about 180 words, so the thinking budget is kept short on purpose. The system prompt gives the model a short chart summary (ascendant, Sun and Moon signs, the running dasha, every planet with sign and house), the conditions under "Newly active now" and "No longer active" with their factual `detail` lines, and four rules: never invent a placement, dosha, date or number not listed; no guaranteed outcomes and no medical, legal or financial guarantees; calm and practical, a heads-up and not doom; a remedy that is traditional and harmless.

```
// lib/alerts/compose.ts
export function parseAlertCopy(text: string): AlertCopy | null {
  const title = text.match(/^s*TITLE:s*(.+)$/m)?.[1]?.trim();
  const body = text.match(/^s*BODY:s*(.+)$/m)?.[1]?.trim();
  const detailIndex = text.search(/^s*DETAIL:s*$/m);
  const detail = detailIndex === -1 ? undefined : text.slice(text.indexOf("\n", detailIndex) + 1
  if (title && body && detail) {
    return { title: title.slice(0, 60), body: body.slice(0, 200), detail };
  }
  return parseAlertJson(text);
}
```

The reply is line-delimited rather than JSON because `detail` is markdown. Asked for JSON, the model sometimes writes real newlines inside the string, which is not valid JSON, and the reply is lost; the comment records two of a batch of seven live readings coming back that way. Delimited text has no escaping to get wrong. JSON is still accepted as a second parse for the old shape. When neither parses, or the call throws, `fallbackCopy` still names the condition: "Sade Sati (peak phase) is active", or "has eased" when only endings remain.

THE APNS CLIENT

`lib/push/apns.ts` talks to Apple directly over HTTP/2 with token-based authentication; there is no push library. `isPushConfigured()` is true only when all of `APNS_KEY_ID`, `APNS_TEAM_ID`, `APNS_BUNDLE_ID` and `APNS_PRIVATE_KEY` are set. Without them the jobs still write readings and alerts; they just send nothing, and the summary says `pushConfigured: false`. `APNS_PRIVATE_KEY` is the full text of the `.p8` file; the env accessor turns literal `\n` sequences back into newlines so the key survives a single-line variable.

```
// lib/push/apns.ts
const HOSTS: Record<PushEnvironment, string> = {
  sandbox: "https://api.sandbox.push.apple.com",
  production: "https://api.push.apple.com",
};

function providerToken(): string {
  const now = Math.floor(Date.now() / 1000);
  if (cachedToken && now - cachedToken.issuedAt < 45 * 60) return cachedToken.value;
  const header = base64url(JSON.stringify({ alg: "ES256", kid: env.apnsKeyId() }));
  const payload = base64url(JSON.stringify({ iss: env.apnsTeamId(), iat: now }));
  const key = createPrivateKey(env.apnsPrivateKey());
  const signature = signWithKey("sha256", Buffer.from(`${header}.${payload}`), {
    key,
    dsaEncoding: "ieee-p1363",
  });
  ...
}
```

The provider token is a JWT signed with the `.p8` key: `kid` in the header, `iss` (the team id) and `iat` in the payload. Apple accepts one for an hour and rate-limits regeneration, so the module keeps it for 45 minutes across sends and warm invocations. The one subtlety is `dsaEncoding: "ieee-p1363"`: ES256 wants the raw `r` and `s` pair, and Node signs DER by default. `ApnsClient` keeps one HTTP/2 session per environment, so a run that notifies many users pays for one TLS handshake; the session's `error` handler exists because an unhandled socket error would take the process down.

`send` posts to `/3/device/<token>` with `apns-topic` set to the bundle id, `apns-push-type: alert` and `apns-priority: 10`. The payload is an `aps` block (title and body, the default sound, a `thread-id` defaulting to `sanchara-alerts`, `interruption-level: active`) plus two custom keys the app reads on a tap: `alert_id` and `kind`. A ten-second timeout cancels the stream. The result reports `unregistered: true` on a 410 or the reasons `Unregistered` and `BadDeviceToken`; both jobs delete those tokens after the loop, because Apple only tells you a token is dead when you use it.

The job never chooses the environment. Each `device_tokens` row carries the `environment` the app reported at registration, and `send` opens the session for that row. A sandbox token and a production token for the same user coexist, each going to the only host that will accept it.

REGISTERING A DEVICE

`POST /api/devices` takes `{ token, environment }`. The token must match `/^[0-9a-fA-F]{64,200}$/ :` APNs token length has grown over the years, so the route checks the alphabet and a sane range rather than pinning 64 characters. The environment is `sandbox` only when the body says exactly that; anything else

becomes `production`. The row is upserted on `token` with `platform: "ios"` and the user id from the verified session, never from the body. It is written with the admin client on purpose: a device handed from one account to another has to change owner, and the owner-only RLS policy would correctly refuse that update. `DELETE` matches both the token and the session user, so one account cannot unregister another's device.

9.5 The clients

THE WEB APP

There is no web surface for readings or alerts in 0.3.0: nothing under `app/` or `components/` reads `daily_readings` or `alerts`. The web app's part is the route handlers it hosts. The inbox is an iOS feature, and the phone reads the tables straight through PostgREST under RLS.

PUSHSTORE AND THE ENTITLEMENT

`Push.swift` holds three things. `PushStore` is a `@MainActor` observable singleton with the device token, an authorization flag, and `pending`, the notification just tapped. `SancharaAppDelegate` exists because SwiftUI has no hook for APNs callbacks: it receives the token, turns a tap into a `TappedNotification` (an id and a kind, `daily` or `alert`, from the payload's custom keys), and shows a banner even when the app is open, since these are infrequent and time-relevant. And `ApsEnvironment` answers the one fact the server needs right about a device.

```
// mobile/Sanchara/Sources/Services/Push.swift
static func parse(profileText: String) -> String {
    guard let open = profileText.range(of: "<plist"),
          let close = profileText.range(of: "</plist>", range: open.upperBound..profileText.end)
    else { return sandbox }
    let plist = String(profileText[open.lowerBound..close.upperBound])
    guard let data = plist.data(using: .utf8),
          let root = try? PropertyListSerialization.propertyList(from: data, format: nil) as? [String: Any],
          let entitlements = root["Entitlements"] as? [String: Any],
          let value = entitlements["aps-environment"] as? String
    else { return sandbox }
    return value == production ? production : sandbox
}
```

The environment is read from the signing entitlement, not the build configuration. The 0.2.0 changelog records why: a Release build run from Xcode still carries `aps-environment: development` and mints a sandbox token, and reporting "production" for it made every push to that device fail silently. So the app reads `embedded.mobileprovision` from its own bundle, which is what APNs reads too. That file is a CMS-signed blob with a plist inside. Rather than parse CMS, the code decodes the bytes as Latin-1 (every byte maps to a character, so the binary envelope cannot make the decode fail), finds the span between `<plist` and `</plist>`, and reads the entitlements dictionary. `production` is returned only when the profile says exactly that; a simulator build has no profile and cannot receive pushes, so sandbox is the harmless answer for every failure. `mobile/project.yml` sets the entitlement to `development`; the README and the TestFlight workflow switch it to `production` before archiving, and the app reports whichever it finds.

Registration is asked for in `ChatView`, not at launch, with the reason in the code: by then the user has a chart, so "we'll tell you when a dosha starts" means something. `requestAuthorization` asks once and calls `registerForRemoteNotifications`; the delegate hands the token to `received`, which formats it as hex and calls `register`, which posts it with `ApsEnvironment.current` whenever there is a session. A failed registration is swallowed; the next launch retries. `AuthStore.signOut` calls `unregister` before the session is gone, so the next person to sign in on that phone does not receive the previous account's readings.

THE TWO STORES

`DailyStore` loads the newest 60 rows of `daily_readings`, ordered by `for_date` then `created_at`, both descending, and groups them into `DailyReadingDay` values: newest day first, morning above night within a day, each labelled "Today", "Yesterday" or a long date. After every successful load it saves the newest reading into the App Group cache as a `CachedReading` and calls `WidgetRefresh.reload()`, because the widget and the Siri snippet read this and neither can run the query themselves. `AlertsStore` loads the newest 50 alerts. Both stores expose `unreadCount`, `open(id:)` and `markRead`. `open` looks in the loaded list first and reloads only on a miss, because the push usually arrives before the app has reloaded. `markRead` writes `read_at`, the only write RLS allows the app, and a failure leaves the badge until the next load.

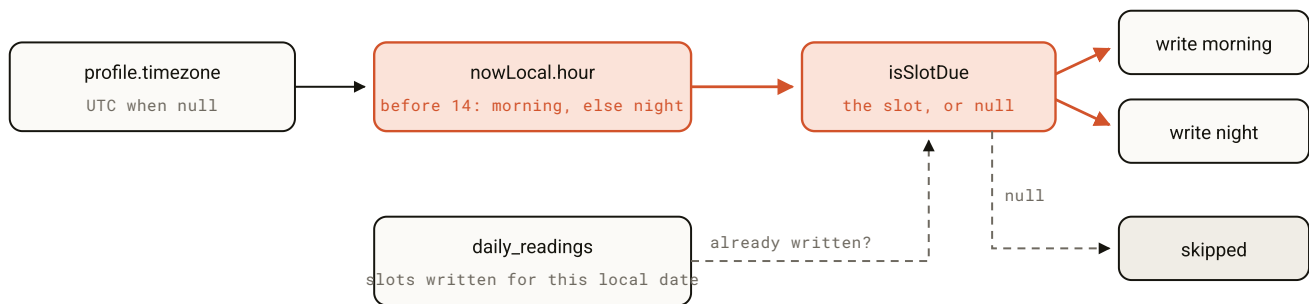
THE INBOX

`InboxView` is one sheet with two tabs, Daily and Alerts, each with an unread badge, so the bell means "things for you". The Daily tab is sectioned by day with a sun or moon glyph per row; the Alerts tab is a flat list with a severity dot (`severityColor`: the accent for `warning`, amber for `caution`, muted otherwise). A tap opens `DailyReadingView` or `AlertDetailView`, which render the markdown `detail` and end with one button, "Ask Sanchara about this", that closes the inbox, starts a new reading and sends a question built from the title and body. The sheet's `.task` marks the item read as it opens.

9.6 How it works

A SCHEDULED RUN

1. Vercel Cron sends `GET /api/cron/daily` with the bearer secret. The handler checks it and calls the two jobs in order: readings first, then the sweep.
2. Each job opens one admin client and one `ApnsClient`, loads up to 500 profiles with a chart, and loops. The readings job decides a slot per user and writes and pushes only when it is due; the sweep diffs, records, partitions, and alerts only on a change.
3. Both clients are closed in `finally`, and the handler answers with the two summaries. A top-level error becomes a 500 with a logged cause; a per-user failure is only a count.



A second run in the same half of a local day finds the slot written and skips. A slot the user has moved past is never backfilled.

The slot decision, per user, on every run. The unique index on `(user_id, for_date, slot)` backs the same decision at the database, so two runs racing cannot both write.

WHAT THE TWO UTC RUNS MEAN IN OTHER PLACES

The schedule was chosen for India, where the two runs land at 08:00 and 21:00. Everyone else gets whatever those two instants are on their own clock, run through `slotForHour` (offsets as of late September):

ZONE	02:30 UTC IS	15:30 UTC IS	READINGS PER DAY
India, UTC+5:30	08:00, morning	21:00, night	2
London, UTC+1	03:30, morning	16:30, night	2
New York, UTC-4	22:30 the day before, night	11:30, morning	2
Los Angeles, UTC-7	19:30 the day before, night	08:30, morning	2
Tokyo, UTC+9	11:30, morning	00:30 the next day, morning	1
Sydney, UTC+10	12:30, morning	01:30 the next day, morning	1

A user far enough east has both runs land in the morning half of two consecutive local days: the second writes tomorrow's morning reading just after midnight, and the next morning's run finds it written and skips. They never get a night reading. The London user's morning reading is written at 03:30 and waits on the lock screen until they wake. `docs/PUSH_ALERTS.md` names the remedy: on a Vercel plan that allows sub-daily crons, `0 * * * *` makes every timezone exact at no extra model cost, because each hourly run writes only for users whose current slot is missing. The code is already written for that schedule; only `vercel.json` would change.

A CONDITION STARTS

Saturn enters the sign twelfth from a user's natal Moon. `detectConditions` now returns a `sade_sati` signature carrying the phase and the sign, a `warning` with scope `transit`. No open row has that signature, so it is in `started` and is inserted with today's local date; it is transit, so the partition keeps it; it is notifiable; Opus writes a title, a sentence and a page; the `alerts` row is inserted with `kinds: ["sade_sati"]`; and every device gets the push in the `sanchara-alerts` thread. Tomorrow the same signature is open and current, so it is in neither list and nothing happens. When Saturn moves on, the row is closed, the next phase's signature opens, and the alert reads as a change of phase rather than a repeat.

A TAP

The delegate's `didReceive` reads `alert_id` and `kind` from the payload and sets `PushStore.shared.pending`. `ChatView` observes it, clears it, switches `inboxTab` to match, calls the store's `open(id:)`, and presents the inbox. Because `open` sets `selected`, the detail sheet is already up when the inbox appears.

9.7 Performance and cost

- **Idempotent by construction.** The unique index on `daily_readings` and the partial unique index on `transit_conditions` mean an extra run, a retry, or two runs racing write nothing twice. The route's own comment says it: extra runs mostly find nothing to do.
- **No model call for a skipped slot.** The slot check happens before the transit chart and before the model, so a run that lands in an already-written half of a user's day costs one small query.
- **The chart block is cached.** The stable system prompt carries `cache_control` with a one-hour TTL and is byte-identical for a user on every run, and it matches the chat's chart block, so a reading and a conversation in the same hour share it.
- **Sonnet for the reading, Opus for the alert.** The reading runs twice a day for everyone; the alert runs only on a change, so the more expensive model is reserved for the rarer message. Both run at low effort with a 1,600-token cap.
- **One APNs session per environment per run**, and one provider token per 45 minutes, shared across every device in the loop.
- **Dead tokens are pruned as they are found**, so the loop does not grow with every uninstalled app.
- **Bounded work.** Both jobs cap the profile query at 500 and the route at five minutes. There is no pagination; past 500 users with charts, the limit is the next thing to change.
- **The phone caches the newest reading** in the App Group so the widget and Siri answer with no network, and the cache's `isFresh` keeps a widget from insisting on yesterday's morning reading.

9.8 Edge cases and tests

CONDITION	BEHAVIOUR
No <code>CRON_SECRET</code> configured	Both cron routes answer 401 to everyone, including Vercel. The job cannot be left open by omission
No APNs variables	Readings and alerts are still written and appear in the inbox; <code>pushesSent</code> is 0 and <code>pushConfigured</code> is false
Profile has no timezone	Treated as UTC for the slot and the stored date
Profile has no Vedic chart	Skipped by both jobs; nothing is written
Second run in the same half of a local day	<code>isSlotDue</code> returns null; counted as <code>skipped</code> ; no chart, no model call
Two runs race on the same slot	The insert fails with <code>23505</code> and is treated as "already done", not as a failure
Evening, and no morning reading exists	Tonight's reading is written; the morning is never backfilled
Model call fails or returns something unparseable	A deterministic fallback is stored: a greeting for a reading, the condition's name for an alert. The slot or alert is never empty
Model replies in the old JSON shape	Still accepted by <code>parseAlertJson</code> , including inside a fenced or chatty reply
Natal condition's signature changes for an existing user	The row is opened and the old one closed, silently. No alert, no push
Natal count query fails	Fails closed: the user is treated as having natal rows, so natal conditions cannot alert; transit alerts still go out
A user's first sweep	Every standing condition, natal included, is news and may alert once, subject to severity
Only <code>info</code> conditions changed	Recorded in <code>transit_conditions</code> ; no alert row, no push
More than six notifiable changes	The first six are named; all are still recorded
One user throws mid-loop	Counted in <code>failures</code> , logged with the user id; the loop continues
Apple says the token is unregistered or bad	Deleted after the loop; no retry
APNs does not answer in ten seconds	The stream is cancelled and the send reported as a timeout; the token is kept
Sandbox token sent to the production host, or the reverse	Cannot happen from the job: the host comes from the row, and the row's environment from the entitlement the app was signed with
Release build run from Xcode, or a simulator	Reports <code>sandbox</code> : the embedded profile says <code>development</code> , or there is no profile at all
Body claims a different <code>user_id</code>	Ignored; the session user owns the row
Device changes hands between accounts	The admin-client upsert moves the row to the new owner; sign-out unregistered it first anyway
Push arrives before the inbox has loaded	<code>open(id:)</code> misses, reloads, and finds it

TESTS

On the server, five Vitest files, 24 tests, none of which touch a network, a database or the model:

- `lib/alerts/slots.test.ts` : `slotForHour` splits the day at 14; `isSlotDue` returns the current slot when missing, null when written, and never backfills.
- `lib/alerts/run.test.ts` : `partitionForAlerts` alerts natal conditions only on a first read and keeps transit conditions alertable either way; `deriveHadNatalRows` is false at zero, true at three, and true when the count query errors (the test is named "fails CLOSED").
- `lib/alerts/compose.test.ts` : `parseAlertCopy` reads the delimited shape with newlines intact, still accepts JSON, returns null for a refusal, and rejects a reply missing its detail; `parseAlertJson` finds the object in a fenced reply; `fallbackCopy` names the condition and reads as relief when one has ended.
- `lib/alerts/predictions.test.ts` : `fallbackPrediction` greets by first name, keeps the body under 140 characters, ends with "In simple words", and gives night a different title from morning.
- `app/api/devices/route.test.ts` : 401 with nothing written for no session; 400 for a malformed token; the row is stored against the session user, not the body, with `onConflict: "token"` ; an unknown environment defaults to production; `DELETE` filters by both token and user.

On the phone, two XCTest files:

- `ApsEnvironmentTests` builds a fake profile with non-text bytes either side of the plist, as CMS wrapping produces, and checks that development is sandbox, production is production, a profile without the entitlement is sandbox, no plist is sandbox, a wrong-case value is not trusted as production, and the test bundle itself (no embedded profile, the simulator case) reports sandbox.
- `DailyReadingTests` : newest day first, morning before night within a day, "Today" and "Yesterday" labels, and the slot flags.

WHAT IS NOT TESTED

The orchestration in `runAlertsForUser` and `writeDueReading`, the diff against open rows, the APNs client and the JWT signing have no automated tests. They are exercised by calling `/api/cron/daily` with the secret and reading the summary, as `docs/PUSH_ALERTS.md` describes.

9.9 Study notes

READ IN THIS ORDER

1. `lib/alerts/slots.ts` , the whole timezone story in twenty lines.
2. `lib/astrology/doshas.ts` , the header comment on `signature` , then one natal and one transit detector.
3. `lib/alerts/run.ts` , top to bottom: `partitionForAlerts` , `deriveHadNatalRows` , then `runAlertsForUser` and the comments at each write.
4. `lib/alerts/predictions.ts` , `writeDueReading` and the three-part prompt in `composePrediction` .
5. `lib/alerts/compose.ts` , the rules in the system prompt and why `parseAlertCopy` is not JSON.
6. `lib/push/apns.ts` , `providerToken` and `send` .
7. `supabase/migrations/0004_alerts_push.sql` , for the partial unique index and its comment.
8. `mobile/Sanchara/Sources/Services/Push.swift` , `ApsEnvironment.parse` and the comment above `PushStore.environment` .
9. `mobile/Sanchara/Sources/Features/Alerts/DailyStore.swift` , `group` and the cache write in `load` .

CHECK YOURSELF

Why is the slot decided by the user's local hour rather than by which cron fired?

One fixed UTC schedule then serves every timezone, and the same code works unchanged under an hourly schedule. The cron decides when to look; the user's clock decides what is due.

Why are natal rows still inserted into `transit_conditions` if they can never alert?

The diff is against open rows. A natal condition never recorded is absent from the open set on every run, so it would be in `started` forever. Recording it once is what makes it silent.

What does `deriveHadNatalRows` return when the count query fails, and why?

True. That routes natal conditions to silent, which costs nothing; the other direction is a push saying a birth condition began today.

Why does the reply use `TITLE:` , `BODY:` and `DETAIL:` lines instead of JSON?

The detail is markdown with real newlines, and the model sometimes writes them raw inside a JSON string, which does not parse. Delimited lines have no escaping to get wrong.

Where does the job learn which APNs host to use for a device?

From the `environment` column of that device's row, set at registration from the `aps-environment` entitlement in the app's embedded provisioning profile. Neither the job nor the build configuration guesses.

Why does the device route write with the admin client when RLS already lets a user manage their own tokens?

A phone that changes hands has to move its token to the new account, and an owner-only policy correctly refuses that update. The user id still comes from the session, never the body.

A user in Tokyo never gets a night reading. Why, and what fixes it?

Both UTC runs land before 14:00 local on two consecutive days, so both are morning slots and the second is skipped. An hourly cron fixes it with no code change, because `isSlotDue` already writes only what is missing.

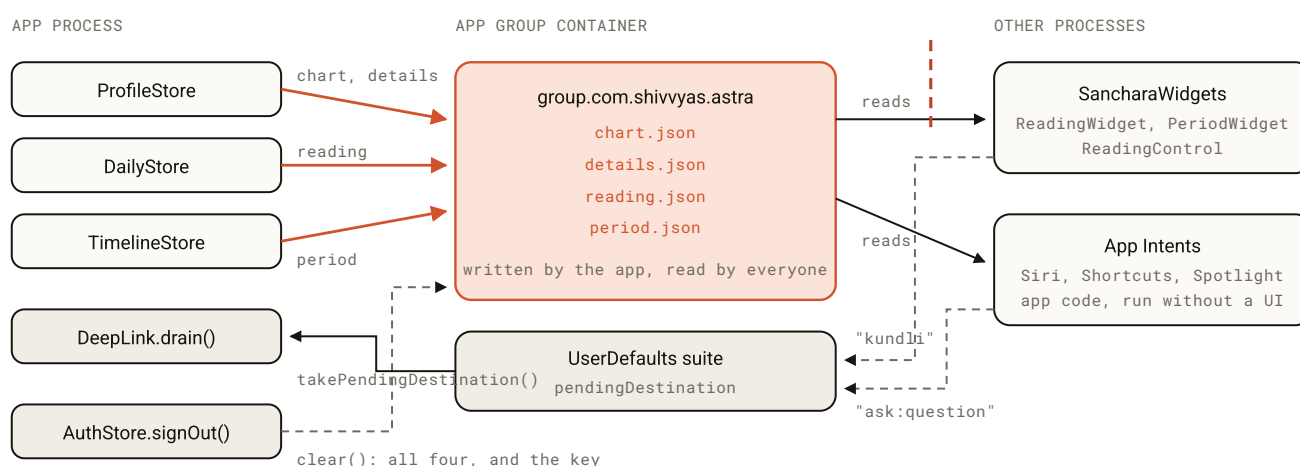
10 On-device answers, Siri, and widgets

A birth chart is fixed at birth and about four kilobytes. Once the phone holds a copy, the questions that are lookups ("where is my Saturn", "what is in my seventh", "which dasha am I in") can be answered by the phone itself, out loud through Siri, on the Home Screen, or in the chat, with no session, no network and no paid call. One cache in an App Group makes all three surfaces possible.

This chapter describes the code on `main` on 30 September 2026, version 0.3.0. `ChartCache` writes the chart, the birth details, the latest daily reading and the current dasha period into the App Group container after every successful load in the app. The widget extension and the App Intents only ever read from it. On top of the cache sit Apple's on-device model, which restates the chart when a question is a retrieval and hands everything else to Claude, and a small deep-link protocol that lets a process with no UI tell the app where to land.

10.1 Intent

A large share of what people ask Sanchara is not interpretation. The doc comment at the top of `mobile/Sanchara/Sources/Features/Chat/OnDeviceReasoner.swift` puts it plainly: every one of those questions "was a paid Claude call answering from a chart the phone could have read itself". The same split runs through Siri. Positions and periods are records, so an intent answers them instantly and for nothing. Readings are written by Claude, so an intent can hand back one already written but never commissions a new one: a voice phrase should not silently spend money or take thirty seconds. The widgets follow the rule in its strictest form. They never fetch. A widget that waits on a network call is a widget that shows a placeholder at exactly the moment someone glanced at it.



The extension is a separate process with no session. The intents are compiled into the app target, so iOS runs them inside the app,

One container, four files, one key. Only the widget extension sits across a real process boundary; the App Intents are app code that the system runs in the background, but they hold no view and never wait on the network, so they read the same cache.

10.2 Files and folders

mobile/Sanchara/Sources/	
├─ Services/	
│ ├─ ChartCache.swift	the App Group cache: chart, details, reading, period, destination
│ └─ WidgetRefresh.swift	asks WidgetKit to rebuild a timeline after a cache write
├─ Features/Chat/	
│ ├─ ChartFacts.swift	the chart restated as neutral rows for the on-device model
│ ├─ OnDeviceReasoner.swift	the router, the classifier and the lookup tool
│ ├─ OnDeviceSuggestions.swift	follow-up questions written on the device, with starters
│ ├─ ChatStore.swift	where a chat turn meets the router
│ └─ ChatView.swift	the on-device footer, suggestion chips, deep-link handling
├─ Features/Kundli/ChartData.swift	NatalChart, DerivedFacts, PeriodProgress (shared with the e
└─ Intents/	
├─ SancharaIntents.swift	five App Intents Siri, Shortcuts and Spotlight can run
├─ SancharaEntities.swift	DailyReadingEntity and PlacementEntity, indexed for Spotlig
├─ SancharaShortcuts.swift	the phrases that work with nothing set up
├─ IntentData.swift	cache first, network second, never throws
├─ ReadingSnippet.swift	the reading as a card inside Siri (iOS 26)
├─ OpenKundliIntent.swift	the one intent that opens the app; shared with the extensio
└─ DeepLink.swift	where an intent, a control or a widget tap says the app sho
mobile/SancharaWidgets/	
├─ ReadingWidget.swift	kind SancharaReading, rebuilt at 06:00 and 18:00
├─ PeriodWidget.swift	kind SancharaPeriod, rebuilt after midnight, links sanchara
├─ ReadingControl.swift	the Control Center button and the @main WidgetBundle
├─ Info.plist	com.apple.widgetkit-extension
└─ SancharaWidgets.entitlements	the App Group, and nothing else
mobile/project.yml	which app files the extension also compiles
docs/SIRI_AND_WIDGETS.md	portal setup, the phrases, and a manual gate on the model's
mobile/Sanchara/Tests/	
├─ OnDeviceRoutingTests.swift	the cheap filter, the house parser, the rows
├─ ChartFactsTests.swift	the derived rows, and the fallback without them
├─ DerivedFactsTests.swift	decoding a chart cached before derived facts existed
├─ SuggestionTests.swift	starters where the model is missing
└─ PeriodProgressTests.swift	two of its tests cover the period cache

Nothing here has a web counterpart. The web app has no widget, no Siri surface and no on-device model; a question typed into `components/Chat.tsx` always goes to `/api/chat`. The iOS chat is the only client that routes.

10.3 Model

WHAT THE CACHE HOLDS

```
// mobile/Sanchara/Sources/Services/ChartCache.swift
final class ChartCache: @unchecked Sendable {
    static let shared = ChartCache()
    static let appGroup = "group.com.shivvyas.astra"

    private let queue = DispatchQueue(label: "com.shivvyas.astra.chartcache")
    private let directory: URL

    private init() {
        let shared = FileManager.default.containerURL(
            forSecurityApplicationGroupIdentifier: Self.appGroup
        )
        // Without the entitlement (a simulator build with no provisioning, say)
        // the app still works, it just stops sharing with the widget.
        directory = shared ?? URL.cachesDirectory
        try? FileManager.default.createDirectory(at: directory, withIntermediateDirectories: true)
    }

    private func write<T: Encodable>(_ value: T, to name: String) {
        queue.async {
            guard let data = try? JSONEncoder().encode(value) else { return }
            try? data.write(to: self.url(name), options: [.atomic, .completeFileProtection])
        }
    }

    private func read<T: Decodable>(_ type: T.Type, from name: String) -> T? {
        queue.sync {
            guard let data = try? Data(contentsOf: url(name)) else { return nil }
            return try? JSONDecoder().decode(type, from: data)
        }
    }
}
```

Four JSON files live in the group container, each behind a save-and-load pair. Writes are asynchronous on one serial queue and reads are synchronous on the same queue, so a read issued after a write sees the write. Every file is written atomically with complete file protection. Nothing secret goes in: no token, no session. Those stay in the Keychain, where supabase-swift already keeps them.

FILE	TYPE	WRITTEN BY	READ BY
chart .json	NatalChart , the Vedic half of birth_profiles.chart	ProfileStore.loadChar t()	ProfileStore on launch, IntentData , PlacementQuery , PeriodProvider as a fallback
detail ls.json	BirthProfileDetails	ProfileStore.load()	ProfileStore on launch, IntentData
reading .json	CachedReading : id, slot, title, body, forDate, savedAt	DailyStore.load() , IntentData.fetchReadi ngs()	ReadingProvider , IntentData.latestReading()
period .json	CachedPeriod : lord, antardasha, four ISO dates, theme, nowMeaning, savedAt	TimelineStore.cache(_ :)	PeriodProvider

`CachedReading.isFresh` is true when `savedAt` is today or within the last 86,400 seconds: "A widget should not keep insisting on yesterday's morning reading." `CachedPeriod` carries what the chart alone cannot, the server's one-line `theme` for the mahadasha and the `nowMeaning`, which only `/api/timeline` writes. A fifth value is not a file. `pendingDestination` is one short string in `UserDefaults(suiteName: appGroup)`, because it is written from the widget extension milliseconds before the app launches and the suite gives both processes the same store without coordination. `takePendingDestination()` reads and clears in one step, so a destination is never handled twice. `clear()` removes all four files and the key, and runs on sign-out.

THE CHART THE PHONE REASONS OVER

`NatalChart` in `mobile/Sanchara/Sources/Features/Kundli/ChartData.swift` mirrors `lib/astrology/types.ts`: ascendant, nine `ChartPlanet` rows (name, sign, degree, house, retrograde, nakshatra), Moon and Sun signs, ayanamsa, an optional `DashaInfo` and an optional `DerivedFacts`. The derived block mirrors `lib/astrology/derived.ts`: per planet the houses it rules, its dignity, combustion and distance from the Sun, aspects and conjunctions; per house its lord and where that lord sits; per running dasha level the lord's placement. The top-level block is optional for the reason the type's comment gives: "A widget or a Siri intent must never fail to decode a chart it has already stored." Doshas are deliberately not carried into the app's type; they are interpretive, and the on-device model is only allowed to restate records.

THE TWO ENTITIES

`SancharaEntities.swift` defines the only two things Siri, Shortcuts and Spotlight may know about. `DailyReadingEntity` is what a reading intent returns and what Spotlight indexes; its `attributeSet` puts the body in `contentDescription` so a half-remembered phrase finds the reading again. `PlacementEntity` is one body's position built from a `ChartPlanet`, with the Sanskrit sign from `Rashi(english:)`, and a `spoken` sentence: "Saturn is in Meena, Pisces, at 3°15', in house 5, nakshatra Uttara Bhadrapada. It is retrograde." The file's comment sets the boundary: "Nothing interpretive is an entity. A prediction is not a record and should not be searchable as though it were one."

10.4 The server side

There is none. No route handler, table or cron job exists for the widget, for Siri or for the on-device model. Everything they read was put in the cache by the app from three sources that belong to other chapters: the `birth_profiles.chart` jsonb column, read directly through supabase-swift under the owner-only RLS policy (roughly 4 KB, changing only when birth details are re-saved); the `daily_readings` table, read the same way; and `GET /api/timeline`, whose payload carries the period bands with their themes and the `now` meaning. The on-device model is Apple's, not Sanchara's: `FoundationModels` is a system framework, no weights ship in the bundle, and no request leaves the phone. Where the model declines or is absent, the question goes to `/api/chat` exactly as it did before the feature existed.

10.5 The clients

FILLING THE CACHE

Three stores write, each at the moment its data is confirmed. `ProfileStore.load()` first seeds `chart` and `details` from the cache so the kundli draws immediately, then fetches and saves both on success; if the fetch fails and the cache had both, the state is still `.ready`. `DailyStore.load()` saves the newest of 60 readings and calls `WidgetRefresh.reload()`. `TimelineStore.cache(_:)` finds the band marked `isCurrent`, picks the antardasha whose range contains `payload.today`, saves a `CachedPeriod` and calls `WidgetRefresh.reloadPeriod()`. Because the period file is only written when a timeline arrives, `ChatView` calls `TimelineStore.warmCache()` once per launch, so the period widget fills in without the timeline screen ever being opened. `WidgetRefresh` is two static calls to `WidgetCenter.shared.reloadTimelines(ofKind:)`, for `"SancharaReading"` and `"SancharaPeriod"`, and its comment carries the discipline: reloads are rate-limited by the system, "so this is called when a reading actually changes, never on every load."

CHARTFACTS: THE CHART AS A REFERENCE TABLE

`ChartFacts` turns positions into sentences and does nothing else. It has no dependency on `FoundationModels`, which is what makes it testable on any machine. The wording is the load-bearing part. Apple's on-device model refuses anything it reads as fortune telling, a result found by testing against the real model, so every row uses a neutral vocabulary and never names a tradition, a practice or a meaning:

CHART CONCEPT	WORD IN THE ROW	EXAMPLE
house	section	"section 5"
nakshatra	segment	"segment Uttara Bhadrapada"
lordship	controls, controlled by	"Controls sections 7, 12"
dignity	strength rating	exalted "highest", debilitated "lowest", moolatrikona or own sign "high", otherwise "standard"
combustion	within N degrees of Sun	"Within 5 degrees of Sun"
aspect, conjunction	linked to sections, filed alongside	"Linked to sections 1", "Filed alongside Jupiter"
ascendant, Sun, Moon	reference point, Entry A, Entry B	"Reference point: Scorpio 14°30', which sets section 1."
mahadasha, antardasha	current major period, current minor period	"Current major period: Venus, running 1 April 2012 to 1 April 2032."

Four lookups exist. `body(named:)` returns one planet's row and demands at least two characters, because `hasPrefix("")` is true of every row and an empty argument from the model "would otherwise come back as whichever body happened to be first, a wrong answer delivered with complete confidence". `section(_:)` lists everything filed under one house, names its controller when derived facts exist, and says "No entries are filed under it." when nothing is. `currentPeriod()` renders both dasha levels with their date ranges and, from the derived block, where the running lord is filed. `header()` is the three lines every answer is indexed by, and `everything()` joins them all. Two helpers are shared beyond the file: `sectionNumber(in:)` finds a house number written as "seventh", "house 7", "7th house" or a bare "12th", and only accepts 1 to 12; `calendarDay(_:format:)` parses and formats a `yyyy-MM-dd` string in UTC, because formatting a bare ISO date anywhere west of Greenwich hands back the day before, "the kind of wrong that nobody reports and everybody notices."

ONDEVICEREASONER: THE ROUTER

```
// mobile/Sanchara/Sources/Features/Chat/OnDeviceReasoner.swift
func route(_ question: String) async -> Route {
    #if canImport(FoundationModels)
    if #available(iOS 26.0, *), Self.isAvailable {
        // A question that names no part of the table cannot be a lookup, so
        // the cheap check runs before the model does.
        guard Self.mentionsTheTable(question) else { return .escalate }
        guard await isLookup(question) else { return .escalate }
        if let answer = await answer(question) { return .answered(answer) }
    }
    #endif
    return .escalate
}
```

Three gates, cheapest first. `mentionsTheTable` is a string scan with no model call: true if the question contains one of the nine body names, a house number `sectionNumber(in:)` can find, or one of a short list of period and derived words ("dasha", "mahadasha", "antardasha", "period", "lagna", "ascendant", "rising",

"nakshatra", "moon sign", "sun sign", "rashi", "lord", "ruler", "rules", "exalted", "debilitated", "combust"). The comment records two words tried and dropped, "aspect" and "strength": every question they usefully caught already routed via a body name or an ordinal, so they added only false positives such as "what aspects of my life".

`isLookup` is the first model call. It opens a `LanguageModelSession` on `SystemLanguageModel(guardrails: .permissiveContentTransformations)` with the instruction "You sort incoming questions into two buckets. You never answer them." and asks for a `@Generable` struct, `Classification`, whose single field is `isRetrieval: Bool`. Constrained decoding means the answer is always one of two values, never prose to be parsed; sampling is greedy. Any thrown error, a refusal included, returns false, and the question goes to Claude, "which is where it went before".

`answer` is the second model call, and it uses a tool rather than a stuffed prompt. `TableLookupTool` is registered as `lookupTableEntry` with four optional arguments: `body` (constrained to the nine names by `.anyOf`), `section` (constrained to `1...12`), `currentPeriod` and `wholeTable`. The instructions tell the model to call the tool before answering, report only what it returned, say the table does not record something when the tool returns nothing, never explain what an entry means, and answer in two or three plain sentences. Generation runs at temperature 0.3 with at most 220 response tokens. Every tool result is prefixed with the header, because "which section is that" is the follow-up to almost every row.

```
// mobile/Sanchara/Sources/Features/Chat/OnDeviceReasoner.swift
let text = response.content.trimmingCharacters(in: .whitespacesAndNewlines)
// An answer that never consulted the table is a guess. Drop it.
guard !text.isEmpty, tool.wasCalled else { return nil }
return text
```

`wasCalled` is a flag behind an `NSLock`, written on whatever executor the framework calls the tool from and read on the main actor afterwards. An answer composed without a single lookup is thrown away rather than shown.

WHERE THE CHAT MEETS THE ROUTER

```
// mobile/Sanchara/Sources/Features/Chat/ChatStore.swift
// A lookup the phone can answer from the chart it already holds
// never reaches the network. The opening reading and an explicit
// "read this properly" always do.
if !silent, !forceServer, mode == .vedic, let reasoner {
    if case .answered(let local) = await reasoner.route(text) {
        guard !Task.isCancelled else { return }
        if let last = messages.indices.last, messages[last].role == .assistant {
            messages[last].content = local
            messages[last].source = .onDevice
        }
        isStreaming = false
        return
    }
}
```

`ChatStore.chart` has a `didSet` that builds the reasoner, and its comment limits the scope: "Vedic only: the router reads a Vedic chart, so a Western or numerology question has nothing local to answer from and goes straight to Claude." A locally answered turn is marked `ChatMessage.Source.onDevice` and is never saved to the conversation. `ChatView` renders it with a footer, "Read from your chart on this device", and under the last one a button, "Ask for a full reading", which calls `expandLastAnswer()`: the local answer is removed and the same question re-sent with `forceServer: true`. The disclosure and the escape hatch are both required; if the router guesses wrong, one tap undoes it.

ONDEVICESUGGESTIONS: WHAT TO ASK NEXT

`SuggestionEngine` owns the chips under a reading. `ChatView` calls `refresh(after:mode:)` whenever streaming stops with a non-empty assistant turn. On a device where `SystemLanguageModel.default.availability == .available`, the engine sends the last 1,200 characters of the reading (the tail carries the conclusion) to a session instructed "You rewrite and summarise text. You never add facts of your own, and you never answer the questions you raise." and asks for a `@Generable FollowUpQuestions`: three first-person questions the passage leaves unanswered, trimmed, capped at 90 characters, cut to three. The prompt never mentions astrology: asked to act as an astrologer's assistant the model refuses outright; asked to list what a passage leaves unanswered, it answers well. Everywhere else, and whenever generation throws, `starters(for:)` supplies three written questions per `ChatMode`, so the feature degrades rather than disappears. `onDeviceStatus` turns the availability enum into a sentence for the profile screen, and `OnDeviceReasoner.isAvailable` reads the same flag.

THE INTENTS

INTENT	AVAILABLE	OPENS THE APP	WHAT IT DOES
<code>TodaysReadingIntent</code>	iOS 18	no	Returns a <code>DailyReadingEntity</code> from <code>IntentData.latestReading()</code> and speaks "title. body". Throws <code>noReadingYet</code> when there is none.
<code>ShowReadingCardIntent</code>	iOS 26	no	The same reading as a card: speaks the title and shows <code>ReadingSnippetIntent</code> , which renders <code>ReadingCard</code> . A separate intent because <code>perform()</code> has one concrete return type.
<code>ChartPositionIntent</code>	iOS 18	no	Takes a <code>PlacementEntity</code> parameter ("Where is Saturn in my kundli") and speaks its sentence. The entity query already resolved it from the cache.
<code>CurrentPeriodIntent</code>	iOS 18	no	Reads <code>IntentData.chart()?.dasha</code> and speaks both levels with their end months in UTC. Throws <code>noChartYet</code> without a chart.
<code>AskSancharaIntent</code>	iOS 26	no	Free-form question through <code>OnDeviceReasoner.route</code> ; speaks an answer, or queues the question via <code>DeepLink.shared.queue(question:)</code> and says it needs a full reading.
<code>OpenKundliIntent</code>	iOS 17	yes	Writes "kundli" to <code>pendingDestination</code> and returns. The only intent that opens the app, because "a diagram is the answer, and Siri has nowhere to put one."

Errors are sentences, not codes, because Siri reads them aloud: `SancharaIntentError.noReadingYet` says "There's no reading yet. Sanchara writes one each morning and night." `IntentData` is the data layer for all of them: `chart()` and `details()` read the cache and nothing else, since a chart fixed at birth makes a cache hit always correct; `latestReading()` returns the cached reading if `isFresh`, otherwise the first row of a ten-row `daily_readings` query, which also rewrites the cache "for the widget, which cannot make this call". Nothing throws; a failed query is an empty list.

`SancharaShortcuts` is the `AppShortcutsProvider`: six `AppShortcut`s, the last two behind `#available(iOS 26.0, *)`. Every phrase contains `\(.applicationName)`, an App Intents rule that also covers a user who renames the app. "What's my reading in Sanchara", "What dasha am I in on Sanchara", "Where is my Saturn in Sanchara" (the parameter is spoken inside the phrase), "Open my Sanchara kundli", "Show my Sanchara reading" and "Ask Sanchara about my chart" all work with nothing set up. Ten is Apple's ceiling; the file's comment says fewer is better.

DEEPLINK AND THE SANCHARA:// SCHEME

```
// mobile/Sanchara/Sources/Intents/DeepLink.swift
func drain() {
    guard let raw = ChartCache.shared.takePendingDestination() else { return }
    if raw == "kundli" {
        pending = .kundli
    } else if raw == "timeline" {
        pending = .timeline
    } else if raw.hasPrefix("ask:") {
        let question = String(raw.dropFirst("ask:".count))
        if !question.isEmpty { pending = .ask(question) }
    }
}
```

Intents run without a UI, sometimes in a different process, so none of them can present anything. They write a destination into the App Group and the app drains it when it next comes to the front, mirroring

`PushStore.pending` on purpose: "one habit, two sources." A widget tap is the other way in. `PeriodWidgetView` sets `.widgetURL(URL(string: "sanchara://timeline"))`, the scheme is declared under `CFBundleURLTypes` in `mobile/project.yml`, and `RootView.onOpenURL` hands a URL whose scheme is `sanchara` to `DeepLink.shared.handle(url:)` and everything else to the auth store, which would otherwise report an expired link. `handle(url:)` accepts the hosts `timeline` and `kundli` and ignores unknown paths rather than guessing. `ChatView` consumes the destination on `onAppear` (a cold launch sets it before the view exists), on `scenePhase == .active` (which calls `drain()`), and on `onChange(of: DeepLink.shared.pending)`. `.kundli` and `.timeline` present the matching sheet; `.ask` starts a new reading and sends the question with `forceServer: true`.

THE WIDGET EXTENSION

```
// mobile/SancharaWidgets/ReadingWidget.swift
/// The next 6am or 6pm. Asking to refresh twice a day, rather than every
/// fifteen minutes, is what keeps the widget inside its energy budget while
/// still being right whenever anyone looks.
static func nextBoundary() -> Date {
    let calendar = Calendar.current
    let now = Date()
    for hour in [6, 18] {
        if let candidate = calendar.date(bySettingHour: hour, minute: 0, second: 0, of: now),
            candidate > now {
            return candidate
        }
    }
    let tomorrow = calendar.date(byAdding: .day, value: 1, to: now) ?? now.addingTimeInterval(86400)
    return calendar.date(bySettingHour: 6, minute: 0, second: 0, of: tomorrow)
    ?? now.addingTimeInterval(43_200)
}
```

`SancharaWidgets` is the `@main WidgetBundle` in `ReadingControl.swift`. It vends `ReadingWidget`, `PeriodWidget` and, on iOS 18, `ReadingControl`. Both widgets use a `StaticConfiguration` and a provider whose timeline holds exactly one entry. `ReadingWidget` (kind `SancharaReading`; small, medium and `accessoryRectangular`) reads `loadReading()` and rebuilds at the next 06:00 or 18:00, the cadence of the content. The Home Screen families draw the shared `ReadingCard` with a body limit of 3 lines on small and 5 on medium; the Lock Screen family draws two lines of its own because the Lock Screen tints everything to one colour. The empty state names the next action: "Open Sanchara to get your first reading."

`PeriodWidget` (kind `SancharaPeriod`; the same three families plus `accessoryInline`) prefers `loadPeriod()` and, failing that, the `dasha` pair inside the cached chart with no theme, "shown without a theme rather than as an empty state, the dates are true". It computes today as a UTC calendar day, the API's convention, and derives the bar and the "N years left" caption from `PeriodProgress.fraction` and `yearsRemaining` in `ChartData.swift`, which lives there "because the period widget needs it too, and this file is one of the few compiled into both targets". The timeline rebuilds after the next local midnight so the bar moves by a day when a day has passed. `ReadingControl` is a `ControlWidget` of kind `SancharaReadingControl`: one `ControlWidgetButton` whose action is `OpenKundliIntent()`, labelled "Kundli", displayed as "Open kundli". The type and kind keep "Reading" in their names and the file's leading comment still describes the reading; what the button does today is open the kundli, from the extension's own process.

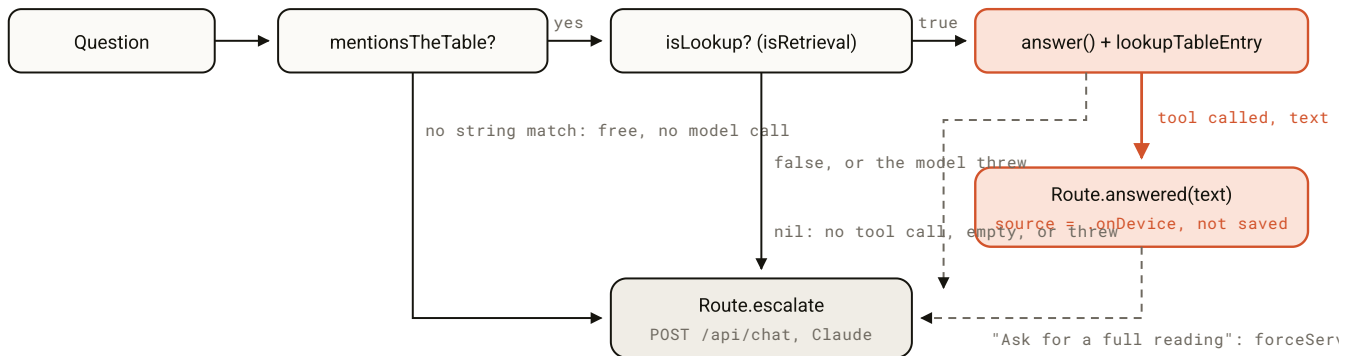
WHAT THE EXTENSION COMPILES, AND WHY EACH FILE IS DEPENDENCY-FREE

```
# mobile/project.yml
SancharaWidgets:
  type: app-extension
  sources:
    - path: SancharaWidgets
    # Compiled into the extension as well as the app. Each of these is
    # dependency-free on purpose: pulling in anything that imports Supabase
    # would drag the whole client into a process that has no session to use
    # it with.
    - path: Sanchara/Sources/Design/Theme.swift
    - path: Sanchara/Sources/Design/ReadingCard.swift
    - path: Sanchara/Sources/Services/ChartCache.swift
    - path: Sanchara/Sources/Features/Kundli/ChartData.swift
    - path: Sanchara/Sources/Features/Intake/BirthProfileDetails.swift
    - path: Sanchara/Sources/Intents/OpenKundliIntent.swift
```

FILE	IMPORTS	WHY THE EXTENSION NEEDS IT
Design/Theme.swift	SwiftUI	The four colour tokens the widget backgrounds and text use
Design/ReadingCard.swift	SwiftUI	The card both the widget and the Siri snippet draw
Services/ChartCache.swift	Foundation	The only door into the container
Features/Kundli/ChartData.swift	Foundation	NatalChart to decode chart.json ; PeriodProgress for the bar
Features/Intake/BirthProfileDetails.swift	Foundation	The type behind details.json ,so ChartCache compiles
Intents/OpenKundliIntent.swift	AppIntents	The control's action, run in the extension's process; it imports nothing but ChartCache

10.6 How it works

PRECONDITIONS: iOS 26, FoundationModels available, mode == .vedic, not the opening reading, not "ask for a full reading"



Cheapest gate first. Only questions that survive the string scan cost a model call, only retrievals cost a second, and only an answer that actually consulted the table is shown. Every failure path ends where the question went before the feature existed.

A QUESTION IN THE CHAT

1. `ChatStore.send` appends the user turn and an empty assistant turn, then, when the preconditions hold, awaits `reasoner.route(text)`.
2. `mentionsTheTable` rejects "should I take the job" without a model call. "Where is my Saturn" passes on the body name; "who rules my 7th" passes on the ordinal and on "rules". `isLookup` then escalates "what does my Saturn mean" as not a retrieval.
3. `answer` opens a session with `TableLookupTool`. The model calls `lookupTableEntry(body: "Saturn")`, receives the header and Saturn's row, and restates them in two or three sentences.
4. The assistant turn is filled and marked `.onDevice`; `isStreaming` goes false; `SuggestionEngine.refresh` runs on the answer like any other reading. Nothing was sent anywhere. If any gate fails, `SancharaAPI.chatStream` streams the reading from `/api/chat` and the turn is saved server-side.

A SIRI PHRASE

1. "Where is my Saturn in Sanchara" matches the `ChartPositionIntent` shortcut. iOS resolves the spoken parameter through `PlacementQuery.entities(matching:)`, which matches the lowercased string against the planet name or Sanskrit sign of each placement in `ChartCache.shared.load()`. `perform()` returns the entity and its sentence; the app is not opened.
2. "Ask Sanchara about my chart" runs `AskSancharaIntent`, which reads the chart from the cache and runs the same three gates. On `.escalate` it writes `"ask:" + question` to `pendingDestination` and says the question is saved. The next time `ChatView` becomes active, `drain()` turns it into `.ask(question)` and the view sends it with `forceServer: true`.
3. "What's my reading in Sanchara" runs `TodaysReadingIntent`. A fresh cached reading answers with no query; a stale one triggers a ten-row fetch that also rewrites the cache.

A WIDGET REFRESH, A TAP, AND SIGN-OUT

1. `DailyStore.load()` writes `reading.json` and calls `WidgetRefresh.reload()`. `WidgetKit`, within its rate limit, asks `ReadingProvider.getTimeline` for a new timeline; the provider reads the cache synchronously and returns one entry with a `.after(nextBoundary())` policy. With the app closed, the night cron job's reading reaches the widget only after the app, or an intent's fetch, has rewritten the cache.
2. A tap on the period widget opens `sanchara://timeline`; `RootView` recognises the scheme and `handle(url:)` sets `pending = .timeline` directly, with no App Group hop. A tap on the Control Center button runs `OpenKundliIntent` inside the extension, which writes `"kundli"` to the group's `UserDefaults`; `openAppWhenRun = true` brings the app forward, and `ChatView` presents the kundli sheet on `onAppear` or on becoming active.
3. `AuthStore.signOut()` unregisters the push token, signs out of Supabase, then calls `ChartCache.shared.clear()`, because the cached files "are on disk in the app group, and the widget would go on showing them to whoever signs in next." The next widget rebuild draws the empty state.

10.7 Performance and cost

- **Zero network on every read path.** Widgets, entity queries and `IntentData.chart()` read a JSON file from disk synchronously. The only intent that can touch the network is a reading intent with a stale cache, capped at ten rows.
- **Model calls are gated by a string scan.** The two on-device sessions run only for questions that name a body, a house or a period word, so an interpretive question costs nothing locally and goes straight to the server.
- **Every on-device answer is a Claude call not made,** and it is never written to `messages` on the server, so there is no storage cost either.
- **Two widget rebuilds a day, plus one after midnight.** Explicit reloads are sent only when a cache file actually changed.
- **Bounded generation.** The lookup answer is capped at 220 tokens at temperature 0.3; the classifier is greedy and returns one Boolean; the suggestion prompt sees at most 1,200 characters.
- **No extension bloat.** The widget binary compiles six app files and no package. Keeping the Supabase client out is what keeps its launch inside `WidgetKit`'s budget.

10.8 Edge cases and tests

CONDITION	BEHAVIOUR
No App Group entitlement (a simulator build without provisioning)	<code>ChartCache</code> falls back to the caches directory. The app works; sharing with the widget stops.
Chart cached before derived facts shipped, or a server key the app does not declare	<code>derived</code> decodes as nil and every row falls back to the position line; <code>conditions</code> is ignored and the decode succeeds.
Device west of Greenwich	Dates parse and format in UTC in <code>ChartFacts.calendarDay</code> , <code>PeriodProgress</code> and <code>PeriodProvider.todayISO</code> , so a dasha ending on the 1st never reads as the 31st.
Model refuses, is not downloaded, or Apple Intelligence is off	<code>isLookup</code> returns false or <code>answer</code> returns nil; the question goes to Claude. The profile screen says which state the device is in.
Model answers without calling the tool, or passes a one-letter body name	Dropped: <code>wasCalled</code> is false; <code>body(named:)</code> returns nil rather than the first row.
Router answers a question that wanted a reading	"Ask for a full reading" removes the local answer and re-sends with <code>forceServer</code> .
Western or numerology mode	Never routed; the reasoner only reads a Vedic chart.
Siri asked before any chart exists, or asked for meaning	<code>noChartYet</code> spoken as a sentence; an interpretive question is queued as <code>ask:</code> and sent to Claude on next launch. Siri never spends money or waits.
Widget added before any reading or timeline is loaded	An empty state naming the next action; the period widget draws the dasha pair from the cached chart with no theme.
Widget tap on a cold launch, or an unknown <code>sanchara://</code> host	<code>onAppear</code> handles the destination set before <code>onChange</code> could fire; unknown hosts are ignored rather than guessed.
Sign-out on a shared device	All four files and the pending key are removed; the next widget rebuild shows the empty state.

TESTS

All in `mobile/Sanchara/Tests`, all runnable on the simulator, none needing Apple Intelligence. The model itself is not tested: it cannot be on a machine without Apple Intelligence, and its answers are not deterministic. What is tested is everything that decides whether it runs.

- `OnDeviceRoutingTests`: the cheap filter admits "where is my saturn", "what's in house 12", "what is my lagna" and the derived vocabulary ("which planet rules my chart", "is anything combust"), and rejects "should I take the job", "when will I get married" and the two dropped words. `sectionNumber` reads "7th house", "house 12", "my fourth house" and "the 1st", and rejects "my 40th birthday". A body row carries sign, degree, section, segment and retrograde; lookups are case-insensitive; "Pluto" and "" return nil; an empty section says so; the period row names both lords and "1 April 2032". `testNothingHandedToTheModelNamesAstrology` sweeps `everything()` for fourteen forbidden words including "house", "planet", "dasha" and "nakshatra". If the framing slips, the feature stops answering silently, and this is the test that notices.

- `ChartFactsTests` : with a derived block, a body row says "Controls sections 7, 12", "Strength rating: highest" and "Linked to sections 1"; a combust Mars 4.6 degrees from the Sun renders "Within 5 degrees of Sun" and "Filed alongside Jupiter"; a section row names its controller; the period row names where the running lord is filed. Without a derived block every row still renders and the "controlled by" and "filed in section" sentences stay silent. A second forbidden-word sweep covers every row builder.
- `DerivedFactsTests` : a chart JSON with no `derived` key decodes with it nil; one with the block decodes the planet, house and dasha values in place; an undeclared `conditions` key does not fail the decode; a `DerivedPeriod` with no `placement` decodes to nil.
- `SuggestionTests` : every mode has three non-empty starters; where the model is unavailable `refresh` falls straight back to them with `isThinking` false; `onDeviceStatus` is never empty; where the model reports itself available, `refresh` settles within a minute on at most three usable questions.
- `PeriodProgressTests` : `testCachedPeriodSurvivesTheRoundTrip` writes a `CachedPeriod` through the real cache and reads it back equal; `testClearRemovesTheCachedPeriod` proves `clear()` removes it.

A GATE NO TEST CAN RUN

`docs/SIRI_AND_WIDGETS.md` section 6 describes a manual check against the real model on a device with Apple Intelligence on: six lookup questions, each of which must be answered on device, not refused, and restated rather than interpreted. As of this version the document marks its status as not yet run. The vocabulary in `ChartFacts` was arrived at by testing, and the table has grown since; the guardrail can trip back into "May contain sensitive content" on wording no unit test can see.

10.9 Study notes

READ IN THIS ORDER

1. `mobile/Sanchara/Sources/Services/ChartCache.swift` , for the container, the four files, the queue and the pending key. Everything else reads or writes through it.
2. `mobile/project.yml` , the `SancharaWidgets` target, for the six app files the extension compiles and the comment on why.
3. `mobile/Sanchara/Sources/Features/Chat/ChartFacts.swift` , for the neutral vocabulary and the UTC date rule.
4. `mobile/Sanchara/Sources/Features/Chat/OnDeviceReasoner.swift` , for the three gates and the tool.
5. `mobile/Sanchara/Sources/Features/Chat/ChatStore.swift` , `send` and `expandLastAnswer` , for where the router meets a turn.
6. `mobile/Sanchara/Sources/Intents/SancharaIntents.swift` then `SancharaShortcuts.swift` , for what Siri can do and the phrases.
7. `mobile/Sanchara/Sources/Intents/DeepLink.swift` and `RootView.swift` , for the two ways a destination arrives.
8. `mobile/SancharaWidgets/ReadingWidget.swift` and `PeriodWidget.swift` , for the timeline policies and the fallbacks.
9. `mobile/Sanchara/Tests/OnDeviceRoutingTests.swift` , which reads as the routing rules in plain English.

CHECK YOURSELF

Why does the widget never fetch, even when its cache is a day old?

A widget that waits on the network shows a placeholder at the moment someone looks. The app and the intents refresh the cache; the widget only reads, and asks to be rebuilt at the next 06:00 or 18:00.

Which intent opens the app, and why is it the only one compiled into the widget extension?

`OpenKundliIntent` . A diagram is the answer and Siri cannot show one. The Control Center button runs it from the extension's process, so it must be compiled there, and it must import nothing but `ChartCache` .

Why does the on-device table say "section" and "segment" instead of "house" and "nakshatra"?

Apple's on-device model refuses anything it reads as fortune telling. Framed as a reference table of positions it answers; `testNothingHandedToTheModelNamesAstrology` pins the wording.

What happens to an on-device answer that never called `lookupTableEntry` ?

It is discarded. `wasCalled` is false, `answer` returns nil, and the question escalates to Claude. An answer composed from memory is a guess.

Why is `pendingDestination` in `UserDefaults` rather than a JSON file like the rest of the cache?

It is one short string written from the extension milliseconds before the app launches; the shared suite gives both processes the same store with no coordination, and `takePendingDestination` reads and clears it in one step.

Why does a Siri question that needs interpretation get queued instead of answered?

Answering would mean a thin restatement or a paid call the person did not ask for and a wait Siri will not sit through. The question is written as `ask:` and the app sends it to Claude with `forceServer` on next launch.

Why must `NatalChart.derived` be optional?

A chart cached in the App Group before derived facts shipped has no `derived` key, and the widget and intent processes cannot report a decode failure; a required field would turn an old cache into a blank widget.

11 Profile, account, and admin

The profile is the birth chart's front door: the details that computed it, a photo, the kundli, and the two account controls that end things. Beside it sits an admin area that can read everything about a user and change nothing.

11.1 Intent

This chapter describes the profile, account deletion and the admin area as they are on `main` on 30 September 2026, version 0.3.0. The profile screen has one job on both clients: show the birth details on file and let a person change them or their photo. Changing the birth data recomputes the chart; that sentence appears verbatim on the web page and in the iOS header, because it is the one consequence a person should know before they edit. Account deletion exists because App Store Review Guideline 5.1.1(v) requires it, and it was built as one route that both clients call. The admin area exists so the author can see what a user's chart, profile and transcripts look like when something goes wrong, without a database console and without any way to alter a user's data from the browser.

Three decisions shape the code. A photo is optional and never travels alone: it rides as one part of the same multipart request as the birth details, and the server overwrites the stored avatar only when a photo part is present. Deletion takes no user id from the request, so a caller can only ever delete themselves, and the database's cascades do the rest. And the admin has read access through the service-role client, but the only actions on offer send a password reset email or mint a one-time login link; nothing on any admin page writes to a user's rows.

11.2 Files and folders

<code>app/(app)/app/profile/page.tsx</code>	the web profile: IntakeForm with initial values
<code>app/(app)/app/intake/actions.ts</code>	saveIntake: avatar upload, then saveBirthProfile
<code>app/api/profile/route.ts</code>	POST: the same work for the phone, returns JSON
<code>app/api/profile/route.test.ts</code>	
<code>app/api/account/route.ts</code>	DELETE: deletes the calling user
<code>app/api/account/route.test.ts</code>	
<code>lib/data/birthProfile.ts</code>	BirthProfileRow, getBirthProfile, saveBirthProfile
<code>components/IntakeForm.tsx</code>	the form, including the photo control
<code>components/AppShell.tsx</code>	sidebar: Profile link and Sign out
<code>components/AdminShell.tsx</code>	admin sidebar: Users and Sign out
<code>app/admin/</code>	
<code>├─ layout.tsx</code>	passthrough, force-dynamic
<code>├─ login/page.tsx</code>	email and password, nothing else
<code>├─ actions.ts</code>	adminSignIn, adminSignOut, sendPasswordReset, generateLogi
<code>├─ page.tsx</code>	the user list
<code>├─ users/[id]/page.tsx</code>	one user: account, birth profile, chats
<code>└─ conversations/[id]/page.tsx</code>	one transcript
<code>lib/admin/</code>	
<code>├─ guard.ts</code>	isAdminUser, requireAdmin (chapter 4)
<code>└─ data.ts</code>	adminListUsers, adminGetUser, adminGetTranscript
<code>supabase/migrations/</code>	
<code>├─ 0001_init.sql</code>	admins, profiles, birth_profiles, cascades

```

└─ 0002_avatar.sql birth_profiles.avatar_url
mobile/Sanchara/Sources/
├─ Features/Profile/ProfileView.swift the sheet: header, details, kundli, account
├─ Features/Profile/AvatarBadge.swift the top-bar control: photo or initials
├─ Features/Intake/PhotoField.swift SancharaPhotoField and UIImage.avatarJPEG
├─ Features/Intake/ProfileStore.swift details and chart, cached to the App Group
├─ Features/Intake/BirthProfileDetails.swift
└─ Services/SancharaAPI.swift saveProfile, deleteAccount, BirthProfileInput
mobile/Sanchara/Tests/ProfilePhotoTests.swift

```

The profile has no route of its own for reading. The web page reads through `getBirthProfile` with the cookie client, and the phone reads `birth_profiles` directly over PostgREST under the owner-only policy. Only writes go through a route or an action, because a write computes the chart with the Swiss Ephemeris and that runs on the server.

11.3 Model

BIRTH_PROFILES

One row per user, enforced by `unique (user_id)`. The columns are the eight birth fields, the computed `chart` as jsonb, `updated_at`, and since migration 0002 a nullable `avatar_url`. `user_id` references `auth.users(id)` on delete cascade.

```

// lib/data/birthProfile.ts
export type BirthProfileRow = {
  id: string;
  user_id: string;
  first_name: string;
  last_name: string;
  birth_date: string;
  birth_time: string;
  place_name: string;
  lat: number;
  lng: number;
  timezone: string;
  avatar_url: string | null;
  chart: { vedic: unknown; western: unknown } | null;
};

```

The iOS counterpart is `BirthProfileDetails`, a `Codable` struct whose coding keys map the snake-case columns. It deliberately omits the chart: `ProfileStore` fetches the details and the roughly 4 KB chart in two queries, the second only once the first says a chart exists. Two computed properties matter here. `fullName` joins and trims the names. `birthTimeShort` takes the first five characters of the Postgres `time`, which arrives as `HH:mm:ss`, giving the `HH:mm` shape the picker and `POST /api/profile` both use. The web page does the same with `String(p.birth_time).slice(0, 5)`.

THE AVATAR

The photo lives in a Supabase Storage bucket called `avatars`, at the path `<user_id>/avatar.<ext>` where the extension is `png`, `webp` or `jpg` by content type. What the row stores is the bucket's public URL for that path with a `?v=<timestamp>` query appended; the comment says "cache-bust so a replaced photo shows immediately". Nothing in the migrations creates the bucket or a storage policy: uploads use the service-role client, and reads are plain fetches of the public URL by the browser, `AsyncImage`, and the admin pages. The code therefore assumes the bucket exists and is public, which is a one-time dashboard step rather than a migration.

THE UPLOAD CONTRACT

```
// mobile/Sanchara/Sources/Services/SancharaAPI.swift
struct BirthProfileInput {
    var firstName = ""
    var lastName = ""
    var birthDate = ""           // yyyy-MM-dd
    var birthTime = ""          // HH:mm
    var placeName = ""
    var lat: Double = 0
    var lng: Double = 0
    var timezone = ""
    /// Optional profile photo, already downscaled to JPEG. The route uploads it
    /// to the `avatars` bucket and only overwrites the stored URL when one is
    /// sent, so leaving this nil keeps the existing photo.
    var photoJPEG: Data?
}
```

`multipartBody(boundary:)` writes the eight text fields as form parts named exactly as the columns, then, only if `photoJPEG` is non-empty, a part named `photo` with filename `avatar.jpg` and content type `image/jpeg`. The same field names are what the web `IntakeForm` posts, with a file input named `photo` that accepts PNG, JPEG and WebP. One server contract, two producers.

ADMINS AND THE ADMIN ROWS

`admins` has two columns, `user_id` and `created_at`, and no write policy. `AdminUserRow` in `lib/admin/data.ts` is what the user list renders: id, email, created and last sign-in from Auth, first and last name, place, birth date and avatar from `birth_profiles`, and a `conversationCount`.

11.4 The server side

SAVING THE PROFILE

Two entry points do the same work. `saveIntake` in `app/(app)/app/intake/actions.ts` is a server action the web form posts to; it reads the user from the cookie client, redirects to `/login` if there is none, uploads the photo if one was sent, calls `saveBirthProfile`, and redirects to `/app`. `POST /api/profile` exists because, as its comment says, a native client cannot invoke a server action. It does the same work with the bearer-aware client, validates the eight required fields and that `lat` and `lng` are finite numbers (400 otherwise), and returns `{ ok: true }` instead of redirecting. Both contain an identical `uploadAvatar`.

```
// app/api/profile/route.ts
let avatarUrl: string | null = null;
const photo = form.get("photo");
if (photo instanceof File && photo.size > 0 && photo.type.startsWith("image/")) {
  avatarUrl = await uploadAvatar(user.id, photo);
}
```

```
// lib/data/birthProfile.ts, inside saveBirthProfile
// Only overwrite the avatar when a new one was uploaded.
if (input.avatarUrl) row.avatar_url = input.avatarUrl;

const { error } = await supabase.from("birth_profiles").upsert(row, { onConflict: "user_id" });
```

These two fragments are the whole avatar rule. No photo part, or an empty one, or one that is not an image, means `avatarUrl` stays null; and a null `avatarUrl` means the upsert row has no `avatar_url` key at all, so the existing value is left standing. `saveBirthProfile` always recomputes both charts (`computeChart` for vedic and western in parallel) and writes them with the row, so any save, including one whose only purpose is a new photo, is a chart computation. An upload that fails returns null from `uploadAvatar` and the save proceeds without touching the photo; the failure is not reported to the client.

DELETING THE ACCOUNT

```
// app/api/account/route.ts
export async function DELETE(request: Request) {
  const supabase = await createRouteSupabase(request);
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) return new Response("Unauthorized", { status: 401 });

  const admin = createAdminSupabase();
  const { error } = await admin.auth.admin.deleteUser(user.id);
  if (error) {
    console.error("account deletion error", error);
    return Response.json({ error: "Could not delete your account. Please try again." }, { status: 400 });
  }

  return Response.json({ ok: true });
}
```

Sixteen lines, and the comment above them carries the two rules: the user id comes from the verified session, never from the request body, so a caller can only ever delete themselves; and the rows cascade from `auth.users`. What actually goes, and in what order:

1. `auth.admin.deleteUser` removes the row in `auth.users`. This is the only statement the route issues.
2. Postgres runs the `on delete cascade` on every foreign key that points at it. In schema order that is `admins`, `profiles`, `birth_profiles`, `conversations`, `device_tokens`, `transit_conditions`, `alerts`, `daily_readings`, `life_events`, `life_event_scans` and `period_readings`.

3. `messages` cascades from `conversations` , one level down, so transcripts go with their conversations.

Two things do not go. The avatar object in the `avatars` bucket is not deleted; no code removes it, and the public URL that pointed to it was in the row that just cascaded away. And the Apple token is not revoked, the gap described in 4.6. Both are worth knowing before an App Store review that exercises deletion.

WHAT THE ADMIN CAN READ

`lib/admin/data.ts` has three functions, all built on `createAdminSupabase` , all read-only.

- `adminListUsers` calls `auth.admin.listUsers` for the first page of up to 1000 users, selects six columns from every `birth_profiles` row and the `id`, `user_id` pair of every conversation, joins them in memory into `AdminUserRow` , and sorts newest first. Three queries regardless of user count.
- `adminGetUser` calls `auth.admin.getUserById` , selects the whole `birth_profiles` row for that user, and lists their conversations newest first with id, tradition, title and created date. The returned `user` carries a `confirmed` boolean derived from `email_confirmed_at` .
- `adminGetTranscript` selects one conversation and its messages in creation order, with role, content and timestamp.

The `chart` jsonb comes back as part of `select("*")` in `adminGetUser` , but the page renders only the six birth fields from it; the chart is never shown. Nothing in the file inserts, updates or deletes.

WHAT THE ADMIN CAN DO

`app/admin/actions.ts` has four server actions. `adminSignIn` and `adminSignOut` are covered in 4.8. The other two both begin with `await requireAdmin()` :

- `sendPasswordReset` calls `resetPasswordForEmail` on the cookie client with a redirect to `/login` . The comment says why this is the shape: the admin never sees or stores the password. The user page repeats this next to the buttons: "Passwords are stored only as secure one-way hashes and cannot be displayed."
- `generateLoginLink` calls `auth.admin.generateLink` with type `magiclink` for the user's email and a redirect to `/app` , then redirects back to the user page with the link in the query string, where it is printed with the instruction to open it in a private window. This is the one capability that lets the author act as a user, and it acts through a normal session under RLS, not through the admin client.

11.5 The clients

WEB

`app/(app)/app/profile/page.tsx` is a server component. It calls `getBirthProfile` , redirects to `/app/intake` if there is no row, and renders a heading, the sentence about recomputing, a "Download kundli (PDF)" link to `/api/kundli` , and the `IntakeForm` with `action={saveIntake}` and every field pre-filled from the row. The form is the same component intake uses (chapter 5); an `initial` prop switches its button label to "Update my details" and seeds the photo preview from `avatarUrl` . Choosing a file replaces the preview with an object URL before anything is uploaded.

The sidebar in `AppShell` has the Profile link, highlighted when the path is `/app/profile` , and the Sign out form at the bottom. There is no account deletion on the web. The route accepts the cookie client and would

work, but no page calls it; deletion was built for the App Store requirement and only the phone offers it today.

IOS: PROFILEVIEW

`ProfileView` is a sheet in a `NavigationStack`, presented over the chat, and its header comment names it the native counterpart of the web page. It takes a `ProfileStore` and reads `AuthStore` from the environment. Top to bottom:

- **The header:** a 64-point circular `PhotosPicker` showing the current photo via `AsyncImage`, or "Add photo", with a small camera badge and a progress overlay while uploading; then `fullName` in the light 28-point face and the recompute sentence.
- **The detail card:** three rows, Born, Place, Timezone. `formattedBirth` parses `birthDate` with a POSIX formatter and renders it long-style with `birthTimeShort`; the comment gives the target shape, "15 June 1995 at 10:30", the moment as it was where they were born.
- **Buttons:** "View your kundli" (disabled and dimmed until `profile.chart` is loaded), "Edit birth details", and, only when there is no chart yet, "Download kundli (PDF)". The comment explains the ordering: the kundli leads now that there is one to look at in the app, and the PDF is reachable from inside the chart instead of as a second competing button.
- **Suggestions:** a status line from `SuggestionEngine.onDeviceStatus` and a note that follow-up questions are written on the device.
- **Account:** Sign out, which calls `auth.signOut()` and dismisses; and "Delete my account" in the accent colour, which sets `confirmDelete` and raises an alert with a Cancel and a destructive Delete.

"Edit birth details" presents `IntakeView` with the current details; on completion the sheet reloads the store and closes. That path is chapter 5's. Changing the photo is deliberately not that path. The header comment says it: routing a photo change through the birth-details form would mean re-submitting a chart to swap a picture.

IOS: THE PHOTO

```
// mobile/Sanchara/Sources/Features/Intake/PhotoField.swift
func avatarJPEG(maxDimension: CGFloat = 1024, quality: CGFloat = 0.8) -> Data? {
    let longest = max(size.width, size.height)
    guard longest > 0 else { return nil }

    let scale = min(1, maxDimension / longest)
    let target = CGSize(width: (size.width * scale).rounded(), height: (size.height * scale).rou
    ...
    return resized.jpegData(compressionQuality: quality)
}
```

`UIImage.avatarJPEG` is the one image function in the app. It scales the longest side down to 1024 points, never up (`min(1, ...)`), renders at scale 1 so the output is not multiplied by the screen density, and encodes JPEG at 0.8. The comment gives the reason: a modern phone photo is several megabytes and is shown at 72 points, and sending the original would make intake feel broken on a slow connection for no visible gain. `SancharaPhotoField`, the intake form's circular control, wraps `PhotosPicker`, which runs out

of process; the app never asks for photo library permission because only the one chosen image is handed over. `ProfileView` uses `PhotosPicker` directly for the same reason.

`AvatarBadge` is the 28-point control in the chat top bar: the photo when there is one, otherwise the first letters of the first and last name, and when even those are absent a star glyph. The comment's rule is "never an empty grey circle". `AsyncImage` uses the initials circle as its placeholder, so the badge never flashes empty while the image loads.

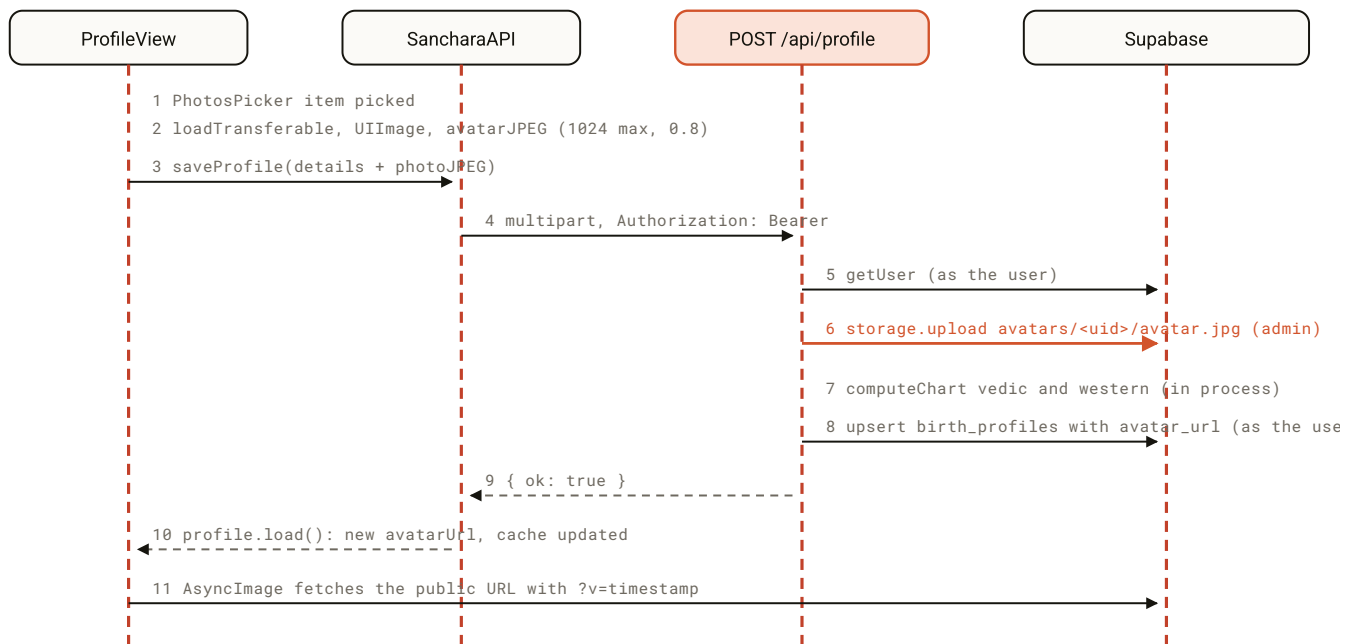
IOS: WHAT IS CACHED

`ProfileStore.load()` begins by reading the chart and the details from `ChartCache` in the App Group, so the profile sheet and the badge draw at once; the network fetch that follows refreshes both and writes them back. If the fetch fails and a cache exists, the state is still `ready`; only a device that has never loaded a profile sees the failure screen. On sign-out `ChartCache.clear()` removes the chart, details, reading and period files, so the next account on the device does not inherit this one's name or photo URL. The avatar image itself is fetched by `AsyncImage` from the public URL and is not written to the App Group.

THE ADMIN PAGES

Three pages, each opening with `await requireAdmin()` and wrapping itself in `AdminShell`, a sidebar with one menu entry, Users, and a Sign out form. `app/admin/page.tsx` lists users newest first with avatar or initials, name or "(no profile yet)", email, join date and chat count. `app/admin/users/[id]/page.tsx` shows an Account card (names, email, joined, last sign-in, email confirmed, user id), the two action buttons, a Birth profile card (date, time, place, timezone, latitude, longitude) or "No birth details entered yet", and the list of chats linking to `app/admin/conversations/[id]/page.tsx`, which renders the transcript as bubbles labelled User and Sanchara. Every page is read from `lib/admin/data.ts`; there is no form on any of them that changes a user's row.

11.6 How it works



A photo change is a full profile save. The upload uses the admin client because there is no storage policy; the row write uses the user's own client and RLS. The chart is recomputed even though nothing about the birth changed.

CHANGE A PHOTO

`ProfileView.uploadPickedPhoto` runs when `photoItem` changes. It loads the item as `Data`, decodes a `UIImage`, and calls `avatarJPEG`; if any step fails the message is "That image could not be read. Try another." It then builds a `BirthProfileInput` from the details already on file, using `birthTimeShort` for the time, attaches the JPEG, and calls `SancharaAPI.saveProfile`. The method's comment states the reason for sending the whole profile: the route takes it as a whole and only replaces the avatar when a photo part is present. On success `profile.load()` refreshes the details, and the new URL, with its fresh `?v=` query, makes `AsyncImage` fetch the replaced image rather than a cached one. A `defer` block clears the picker item and the uploading flag on every exit.

DELETE THE ACCOUNT

1. The alert's destructive button calls `deleteAccount()`, which sets `isDeleting` so the button cannot be tapped twice.
2. `SancharaAPI.deleteAccount()` sends `DELETE /api/account` with the bearer token and checks the status; a non-2xx becomes an error shown under the buttons.
3. The route deletes the auth user and the cascades run (11.4).
4. On success the view calls `auth.signOut()`. Its first step, `PushStore.shared.unregister()`, has nothing left to act on: the user is gone and the `device_tokens` row has already cascaded away. The call is wrapped in `try?`, so whatever the server answers, nothing is lost and nothing is shown. Then the Supabase session is signed out and `ChartCache` is cleared, which removes the deleted person's chart from the widget and Siri.
5. The sheet dismisses and `RootView` shows `AuthView`.

AN ADMIN SESSION

1. The author opens `/admin`. The middleware finds no session and redirects to `/admin/login`.
2. `adminSignIn` signs in with a password, checks `admins`, and redirects to `/admin`. A non-admin account is signed out again and told "Not authorized".
3. The users page calls `requireAdmin` then `adminListUsers`; each user links to their page, each chat to its transcript. Every read goes through the service-role client because the rows belong to other people.
4. To see the app as a user, "Generate login link" mints a magic link and prints it. Opened in a private window, it goes through `/auth/callback` like any other link and produces an ordinary session for that user. From then on the private window is that user, under RLS, with no admin powers.

11.7 Performance and cost

- **A photo change costs a chart.** `saveBirthProfile` always runs `computeChart` twice, so swapping a picture invokes the ephemeris. It runs in process inside the route and completes in the same request, so the cost is server time rather than a slow round trip, but it is the reason the profile route declares `runtime = "nodejs"`. Splitting a photo-only endpoint off would remove it.
- **Uploads are bounded on the phone.** 1024 points on the longest side at 0.8 quality; `testDownscalesLargePhotos` asserts a 3000 by 2000 source comes out under half a megabyte. The web form sends whatever file was chosen, with no client-side resize, and the route accepts any `image/*` of any size.
- **Reads are cached on the phone.** The profile sheet draws from the App Group cache before the network answers, and the details query selects nine columns without the chart, which is fetched separately and only when needed.
- **The public URL carries a cache-buster.** The `?v=` timestamp is stored in the row, not computed on read, so every client sees the same URL until the next upload and the image is cached by the browser and by `URLSession` until then.
- **Admin reads are three queries for the list**, regardless of user count, joined in memory; and `listUsers` asks for one page of 1000, so past that the list is silently truncated. The user page is three more, and the chart jsonb rides along in `select(*)` unused.
- **Deletion is one statement.** The cascades run inside Postgres; the route does not enumerate tables and does not need to change when a table is added, provided the new table references `auth.users` with `on delete cascade` as all eleven do today.

11.8 Edge cases and tests

CONDITION	BEHAVIOUR
Profile page opened with no birth row	Web redirects to <code>/app/intake</code> ; iOS <code>ProfileStore</code> goes to <code>needsIntake</code> and <code>SignedInView</code> shows intake instead
Save with no photo part	<code>avatarUrl</code> is null, the upsert row omits <code>avatar_url</code> , the existing photo stays
Save with an empty file input on the web	<code>photo.size > 0</code> fails; treated as no photo
A non-image file named <code>photo</code>	<code>type.startsWith("image/")</code> fails; treated as no photo
Storage upload fails	<code>uploadAvatar</code> returns null; the birth details still save; the failure is logged nowhere and not reported
Picked image cannot be decoded on iOS	"That image could not be read. Try another." and no request is sent
Same photo path uploaded twice	<code>upsert: true</code> on the storage call replaces the object; the new <code>?v=</code> query defeats caches
Photo changed from JPEG to PNG on the web	A second object at <code>avatar.png</code> is created; the old <code>avatar.jpg</code> is left in the bucket
Missing required field on <code>POST /api/profile</code>	400 with the field name; <code>saveBirthProfile</code> not called
<code>lat</code> not numeric	400; <code>saveBirthProfile</code> not called
Chart computation throws	500 with "Could not compute your chart. Please try again."; the avatar, if any, was already uploaded
Delete without a session	401; <code>deleteUser</code> never called
<code>deleteUser</code> returns an error	500 with a generic message; the error is logged; nothing was deleted because the one statement failed
Delete succeeds, then push unregister fails	Expected: the user is gone and the token row cascaded; <code>try?</code> swallows it
Avatar object after deletion	Remains in the bucket under the old user id; nothing references it
Non-admin account signs in at <code>/admin/login</code>	Signed out immediately, "Not authorized"
Admin row removed mid-session	The next page load's <code>requireAdmin</code> redirects to login
User with no birth profile in the admin list	"(no profile yet)" with initials from the email
More than 1000 users	The list shows the first page only

TESTS

- `app/api/account/route.test.ts` mocks `createRouteSupabase` and `createAdminSupabase` and pins three things: an unauthenticated request returns 401 and `deleteUser` is not called; an authenticated

request returns 200 with `{ ok: true }` and `deleteUser` is called with exactly the session's user id ("deletes only the calling user"); and an error from `deleteUser` becomes a 500.

- `app/api/profile/route.test.ts` pins the 401 with no user, a 200 for a valid payload with `saveBirthProfile` receiving the parsed `lat` as a number and the bearer client as its second argument, a 400 when `timezone` is missing, and a 400 when `lat` is not a number.
- `mobile/Sanchara/Tests/ProfilePhotoTests.swift`, whose header says what it guards: the contract with `/api/profile`, which only overwrites the avatar when a `photo` part is present.
`testDownscalesLargePhotos` : a 3000 by 2000 image comes out with its longest side at 1024, its aspect ratio at 1.5, and under 500,000 bytes. `testLeavesSmallPhotosAlone` : a 400 by 400 image is not upscaled. `testMultipartCarriesThePhotoWhenOneIsPicked` : the body contains a part named `photo` with filename `avatar.jpg`, the `image/jpeg` content type, the image bytes, the text fields, and a terminating boundary. `testMultipartOmitsThePhotoPartWhenNoneIsPicked` : with no JPEG there is no `photo` part at all, with the assertion message "sending an empty photo part would clear a profile photo the user meant to keep".

Not covered by a test: `uploadAvatar` itself, the admin data functions, and the admin pages. The first would need a storage stub; the second and third depend on the Auth admin API.

11.9 Study notes

READ IN THIS ORDER

1. `lib/data/birthProfile.ts` , for `saveBirthProfile` and the one line that decides whether the avatar changes.
2. `app/api/profile/route.ts` , then `app/(app)/app/intake/actions.ts` , to see the same work done twice for two callers.
3. `mobile/Sanchara/Sources/Services/SancharaAPI.swift` , the `BirthProfileInput` struct and `multipartBody` at the bottom.
4. `mobile/Sanchara/Sources/Features/Intake/PhotoField.swift` , for `avatarJPEG` .
5. `mobile/Sanchara/Sources/Features/Profile/ProfileView.swift` , `uploadPickedPhoto` and `deleteAccount` in particular.
6. `app/api/account/route.ts` and `supabase/migrations/0001_init.sql` side by side, following every `on delete cascade` .
7. `lib/admin/data.ts` and `app/admin/actions.ts` , to confirm for yourself what the admin can and cannot do.
8. `mobile/Sanchara/Tests/ProfilePhotoTests.swift` , which reads as the upload contract in plain English.

CHECK YOURSELF

How does the server know whether to keep or replace the avatar?

By the presence of a `photo` part that is a non-empty image. If there is none, `avatarUrl` is null and `saveBirthProfile` omits `avatar_url` from the upsert, so the column keeps its value.

Why does the iOS photo change send all eight birth fields?

Because the route takes the profile as a whole and there is no photo-only endpoint. The view builds the input from the details already on file so nothing changes but the picture.

What does `DELETE /api/account` delete, and what does it leave?

It issues one `auth.admin.deleteUser` ; every table referencing `auth.users` cascades, and `messages` cascades from `conversations` . It leaves the avatar object in storage and does not revoke the Apple token.

Why does the profile save recompute the chart when only the photo changed?

Because `saveBirthProfile` has one path and it always computes both charts before the upsert. The cost is accepted for simplicity; a separate avatar endpoint would avoid it.

Which admin action changes a user's row?

None. The two actions send a password reset email and mint a magic link; both go through Auth, and neither touches a table.

Why is the upload done with the admin client while the row write is done as the user?

There is no storage policy for users, so the bucket write needs the service role; the path is built from the verified user id. The row is under RLS, so the user's own client is enough and keeps the rule that a user writes only their own row.

12 The landing page and the web shell

Four colours on `:root`, one photograph of a planet, and a sidebar that lists past readings. Everything a person sees before and around a reading is in this chapter, and the native app copies its tokens from the same stylesheet so that a screen on the phone and a page in the browser read as one product.

This chapter describes the web surface of Sanchara as it is on `main` on 30 September 2026, version 0.3.0. It covers the root layout, the marketing page at `/`, the signed-in shell under `/app`, the auth form, the two build-time configuration files that shape the whole site, and, for comparison, the iOS design system that translates the site's tokens into SwiftUI. The chapters before this one each own a feature; this one owns the frame the features sit in.

12.1 Intent

The web app has two audiences and one theme. A stranger arriving at `/` should understand in one screen that the astrology is computed, not guessed, and be one tap from an account. A signed-in person should never see the marketing again: `/` sends them straight to `/app`, where the shell shows their readings, a button to start another, and a link to their profile. Both audiences see the same dark cosmic theme, because there is only one. The site has no light mode, and the iOS app is pinned dark in its `Info.plist` for the same reason: a reading card that turned white inside Siri while the app stayed black would look like someone else's product.

Three decisions shaped the code. The theme is four CSS custom properties and nothing else, so it can be copied by hand into Swift and checked by a test. The landing page's animation is loaded on the client only, after the HTML has arrived, so the first paint is a photograph and never a blank canvas. And the shell is the only client component in the signed-in frame; every page inside it is a server component that reads Supabase directly and hands props down.

12.2 Files and folders

app/	
├─ layout.tsx	html, body, metadata, viewport; imports globals.css
├─ globals.css	Tailwind 4 import, the four tokens, animations, .prose-reading
├─ page.tsx	"/": redirect to /app when signed in, else the landing
├─ (auth)/	
│ ├─ layout.tsx	centred card with the glow behind it
│ ├─ actions.ts	authenticate, sendMagicLink, resendConfirmation, signOut
│ ├─ login/page.tsx	AuthForm, "Welcome to Sanchara"
│ ├─ signup/page.tsx	AuthForm, "Create your account" (same action)
│ └─ check-email/page.tsx	confirmation screen with a resend button
├─ (app)/	
│ ├─ layout.tsx	force-dynamic; lists conversations; renders AppShell
│ └─ app/	
│ ├─ page.tsx	the signed-in home
│ ├─ chat/page.tsx	Chapter 6
│ └─ intake/	Chapter 5

	└─ profile/page.tsx	Chapter 11
	└─ admin/	AdminShell and the admin pages (Chapter 4)
	└─ api/	route handlers, one folder per feature
	└─ auth/callback/route.ts	exchanges the magic-link code for a session
components/		
	└─ AppShell.tsx	client: sidebar, mobile drawer, sign-out form
	└─ AuthForm.tsx	server: one form, two server actions
	└─ marketing/	
	└─ CosmicLanding.tsx	client: dynamic import of CosmicHero, SSR off
	└─ CosmicHero.tsx	client: GSAP intro and scroll reveals
	└─ Hero.tsx	unused since the cinematic hero landed
	└─ HowItWorks.tsx	unused
	└─ CTA.tsx	unused
middleware.ts		
next.config.ts		
postcss.config.mjs		
public/images/planet-sun.jpg		
public/images/starfield.jpg		
mobile/Sanchara/Sources/Design/		
	└─ Theme.swift	the four tokens, hairline, field fill, radius, easing
	└─ Components.swift	SancharaField, SancharaPrimaryButton, SancharaSecondaryButton,
	└─ ReadingCard.swift	the card shared by the Siri snippet and the widget
mobile/Sanchara/Tests/ThemeTests.swift		

Three files in `components/marketing/` are dead. `Hero.tsx`, `HowItWorks.tsx` and `CTA.tsx` are the first landing page, built with `next/image`; nothing imports them today. The history of the folder tells the story: a WebGL scene built with `three.js` and GSAP replaced them, then commit `c0a209d` ("Hero: remove 3D WebGL, keep planet image with animated text") took the scene out again and kept the photograph and the animated headline. `three` and `@types/three` are still in `package.json`, and the comment at the top of `CosmicLanding.tsx` still says WebGL, but no file imports `three.js`, so nothing of it reaches a bundle.

12.3 Model

THE THEME IS FOUR NUMBERS

```
/* app/globals.css */
@import "tailwindcss";

:root {
  --bg: #0a0a0b;
  --fg: #f4f1ea;
  --muted: #9a978f;
  --accent: #e8663d;
}

@theme inline {
  --color-bg: var(--bg);
  --color-fg: var(--fg);
  --color-muted: var(--muted);
  --color-accent: var(--accent);
  --font-display: ui-sans-serif, system-ui, sans-serif;
}
```

Near-black, warm off-white, a grey with the same warmth, and one orange. The `@theme inline` block is how Tailwind 4 learns about them: it turns each `--color-*` variable into a utility family, so `bg-bg`, `text-fg`, `text-muted` and `bg-accent` exist without a configuration file. There is no `tailwind.config.*` in the repo. `postcss.config.mjs` is one line, registering `@tailwindcss/postcss`, and everything else about the theme is CSS.

The rest of `globals.css` is small and each rule carries its reason. `html, body` set the background and foreground from the tokens and `overflow-x: hidden` with `max-width: 100%`, because a landing page with a headline wider than a phone otherwise scrolls sideways. Form controls get `max-width: 100%; min-width: 0`, with the comment that iOS date and time inputs have a large intrinsic width that otherwise overflows the screen. Below 768 pixels every input is forced to `16px`, because iOS Safari zooms the page when a smaller control takes focus. Then the animations: `fade-up`, `fade-in`, `glow-pulse`, `heartbeat`, `blink` and `shimmer`, exposed as `.animate-*` classes with three delay steps of 0.08, 0.16 and 0.24 seconds. `.prose-reading` styles the Markdown that the chat renders (chapter 6). A `prefers-reduced-motion` block turns off `fade-up`, `fade-in` and `glow`; the heartbeat, shimmer and caret animations are not in that block and keep running.

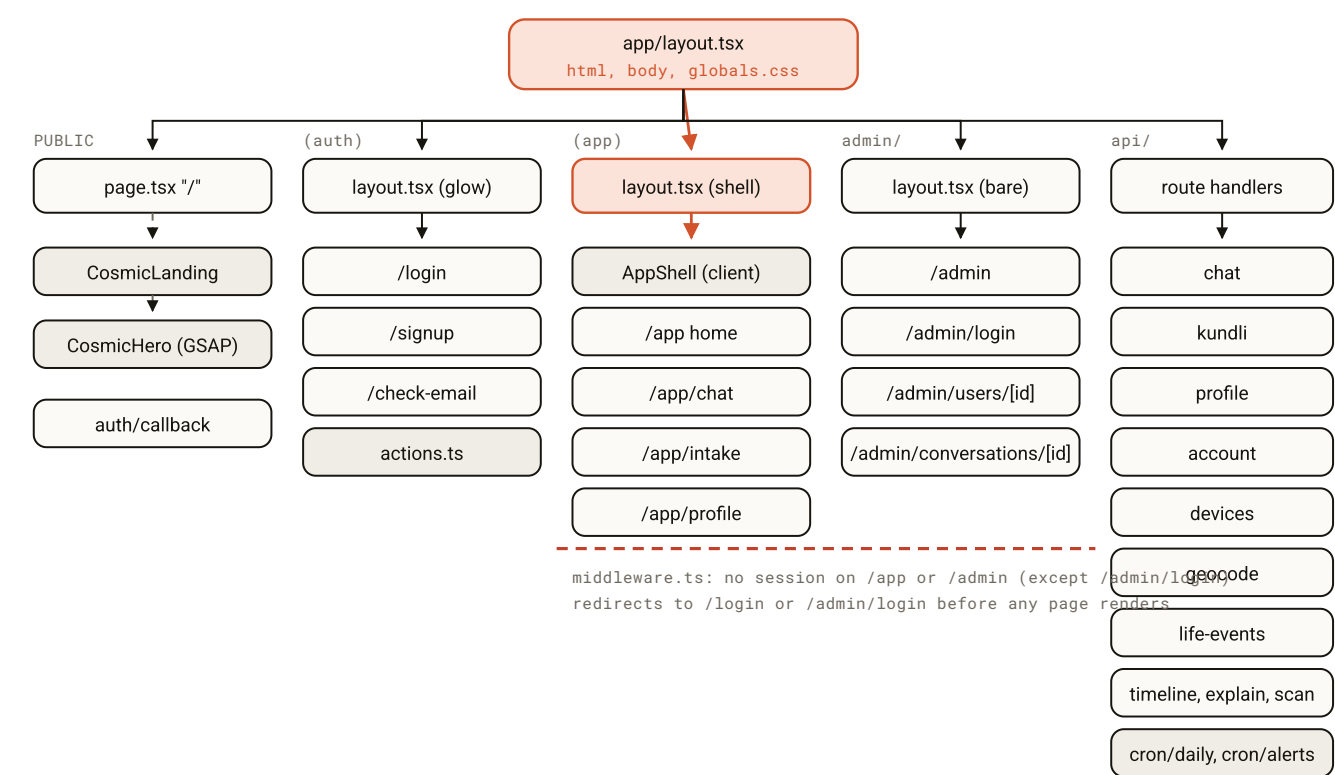
THE SWIFT COPY

```
// mobile/Sanchara/Sources/Design/Theme.swift
enum Theme {
    /// `--bg: #0a0a0b`
    static let bg = Color(hex: 0x0A0A0B)
    /// `--fg: #f4f1ea`
    static let fg = Color(hex: 0xF4F1EA)
    /// `--muted: #9a978f`
    static let muted = Color(hex: 0x9A978F)
    /// `--accent: #e8663d`
    static let accent = Color(hex: 0xE8663D)

    /// Tailwind `border-white/10`, used for every hairline on the site.
    static let hairline = Color.white.opacity(0.10)
    /// Tailwind `bg-white/[0.04]`, the fill behind form controls.
    static let fieldFill = Color.white.opacity(0.04)
    /// Tailwind `rounded-lg` is 0.5rem.
    static let cornerRadius: CGFloat = 8
    static let chartLine = Color.white.opacity(0.22)
    /// The site's easing: `cubic-bezier(0.22, 1, 0.36, 1)`
    static let ease = Animation.timingCurve(0.22, 1, 0.36, 1, duration: 0.6)
}
```

The file's own header says it is "translated verbatim from `app/globals.css`", and the doc comment above each token quotes the CSS line it copies. The `Color(hex:)` initialiser exists so a token can be written the way it appears in CSS, as a 24-bit literal in sRGB. Two things are added that the CSS never names as tokens: the hairline and the field fill are Tailwind opacity utilities on the site (`border-white/10` , `bg-white/[0.04]`), and Swift has to give them a name. `chartLine` is heavier than the hairline, at 0.22 white, and the comment explains why: the kundli square "all but vanished on an OLED screen in daylight" at 0.10. `Text.eyebrow()` reproduces the site's eyebrow style (`text-xs uppercase tracking-[0.3em] text-muted`) as 12 points with 3.6 points of tracking, which is 0.3em at that size.

THE ROUTE TREE



Muted boxes are client components, server actions, or the cron routes only Vercel calls. Every page is a server component. The api/ branch is covered feature by feature in chapters 5 to 11; cron/ is chapter 9.

One root layout, then five branches. The two route groups in parentheses add a layout without adding a path segment: (auth) centres a card, (app) wraps every page in the shell. The admin layout is a passthrough so the login page renders bare.

12.4 The server side

THE ROOT LAYOUT

```
// app/layout.tsx
export const metadata: Metadata = {
  title: "Sanchara: Your chart, read by the stars",
  description: "Sign up, share your birth details, and chat with an astrologer powered by your r
};

export const viewport = {
  width: "device-width",
  initialScale: 1,
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body className="min-h-screen antialiased">{children}</body>
    </html>
  );
}
```

That is the whole file, plus the `import "../globals.css"` above it. It is a server component with no data access; the title and description are the only metadata the site declares, and no page overrides them. The viewport export is what keeps iOS from rendering the site at a desktop width and shrinking it.

THE LANDING ROUTE

```
// app/page.tsx
export default async function Landing() {
  const supabase = await createServerSupabase();
  const { data: { user } } = await supabase.auth.getUser();
  if (user) redirect("/app");
  return <CosmicLanding />;
}
```

`/` is a server component that does one thing before rendering: it asks Supabase who the visitor is, using the cookie client from `lib/supabase/server.ts`. A signed-in visitor is redirected on the server and never receives the landing HTML. Everyone else receives `<CosmicLanding />`, which is a client component, so the server's contribution to the page is the root layout and whatever the client component's fallback renders on the server (12.5).

THE MIDDLEWARE

`middleware.ts` runs `updateSession` from `lib/supabase/middleware.ts` on every request except static assets, images, the favicon and `.well-known/`. It creates a server client over the request cookies, calls `getUser()` so the session cookie is refreshed, and then applies one rule: a request to a path starting with

`/app` , or with `/admin` other than `/admin/login` , with no user is redirected to `/login` or `/admin/login` . This is why none of the pages under `(app)` check for a session themselves. They check for a birth profile, which is a different question, and one the middleware cannot answer.

THE SIGNED-IN LAYOUT AND HOME

```
// app/(app)/layout.tsx
export const dynamic = "force-dynamic";

export default async function AppLayout({ children }: { children: React.ReactNode }) {
  const conversations = await listConversations();
  return <AppShell conversations={conversations as never}>{children}</AppShell>;
}
```

`force-dynamic` tells Next never to cache any page under this layout; every request renders fresh, which is right for a frame whose sidebar lists the person's own conversations. `listConversations` (in `lib/data/chat.ts`) selects `id`, `tradition`, `title`, `created_at` ordered newest first, with no user filter in the query: Row Level Security supplies it (chapter 13). The `as never` cast is there because the query returns loosely typed rows and the shell declares a stricter `Conv` type; it is a cast, not a validation, and if a column were renamed the shell would receive `undefined` rather than an error.

The home at `app/(app)/app/page.tsx` calls `getBirthProfile()` and branches on the result. With no profile it shows "Let's build your chart" and a button to `/app/intake` in the foreground colour. With a profile it shows the avatar if there is one, "Hello," followed by the first name, an accent-coloured "Start a reading" button to `/app/chat` , and a plain link to `/api/kundli` that downloads the PDF (chapter 7). The avatar is a plain `<img>` , with an ESLint disable comment above it, because it is a public Supabase Storage URL and `next/image` would need the host allow-listed. Behind both states sits `Glow` , a blurred radial gradient from a violet at 22% opacity through the accent at 6% to transparent. The same gradient string appears in `app/(auth)/layout.tsx` and `app/(app)/app/intake/page.tsx` ; it is copied, not shared.

THE TWO CONFIGURATION FILES

```
// next.config.ts
const nextConfig: NextConfig = {
  // swisseph-wasm ships a .wasm binary; keep it external so Next bundles it for the server runtime
  serverExternalPackages: ["swisseph-wasm"],

  async headers() {
    return [
      {
        source: "/.well-known/apple-app-site-association",
        headers: [{ key: "content-type", value: "application/json" }],
      },
    ];
  },
};
```

Two settings, each for a different chapter. `serverExternalPackages` keeps the Swiss Ephemeris package out of the bundler so its `.wasm` file is loaded from `node_modules` at run time (chapter 2). The header fixes a platform quirk: Apple requires the app-site-association file to be served as JSON, the file has no extension, and Next would otherwise serve it as `application/octet-stream`. That file is what lets iOS open universal links in the app (chapter 14). The middleware matcher excludes `.well-known/` so the session code never runs for it.

12.5 The clients

THE LANDING, IN TWO CLIENT COMPONENTS

```
// components/marketing/CosmicLanding.tsx
"use client";
import dynamic from "next/dynamic";

const CosmicHero = dynamic(() => import("./CosmicHero"), {
  ssr: false,
  loading: () => (
    <div className="min-h-[100svh] bg-cover bg-center" style={{ backgroundImage: "url(/images/pl
  )",
  });

export function CosmicLanding() {
  return <CosmicHero />;
}
```

`CosmicLanding` exists only to do this import. `ssr: false` means the server never renders `CosmicHero`; the HTML for `/` contains the `loading` element, a full-height div with the planet photograph as its background, and nothing else. The browser downloads the hero's chunk, which carries GSAP and its ScrollTrigger plugin, and swaps it in. The design intent in the file's comment is that "nothing looks blank" while that happens, and the photograph does that job.

`CosmicHero` is data plus one effect. The data is a `HERO` constant (eyebrow "Computed astrology, never guessed", title "SANCHARA", one line of copy, two links: "Get your reading" to `/signup` and "Log in" to `/login`) and three `SECTIONS` ("THE SCIENCE", "YOUR CHART", "YOUR FUTURE"), the last with a "Create your free account" button. The hero section is `min-h-[100svh]` with the planet photograph as a CSS background; each dark section is another full viewport in `bg-bg`. The title is split into one `<span class="ch">` per character and starts with `visibility: hidden` as an inline style, so the letters are not seen before they animate.

```
// components/marketing/CosmicHero.tsx
useEffect(() => {
  const reduce = window.matchMedia?.("(prefers-reduced-motion: reduce)").matches;
  if (reduce) {
    if (titleRef.current) titleRef.current.style.visibility = "visible";
    return;
  }
  gsap.registerPlugin(ScrollTrigger);
  const ctx = gsap.context(() => {
    if (titleRef.current) {
      gsap.set(titleRef.current, { visibility: "visible" });
      gsap.from(titleRef.current.querySelectorAll(".ch"), {
        y: 120, opacity: 0, duration: 1.1, stagger: 0.05, ease: "power4.out",
      });
    }
    gsap.utils.toArray<HTMLElement>(".reveal").forEach((el) => {
      gsap.from(el.children, {
        y: 60, opacity: 0, duration: 1, stagger: 0.12, ease: "power3.out",
        scrollTrigger: { trigger: el, start: "top 78%" },
      });
    });
  }, root);
  return () => ctx.revert();
}, []);
```

The effect reads the reduced-motion preference first and, if set, makes the title visible and does nothing else: no plugin registration, no tweens, no scroll listeners. Otherwise the eight letters rise 120 pixels over 1.1 seconds with a 50 millisecond stagger, and each `.reveal` block's children rise 60 pixels when the block's top crosses 78% of the viewport height. Everything is created inside `gsap.context` scoped to the root element, so the cleanup is one call to `ctx.revert()` when the component unmounts. No three.js scene is created anywhere; the "cosmic" in the name is the photograph and the black sections.

THE SHELL

`AppShell` is the one client component in the signed-in frame, and it is a client component for three reasons only: it holds the open state of the mobile drawer, it reads the `?c=` search parameter to highlight the active conversation, and it reads the pathname to highlight Profile. It receives the conversation list as a prop and fetches nothing.

The navigation column is built once as `nav` and placed twice. On a medium screen and up it is a 256 pixel aside with a hairline right border. Below that it is hidden, a header with a hamburger button (labelled "Open menu" for assistive technology) appears, and tapping it renders a fixed overlay: a 60% black backdrop that closes on click, and a drawer 288 pixels wide, capped at 82% of the screen. Every link in the column closes the drawer on click, so navigating on a phone never leaves it open over the new page. The column holds the brand link to `/app`, a "New reading" link to `/app/chat`, a Profile link, the "Past readings" list, and a sign-out form whose `action` is the `signOut` server action imported from `app/(auth)/actions.ts`. A conversation is highlighted when its id matches `?c=` and the pathname is exactly `/app/chat`; a conversation with no title is shown as "Reading". The outer frame is `h-dvh` with `overflow-hidden`, and `<main>` is `min-h-0 flex-1`, which is what lets the chat transcript scroll inside the frame instead of the page scrolling under the sidebar.

THE AUTH FORM

```
// components/AuthForm.tsx (props)
{
  title: string;
  subtitle?: string;
  primaryLabel: string;
  primaryAction: (fd: FormData) => void;
  magicAction?: (fd: FormData) => void;
  footer: React.ReactNode;
  error?: string;
  notice?: string;
}
```

`AuthForm` has no `"use client"` directive and no state. It is a server component rendering one form with an email field, a password field, a primary button with `formAction={primaryAction}` and, when `magicAction` is given, a second button "Email me a magic link instead" with its own `formAction`. Two buttons in one form, each bound to a different server action, is how a single email field serves both a password sign-in and a magic link without any client JavaScript. `/login` and `/signup` both render it with the same `authenticate` action; only the title and subtitle differ, because the action itself decides whether to sign in or sign up (chapter 4). Errors arrive as `?error=` in the URL and are printed in the accent colour above the form; `?sent=1` prints the magic-link notice in muted text. The inputs use the same recipe as every field on the site: `bg-white/[0.04]`, `border-white/10`, `rounded-lg`, and the border turns accent on focus.

THE IOS DESIGN SYSTEM

The Swift files under `mobile/Sanchara/Sources/Design/` are the same recipe written for SwiftUI.

`SancharaField` is the input: 16 point text, 12 by 14 points of padding, `Theme.fieldFill` behind it, a `Theme.hairline` stroke, and an 8 point radius. Its doc comment notes that the web forces 16 pixels on phones so Safari does not zoom, that native has no such bug, and that "the size is the look".

`SancharaPrimaryButton` is `bg-fg text-bg rounded-lg`: foreground fill, background-coloured text, full width, a spinner in place of the label while loading and disabled while it shows. `SancharaSecondaryButton` is the outlined button with white at 0.15 for its stroke and muted 14 point text. `SancharaSecureField` adds an eye toggle the web form does not have, with the comment that choosing a password you cannot see "is the most common reason a sign-up attempt fails twice".

`ReadingCard` is the one component shared by the Siri snippet and the Home Screen widget (chapter 10): an eyebrow at 10 points with 2 points of tracking, a serif headline in `Theme.fg`, and body text at 13 points in `Theme.muted` with a line limit that defaults to 6 because "widgets get less room than a Siri snippet does". It sets its own colours rather than reading the environment, and its header comment gives the reason this chapter opened with: the app is dark-only by design, `UIUserInterfaceStyle: Dark` in `Info.plist` and in `mobile/project.yml`, and a card that went light inside Siri would read as a different product. Its accessibility label concatenates the three strings so VoiceOver reads the card as one element.

12.6 How it works

AN ANONYMOUS VISIT TO /

1. The middleware runs. There is no session cookie, the path is not under `/app` or `/admin`, so the request passes through.
2. `app/page.tsx` renders on the server, calls `getUser()`, finds nobody, and returns `<CosmicLanding />`.
3. The HTML that leaves the server is the root layout around the `loading` fallback: a full-viewport div showing `planet-sun.jpg`. No headline, no links.
4. The browser hydrates, requests the `CosmicHero` chunk, and mounts it. The title's spans exist but are hidden.
5. The effect runs. Without a reduced-motion preference GSAP makes the title visible and the letters rise; three ScrollTriggers wait for the dark sections. With the preference set, the title is simply shown.
6. "Get your reading" and "Log in" are ordinary `next/link` anchors to `/signup` and `/login`. Both routes render `AuthForm` inside the `(auth)` layout's centred grid with the glow behind it.

A SIGNED-IN VISIT TO /

1. The middleware refreshes the session cookie by calling `getUser()`.
2. `app/page.tsx` calls `getUser()` again, finds a user, and `redirect("/app")` throws before any component renders. The landing chunk is never requested.
3. `app/(app)/layout.tsx` renders: `listConversations()` runs under RLS for that user and the result is handed to `AppShell`.
4. `app/(app)/app/page.tsx` renders inside it: `getBirthProfile()` runs, and if the stored Vedic chart predates the whole-sign change, `ensureCurrentChart` recomputes both traditions and rewrites the row before the page is returned (chapter 2).
5. The home shows either the intake prompt or the greeting with "Start a reading".

SIGNING OUT FROM THE SHELL

The "Sign out" button is inside a `<form action={signOut}>`. Submitting it calls the server action, which creates the cookie client, calls `supabase.auth.signOut()`, and `redirect("/login")`. The next request to anything under `/app` has no session and the middleware sends it to `/login` as well. No client code is involved beyond the form submission.

12.7 Performance and cost

- **The landing ships one animation library.** The client chunk for the hero carries GSAP 3.15 and ScrollTrigger and nothing else of note. `three.js` 0.169 is installed and unused, and Next bundles only what is imported, so it costs install time, not bytes on the wire. Removing it from `package.json` would be a one-line cleanup with no behaviour change.
- **The first paint is a 141,888 byte JPEG.** `CosmicHero` and its fallback both load `planet-sun.jpg` as a CSS background, so `next/image` does no resizing or format conversion for it. The unused `Hero.tsx` did use `next/image` with `priority`; that optimisation went away with the component. `starfield.jpg` is referenced only by the unused components and is never requested.

- **Nothing above the fold is server-rendered on /.** With `ssr: false` the headline and the two calls to action exist only after hydration. A crawler that does not run JavaScript sees a photograph and the metadata title. That is a trade the code makes for a paint that is never blank; it is worth knowing when reading a search report.
- **Two session checks per request.** The middleware calls `getUser()` on every matched request and `app/page.tsx` calls it again. With a session cookie present, each is a verification round trip to Supabase Auth.
- **Every /app page is dynamic.** `force-dynamic` on the layout means each render runs `listConversations()`; the home, chat and profile pages each add `getBirthProfile()`. Two queries per navigation, both indexed (`conversations (user_id, created_at desc)`, `birth_profiles` unique on `user_id`), and the shell itself makes no client fetch.
- **The chart upgrade is on the read path.** A stale stored chart costs two Swiss Ephemeris computations and one update the first time its owner loads any page that reads the profile. It happens once per profile.
- **Reduced motion is free.** The preference short-circuits the effect before GSAP registers its plugin, so nothing is created that then has to be reverted.
- **The theme costs nothing to keep in sync.** Four hex values copied by hand and pinned by four unit tests. There is no build step between the CSS and the Swift.

12.8 Edge cases and tests

CONDITION	BEHAVIOUR
Signed-in person opens /	Server redirect to <code>/app</code> ; the landing chunk is never sent
Signed-out person opens <code>/app/anything</code>	Middleware redirects to <code>/login</code> before the page renders
Signed-out person opens <code>/admin/login</code>	Allowed; every other <code>/admin</code> path redirects there
Signed in, no birth profile	Home shows the intake prompt; <code>/app/chat</code> redirects to <code>/app/intake</code> when <code>profile.chart</code> is missing
Signed in, profile without an avatar	The <code></code> is not rendered at all, so there is no broken image
Conversation with a null title	Listed as "Reading" in the sidebar and in the link's <code>title</code> attribute
prefers-reduced-motion	The hero shows the title without GSAP; CSS disables <code>fade-up</code> , <code>fade-in</code> and <code>glow</code> . <code>heartbeat</code> , <code>shimmer</code> and the caret <code>blink</code> are not disabled
JavaScript disabled, or before hydration	/ shows the planet photograph only; no headline or links are in the HTML
Phone narrower than 768px	Sidebar hidden; hamburger opens a drawer capped at 82% width; any link closes it
iOS date or time input	<code>max-width: 100%</code> and <code>appearance: none</code> stop the intrinsic width overflowing; 16px text stops Safari zooming on focus
Headline wider than the viewport	<code>break-words</code> on the title plus <code>overflow-x: hidden</code> on <code>body</code> ; no sideways scroll
Request for <code>/.well-known/apple-app-site-association</code>	Served as <code>application/json</code> ; the middleware does not run for it
The Next data cache	Never used under <code>/app</code> : <code>force-dynamic</code> on the layout

TESTS

- `app/smoke.test.ts` : one test, "layout module loads", which imports the default export of `app/layout.tsx` and asserts it is a function. The import is named `metadata` in the test, but it is the default export, which is `RootLayout` ; the metadata object itself is not asserted on. This is the only web test that touches anything in this chapter.
- `mobile/Sanchara/Tests/ThemeTests.swift` : `testBackgroundMatchesCSSToken` , `testForegroundMatchesCSSToken` , `testMutedMatchesCSSToken` and `testAccentMatchesCSSToken` . Each converts the SwiftUI `Color` to a `UIColor` , reads its components, rounds them to 0 to 255, and compares with the hex bytes from `globals.css` . The class comment states the premise: a careless edit to `Theme.swift` "fails the build rather than quietly drifting". The tests pin the Swift side only; an edit to the CSS would not fail them.
- No test renders `CosmicHero` , `AppShell` or `AuthForm` . The behaviour in the table above is read from the code, not proven by a suite.

12.9 Study notes

READ IN THIS ORDER

1. `app/globals.css` , for the four tokens and every rule that exists because of a phone.
2. `app/layout.tsx` and `app/page.tsx` , the whole server side of the public site in thirty lines.
3. `components/marketing/CosmicLanding.tsx` , then `CosmicHero.tsx` , for the dynamic import and the one effect.
4. `middleware.ts` and `lib/supabase/middleware.ts` , the guard that lets the pages skip their own session checks.
5. `app/(app)/layout.tsx` and `components/AppShell.tsx` , the frame and why it is the only client component in it.
6. `app/(app)/app/page.tsx` , the two states of the home.
7. `components/AuthForm.tsx` with `app/(auth)/login/page.tsx` , two buttons and two server actions in one form.
8. `mobile/Sanchara/Sources/Design/Theme.swift` and `ThemeTests.swift` , the copy and the test that pins it.
9. `next.config.ts` , last, because both of its settings belong to other chapters.

CHECK YOURSELF

Why is the hero loaded with `ssr: false` , and what does the visitor lose?

GSAP needs the DOM and the code was written for a browser-only scene, so the component is rendered only on the client. The visitor loses server-rendered headline text: before hydration the page is the photograph alone.

Where does the sidebar's list of readings come from, and why does the query have no `user_id` filter?

From `listConversations()` in the `(app)` layout, run on the server with the cookie client. Row Level Security restricts the rows to the signed-in user, so the query does not need to.

Why do the `(app)` pages not check for a session?

The middleware redirects any request under `/app` without a user to `/login` before a page renders. The pages check for a birth profile instead, which is the question the middleware cannot answer.

What would happen if someone changed `--accent` in `globals.css` ?

The site would change and the iOS app would not. `ThemeTests` compare Swift against constants copied from the CSS, not against the CSS itself, so the test would still pass until `Theme.swift` was edited to match.

Why are there two `getUser()` calls on a signed-in visit to `/` ?

The middleware calls one to refresh the cookie and decide whether to redirect protected paths; `app/page.tsx` calls another to decide whether to redirect the landing itself. The landing is not a protected path, so the middleware cannot make that second decision.

Why is the reading card in iOS given explicit colours instead of the environment's?

It renders inside Siri and inside a widget, where the environment's colour scheme may be light. The app is dark-only, and a card that changed with its host would look like another product.

PART THREE

Under the hood

Every table and policy, the iOS project and how it is built, how the system is set up, deployed and released, and a catalogue of what breaks and what catches it.

- 13. The data model
- 14. The iOS app
- 15. Setup, deployment, and release
- 16. What breaks, and how it is handled

13 The data model

Twelve tables in seven numbered SQL files, one storage bucket, and two keys. Every table hangs off `auth.users` and cascades when the user goes. Every table has Row Level Security on, and the shape of its policies says who may write it: the person, or only the server.

This chapter reads `supabase/migrations/` in order, as it stands on `main` on 30 September 2026, version 0.3.0. Migrations are the schema's history, and reading them in sequence is reading the product's: a chat with a chart, then a photo, then numerology, then push alerts, then twice-daily readings, then the life timeline and its explanations. After the seven files there is a summary of every table with its RLS shape and every piece of code that reads or writes it, a section on the `avatars` bucket that no migration creates, and an explanation of the two Supabase keys in terms of the policies. Where a policy and a route handler disagree about who should write a table, the route handler's comment usually says why, and the comment is quoted.

13.1 Where the schema lives and how it is applied

```
supabase/migrations/
├─ 0001_init.sql           admins, profiles, birth_profiles, conversations, messages
├─ 0002_avatar.sql        birth_profiles.avatar_url
├─ 0003_numerology_mode.sql conversations.tradition may be 'numerology'
├─ 0004_alerts_push.sql   device_tokens, transit_conditions, alerts
├─ 0005_daily_readings.sql daily_readings
├─ 0006_life_events.sql   life_events, life_event_scans
└─ 0007_period_readings.sql period_readings

lib/supabase/
├─ client.ts              browser client, anon key, "use client"
├─ server.ts              cookie client for server components and actions, anon key
├─ middleware.ts          cookie refresh and the /app and /admin guard
├─ route.ts               bearer-or-cookie client for route handlers, anon key
└─ admin.ts               service-role client, "server-only"

docs/GO_LIVE_CHECKLIST.md the production facts this chapter quotes
docs/SUPABASE_SETUP.md    project creation and the first admin row
```

The files are plain SQL with a four-digit prefix. There is no migration runner in the codebase and no CI step that applies them. The README gives two ways to apply them: `supabase link` followed by `supabase db push`, or pasting each file into the dashboard's SQL editor, "lowest number first". `docs/SUPABASE_SETUP.md` still describes running only `0001_init.sql`; the README's instruction is the current one and it means all seven.

MIGRATIONS ARE APPLIED BY HAND, BEFORE THE CODE THAT NEEDS THEM

The README states the rule: "After deploying a version that adds a migration, apply it to the production Supabase project before traffic reaches the new code." `docs/GO_LIVE_CHECKLIST.md` records what happens when that is missed. As of 16 September 2026 the production database did not have the tables from `0006_life_events.sql` and `0007_period_readings.sql`, although the timeline code that reads them had shipped. The checklist's fix is to run the two files in the SQL editor, in order, exactly as they are in the repo, and confirm with a query against `information_schema.tables` for `life_events`, `life_event_scans` and `period_readings`. Since `main` deploys to production on every push (chapter 15), a commit that adds `0008` is live before anyone has had a chance to apply it, and every route that touches the new table returns an error until someone does.

13.2 The migrations, in order

0001 · INIT

The first file enables `pgcrypto` (for `gen_random_uuid()`) and creates the five tables the original product needed: who is an admin, a profile row per auth user, the birth details with the computed chart beside them, and conversations with their messages.

TABLE	COLUMNS	KEYS AND INDEXES
admins	user_id uuid, created_at timestamptz	PK user_id, references auth.users(id) on delete cascade
profiles	id uuid, display_name text, created_at	PK id, references auth.users(id) on delete cascade
birth_profiles	id uuid, user_id uuid not null, first_name, last_name, birth_date date, birth_time time, place_name, lat, lng double precision, timezone text, chart jsonb, updated_at	PK id default gen_random_uuid(); unique (user_id); FK to auth.users cascade
conversations	id uuid, user_id uuid not null, tradition text checkin ('vedic', 'western'), title text, created_at	PK id; index (user_id, created_at desc); FK cascade
messages	id uuid, conversation_id uuid not null, role text checkin ('user', 'assistant'), content text not null, created_at	PK id; index (conversation_id, created_at); FK to conversations(id) on delete cascade

Two functions and one trigger. `is_admin()` is `language sql stable security definer` with `set search_path = public`, and returns whether a row in `admins` matches `auth.uid()`. `handle_new_user()` is a `plpgsql security definer` trigger function that inserts a `profiles` row for a new auth user, taking `display_name` from `raw_user_meta_data->>'display_name'` or, failing that, the part of the email before

the @. The trigger `on_auth_user_created` fires after insert on `auth.users` for each row. This is the only code anywhere that writes `profiles`, and, as the summary in 13.4 shows, nothing reads it either.

```
-- supabase/migrations/0001_init.sql
-- RLS: owner-only. Admin reads happen via the service-role client, which bypasses RLS.
create policy "profiles self" on public.profiles
  for all using (id = auth.uid()) with check (id = auth.uid());

create policy "birth self" on public.birth_profiles
  for all using (user_id = auth.uid()) with check (user_id = auth.uid());

create policy "conv self" on public.conversations
  for all using (user_id = auth.uid()) with check (user_id = auth.uid());

create policy "msg self" on public.messages
  for all using (
    exists (select 1 from public.conversations c
            where c.id = messages.conversation_id and c.user_id = auth.uid())
  ) with check (
    exists (select 1 from public.conversations c
            where c.id = messages.conversation_id and c.user_id = auth.uid())
  );

-- admins table: readable only by admins; never writable via API (no insert/update/delete policy
create policy "admins read self-or-admin" on public.admins
  for select using (user_id = auth.uid() or public.is_admin());
```

Four tables get one `for all` policy each, which covers select, insert, update and delete with the same predicate. `messages` has no `user_id` of its own; its policy reaches through `conversations`, so a message is yours if its conversation is. `admins` gets a select policy only. There is no insert policy and none is meant to exist: the file's first comment says "presence in this table = admin. No self-service path", and `docs/SUPABASE_SETUP.md` has the first admin row inserted by hand in the SQL editor. The `is_admin()` function is used nowhere in TypeScript; it exists for that one policy, and the application checks adminship with the service-role client instead (13.6).

0002 · AVATAR

One statement: `alter table public.birth_profiles add column if not exists avatar_url text`. The column holds a public URL into the `avatars` bucket (13.5), not the object path, and it is nullable because a photo is optional. The `if not exists` makes the file safe to run twice.

0003 · NUMEROLOGY_MODE

```
-- supabase/migrations/0003_numerology_mode.sql
alter table public.conversations drop constraint if exists conversations_tradition_check;
alter table public.conversations
  add constraint conversations_tradition_check check (tradition in ('vedic', 'western', 'numerology'))
```

The check constraint from `0001` is dropped by its generated name and recreated with a third value. This is the whole database side of the numerology mode (chapter 6): a conversation records which of the three lenses it was read through, and the constraint is what stops a typo in a client from creating a fourth.

0004 · ALERTS_PUSH

The first migration written for the iOS app and the scheduled job. Its header comment sets the pattern every later table follows: "The daily job (`/api/cron/alerts`) writes both tables with the service-role client, which bypasses RLS. The policies below exist so the app can read its own alerts and manage its own device tokens directly."

TABLE	COLUMNS	KEYS AND INDEXES
<code>device_tokens</code>	<code>token text</code> , <code>user_id uuid not null</code> , <code>platform text</code> default <code>'ios'</code> check in (<code>'ios'</code>), <code>environment text</code> default <code>'production'</code> check in (<code>'sandbox'</code> , <code>'production'</code>), <code>updated_at</code>	PK <code>token</code> ; index (<code>user_id</code>); FK cascade
<code>transit_conditions</code>	<code>id uuid</code> , <code>user_id</code> , <code>kind text</code> , <code>signature</code> <code>text</code> , <code>label text</code> , <code>severity</code> check in (<code>'info'</code> , <code>'caution'</code> , <code>'warning'</code>), <code>scope</code> check in (<code>'natal'</code> , <code>'transit'</code>), <code>detail text</code> , <code>started_on date</code> default today in UTC, <code>ended_on</code> <code>date</code>	PK <code>id</code> ; unique partial index <code>transit_conditions_open_signature</code> (<code>user_id</code> , <code>signature</code>) where <code>ended_on</code> is null; index (<code>user_id</code> , <code>ended_on</code>); FK cascade
<code>alerts</code>	<code>id uuid</code> , <code>user_id</code> , <code>severity</code> check as above, <code>kinds text[]</code> default <code>'{}'</code> , <code>title</code> , <code>body</code> , <code>detail</code> , <code>created_at</code> , <code>read_at timestamptz</code>	PK <code>id</code> ; index (<code>user_id</code> , <code>created_at</code> <code>desc</code>); FK cascade

The token is the primary key of `device_tokens`, not the user, because one person can have several phones and one phone can change hands. `environment` exists because a token minted by a development build is only valid against the APNs sandbox host and vice versa; the comment on the column says so, and the go-live checklist notes that rows registered before the app reported its environment correctly may carry the wrong value.

`transit_conditions` is the state table behind "this changed today". A row is open while `ended_on` is null, and the partial unique index guarantees at most one open row per (`user_id`, `signature`). The daily diff in `lib/alerts/run.ts` compares the engine's current conditions against the open rows, inserts the new ones, and sets `ended_on` on the ones that stopped (chapter 9). The default for `started_on` is `(now() at time zone 'utc')::date`, so the date is the server's, not the user's; `daily_readings` makes the opposite choice on purpose.


```

-- supabase/migrations/0004_alerts_push.sql
create policy "device tokens self" on public.device_tokens
  for all using (user_id = auth.uid()) with check (user_id = auth.uid());

create policy "conditions self read" on public.transit_conditions
  for select using (user_id = auth.uid());

create policy "alerts self read" on public.alerts
  for select using (user_id = auth.uid());

-- Marking an alert read is the only write the app makes.
create policy "alerts self update" on public.alerts
  for update using (user_id = auth.uid()) with check (user_id = auth.uid());

```

Three shapes in one file. Device tokens are fully owner-writable. Conditions are owner-readable and nothing else. Alerts are owner-readable and owner-updatable, so the app can set `read_at`, but no client can insert one; the cron job inserts with the service role. Note that the update policy is on the whole row, not on the column: a client could rewrite an alert's title with its own session. The policy states an intent ("the only write the app makes") that the database does not enforce column by column.

0005 · DAILY_READINGS

TABLE	COLUMNS	KEYS AND INDEXES
<code>daily_readings</code>	<code>id uuid, user_id, for_date date, slot text</code> <code>check in ('morning', 'night'), tradition text</code> <code>default 'vedic', title, body, detail,</code> <code>created_at, read_at</code>	<code>PK id; unique (user_id,</code> <code>for_date, slot); index (user_id,</code> <code>for_date desc); FK cascade</code>

The two comments in this file are the design. On `for_date`: "The user's own local date, not UTC: 'my reading for the 26th' has to mean their 26th." On the unique constraint: "One reading per slot per day. This is also what makes the job safe to run more often than expected, or twice by accident." The job in `lib/alerts/predictions.ts` reads which slots already exist for the user's local date and inserts the missing one; a second run the same morning finds the row and does nothing. Policies are select and update for the owner, the same shape as `alerts`, and for the same reason: the app marks a reading read, the server writes it.

0006 · LIFE_EVENTS

TABLE	COLUMNS	KEYS AND INDEXES
life_events	id uuid , user_id , occurred_on date , precision text default 'day' check in ('day', 'month', 'year') , title text check length 1 to 120, note text check length at most 2000, source text default 'manual' check in ('manual', 'extracted') , created_at	PK id ;index (user_id, occurred_on) ;FK cascade
life_event_scans	user_id uuid , scanned_at timestamptz , found_count integer default 0	PK user_id ;FK cascade

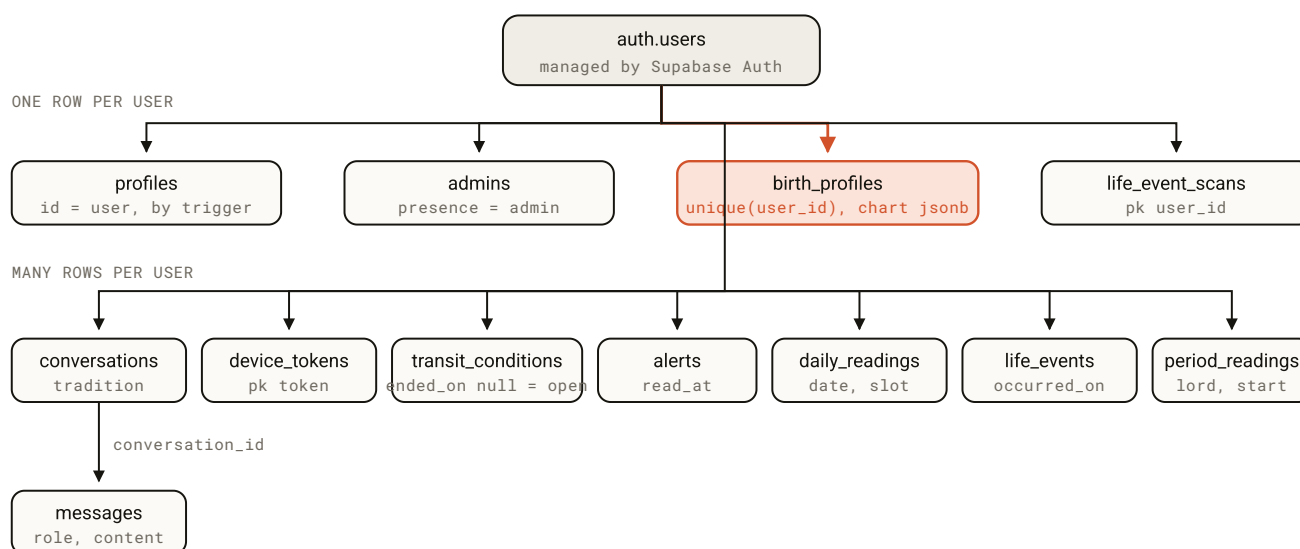
The header comment marks this as "the one table in the schema holding what the user tells us about their own life rather than what the ephemeris computes, so it is owner-only in both directions: no admin read policy, no service-role writes outside the extraction job acting for that same user." Both tables get a single `for all` owner policy, and in code they are written only through the bearer-or-cookie client, never the admin one. `precision` exists because people remember "March 2021", not an hour; `source` distinguishes a typed event from one proposed by mining the person's own chat history, "so a bad extraction run can be undone without touching hand-entered events". `life_event_scans` is one row per user remembering that the mining ran and how many candidates it found, so the offer to scan appears once (chapter 8).

0007 · PERIOD_READINGS

TABLE	COLUMNS	KEYS AND INDEXES
period_readings	user_id , lord text , period_start date , theme text check length 1 to 80, meaning text check length 1 to 800, events_hash text , generated_at	PK (user_id, lord, period_start) ;FK cascade

One row per mahadasha, plus one row with `lord = 'now'` for the current mahadasha and antardasha pair, keyed on the antardasha's start "so it goes stale by itself when the sub-period changes". `events_hash` records which pinned moments the text was written against; when the events inside a period change, the hash no longer matches and the explain route rewrites that period on the next open. The composite primary key is what the explain route's `upsert(rows, { onConflict: "user_id,lord,period_start" })` relies on. The policy is one `for all` owner rule, with the comment "Written by the explain route acting as the user, never by admin tooling." No table in the schema has a `touch_updated_at` style trigger; every timestamp is a column default or a value the application writes.

13.3 The entity relationships



Every arrow is a foreign key with on delete cascade. Deleting the auth user (DELETE /api/account) removes every row in this picture. The avatars bucket is not a table and is not in the cascade: nothing in the code deletes a stored photo.

A star, not a graph. Every table references `auth.users` directly except `messages`, which references `conversations`. There are no foreign keys between feature tables; `period_readings` and `life_events` are related only by dates and a hash the application computes.

The shape matters for two operations. Account deletion (`app/api/account/route.ts`) is one call, `admin.auth.admin.deleteUser(user.id)`, and the cascades do the rest; the route's comment lists profiles, birth profiles, conversations and messages, and the same `on delete cascade` is on every other table too. And any query the application makes needs at most one join, `messages` to `conversations`, which is the join the `msg self` policy performs on every access to a message.

13.4 Every table, its RLS shape, and who touches it

The table below was built by reading every `.from("...")` call under `app/` and `lib/`, excluding tests, and noting which client made it. "User client" means the cookie client (`lib/supabase/server.ts`) or the bearer client (`lib/supabase/route.ts`), both of which evaluate RLS as the signed-in user. "Admin client" means `createAdminSupabase()`, which bypasses RLS. The last column lists the iOS stores that query the table directly over PostgreSQL with the user's own session, which is the same trust level as the user client and the reason the policies exist at all.

TABLE	RLS SHAPE	READ BY	WRITTEN BY	IOS DIRECT
admins	Select for self or any admin; no write policy	lib/admin/guard.ts isAdminUser (admin client)	Nothing in code; a hand-written insert	No
profiles	Owner for all	Nothing	The on_auth_user_created trigger only	No
birth_profiles	Owner for all	lib/data/birthProfile.ts (user); app/api/chat , app/api/kundli , lib/timeline/load.ts (user); lib/alerts/run.ts , lib/alerts/predictions.ts , lib/admin/data.ts (admin)	saveBirthProfile upsert on user_id and ensureCurrentChart update, both user client, from the intake action and app/api/profile	ProfileStore reads
conversations	Owner for all	lib/data/chat.ts , app/api/timeline/scan , lib/timeline/load.ts (user); lib/admin/data.ts (admin)	lib/data/chat.ts getOrCreateConversation insert (user)	ChatStore reads
messages	Owner via conversations, for all	lib/data/chat.ts , app/api/timeline/scan , lib/timeline/load.ts count (user); lib/admin/data.ts (admin)	lib/data/chat.ts appendMessage insert (user)	ChatStore reads
device_tokens	Owner for all	lib/alerts/run.ts , lib/alerts/predictions.ts (admin)	app/api/devices upsert on token and delete (admin); the alert and prediction senders delete dead tokens (admin)	No; the app calls the route
transition_conditions	Owner select only	lib/alerts/run.ts (admin)	lib/alerts/run.ts insert and update ended_on (admin)	No
alerts	Owner select and update	Nothing under app/ or lib/	lib/alerts/run.ts insert (admin)	AlertsStore reads and sets read_at
daily_readings	Owner select and update	lib/alerts/predictions.ts reads existing slots (admin)	lib/alerts/prediction s.ts insert (admin)	DailyStore and IntentData read; DailyStore sets read_at
life_events	Owner for all	lib/timeline/load.ts , app/api/timeline/scan (user)	app/api/life-events insert and delete (user)	No; the app calls the routes

TABLE	RLS SHAPE	READ BY	WRITTEN BY	IOS DIRECT
<code>life_events</code>	Owner for all	<code>lib/timeline/load.ts</code> (user)	<code>app/api/timeline/scan</code> upsert (user)	No
<code>period_readings</code>	Owner for all	<code>lib/timeline/load.ts</code> (user)	<code>app/api/timeline/explain</code> upsert (user)	No

Four things in that table are worth a second look.

- `profiles` **is write-only**. The trigger fills it and nothing reads it. The name the app shows comes from `birth_profiles.first_name`; the admin page shows the email from `auth.users`. The table is a leftover from the first design and costs one insert per sign-up.
- `device_tokens` **has an owner policy that no code uses**. The devices route writes with the admin client, and its comment explains: "a device handed from one account to another has to change owner, and the owner-only RLS policy would (correctly) refuse that update. The user id still comes from the verified session, never from the body." The policy remains so a client could read its own registrations; today none does.
- `alerts` **is never read on the web**. Only the iOS app lists alerts and marks them read, directly against PostgREST. The web has no inbox (chapter 9).
- **The timeline tables are the only ones written as the user from a route handler**. `life_events`, `life_event_scans` and `period_readings` are inserted, upserted and deleted through the bearer-or-cookie client, so RLS is the authorisation. Every other server-side write goes through the admin client, and every one of those routes takes the user id from the verified session first.

A REJECTED WRITE DOES NOT THROW

`supabase-js` resolves with an `error` field when RLS or a constraint rejects a write; it does not throw. The comment in `lib/data/birthProfile.ts` spells this out and is why every write in `lib/data/` checks `error` and turns it into a `throw`. A read blocked by RLS is worse: it returns an empty result that looks exactly like "no rows yet". That is what a missing policy, or a missing session on a bearer client, looks like from a route handler, and it is the first thing to suspect when a signed-in user sees an empty home.

13.5 The avatars bucket

The one piece of storage is not in any migration. The README's architecture sketch lists "Storage (avatars)" beside Postgres and Auth, and points at `docs/SUPABASE_SETUP.md` for "the storage bucket", but that document has no bucket step, so the bucket is created in the dashboard by hand. The code tells you what it must look like.

```
// app/(app)/app/intake/actions.ts (the same function is in app/api/profile/route.ts)
async function uploadAvatar(userId: string, file: File): Promise<string | null> {
  const ext = file.type === "image/png" ? "png" : file.type === "image/webp" ? "webp" : "jpg";
  const path = `${userId}/avatar.${ext}`;
  const bytes = Buffer.from(await file.arrayBuffer());
  const admin = createAdminSupabase();
  const { error } = await admin.storage
    .from("avatars")
    .upload(path, bytes, { contentType: file.type, upsert: true });
  if (error) return null;
  const { data } = admin.storage.from("avatars").getPublicUrl(path);
  // cache-bust so a replaced photo shows immediately
  return `${data.publicUrl}?v=${Date.now()}`;
}
```

Four facts follow. The upload is made with the service-role client, so the bucket needs no storage policies for writes and none exist in the repo. The object name is `{user_id}/avatar.{ext}` with `upsert: true`, so each person has at most one photo per extension, and a person who uploads a PNG after a JPEG has two objects with only the newer one referenced. The URL stored in `birth_profiles.avatar_url` comes from `getPublicUrl`, which only works if the bucket is public; the home page, the profile page and the admin list all render it in a plain `<img>` with no token. And a failed upload returns `null`, which `saveBirthProfile` reads as "keep the old photo": the comment there says "Only overwrite the avatar when a new one was uploaded". The same function is duplicated in `app/api/profile/route.ts` for native clients, byte for byte. Nothing in the code deletes an object, on account deletion or otherwise.

13.6 The anon key and the service-role key

Supabase issues two keys for a project and the code names them by their environment variables.

`NEXT_PUBLIC_SUPABASE_ANON_KEY` is public by design; the README says it is "safe to ship in the iOS binary", and the `NEXT_PUBLIC_` prefix means Next inlines it into browser bundles. `SUPABASE_SERVICE_ROLE_KEY` is read only through `lib/env.ts`, which imports `server-only` so any client component that reaches for it fails the build. The policies in this chapter are what make the first key safe and the second one necessary.

CLIENT	FILE	KEY	RUNS AS	USED FROM
Browser	<code>lib/supabase/client.ts</code>	anon	The signed-in user, via the session in cookies	Nothing today; no client component imports it
Cookie	<code>lib/supabase/server.ts</code>	anon	The signed-in user, via the session in cookies	Server components and server actions; <code>lib/supabase/middleware.ts</code> builds the same kind of client over the request's cookies
Bearer	<code>lib/supabase/routes.ts</code>	anon, plus Authorization: Bearer forwarded	The user whose JWT the iOS app sent; falls back to the cookie client	Every <code>app/api/*</code> route that acts for a user; cron has no session and geocode needs none
Admin	<code>lib/supabase/admin.ts</code>	service role	Nobody: RLS is bypassed	Cron jobs, admin pages, device registration, avatar upload, account deletion

With the anon key, every query runs as the `authenticated` role if a valid JWT is attached, or as `anon` if none is. Every policy in the schema compares a column to `auth.uid()`. For the `anon` role that function returns null, no row matches, and every table reads as empty and refuses every write. So the anon key on its own can do nothing, which is what makes it publishable. With a JWT attached the same key sees exactly the rows whose `user_id` (or `id`, or conversation) is the token's subject, which is why `listConversations` and `getBirthProfile` need no user filter and why the iOS stores can query PostgREST directly.

The service role bypasses RLS entirely. Reading the policies tells you where it is unavoidable:

- `transit_conditions`, `alerts` and `daily_readings` have no insert policy. Their writer is the cron job in `lib/alerts/`, which has no user session because no user is present at 02:30 UTC. It runs as the service role and iterates every `birth_profiles` row, which an owner policy would never let one session do.
- `admins` has no write policy, and its read policy calls `is_admin()`. `requireAdmin()` could read it as the signed-in user; instead `isAdminUser` uses the admin client, and `lib/admin/data.ts` then reads other people's `birth_profiles`, `conversations` and `messages`, which no policy allows and which is the point of an admin page. Every admin function calls `requireAdmin()` first; the service role is behind a check, never in front of one.
- `device_tokens` needs to change owner, and the owner policy forbids that, so the devices route uses the admin client after verifying the session.
- Account deletion and the avatar upload are Auth and Storage operations, not table writes; `auth.admin.deleteUser` exists only on the service role, and the bucket has no policies because only the service role writes it.

Everything else runs as the user. The rule in the code is simple to state and easy to check with a grep for `createAdminSupabase`: outside tests it appears in ten files, every one of them either imports `server-only` itself or is a route handler or server action, and the comment in `lib/supabase/admin.ts` is the rule in one line: "Bypasses RLS. Only ever import from server code behind an admin auth check."

WHAT THE POLICIES DO NOT CHECK

An owner policy proves the row belongs to the caller. It does not check columns: the update policies on `alerts` and `daily_readings` permit rewriting the title as well as `read_at`, and the `for_all` policy on `birth_profiles` lets a client write any `chart` JSON it likes. The application writes `chart` only from the Swiss Ephemeris on the server, but the database would accept a hand-made one from a valid session. The schema trusts its owners with their own rows completely; it trusts them with nobody else's at all.

13.7 Study notes

READ IN THIS ORDER

1. `supabase/migrations/0001_init.sql` , for the trigger, the five policies and the comment about admin reads.
2. `supabase/migrations/0004_alerts_push.sql` , for the three policy shapes in one file and the partial unique index.
3. `supabase/migrations/0005_daily_readings.sql` , two comments that are the design.
4. `supabase/migrations/0006_life_events.sql` and `0007_period_readings.sql` , the owner-only-in-both-directions tables.
5. `lib/supabase/route.ts` , how one function serves cookies and bearer tokens.
6. `lib/supabase/admin.ts` and `lib/env.ts` , the service role and `server-only` .
7. `app/api/devices/route.ts` , the one route whose comment argues against its own table's policy.
8. `lib/data/birthProfile.ts` , for the note on rejected writes resolving rather than throwing.
9. `docs/GO_LIVE_CHECKLIST.md` , section 3, for what an unapplied migration looks like in production.

CHECK YOURSELF

Why can the cron job not use the cookie or bearer client?

It has no session: no user is signed in when Vercel calls the route, and it must read every user's `birth_profiles` row and insert readings for each. Only the service role sees across users, so the job is written for it and the tables it writes have no insert policies at all.

A user's `listConversations()` returns an empty list although they have chats. What are the first two suspects?

A missing or expired session on the client that made the call (RLS returns nothing, not an error), or a policy that does not match the row. Neither throws; both look like "no data".

Why is the token, not the user, the primary key of `device_tokens` ?

One person can have several phones and one phone can be signed into a different account later. A token must belong to exactly one user at a time, and moving it is an upsert on the token, which is why the route writes it as the service role.

What does `unique (user_id, for_date, slot)` on `daily_readings` buy?

Idempotence for the twice-daily job. A second run for the same local date and slot finds the row and writes nothing, so running the job more often, or twice by accident, is safe.

Which tables does the service role never touch, and why does that matter?

`life_events` , `life_event_scans` and `period_readings` . They hold what the person said about their own life, so the migration deliberately gives them no admin path; their routes act as the user and RLS is the whole authorisation.

Why does deploying a commit that adds a migration need a manual step, and in what order?

Nothing applies migrations automatically and `main` deploys on push. The SQL must be run against production, lowest number first, before the code that reads the new table takes traffic; otherwise every route that touches it errors until it is.

14 The iOS app

Sanchara for iOS is a SwiftUI app that talks to the same route handlers and the same Supabase project as the web app, holds no secret the web does not also publish, and is built from a 127-line YAML spec rather than a checked-in Xcode project. Everything about how it is assembled, configured, signed and shipped follows from that.

This chapter describes the code on `main` on 30 September 2026, version 0.3.0. It covers the plumbing under the features: the generated project and its three targets, the generated configuration file, the entry point and the routing state machine, the design tokens, the API client and its streaming reader, the plist and entitlement choices, how the XCTest suite is organised, and the two GitHub Actions workflows that build it on every push and ship it to TestFlight on every tag. The features themselves (chat, kundli, timeline, alerts, on-device answers) live in their own chapters; chapter 10 in particular covers the widget extension and the App Group cache.

14.1 One spec, three targets

The Xcode project is generated. `mobile/project.yml` is the whole description of the app for XcodeGen, and `.gitignore` excludes `mobile/Sanchara.xcodeproj/`, `mobile/build/`, `mobile/SourcePackages/` and the generated sources folder. The comment in the ignore file is the rule: "Run `cd mobile && xcodegen generate` after changing the spec." The practical consequence is that adding a Swift file means regenerating the project, and that two engineers can never diverge on a `project.pbxproj` merge, because there is no such file to merge.

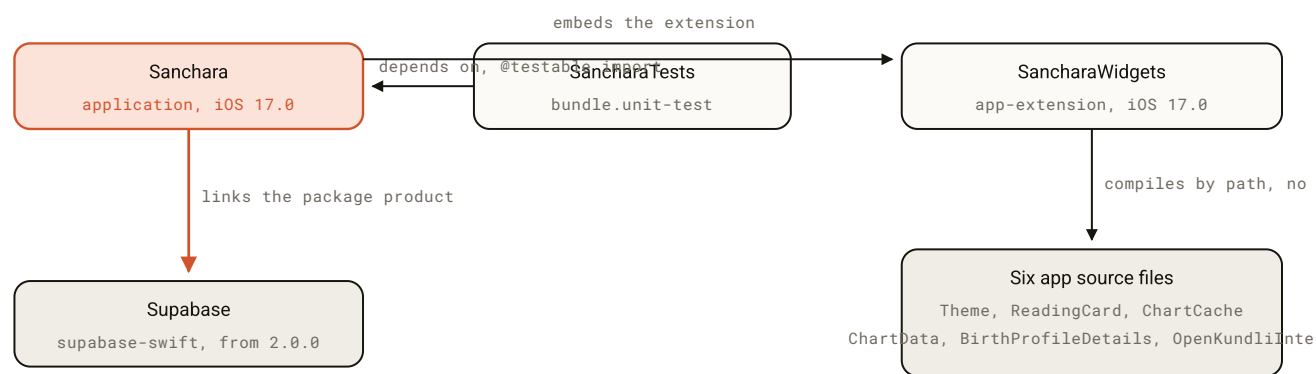
```
# mobile/project.yml
name: Sanchara
options:
  bundleIdPrefix: com.shivvyas
  deploymentTarget:
    iOS: "17.0"
  createIntermediateGroups: true
  groupSortPosition: top

packages:
  Supabase:
    url: https://github.com/supabase/supabase-swift
    from: "2.0.0"

settings:
  base:
    SWIFT_VERSION: "5.0"
    ENABLE_USER_SCRIPT_SANDBOXING: YES
    CODE_SIGN_STYLE: Automatic
```

The base settings also name the development team, which this handbook does not reproduce. Signing is automatic everywhere: locally through Xcode, on CI by turning signing off, and in the TestFlight workflow

through an App Store Connect API key. One package is declared, supabase-swift at 2.0.0 or later, and only the app target links it.



The extension declares no package dependency. Anything it needs from the app is listed file by file, and each file imports only

Three targets, one package. The tests import the app; the app links Supabase and embeds the widgets; the widgets reach into the app's sources by path and nowhere else.

TARGET	TYPE	BUNDLE ID	SOURCES	DEPENDS ON
Sanchara	application	com.shivvyas.astra	Sanchara/Sources , Sanchara/Resources as a resources phase	the Supabase package product; the SancharaWidgets target
SancharaW idgets	app-extension	com.shivvyas.astra.widgets	SancharaWidgets plus six app files by path	nothing
SancharaT ests	bundle.unit-test	com.shivvyas.astra.tests	Sanchara/Tests	the Sanchara target

Three per-target settings are worth knowing. `GENERATE_INFOPLIST_FILE` is `NO` for the app and the extension, because each keeps a real `Info.plist` that the TestFlight workflow edits with PlistBuddy; the test bundle lets Xcode generate its own. `TARGETED_DEVICE_FAMILY` is `"1"`, iPhone only. And the extension sets `SKIP_INSTALL: YES`, the standard way to keep an embedded extension out of the archive's products list. The extension carries its own `deploymentTarget` of iOS 17.0 with a comment noting that widgets and controls arrived on that schedule while the app itself still runs on iOS 17; the newer surfaces (controls on iOS 18, snippets and the on-device model on iOS 26) are gated in code with `#available`, not by raising the floor.

14.2 Configuration and the icon

MAKE-CONFIG.SH AND THE GENERATED APPCONFIG

```
# mobile/scripts/make-config.sh
read_var() { grep -E "^$1=" "$ENV_FILE" | head -1 | cut -d= -f2- | tr -d '"' | tr -d "'" | tr -d '\n'

SUPABASE_URL="$(read_var NEXT_PUBLIC_SUPABASE_URL)"
SUPABASE_ANON_KEY="$(read_var NEXT_PUBLIC_SUPABASE_ANON_KEY)"
API_BASE_URL="{SANCHARA_API_BASE_URL:-https://astra.shivvyas.com}"

cat > "$OUT" <<SWIFT
// Generated by mobile/scripts/make-config.sh, do not edit, do not commit.
import Foundation

enum AppConfig {
    static let supabaseURL = URL(string: "$SUPABASE_URL")!
    static let supabaseAnonKey = "$SUPABASE_ANON_KEY"
    static let apiBaseURL = URL(string: "$API_BASE_URL")!
}
SWIFT
```

The app has no secrets file and no build-time environment; it has one generated Swift file, `mobile/Sanchara/Sources/Generated/AppConfig.swift`, written by `mobile/scripts/make-config.sh` from the repo's `.env.local`. The script reads exactly two variables, the same two the web app ships to browsers: `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`. Its header comment states the boundary: those values "are designed to ship in clients (RLS is the security boundary, not secrecy)", while `SUPABASE_SERVICE_ROLE_KEY` and `ANTHROPIC_API_KEY` "are never read here and must never reach the app binary." The script fails loudly if either value is missing. A third value, `SANCHARA_API_BASE_URL`, is read from the shell environment rather than the file and defaults to the production host, which is how a developer points a build at a local or preview API without touching the checked-in code. The output folder is gitignored, so the generated file exists on every machine that built the app and in no commit.

Three places consume the three constants. `Supa.client` is constructed from `supabaseURL` and `supabaseAnonKey`. `SancharaAPI.request` builds every URL by appending a path to `apiBaseURL`. And `AuthStore` uses `apiBaseURL` to form the `auth/callback` redirect for magic-link email.

MAKE-ICON.PY

The app icon is not a design file; it is `mobile/scripts/make-icon.py`, a Pillow script that writes `Sanchara/Resources/Assets.xcassets/AppIcon.appiconset/AppIcon.png`. The docstring describes the mark: "a warm disc with a soft corona on the near-black ground, using the same tokens as `app/globals.css`." The three colours are literally the site's tokens (`BG` `0x0A0A0B`, `ACCENT` `0xE8663D`, `FG` `0xF4F1EA`). The script renders on a 4096-pixel canvas and downsamples to 1024 with Lanczos so the edges stay clean at small sizes. It builds the image in four passes: sixty concentric translucent rings blurred into a glow for the corona, the disc itself with a warm vertical falloff so it reads as lit, a thin bright limb along the upper edge masked by its own falloff so it fades toward the equator, and the final resize. The disc sits at 46 percent of the height rather than centred, with a radius of a fifth of the canvas. Changing the icon means

changing numbers in a script and re-running it, which is the same discipline as the generated project and the generated config: nothing binary is hand-made.

14.3 Entry point and routing

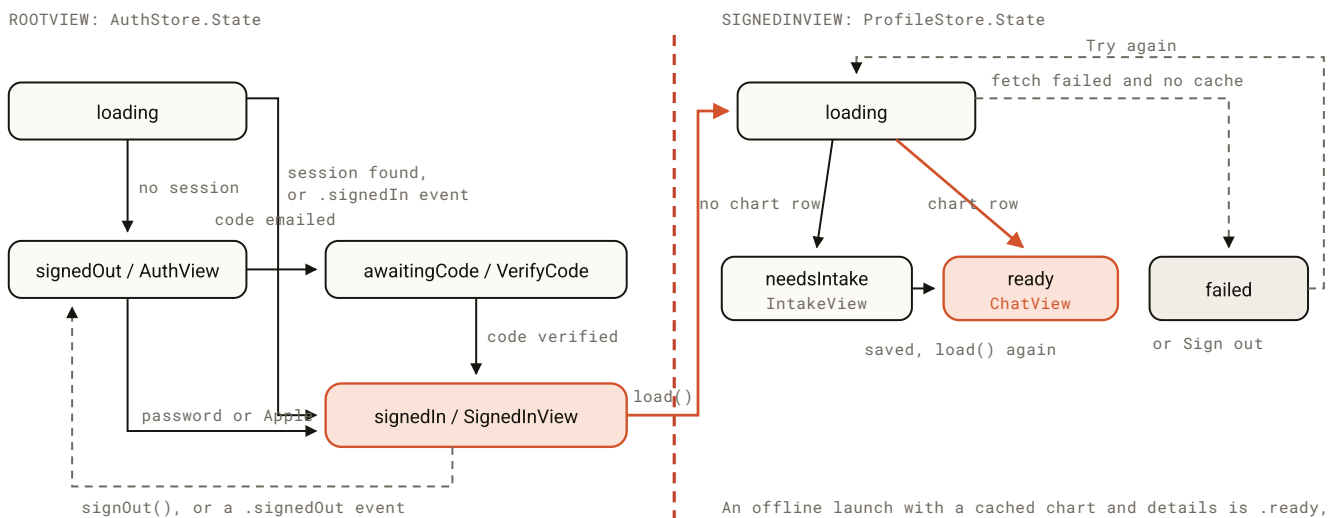
```
// mobile/Sanchara/Sources/SancharaApp.swift
@main
struct SancharaApp: App {
    // APNs callbacks have no SwiftUI equivalent, so the app keeps a delegate.
    @UIApplicationDelegateAdaptor(SancharaAppDelegate.self) private var appDelegate
    @State private var auth = AuthStore()

    var body: some Scene {
        WindowGroup {
            RootView()
                .environment(auth)
                .preferredColorScheme(.dark)
        }
    }
}
```

Sixteen lines. One `AuthStore` is created here and passed down through the environment; nothing else in the app creates one. The app delegate exists for one reason the comment gives, APNs token and notification callbacks, and it lives in `Services/Push.swift`. The dark colour scheme is forced at the root and again in `Info.plist`, which section 14.6 returns to.

`RootView` is a switch over `AuthStore.State`, and `SignedInView` is a switch over `ProfileStore.State`. Between them they are the whole routing policy of the app: there is no tab bar and nothing at the root remembers a screen. `ChatView` owns the one `NavigationStack` that matters and presents history, profile, kundli, timeline and alerts as sheets.


```
// mobile/Sanchara/Sources/RootView.swift
var body: some View {
    Group {
        switch auth.state {
        case .loading:
            LoadingScreen()
        case .signedOut:
            AuthView()
        case .awaitingCode(let email, let purpose):
            VerifyCodeView(email: email, purpose: purpose)
        case .signedIn:
            SignedInView()
        }
    }
    .task { await auth.start() }
    .onOpenURL { url in
        // Two kinds of URL arrive here: Supabase's auth callback, and our
        // own scheme from a widget tap. Only the first is for the auth
        // store; handing it a sanchara:// URL would show "link expired".
        if DeepLink.isDeepLink(url) {
            DeepLink.shared.handle(url: url)
        } else {
            Task { await auth.handle(url: url) }
        }
    }
}
}
```



Two switches, eight states. The left half is identity and never touches the chart; the right half assumes identity and only asks whether a chart exists. A sign-out from anywhere collapses both.

`AuthStore.start()` sets the first state from whether supabase-swift has a session in the Keychain, then loops over `authStateChanges` for the life of the app: `.signedIn` and `.tokenRefreshed` with a session move to `.signedIn`, `.signedOut` moves to `.signedOut`. The `awaitingCode` state carries the email and a `CodePurpose`, either `.signIn` (a passwordless code) or `.confirmSignUp` (confirming the address on a

password account), which decides which Supabase OTP type verifies it. A password sign-in that fails with "not confirmed" lands in `awaitingCode` rather than an error, which is how an unconfirmed account recovers.

`SignedInView` owns a `ProfileStore` and calls `load()` in a `.task`. The store seeds the chart and details from the App Group cache, fetches the `birth_profiles` row that has a non-null `chart`, and lands on `.needsIntake` when there is none or `.ready` when there is one. `IntakeView` is handed a closure that calls `load()` again once the profile is saved, so the same switch carries the person from intake to chat without any navigation code. `.failed` is reached only when the fetch throws and the cache had nothing; it shows the message, a "Try again" button that reloads, and a "Sign out" button. The web app makes the same decision in a redirect: `/api/chat` refuses a user with no chart and the chat page sends them to intake.

14.4 Design: tokens, components, the card

```
// mobile/Sanchara/Sources/Design/Theme.swift
enum Theme {
    /// `--bg: #0a0a0b`
    static let bg = Color(hex: 0x0A0A0B)
    /// `--fg: #f4f1ea`
    static let fg = Color(hex: 0xF4F1EA)
    /// `--muted: #9a978f`
    static let muted = Color(hex: 0x9A978F)
    /// `--accent: #e8663d`
    static let accent = Color(hex: 0xE8663D)

    /// Tailwind `border-white/10`, used for every hairline on the site.
    static let hairline = Color.white.opacity(0.10)
    /// Tailwind `bg-white/[0.04]`, the fill behind form controls.
    static let fieldFill = Color.white.opacity(0.04)
    /// Tailwind `rounded-lg` is 0.5rem.
    static let cornerRadius: CGFloat = 8
    static let chartLine = Color.white.opacity(0.22)
    /// The site's easing: `cubic-bezier(0.22, 1, 0.36, 1)`
    static let ease = Animation.timingCurve(0.22, 1, 0.36, 1, duration: 0.6)
}
```

The web app defines exactly four colours on `:root` in `app/globals.css`, and `Theme` carries the same four hex values with the CSS variable quoted above each. The file's comment gives the reason: matching them exactly "is what makes the native app read as the same product as the site." `ThemeTests` asserts each of the four against its CSS token, so a careless edit fails the build rather than drifting. The `Color(hex:)` initialiser exists so tokens can be written the way they appear in CSS. Two derived values are documented as departures: `chartLine` is heavier than `hairline` because the kundli's rule lines are the content, not a divider, and at 0.10 white "the square all but vanished on an OLED screen in daylight". A `Text.eyebrow()` extension reproduces the site's eyebrow style at 12pt with 3.6 points of tracking, which is 0.3em.

`Components.swift` holds four controls, each a transcription of a Tailwind class list: `SancharaField` (the site's input, 16pt text because that is the size the web forces on phones so Safari does not zoom, and "the size is the look"), `SancharaPrimaryButton` (foreground on background, with a spinner that replaces the title while loading and disables the button), `SancharaSecondaryButton` (a white-at-15-percent border and muted

text), and `SancharaSecureField`, a password field with an eye toggle, because "choosing a password you cannot see is the most common reason a sign-up attempt fails twice."

`ReadingCard.swift` is the third file and the only one that leaves the app: it is compiled into the widget extension and rendered by the Siri snippet, so an eyebrow, a serif headline and a muted body look the same in three places. Its body parameter is named `text` rather than `body`, because `body` is the `View` requirement. It sets its own colours rather than reading the environment, for the reason given in its comment: the app is dark-only, "and a card that went light inside Siri while the app stayed dark would read as a different product."

14.5 Services: the client and the session

ONE SUPABASE CLIENT

```
// mobile/Sanchara/Sources/Services/Supabase.swift
enum Supa {
    static let client = SupabaseClient(
        supabaseURL: AppConfig.supabaseURL,
        supabaseKey: AppConfig.supabaseAnonKey
    )

    /// The current access token, for `Authorization: Bearer` against the
    /// Sanchara API on Vercel. Returns nil when signed out.
    static func accessToken() async -> String? {
        try? await client.auth.session.accessToken
    }
}
```

supabase-swift persists the session in the Keychain by default on Apple platforms, which the comment notes is what the design requires: never `UserDefaults`. The stores read tables directly through this client (`birth_profiles`, `daily_readings`, `conversations`, `messages`) under the owner-only RLS policies, and everything that needs a secret or the ephemeris goes through `SancharaAPI` with the session's access token as a bearer.

```
// mobile/Sanchara/Sources/Services/SancharaAPI.swift
enum APIError: LocalizedError {
    case unauthorized
    case server(String)

    var errorDescription: String? {
        switch self {
            case .unauthorized: "Your session expired. Please sign in again."
            case .server(let message): message
        }
    }
}

private static func request(_ path: String, method: String) async throws -> URLRequest {
    var req = URLRequest(url: AppConfig.apiBaseURL.appendingPathComponent(path))
    req.httpMethod = method
    guard let token = await Supa.accessToken() else { throw APIError.unauthorized }
    req.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
    return req
}

private static func check(_ response: URLResponse, _ data: Data) throws {
    guard let http = response as? HTTPURLResponse else { return }
    switch http.statusCode {
        case 200..<300: return
        case 401: throw APIError.unauthorized
        default:
            let message = String(data: data, encoding: .utf8) ?? "Something went wrong."
            throw APIError.server(message)
    }
}
```

The client is an enum of static functions, one per route handler, each a few lines: `request` builds the URL and attaches the bearer, `URLSession.shared` performs it, `check` maps the status. There are two error cases and no more. A 401 is `unauthorized`, which every screen renders as a sign-in prompt; anything else outside 2xx is `server(message)` carrying the response body verbatim, because the route handlers already write their errors as sentences. A missing token throws before any network call. The route handlers themselves are the same ones the web app calls; the `/api/geocode` call is the one exception to the bearer rule because that route is public, and the comment on it says so.

FUNCTION	ROUTE	NOTES
<code>saveProfile</code>	<code>POST /api/profile</code>	Multipart body built by <code>BirthProfileInput.multipartBody</code> ; the optional JPEG travels as a <code>photo</code> part
<code>kundliPDF</code>	<code>GET /api/kundli</code>	Returns the PDF bytes
<code>deleteAccount</code>	<code>DELETE /api/account</code>	Required by App Store guideline 5.1.1(v)
<code>geocode</code>	<code>GET /api/geocode?q=</code>	Public, no token; returns coordinates with the timezone the ephemeris needs
<code>timeline</code> , <code>explainTimeline</code> , <code>scanLifeEvents</code>	<code>GET /api/timeline</code> , <code>POST /api/timeline/explain</code> , <code>POST /api/timeline/scan</code>	The two POSTs set a 120-second timeout because one model call covers all twelve periods, or a whole transcript
<code>addLifeEvents</code> , <code>deleteLifeEvent</code>	<code>POST</code> and <code>DELETE /api/life-events</code>	Batch add; single delete
<code>registerDevice</code> , <code>unregisterDevice</code>	<code>POST</code> and <code>DELETE /api/devices</code>	The APNs token and its environment
<code>chatStream</code>	<code>POST /api/chat</code>	An <code>AsyncThrowingStream<ChatEvent, Error></code> , below

STREAMING A READING

```
// mobile/Sanchara/Sources/Services/SancharaAPI.swift
if let id = http.value(forHTTPHeaderField: "x-conversation-id") {
    continuation.yield(.conversationId(id))
}

var undecoded = Data()
var batch = ""
for try await byte in bytes {
    undecoded.append(byte)
    guard let piece = String(data: undecoded, encoding: .utf8) else { continue }
    undecoded.removeAll(keepingCapacity: true)
    batch += piece
    if batch.count >= 24 {
        continuation.yield(.text(batch))
        batch = ""
    }
}
if !batch.isEmpty { continuation.yield(.text(batch)) }
continuation.finish()
```

`/api/chat` answers with plain UTF-8 text chunks rather than server-sent events, so the client reads raw bytes with `URLSession.shared.bytes(for:)`. Two things follow. A chunk boundary can split a multi-byte character, so bytes are held in `undecoded` until they decode as a whole string. And byte-at-a-time updates would redraw the transcript thousands of times for one reading, so text is coalesced into batches of at least 24 characters before being yielded. The conversation id arrives first, from the `x-conversation-id` response

header, before any text; `ChatStore` keeps it and sends it back on the next turn so the server appends to the same conversation, exactly as `components/Chat.tsx` does on the web. The request timeout is 120 seconds because readings can think for a while before the first byte. A non-2xx status is read to the end and thrown as `server(body)`; a 401 is `unauthorized`. The stream's `onTermination` cancels the underlying task, which is what lets a new reading supersede one still streaming without leaking the connection.

14.6 Info.plist and the entitlements

`mobile/Sanchara/Info.plist` is short and every key in it is a decision. It is also declared a second time as `info.properties` in `project.yml`, which is the copy with the comments.

KEY	VALUE	WHY
<code>UIInterfaceStyle</code>	<code>Dark</code>	"The app is dark by design, the cosmic theme has no light variant." <code>SancharaApp</code> repeats it with <code>preferredColorScheme(.dark)</code> .
<code>UISupportedInterfaceOrientations</code>	<code>portrait only</code>	One orientation, one device family (iPhone).
<code>CFBundleURLTypes</code>	scheme <code>sanchara</code> , name <code>com.shivvyas.astra</code>	" <code>sanchara://timeline</code> , how a tap on the period widget lands in the app. Universal Links cover the web; this covers the Home Screen."
<code>UILaunchScreen</code>	colour <code>LaunchBackground</code>	A colour set in the asset catalog, so the launch frame is the same near-black as the first screen.
<code>ITSAppUsesNonExemptEncryption</code>	<code>false</code>	Skips the export-compliance question on every TestFlight upload.
<code>CFBundleShortVersionString</code> , <code>CFBundleVersion</code>	<code>1.0</code> , <code>1</code>	Placeholders. The TestFlight workflow overwrites both from the tag and the run number.

`mobile/Sanchara/Sanchara.entitlements` holds four capabilities, each with a comment in `project.yml` pointing at the portal step it needs.

- `aps-environment: development`. The comment is a warning: change it to `production` for TestFlight and App Store builds, because "a token minted under one environment is invalid in the other." The TestFlight workflow makes that edit itself, and `ApsEnvironment.current` in `Services/Push.swift` reads the answer back from the embedded provisioning profile at runtime rather than trusting the build configuration, because a Release build run from Xcode still carries the development entitlement. The server is told `sandbox` unless the profile literally says `production`.
- `com.apple.developer.applesignin: Default`. Sign in with Apple; the App ID and Supabase's Apple provider both need enabling, per `docs/APPLE_SIGN_IN.md`.
- `com.apple.developer.associated-domains: applinks:` and `webcredentials:` for the production host. The first makes the magic-link and confirmation emails open the app; the second lets the system password manager offer saved credentials.
- `com.apple.security.application-groups: group.com.shivvyas.astra`. The cache chapter 10 is built on. The extension's own entitlements file contains this key and nothing else.

14.7 The test suite

One test target, `SancharaTests`, depends on the app and uses `@testable import Sanchara`. There is no UI test target and no test that needs a network, a session or Apple Intelligence: everything that decides something is a pure function or a value type, and the tests read as lists of the rules in plain English. Fourteen files, 95 test methods.

FILE	TESTS	WHAT IT PINS
<code>ApsEnvironmentTests</code>	6	The provisioning-profile parser: development reads as sandbox, production as production, and no plist, no entitlement, an unknown value or a simulator build all fall back to sandbox.
<code>AuthValidationTests</code>	4	Email validation accepts ordinary addresses and rejects malformed ones; code entry keeps only digits; the Apple sign-in nonce is random and hashed.
<code>CandidateResolverTests</code>	9	How a proposed timeline moment becomes a pinned one: dated candidates pass through, undated ones need a year and land on 1 January at year precision, and the payloads from older servers still decode.
<code>ChartFactTests</code>	6	The on-device table's derived rows, the clean fallback without them, and a sweep for forbidden vocabulary.
<code>DailyReadingTests</code>	4	The readings inbox groups newest day first with morning above night, and labels today and yesterday as words.
<code>DerivedFacetsTests</code>	4	<code>NatalChart</code> decodes with or without a <code>derived</code> block, ignores undeclared keys, and tolerates an absent placement.
<code>KundliSnapshotTests</code>	1	Renders the kundli to a PNG for inspection and asserts only that something was drawn; a pixel assertion "that fails whenever anyone touches a colour is a test that gets deleted."
<code>KundliTests</code>	12	Which rashi lands in which house, that the twelve regions tile the square exactly, that planets file under their own house, degrees print as degrees and minutes, every graha has a glyph, and every label anchor sits inside its own house.
<code>OnDeviceRoutingTests</code>	15	The cheap filter, the house-number parser, the rows the table returns, and that nothing handed to the model names astrology.
<code>PeriodProgressTests</code>	8	The period widget's fraction and years-left maths, agreement with the timeline's own progress, and the period cache's round trip and clear.
<code>ProfilePhotoTests</code>	4	Large photos are downscaled and small ones left alone; the multipart body carries a <code>photo</code> part only when one was picked.
<code>SuggestionTests</code>	6	Three starters per mode, fallback when the on-device model is unavailable, and a settled result when it is.
<code>ThemeTests</code>	4	The four colour tokens match <code>app/globals.css</code> exactly.
<code>TimelineTests</code>	12	Band lengths and year labels, progress clamping, which dasha pair a moment fell in, date rendering to the precision given, and that dates parse in UTC rather than the device zone.

Two conventions hold across the suite. Test classes that drive a `@MainActor` type (`SuggestionTests`, `DailyReadingTests`) are themselves `@MainActor`, and the rest are plain `XCTestCase`s over value types.

And tests that depend on the environment skip rather than fail: `SuggestionTests` uses `XCTSkipIf` and `XCTSkipUnless` on `SuggestionEngine.isOnDeviceAvailable`, so the same file passes on a Mac, on the simulator and on a device with Apple Intelligence, asserting a different thing in each.

14.8 CI and TestFlight

THE CI JOB

`.github/workflows/ci.yml` runs on every push to `main` and every pull request, with a concurrency group per ref so a newer push cancels an older run. It has two jobs; the `ios` job runs on `macos-26` with `mobile` as its working directory and does, in order:

1. Selects the newest Xcode 26 on the runner with `xcode-select` and prints its version.
2. Installs XcodeGen with Homebrew.
3. Writes a placeholder `.env.local` with a fake Supabase URL and anon key, then runs `./scripts/make-config.sh`. The comment explains: the generated file "is never committed. The simulator build only needs the values to be present, not real."
4. Runs `xcodegen generate`.
5. Restores `mobile/SourcePackages` from the Actions cache, keyed on the hash of `project.yml`, so the Supabase package is not re-cloned on every run.
6. Picks the first available iPhone simulator by name and runs `xcodebuild test` against it with `CODE_SIGNING_ALLOWED=NO` and `-clonedSourcePackagesDirPath SourcePackages`. Output is teed to `xcodebuild.log` and filtered to errors, warnings from the app's own sources, and the test suite and test case result lines, with `set -o pipefail` so the filter cannot hide a failure.
7. Uploads the full log as an artifact, but only on failure.

No signing, no secrets, no device. The whole XCTest suite runs unsigned on a simulator, which is possible because nothing in it needs an entitlement to be present: `ChartCache` falls back to the caches directory when the App Group container is missing, and the tests that need Apple Intelligence skip.

THE TESTFLIGHT WORKFLOW

`.github/workflows/ios-testflight.yml` runs on every tag matching `v*` and by hand from the Actions tab, in a single concurrency group that does not cancel in progress. It needs five repository secrets, checked up front so a missing one fails in the first step with its name: `SUPABASE_URL` and `SUPABASE_ANON_KEY`, which are baked into `AppConfig.swift` through the same script the CI job uses; and `ASC_KEY_ID`, `ASC_ISSUER_ID` and `ASC_KEY_P8`, an App Store Connect API key with the App Manager role and its `.p8` contents. The header comment states the consequence: "Signing is automatic: with the API key, Xcode creates or refreshes the distribution certificate and profile itself. No certificates are stored here."

```
# .github/workflows/ios-testflight.yml
- name: Set version and build number
  run: |
    BUILD="${GITHUB_RUN_NUMBER}"
    for plist in Sanchara/Info.plist SancharaWidgets/Info.plist; do
      if [[ "${GITHUB_REF_TYPE}" == "tag" ]]; then
        /usr/libexec/PlistBuddy -c "Set :CFBundleShortVersionString ${GITHUB_REF_NAME#v}" "$plist"
      fi
      /usr/libexec/PlistBuddy -c "Set :CFBundleVersion ${BUILD}" "$plist"
    done

- name: Use production push environment
  run: /usr/libexec/PlistBuddy -c "Set :aps-environment production" Sanchara/Sanchara.entitlements
```

After the same Xcode, XcodeGen, config and generate steps as CI, the workflow edits three files in place. The marketing version comes from the tag with its `v` stripped (`v0.3.0` becomes `0.3.0`); on a manual run the value in the plist is kept. The build number is the workflow run number, which App Store Connect requires to be unique per version, and it is written to both the app's and the extension's plist because the two must agree. Then `aps-environment` is switched to `production` in the entitlements file, because "push tokens minted under the development environment are rejected by production APNs, so store builds must carry the production entitlement." The API key is written to the runner's `private_keys` folder with mode 600.

Two `xcodebuild` invocations follow. `archive` builds the `Sanchara` scheme in Release for `generic/platform=iOS` into `build/Sanchara.xcarchive`, with `-allowProvisioningUpdates` and the three authentication-key flags so Xcode can mint the distribution profile. `-exportArchive` then reads `mobile/ExportOptions.plist` and, because that file sets `method` to `app-store-connect` and `destination` to `upload`, sends the build straight to App Store Connect instead of writing an `.ipa`. The plist also sets `automatic signing`, `uploadSymbols` `true` so crash logs symbolicate, and `manageAppVersionAndBuildNumber` `false` so the numbers the workflow just wrote are the ones that ship; it names the team, which this handbook does not print. Both logs are uploaded as an artifact whether or not the run succeeded.

WHY THE VERSION LIVES IN THE TAG

The web app deploys from `main` on every push, and the changelog and `package.json` carry the version. The iOS build number cannot come from those, because two uploads of the same version need different build numbers, and the marketing version should not be edited by hand in two plists. Deriving one from the tag and the other from the run number means a release is `git tag v0.3.0` && `git push --tags` and nothing else on the iOS side.

14.9 Study notes

READ IN THIS ORDER

1. `mobile/project.yml` , for the three targets, the entitlements with their comments, and the six files the extension borrows.
2. `mobile/scripts/make-config.sh` , for the three constants and the two variables it refuses to read.
3. `mobile/Sanchara/Sources/SancharaApp.swift` and `RootView.swift` , for the whole navigation policy in under a hundred lines.
4. `mobile/Sanchara/Sources/Features/Auth/AuthStore.swift` , `State` and `start()` , for how the left half of the state machine moves.
5. `mobile/Sanchara/Sources/Features/Intake/ProfileStore.swift` , for the right half and the offline rule.
6. `mobile/Sanchara/Sources/Design/Theme.swift` , then `app/globals.css` beside it.
7. `mobile/Sanchara/Sources/Services/SancharaAPI.swift` , `request` , `check` and `chatStream` .
8. `mobile/Sanchara/Sources/Services/Push.swift` , `ApsEnvironment` , for why the push environment is read from the profile.
9. `.github/workflows/ci.yml` and `ios-testflight.yml` , for the two build paths, then `mobile/ExportOptions.plist` .

CHECK YOURSELF

Why is there no `Sanchara.xcodeproj` in the repository?

XcodeGen generates it from `mobile/project.yml` , and both CI and the TestFlight workflow run `xcodegen generate` themselves. A generated project cannot suffer a `pbxproj` merge conflict.

Which two environment variables reach the app binary, and which two never do?

`NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` , which the web already ships to browsers and RLS protects. `SUPABASE_SERVICE_ROLE_KEY` and `ANTHROPIC_API_KEY` are never read by `make-config.sh` .

What decides whether a signed-in user sees intake or chat?

`ProfileStore.load()` : a `birth_profiles` row with a non-null `chart` means `.ready` and `ChatView` ; none means `.needsIntake` and `IntakeView` , whose completion closure calls `load()` again.

Why does `chatStream` hold bytes back before yielding text?

The route streams raw UTF-8 and a chunk boundary can split a multi-byte character; bytes are kept until they decode, then coalesced into batches of at least 24 characters so the transcript is not redrawn per byte.

The entitlements file says `aps-environment: development` . How does a TestFlight build end up with `production` ?

The TestFlight workflow edits the file with PlistBuddy before archiving. At runtime `ApsEnvironment.current` reads the embedded provisioning profile rather than the build configuration, so the server is told the environment the token was actually minted under.

Where does the iOS build number come from?

The GitHub Actions run number, written into both the app's and the extension's `Info.plist`; the marketing version comes from the tag with the `v` stripped. `ExportOptions.plist` sets `manageAppVersionAndBuildNumber` to false so App Store Connect keeps them.

Why can the whole XCTest suite run unsigned on a simulator?

Nothing in it needs an entitlement: `ChartCache` falls back to the caches directory without the App Group, and the tests that need Apple Intelligence skip instead of failing.

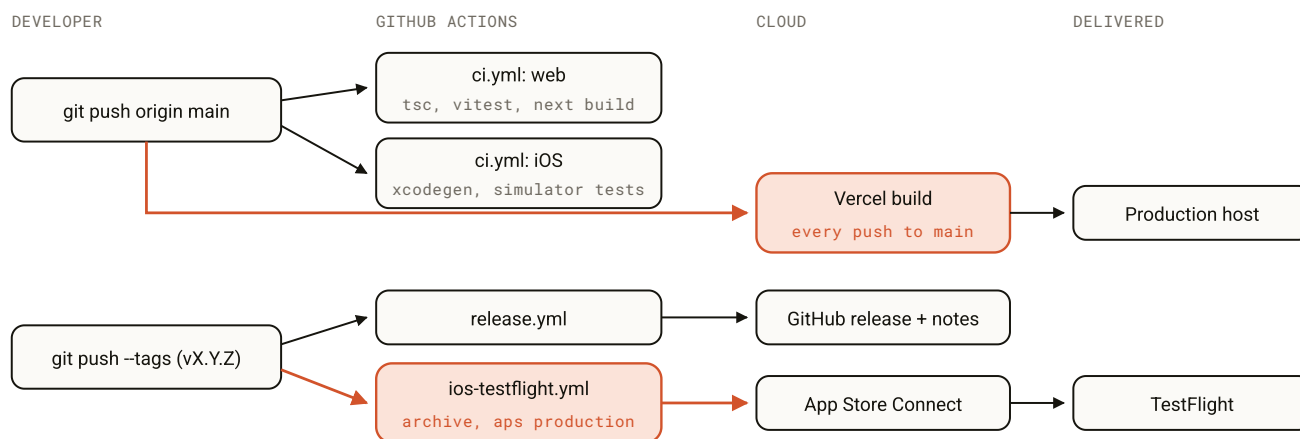
15 Setup, deployment, and release

One repository produces two artifacts: a Next.js app that Vercel builds from every push to `main`, and an iOS archive that GitHub Actions uploads to TestFlight on every version tag. Nothing secret is in git, the production build needs no secrets to compile, and every one-time step that lives outside the repo is written down so it can be done again.

This chapter describes the repository on `main` on 30 September 2026, at version 0.3.0. It is the chapter to read before the first `npm run dev`, before the first deploy to a fresh Vercel project, and before cutting a release. It covers the local setup for the web app and the iOS app, the one-time Supabase and Apple configuration, the Vercel deployment and its two cron schedules, the production checklist that exists because a cron ran for weeks refusing every request, the three GitHub workflows, the release procedure, and the rules about where secrets may live. Where a document under `docs/` names an account, a project reference or a machine, this chapter does not; the procedure is the same for whoever holds the keys.

15.1 The shape of a release

A commit takes two roads out of the repository. Every push to `main` is built by Vercel's Git integration and becomes the production web app; the same push runs `.github/workflows/ci.yml`, which typechecks, tests and builds the web app and builds and tests the iOS app on a simulator. A tag of the form `v*` takes the second road: `release.yml` publishes a GitHub release from the matching section of `CHANGELOG.md`, and `ios-testflight.yml` archives the iOS app with the version from the tag and uploads it to App Store Connect. There is no deploy step in Actions for the web app, and nothing in the repository makes Vercel wait for CI.



Actions and Vercel both start from the same push. Nothing in the repository makes one wait for the other.

Two roads out of the repository. The web app ships on a push; the iOS app ships on a tag. The CI jobs prove the commit; they do not deploy it.

15.2 Local setup

THE WEB APP

The README lists the prerequisites: Node.js 24, a Supabase project, an Anthropic API key, and for iOS, Xcode 26, XcodeGen and an Apple Developer account. The web app starts in three commands: `npm install`, `cp .env.example .env.local` with the values filled in, and `npm run dev`, which serves on port 3000. The migrations under `supabase/migrations/` are applied to the project either with the Supabase CLI (`supabase link` then `supabase db push`) or by pasting each file into the SQL editor, lowest number first. There are seven, and the order matters because later files reference tables the earlier ones create:

<code>supabase/migrations/</code>	
<code>0001_init.sql</code>	admins, profiles, birth_profiles, conversations, messages
<code>0002_avatar.sql</code>	avatar_url and the avatars bucket
<code>0003_numerology_mode.sql</code>	the numerology chat mode
<code>0004_alerts_push.sql</code>	device_tokens, transit_conditions, alerts
<code>0005_daily_readings.sql</code>	daily_readings, one row per user, date and slot
<code>0006_life_events.sql</code>	life_events, life_event_scans
<code>0007_period_readings.sql</code>	period_readings

`.env.example` is the complete list of environment variables, and the comments in it are the documentation. The split between required and optional is enforced in `lib/env.ts`, which is the only place `process.env` is read:

```
// lib/env.ts
import "server-only";

function required(name: string): string {
  const v = process.env[name];
  if (!v) throw new Error(`Missing required env var: ${name}`);
  return v;
}

function optional(name: string): string | undefined {
  const v = process.env[name];
  return v && v.length > 0 ? v : undefined;
}

export const env = {
  supabaseUrl: () => required("NEXT_PUBLIC_SUPABASE_URL"),
  supabaseAnonKey: () => required("NEXT_PUBLIC_SUPABASE_ANON_KEY"),
  supabaseServiceRoleKey: () => required("SUPABASE_SERVICE_ROLE_KEY"),
  anthropicApiKey: () => required("ANTHROPIC_API_KEY"),
  cronSecret: () => optional("CRON_SECRET"),
  apnsKeyId: () => optional("APNS_KEY_ID"),
  apnsTeamId: () => optional("APNS_TEAM_ID"),
  apnsBundleId: () => optional("APNS_BUNDLE_ID"),
  apnsPrivateKey: () => optional("APNS_PRIVATE_KEY")?.replace(/\n/g, "\n"),
};
```

Two details of this file shape everything downstream. Every entry is a function, not a value, so a variable is read when a request needs it and never at module load or at build time. And the file imports `server-only`, so the bundler refuses to include it in anything that ships to a browser. Chapter 16 follows both consequences through the failure modes.

VARIABLE	REQUIRED	WHERE IT COMES FROM	WHAT STOPS WITHOUT IT
<code>NEXT_PUBLIC_SUPABASE_URL</code>	yes	Supabase project settings, API	Every database and auth call
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	yes	Same page, the <code>anon</code> key. Designed to ship in clients; Row Level Security is the boundary	Every database and auth call
<code>SUPABASE_SERVICE_ROLE_KEY</code>	yes	Same page, the <code>service_role</code> key. Bypasses RLS	Admin reads, the cron jobs, device registration, avatar upload, account deletion
<code>ANTHROPIC_API_KEY</code>	yes	The Anthropic console	Readings, alerts, extraction, period meanings
<code>CRON_SECRET</code>	no	Any long random string; Vercel sends it as a bearer token to scheduled routes	The cron route answers 401 to everyone, including Vercel
<code>APNS_KEY_ID</code>	no	The ten-character id of the APNs key	Pushes are skipped; alerts and readings are still written
<code>APNS_TEAM_ID</code>	no	The Apple Developer team	Same
<code>APNS_BUNDLE_ID</code>	no	The app's bundle identifier; <code>.env.example</code> carries it as a default	Same
<code>APNS_PRIVATE_KEY</code>	no	The full contents of the <code>.p8</code> file, BEGIN and END lines included; escaped newlines are unescaped by <code>env.ts</code>	Same

The two `NEXT_PUBLIC_` variables are the only ones a browser may see, and they are the only two the iOS app is ever handed. `SUPABASE_SERVICE_ROLE_KEY` is used by `lib/supabase/admin.ts`, whose comment reads "Bypasses RLS. Only ever import from server code behind an admin auth check."

THE IOS APP

The Xcode project is not committed. `mobile/project.yml` is the XcodeGen spec, and `xcodegen generate` must be run after every change to it and after every new source file. Before that, `mobile/scripts/make-config.sh` writes `mobile/Sanchara/Sources/Generated/AppConfig.swift` from the repo's `.env.local`. The script reads exactly two variables, the Supabase URL and the anon key, and its header says why: those are publishable, and the service-role key and the Anthropic key "are never read here and must never reach the app binary." A third value, `apiBaseUrl`, comes from the `SANCHARA_API_BASE_URL` shell variable if it is set and otherwise defaults to the production API, which is how a debug build is pointed at a local or preview server.


```
cd mobile
./scripts/make-config.sh      # reads the Supabase URL and anon key from ../.env.local
xcodegen generate
open Sanchara.xcodeproj
```

`project.yml` declares three targets: the app, the `SancharaWidgets` extension and a `SancharaTests` unit-test bundle. It also declares every entitlement: associated domains for Universal Links, `aps-environment` for push, Sign in with Apple, and the App Group the widget and Siri read the chart from. Each of those needs a matching capability on the App ID in the Apple Developer portal, which is one-time work covered in the next section. The deployment target is iOS 17; the on-device model features are weak-linked behind `#if canImport(FoundationModels)` and gated at runtime on iOS 26, so the same build runs on both.

15.3 Supabase and Apple, once

Four documents under `docs/` describe configuration that lives in dashboards rather than in git. Each names the project it was written for; the steps do not depend on that. In summary:

- **Project and keys** (`docs/SUPABASE_SETUP.md`). Create a project, copy the URL, the anon key and the service-role key into `.env.local`, enable the Email provider under Authentication, run the migrations, sign up in the app, then make that user an admin with one insert into `public.admins` using the id from the Users page. The admin area at `/admin` has its own login and checks this table.
- **Email codes** (`docs/EMAIL_OTP.md`). Sign-in by email sends a six-digit code, not a link. Supabase decides which to send from the template: a template with only `{{ .ConfirmationURL }}` sends a link and no code, and the app's code screen then has nothing to accept. Two templates, Magic Link and Confirm signup, must include `{{ .Token }}`. Both keep the link as well, so the web app and the Universal Link path keep working. On the app side the flow is `signInWithOTP` then `verifyOTP(email:token:type:)`, with `.email` for passwordless and `.signup` after a password sign-up.
- **Sign in with Apple** (`docs/APPLE_SIGN_IN.md`). The app signs in natively with `signInWithIdToken`, so no web redirect is involved and no environment variable is needed. Two switches must be on: the Apple provider in the Supabase project, with the app's bundle identifier in its Client IDs list, and the Sign In with Apple capability on the App ID, which real devices and TestFlight need for signing. The error `Issuer https://appleid.apple.com is not enabled` means the first switch is off. The document also records a known gap: account deletion does not revoke the Apple token, which Apple's guideline 5.1.1(v) asks for, and lists the three steps closing it would take.
- **Push** (`docs/PUSH_ALERTS.md`). Apply migrations 0004 and 0005, create an APNs key in the Developer portal (it downloads once), and set `CRON_SECRET` and the four `APNS_*` variables in Vercel. The document then explains the twice-daily reading and the dosha sweep, which the alerts chapter covers in full.
- **App Groups** (`docs/SIRI_AND_WIDGETS.md`). Register the group, enable it on both the app's and the widget's App IDs, and let Xcode regenerate the profiles. Without it "the widget shows its empty state forever and Siri says there is no chart", because the two processes are reading different containers.

Finally, the auth redirect URLs. `docs/DEPLOY.md` has Supabase's Site URL set to the production domain and `/auth/callback` on that domain added to the Redirect URLs, with a wildcard pattern for preview deployments if they should authenticate too. The web app's callback route is `app/auth/callback/`.

ORDER OF OPERATIONS

Keys first, migrations second, templates and providers third, admin last. The admin insert needs a user id, which needs a sign-up, which needs the Email provider and the templates. Everything else can be done in any order, and the push and App Group steps can be left until the iOS app needs them; the web app and the API run without them.

15.4 Vercel and the cron schedules

The web app deploys from `main` through Vercel's Git integration. The project's environment variables are set for both Production and Preview; `docs/DEPLOY.md` walks through `vercel link` and `vercel env add` for the four required variables, and `docs/PUSH_ALERTS.md` for the five optional ones. Development is left out of the Vercel store on purpose, because local development reads `.env.local`. After a variable is added, the running functions do not see it until the next deployment; `vercel redeploy --prod` is the step people forget.

Two settings in `next.config.ts` matter for the deployment. `serverExternalPackages: ["swissep-wasm"]` keeps the ephemeris out of the bundler so its `.wasm` binary is shipped intact for the server runtime. And a `headers()` rule serves `/.well-known/apple-app-site-association` as `application/json`, because the file has no extension and Next would otherwise serve it as an octet stream, which Apple rejects; without it Universal Links and the `webcredentials` association silently fail.

The schedule is the whole of `vercel.json`:

```
{
  "crons": [
    { "path": "/api/cron/daily", "schedule": "30 2 * * *" },
    { "path": "/api/cron/daily", "schedule": "30 15 * * *" }
  ]
}
```

02:30 and 15:30 UTC are 08:00 and 21:00 in India, which is where the first users are. The route handler at `app/api/cron/daily/route.ts` declares `runtime = "nodejs"` because the ephemeris is WebAssembly and `maxDuration = 300` because it runs the model once per user per half-day. It checks `Authorization: Bearer` against `CRON_SECRET` and then runs two halves in order: `runDuePredictions`, the readings, then `runDailyAlerts`, the dosha sweep. Each user is handled on their own clock: anything before 14:00 local is the morning slot, anything after is the night slot, and the row is stored against the user's local date under a unique index. That is what makes extra runs harmless, and it is why `docs/GO_LIVE_CHECKLIST.md` says a mixed-timezone user base can move to an hourly schedule at no extra model cost: each hourly run writes only for users whose current slot is still missing. `/api/cron/alerts` still exists for running the dosha sweep on its own by hand.

15.5 The production checklist

`docs/GO_LIVE_CHECKLIST.md` exists because of a specific failure. In mid-September 2026 production had written no daily readings for three weeks and had never sent a dosha alert. The code was deployed and the

crons were firing on schedule, but `CRON_SECRET` was not set, so the route refused every request with a 401, and the four `APNS_*` variables were not set either. Two migrations the timeline feature depended on had also never been applied to the production database. None of that is visible from a green build, which is the point of the checklist. Its steps, generalised:

1. **Log the Vercel CLI into the right account and link the project.** The CLI remembers one login, and `vercel env` against the wrong scope fails in confusing ways. `vercel env ls production` shows what is already set.
2. **Set the secrets.** `CRON_SECRET` from `openssl rand -hex 32`, then `APNS_KEY_ID`, `APNS_TEAM_ID`, `APNS_BUNDLE_ID` and `APNS_PRIVATE_KEY` (the whole `.p8`, BEGIN and END lines included). Then `vercel redeploy --prod`.
3. **Apply any migration the deployed code expects, by hand, in order.** Nothing in the pipeline applies migrations. The README says it plainly: after deploying a version that adds a migration, apply it before traffic reaches the new code. The checklist confirms with a query against `information_schema.tables` for the three timeline tables.
4. **Run the cron by hand.** A `curl` with the bearer secret against `/api/cron/daily` returns JSON with two objects, `predictions` and `alerts`. `predictions.written` should be at least 1; `predictions.skipped` counts users whose current slot was already written, which is the normal result of a second run in the same half-day; `alerts.pushConfigured` says whether the APNs variables were seen. A 401 means the secret is missing or does not match. A query on `daily_readings` grouped by date and slot confirms today's rows.
5. **Check the push environment on each phone.** The app reports `sandbox` or `production` from its signing entitlement when it registers, and the server sends to the matching APNs host. Rows in `device_tokens` written by builds from before that fix may carry the wrong environment; the checklist deletes them by `updated_at` and has each device reopen the app once.
6. **Decide the schedule.** Twice a day at fixed UTC times, or hourly.

TWO FACTS A GREEN DEPLOYMENT CANNOT TELL YOU

Whether the environment variables the cron and the pushes need are present, and whether the database has every table the code will query. The first fails closed and silently (a 401 to a scheduled request that nobody is watching). The second fails loudly but only on the code path that reaches the missing table. Check both after every deploy that touches either.

15.6 Continuous integration

`.github/workflows/ci.yml` runs on every push to `main` and every pull request, under a concurrency group keyed on the ref with `cancel-in-progress`, so a second push to the same branch cancels the run for the first. It has two independent jobs.

THE WEB JOB

On `ubuntu-latest` with Node 24 and the npm cache: `npm ci`, then `npx tsc --noEmit`, then `npm test` (which is `vitest run`), then `npm run build`. The build step carries the comment that explains why the job needs no secrets:

```
# The build needs no secrets: env vars are read at request time, not at
# build time, so a missing key fails the route rather than the build.
- name: Build
  run: npm run build
```

This is the consequence of `env.ts` exposing functions. A `next build` evaluates every module once to discover routes and pre-render what it can; if `env.ts` read `process.env` at the top level, the build would throw on a runner with no variables, and CI would need the production keys just to prove the code compiles. As written, the build succeeds on a bare runner and the first request to a route that needs a missing variable is the thing that fails. The tests, likewise, never need a key: every module that calls the model or the database is stubbed at the boundary, and a single alias in `vitest.config.ts` points `server-only` at the package's empty entry so route handlers and `lib/env.ts` can be imported in a plain Node process.

THE IOS JOB

On `macos-26`, working in `mobile/`: select the newest Xcode 26 on the runner, `brew install xcodegen`, then write a placeholder `.env.local` and run `make-config.sh`. The placeholder is the trick worth knowing. The generated `AppConfig.swift` is never committed, and a build without it does not compile, so the job writes a `.env.local` containing a stand-in URL and a stand-in key, both obviously fake. The comment in the workflow says exactly what is being proved: "The simulator build only needs the values to be present, not real." No test in the iOS suite reaches the network, so nothing ever tries to use them.

Then `xcodegen generate`, a cache of `mobile/SourcePackages` keyed on the hash of `project.yml` (so the Supabase Swift package is fetched once per dependency change, not per run), and one `xcodebuild test` against the first available iPhone simulator with `CODE_SIGNING_ALLOWED=NO`. The output is filtered to errors, warnings in the app's own sources, and test suite lines; the full log is uploaded as an artifact only on failure. The job proves that the spec generates a project, that the app and the widget extension compile, and that the XCTest suite in `mobile/Sanchara/Tests/` passes. It cannot prove signing, push, or anything that needs an entitlement, which is what the TestFlight workflow is for.

15.7 The release and TestFlight workflows

RELEASE.YML

Runs on a pushed tag matching `v*` with `contents: write`. An `awk` program extracts the section of `CHANGELOG.md` whose heading is `## [X.Y.Z]`, stopping at the next version heading or at the link references at the foot of the file. If no section exists the workflow warns and falls back to the commit subjects since the previous tag. Then `gh release create` publishes the release with the tag as its title and the extracted notes as its body. If the release already exists the job exits cleanly, so re-pushing a tag is safe.

IOS-TESTFLIGHT.YML

Runs on the same tags, or by hand from the Actions tab, under a concurrency group named `testflight` that does not cancel in-progress runs, because a half-finished upload is worse than a queued one. The job:

1. Checks that all five repository secrets are set and fails with a named error for each one missing, before any Xcode work starts.

2. Selects Xcode 26, installs XcodeGen, writes `.env.local` from the `SUPABASE_URL` and `SUPABASE_ANON_KEY` secrets, runs `make-config.sh`, and generates the project. These are the same two publishable values a developer's `.env.local` holds; the API base URL stays at the production default.
3. Sets the version. On a tag, `CFBundleShortVersionString` in both the app's and the widget's `Info.plist` becomes the tag without its `v`; on a manual run the value in `project.yml` is kept. `CFBundleVersion` becomes the workflow run number in both, because App Store Connect requires a unique build number per version.
4. Sets `aps-environment` to `production` in `Sanchara/Sanchara.entitlements` with PlistBuddy. The committed spec says `development`, which is right for a debug build on a phone, and wrong for TestFlight: "Push tokens minted under the development environment are rejected by production APNs, so store builds must carry the production entitlement."
5. Writes the App Store Connect API key from `ASC_KEY_P8` to `~/private_keys/AuthKey_<id>.p8` with mode 600.
6. Archives with `-allowProvisioningUpdates` and the three authentication key flags. With an API key, Xcode creates or refreshes the distribution certificate and profile itself, which is why no certificate is stored in GitHub.
7. Exports with `mobile/ExportOptions.plist`, whose method is `app-store-connect` and whose destination is `upload`, so the build goes straight to App Store Connect instead of producing an `.ipa`. Symbols are uploaded; version management is left to the workflow, not to Apple.
8. Uploads the archive and export logs as an artifact whether or not the job succeeded.

REPOSITORY SECRET	VALUE	USED BY
<code>SUPABASE_URL</code>	Same as <code>NEXT_PUBLIC_SUPABASE_URL</code>	Baked into <code>AppConfig.swift</code>
<code>SUPABASE_ANON_KEY</code>	Same as <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Baked into <code>AppConfig.swift</code>
<code>ASC_KEY_ID</code>	App Store Connect API key id	Archive and export authentication
<code>ASC_ISSUER_ID</code>	App Store Connect issuer id	Archive and export authentication
<code>ASC_KEY_P8</code>	Full contents of the downloaded <code>.p8</code>	Written to the runner's <code>private_keys</code> for the archive

The README adds that the API key needs the App Manager role and that the app record must already exist in App Store Connect. The team id the export uses is in `ExportOptions.plist` and `project.yml`; it is an identifier, not a secret, and it is the same value `APNS_TEAM_ID` carries on the server.

15.8 The release procedure

Releases are tagged from `main`. The README's "Releasing" section is four steps:

1. Make sure CI is green.
2. Add a section to `CHANGELOG.md` and bump `version` in `package.json`.
3. Commit, then `git tag -a vX.Y.Z -m "vX.Y.Z"`.
4. `git push origin main --tags`.

The push of `main` deploys the web app through Vercel. The tag triggers both release workflows. Nothing else is needed, and nothing in the workflows reads `package.json`: the web app's version is informational, the iOS version comes from the tag, and the release notes come from the changelog. Keeping the three in step is a discipline, not an enforcement.

`CHANGELOG.md` follows Keep a Changelog 1.1.0 and the project uses Semantic Versioning; the file says so in its first paragraph. Each version has a heading of the form `## [0.3.0] - 2026-09-18`, subsections `Added`, `Changed` and `Fixed` as needed, and a link reference at the foot of the file pointing at the GitHub release for that tag. The `awk` in `release.yml` depends on exactly this shape: the heading must start `## [` followed by the bare version, and the link references must be the last thing in the file. Three versions exist: 0.1.0 and 0.2.0 on 16 September 2026, and 0.3.0 on 18 September. The entries are written for the person using the app, in bold lead-ins followed by a sentence or two of plain language; the commit history is where the engineering detail lives.

VERSIONS ON THE IOS SIDE

`project.yml` pins `CFBundleShortVersionString` at 1.0 and `CFBundleVersion` at 1 for local builds. Those are placeholders. A tagged workflow run overwrites both, and the run number guarantees each upload is distinct. A build archived by hand in Xcode would carry 1.0 (1) and would collide with any earlier hand-archived build; upload through the workflow instead.

15.9 Secrets discipline

The rule in `docs/DEPLOY.md` is short: locally, secrets live only in `.env.local`; in the cloud, only in Vercel's encrypted environment store; and no real key or service-role secret is ever committed. The repository enforces the first two with `.gitignore` and with where values are read.

```
.gitignore, the entries that matter here
.env*                                every env file ...
!.env.example                       ... except the documented template
.vercel                             the CLI's link to a project
mobile/Sanchara.xcodeproj/          generated by XcodeGen
mobile/Sanchara/Sources/Generated/  AppConfig.swift, written from .env.local
mobile/SourcePackages/              resolved Swift packages
mobile/build/                       archives and exports
supabase/.temp/                     the CLI's project reference and versions
xcuserdata/, *.xcuserstate          per-user Xcode state
```

SECRET OR CREDENTIAL	LOCALLY	IN PRODUCTION	REACHES A CLIENT?
Supabase URL and anon key	<code>.env.local</code> , and <code>AppConfig.swift</code> generated from it	Vercel env store; GitHub secrets for the TestFlight build	Yes, by design. The browser bundle and the iOS binary both carry them; RLS is the boundary
Supabase service-role key	<code>.env.local</code>	Vercel env store	Never. Read only in <code>lib/supabase/admin.ts</code> , which imports <code>server-only</code> ; <code>make-config.sh</code> does not read it
Anthropic API key	<code>.env.local</code>	Vercel env store	Never. Read only in <code>lib/anthropic.ts</code> , <code>server-only</code>
<code>CRON_SECRET</code>	Optional in <code>.env.local</code>	Vercel env store; Vercel sends it back on scheduled calls	Never
APNs key id, team id, bundle id, <code>.p8</code>	Optional in <code>.env.local</code>	Vercel env store	Never. Read only in <code>lib/push/apns.ts</code> , <code>server-only</code> . The <code>.p8</code> is also kept offline because Apple lets it download once
App Store Connect API key	Not needed	GitHub secrets; written to the runner for one archive	Never
Signing certificates	Xcode automatic signing	None stored; Xcode mints them from the API key	Never
A user's session	Cookies (web), Keychain via <code>supabase-swift</code> (iOS)	Same	Its own device only; every <code>/api/*</code> call carries it as a cookie or a bearer token

Three code-level habits back the table. First, `server-only` at the top of `lib/env.ts`, `lib/supabase/admin.ts`, `lib/supabase/route.ts`, `lib/anthropic.ts`, `lib/push/apns.ts` and every alerts module turns an accidental client import into a build error. Second, the user id in every mutating route comes from the verified session, never from the request body, so a leaked anon key still cannot act as someone else. Third, `make-config.sh` only knows the names of the two publishable variables; there is no code path by which a server key could be generated into the app.

WHAT A LEAKED REPOSITORY EXPOSES

The bundle identifier, the team id, the production hostname, the App Group name and the URL scheme. All are already visible in the shipped app. No key, no token, no project reference.

15.10 Study notes

READ IN THIS ORDER

1. `.env.example` , for every variable and the comment that explains it.
2. `lib/env.ts` , for required versus optional and why each is a function.
3. `vercel.json` and `app/api/cron/daily/route.ts` , for the schedule and the guard.
4. `.github/workflows/ci.yml` , both jobs; note the two comments about secrets.
5. `.github/workflows/ios-testflight.yml` , top to bottom; it is the whole iOS release.
6. `.github/workflows/release.yml` , for the changelog contract.
7. `mobile/scripts/make-config.sh` and `mobile/project.yml` , for what the app is handed and which entitlements it claims.
8. `docs/GO_LIVE_CHECKLIST.md` , for the failure that motivated it.

CHECK YOURSELF

Why does `npm run build` succeed on a CI runner with no environment variables?

Because `lib/env.ts` exposes functions that read `process.env` when called, inside a request. Nothing reads a variable at module load, so the build evaluates every module without touching one.

What does the iOS CI job write to `.env.local` , and why is that safe?

A placeholder URL and a placeholder key. `make-config.sh` refuses to run without values, the compiler needs the generated file, and no test reaches the network, so the values only need to exist.

Which two things does the TestFlight workflow change in the checked-out tree before archiving?

The version and build number in both `Info.plist` files, and `aps-environment` in the app's entitlements, from `development` to `production` .

A cron fires on schedule and nothing is written. What are the first two things to check?

That `CRON_SECRET` is set in the Production environment and a redeploy happened after it was added, and that every migration the code expects has been applied to the production database.

Why can the anon key ship in the iOS binary when the service-role key cannot?

Every table has Row Level Security, so the anon key can only read and write rows the signed-in user owns. The service-role key bypasses RLS entirely; it is read only in server-only modules.

What in `CHANGELOG.md` does `release.yml` depend on?

A heading of exactly `## [X.Y.Z]` for the tagged version, and the link references being the last block in the file; the `awk` stops at either the next version heading or the first reference line.

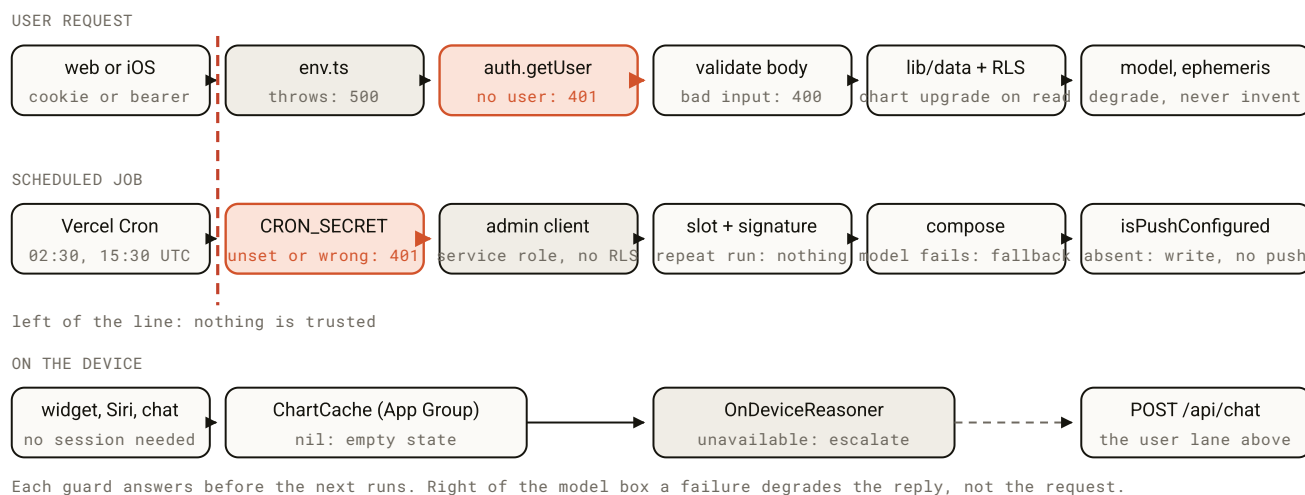
16 What breaks, and how it is handled

A catalogue of the ways the system fails, and for each one the code that meets it. The organising rule is that a failure should cost the user the least thing it can: a push rather than a reading, a reading rather than a chart, a chart upgrade rather than a request, and never a wrong number presented as a right one.

This chapter describes the code on `main` on 30 September 2026, version 0.3.0. Every item below was read from the file it names, and every test named exists in the repository under that name. Where a failure mode has no test, the item says "untested" rather than implying one. Items are grouped by the layer they belong to: configuration, authentication, the model, place and time, stored data, push, the device, untrusted text, and deletion. A table at the end puts the whole catalogue on one page.

16.1 Where the guards sit

Three kinds of request enter the system and each passes a fixed sequence of guards. A user request, from the web app or the iOS app, reaches a route handler, which builds a Supabase client for the caller (reading two environment variables as it does so), asks that client who the user is, validates the body, and only then touches the database and the external services. A scheduled request from Vercel Cron passes one guard, the cron secret, and then runs under the service-role client with its own guards against repetition. A request on the phone from a widget, a Siri intent or the chat's local router never leaves the device unless the on-device path cannot answer, in which case it becomes the first kind.



The three lanes and the guard at each step. The hot boxes are the ones that turn a request away; every box to their right handles a failure by giving less rather than by giving up.

16.2 Configuration

A REQUIRED ENVIRONMENT VARIABLE IS MISSING

Symptom. The build succeeds and the deployment is green; the first request to a route that needs the variable returns a 500, and the function log carries `Missing required env var: NAME`.

Handling. `lib/env.ts` exposes one function per variable. `required()` reads `process.env` when called and throws with the name; `optional()` returns `undefined` for an absent or empty value. Because the read happens inside a request, a missing key fails that request rather than the build, which is the property `.github/workflows/ci.yml` relies on when it runs `next build` with no secrets at all (section 15.6). The variable is first touched where a client is built: `createRouteSupabase` reads the URL and anon key, `createAdminSupabase` the service-role key, `anthropic()` the model key. In a route handler that throw happens before the handler's own `try`, so Next.js turns it into a 500. The file also imports `server-only`, which makes any import from client code a build error rather than a leak; the same alias in `vitest.config.ts` is what lets tests import it.

Test. Untested. Nothing exercises `required()` against an empty environment.

THE CRON ROUTE IS CALLED WITHOUT CRON_SECRET

Symptom. The schedule fires, nothing is written, and the invocation log shows a 401.

`docs/GO_LIVE_CHECKLIST.md` records production in exactly this state for three weeks.

Handling. Both `app/api/cron/daily/route.ts` and `app/api/cron/alerts/route.ts` begin the same way:

```
const secret = env.cronSecret();
if (!secret || request.headers.get("authorization") !== `Bearer ${secret}`) {
  return new Response("Unauthorized", { status: 401 });
}
```

An unset secret and a wrong secret are the same case. The alerts route's comment gives the reason: with no secret the route "refuses everything rather than exposing a job that spends model tokens to anyone who finds the path." Failing closed is deliberate; the cost is that a forgotten variable is silent until someone looks at the logs or the tables, which is what the checklist's `curl` step exists to do.

Test. Untested. There is no `route.test.ts` under either cron directory.

APNS CREDENTIALS ARE ABSENT

Symptom. Locally, or on a deployment before the four `APNS_*` variables were set: readings and alerts appear in the app's inbox but no notification arrives. The cron's JSON reports `alerts.pushConfigured: false` and `pushesSent: 0`.

Handling. `isPushConfigured()` in `lib/push/apns.ts` requires all four values. Both jobs write their rows first and call `pushToDevices` last, and that function returns 0 before it reads any device row when push is not configured. Should `ApnsClient.send` be reached anyway, it returns `{ ok: false, status: 0, reason: "APNsNotConfigured" }` without opening a connection. `.env.example` says the same in one line: "Leave

blank locally: alerts are still written, pushes are skipped." The client also survives Apple being unreachable: the HTTP/2 session installs an error handler so a socket failure cannot become an unhandled event, each request has a ten-second timeout, and the provider token is minted once and reused for 45 minutes because Apple rate-limits regeneration.

Test. Untested. `lib/push/apns.ts` has no test file, and the `pushConfigured` flag in the summaries is not asserted anywhere.

16.3 Authentication

A BAD, EXPIRED OR MISSING BEARER TOKEN

Symptom. The iOS app gets a 401 with the body `Unauthorized` from any authenticated route. The web app, which sends cookies, is unaffected.

Handling. Two functions in `lib/supabase/route.ts`. `bearerTokenFrom` returns `null` when the header is absent, when the scheme is not `bearer` (case-insensitively), or when the scheme is present but the token is empty. `createRouteSupabase` then chooses: no token means the cookie-backed client from `lib/supabase/server.ts`, so a web caller never notices this code exists; a token means a fresh `supabase-js` client that forwards it as an `Authorization` header with `persistSession` and `autoRefreshToken` off. The route does not inspect the token itself. It calls `supabase.auth.getUser()`, and a forged, expired or revoked token yields no user, at which point every mutating route returns 401 before reading the body. For a valid token, Row Level Security evaluates as that user for the rest of the request, which is the reason the anon key can ship in the binary. On the web, pages under `/app` and `/admin` (except `/admin/login`) are redirected to the login page by `updateSession` in `lib/supabase/middleware.ts` when there is no user; the API routes still check for themselves.

Tests. `lib/supabase/route.test.ts`: "extracts the token from an Authorization header", "accepts a lowercase scheme", "returns null when the header is absent", "returns null for a non-bearer scheme", "returns null when the scheme is present but the token is empty". The 401 on a missing user is pinned per route: `app/api/devices/route.test.ts` "rejects an unauthenticated caller", `app/api/account/route.test.ts` "returns 401 and deletes nothing when unauthenticated", `app/api/profile/route.test.ts` "returns 401 when there is no user". An expired token reaching `getUser` is not itself tested; the tests stub the user as absent.

16.4 The model

THE MODEL API FAILS DURING A CHAT READING

Symptom. One of two things, depending on when. Before the stream starts: a 500 whose body reads "Something went wrong preparing your reading. Please try again." After it starts: the reply stops and a final line appears in the transcript, `[The stars are momentarily clouded. Please try again.]`.

Handling. `app/api/chat/route.ts` does all its preparation, the profile read, the chart upgrade, the transit chart, the timeline context, the conversation and the history, inside one `try` that ends in the 500 above. Two of those steps have their own softer handling first: a failed `transitChart` sets `transits` to `undefined` and the reading proceeds without today's sky, and a failed `loadTimeline` is logged and skipped.

`lib/astrology/transits.ts` also deletes a rejected promise from its per-day cache, with the comment "never cache a failure", so one bad ephemeris call does not poison the day for every user in that degree cell.

Once the `ReadableStream` has been returned the status is already 200 and the `x-conversation-id` header has been sent, so a mid-stream error cannot change the response code. The stream's `start` wraps the model call in a `try / catch / finally`: the catch enqueues the apology line, and the `finally` saves whatever assistant text had arrived, if any, and closes the stream. A partial reading is therefore kept in the conversation rather than lost, and the user's own message was already saved before the stream began. On iOS, `ChatStore.send` in `mobile/Sanchara/Sources/Features/Chat/ChatStore.swift` catches a thrown stream error and puts its description into the still-empty assistant bubble, or into `errorMessage` if text had already arrived; a `CancellationError` from a superseding message leaves the transcript as it was.

Tests. Untested on the chat route; there is no test file under `app/api/chat/`. The bounded-history rule the route depends on is tested in `lib/data/history.test.ts`: "keeps everything when it fits the budget", "drops the oldest turns first", "never starts on an assistant turn", "keeps the newest turn even when it alone exceeds the budget", "returns nothing for an empty conversation".

THE MODEL FAILS WHILE COMPOSING A READING OR AN ALERT

Symptom. A push arrives, or a row appears in the inbox, with plain wording: "Your reading for today" and a sentence inviting the user to open the app, or for a dosha change, "Sade Sati is active" with the condition's own detail. Nothing is skipped.

Handling. Both composers treat the model as optional. In `lib/alerts/compose.ts`, `composeAlert` catches any error, and also treats output that `parseAlertCopy` cannot read as a failure, then returns `fallbackCopy(started, ended)`, which names the conditions from the facts the engine already decided. In `lib/alerts/predictions.ts`, `composePrediction` does the same and returns `fallbackPrediction(slot, firstName)`. The parser accepts a delimited `TITLE / BODY / DETAIL` shape and, failing that, the older JSON shape, because markdown inside a JSON string was found to break two replies in seven. The comment above `fallbackPrediction` states the rule: "Never leave the slot empty: a plain prompt to open the app still beats silence."

Tests. `lib/alerts/compose.test.ts`: "still names the condition when the model call fails", "reads as relief when a condition has ended", "returns null when neither shape is present", "rejects a delimited reply missing its detail", "still accepts a JSON reply", "rejects output missing a field". `lib/alerts/predictions.test.ts`: "still says something useful when the model call fails".

16.5 Place and time

A BIRTHPLACE THAT CANNOT BE GEOCODED

Symptom. The user types a town and the suggestion list stays empty, or the geocoder is down.

Handling. `geocodePlace` in `lib/geo.ts` returns an empty array for a query under two characters, for a non-OK response from the geocoder, and for a response with no results; it never throws for those. `GET /api/geocode` is public and returns that array. The web form in `components/IntakeForm.tsx` offers a second path: "Can't find your birthplace? Enter coordinates manually", which takes a latitude and longitude and derives the timezone in the browser with `resolveTimezone`, the same `tz-lookup` call the server uses,

so a chart can still be computed for a village no geocoder knows. The iOS form in `mobile/Sanchara/Sources/Features/Intake/IntakeView.swift` debounces the search by 300 milliseconds, turns a thrown request into an empty list with `try?`, and keeps Save disabled until a place has been picked; it has no manual coordinate entry. On the server, `POST /api/profile` rejects a missing field or a non-finite coordinate with a 400 before any chart is computed, and a chart computation that throws returns a 500 reading "Could not compute your chart. Please try again." The web app's `saveIntake` server action in `app/(app)/app/intake/actions.ts` has no equivalent catch: a compute failure there surfaces as a Next.js error page.

Tests. `app/api/profile/route.test.ts` : "returns 400 when a required field is missing", "returns 400 when lat is not a number", "saves the profile and returns 200 for a valid payload". The empty-result paths of `geocodePlace` are untested; `lib/geo.test.ts` covers only `resolveTimezone`.

TIMEZONE EDGE CASES

Symptom. A reading lands at the wrong hour, or a chart is computed for the wrong instant.

Handling, and where it stops. The timezone is never typed by anyone. It is derived from coordinates by `tz-lookup`, stored with the profile, and reused: the iOS form's `prefill` keeps the stored coordinates and zone on edit because "the birthplace has not moved." `computeChart` in `lib/astrology/chart.ts` parses `birthDate` and `birthTime` with Luxon in that zone and converts to UT before the ephemeris sees them. The scheduled jobs read `profile.timezone || "UTC"`, decide the slot from the user's local hour with `slotForHour` (before 14:00 is morning), and store `for_date` as the user's local date, so "my reading for the 26th" means their 26th. The transit chart is sampled at midday UTC once per day per whole-degree cell of latitude and longitude, which keeps the prompt text stable for the cache; the comment notes only the Moon moves enough within a day to matter. Two edges are not guarded. `tzlookup` throws `RangeError("invalid coordinates")` outside the valid range, and the web form's manual mode calls it with any finite numbers, so a latitude of 95 throws in the browser. And Luxon given an unknown zone name returns an invalid `DateTime`, whose fields are `NaN`, and nothing checks validity before `julday`; in practice the zone always comes from `tz-lookup`, so the path is unreachable through the forms. Users far from the fixed UTC schedule can have both cron runs land in the same half of their day and get one reading instead of two, which `docs/PUSH_ALERTS.md` documents together with the hourly schedule that would fix it.

Tests. `lib/geo.test.ts` : "maps New Delhi coordinates to Asia/Kolkata", "maps New York coordinates to America/New_York". `lib/alerts/slots.test.ts` : "splits the user's local day in two", "returns the current slot when it has not been written", "returns nothing when the current slot already exists", "does not backfill a slot the user has moved past". The invalid-coordinate and invalid-zone paths are untested.

16.6 Stored data

A CHART STORED BEFORE WHOLE-SIGN HOUSES

Symptom. None visible, which is the design. Before 0.3.0 the stored Vedic chart carried Placidus house numbers under whole-sign labels, so the kundli and the PDF could draw a planet in a house whose sign it was not in.

Handling. Every chart now carries `schemaVersion`, and `CHART_SCHEMA_VERSION` in `lib/astrology/constants.ts` is 2. `isChartStale` in `lib/astrology/derived.ts` is true when the stored

version (1 if absent) is lower, or when the chart has no `derived facts`. `ensureCurrentChart` in `lib/data/birthProfile.ts` recomputes both traditions together with `Promise.all`, writes them back in one update, and returns the upgraded row; the comment explains that both are recomputed "so a profile is never half upgraded", and that doing it on read "means no migration and no backfill job." It runs from `getBirthProfile`, which the three server-component pages use, and from the chat route. Any failure, whether the compute throws or the write comes back with an `error`, is logged with its stage and the original profile is returned, because an upgrade that cannot be written is not a reason to fail the request that triggered it. Readers that only need facts do not wait for the upgrade: `factsFor` derives them in process from the stored positions, and `deriveFacts` recomputes houses whole-sign regardless of the stored numbers. The alerts job has a companion guard: `partitionForAlerts` in `lib/alerts/run.ts` never notifies a natal condition once the user has natal rows on record, because a natal condition "cannot start or end, so a change in one is always an artifact of the computation changing."

Tests. `lib/data/birthProfile.test.ts`: "returns the profile unchanged when there is no stored vedic chart", "returns the profile unchanged when the stored chart is already current", "recomputes and writes both traditions together for a stale chart", "swallows a thrown compute/write exception and returns the original profile", "swallows a resolved write error (RLS/constraint) and returns the original profile unchanged", "returns an upgraded profile when the stored chart is stale". `lib/astrology/derived.test.ts`: "recomputes houses whole-sign, ignoring the stored numbers". `lib/alerts/run.test.ts`: "alerts natal conditions on a first read, when no natal rows exist", "never alerts a natal condition once the user has natal rows", "keeps transit conditions alertable in both cases".

A FAILED PROFILE READ THAT LOOKED LIKE A MISSING PROFILE

Symptom. Fixed in 0.3.0. A user with a chart opened the app during a database hiccup and was sent to the intake form as though they had never filled it in.

Handling. `supabase-js` resolves `{ data: null, error }` on a failed read rather than throwing, and the old `getBirthProfile` ignored `error` and returned `data ?? null`. It now throws with the error's message, matching `saveBirthProfile` in the same file, while a genuine no-row result, which `maybeSingle()` reports as `{ data: null, error: null }`, still returns `null`. The three pages that read through it (`app/(app)/app/page.tsx`, `chat/page.tsx` and `profile/page.tsx`) now surface an error instead of the intake form. The chat route does its own read and checks `profileError` explicitly, returning a 500 that says "Something went wrong loading your profile."

Tests. `lib/data/birthProfile.test.ts`: "throws on a genuine read failure instead of reporting it as no profile", "returns null when there is no stored profile".

DUPLICATE CRON RUNS

Symptom. None. The job can be fired twice in a minute, or moved to an hourly schedule, and a user still gets one morning reading, one night reading and one alert per change.

Handling. Three mechanisms. First, `daily_readings` has `unique (user_id, for_date, slot)` from `supabase/migrations/0005_daily_readings.sql`, and `writeDueReading` reads the slots already written for today before deciding anything through `isSlotDue`. If two runs race past that read, the insert fails with Postgres code `23505`, which the code treats as "already done" and counts as `skipped`. Second, `transit_conditions` has a partial unique index, `(user_id, signature) where ended_on is null`, from `0004_alerts_push.sql`, and `runAlertsForUser` diffs today's condition signatures against the open rows,

so an unchanged sky inserts nothing, closes nothing and sends nothing. Third, `deriveHadNatalRows` fails closed: if the count of natal rows errors, the user is treated as already having them, so a transient database error cannot turn a standing natal dosha into a "has begun" push. Both jobs also bound their work to 500 profiles per run and count a failing user in `failures` rather than aborting the run.

Tests. `lib/alerts/slots.test.ts`, the four names above. `lib/alerts/run.test.ts`: "is false when the count query succeeds with zero natal rows", "is true when the count query succeeds with at least one natal row", and the test whose name begins "fails CLOSED", which passes an errored count and expects `true`. The `23505` branch is untested.

ACCOUNT DELETION ORDERING

Symptom. The user taps Delete on the profile screen and confirms.

Handling. `DELETE /api/account` takes the user id from the verified session, never from the body, so a caller can only delete themselves, then calls `admin.auth.admin.deleteUser(user.id)` with the service-role client. Every table references `auth.users(id)` on delete cascade: `admins`, `profiles`, `birth_profiles`, `conversations` (and through it `messages`), `device_tokens`, `transit_conditions`, `alerts`, `daily_readings`, `life_events`, `life_event_scans` and `period_readings`. One call removes everything the database holds, and because `device_tokens` goes with it the next cron cannot push to the phone. If the call fails the route returns a 500 with a message and the account is untouched. On the phone, `ProfileView.deleteAccount` calls the API first and only on success runs `AuthStore.signOut`, which unregisters the device (a no-op now, but the same path sign-out always takes), signs out of Supabase, and clears the App Group cache so the widget stops showing the deleted chart. Two things are not removed: the avatar object in the `avatars` storage bucket, which the route never touches, and the Sign in with Apple grant, which `docs/APPLE_SIGN_IN.md` lists as a known gap against guideline 5.1.1(v).

Tests. `app/api/account/route.test.ts`: "returns 401 and deletes nothing when unauthenticated", "deletes only the calling user", "returns 500 when deletion fails". The cascade itself is a database constraint and is not tested from code.

16.7 Push

A TOKEN MINTED UNDER THE WRONG APNS ENVIRONMENT

Symptom. The cron reports `pushesSent: 0` for a device that is registered, and the function log shows `apns send failed 400 BadDeviceToken`. Before 0.2.0 a Release build run from Xcode produced exactly this: it reported `production` from its build configuration while carrying a development entitlement.

Handling. The environment is read from the truth APNs itself uses. `ApsEnvironment.current` in `mobile/Sanchara/Sources/Services/Push.swift` opens `embedded.mobileprovision`, finds the plist inside the signed envelope by searching for `<plist` and `</plist>`, and returns `production` only when `aps-environment` is exactly that string; anything else, including no profile at all (the simulator), is `sandbox`. The app sends that word with every registration. `POST /api/devices` checks the token is 64 to 200 hex characters, maps anything other than `sandbox` to `production`, and upserts on the token with the admin client so a phone handed to a new account changes owner. `ApnsClient.send` picks the host from the row's environment, and when Apple answers 410, `Unregistered` or `BadDeviceToken`, the row is deleted: "Apple only tells you a token is dead when you use it." A token registered under the wrong host is therefore dropped

on first use and written back correctly at the next launch, since `PushStore.register` runs whenever iOS hands the app a token. The TestFlight workflow removes the remaining way to get this wrong by setting the entitlement to `production` before archiving.

Tests. `mobile/Sanchara/Tests/ApsEnvironmentTests.swift`: `testDevelopmentProfileIsSandbox`, `testProductionProfileIsProduction`, `testProfileWithoutThePushEntitlementIsSandbox`, `testNoPlistAtAllIsSandbox`, `testAnUnknownValueIsNotTrustedAsProduction`, `testSimulatorBuildReportsSandbox`. `app/api/devices/route.test.ts`: "rejects a malformed token", "stores the token against the session user, not the body", "defaults an unknown environment to production", "deletes only this user's row for that token". The dead-token deletion in the jobs is untested.

16.8 The device

A WIDGET WITH NO CACHED CHART

Symptom. A freshly installed app, a signed-out phone, or an App Group that was never enabled. The widget must draw something.

Handling. Neither widget fetches. `ReadingProvider` in `mobile/SancharaWidgets/ReadingWidget.swift` reads `ChartCache.shared.loadReading()`, and when that is `nil` the entry's `reading` is `nil` and the view shows its empty state, "Open Sanchara to get your first reading." `PeriodProvider` in `PeriodWidget.swift` tries three sources in order: the cached period with its theme, then the cached chart's `dasha` without a theme ("the dates are true"), then the empty state "Open Sanchara to map your periods." Both empty states name the next action rather than apologising. `ChartCache` itself, in `mobile/Sanchara/Sources/Services/ChartCache.swift`, falls back to the app's own caches directory when the App Group container is unavailable, so the app keeps working and only the sharing stops; `docs/SIRI_AND_WIDGETS.md` names that as the symptom of a missing App Group capability. Every read is a try?, so a corrupt file reads as absent. The reading cache carries an `isFresh` flag so a widget "should not keep insisting on yesterday's morning reading"; today the Siri snippet in `IntentData.swift` checks it and the widget provider does not. Sign-out calls `clear()`, which removes all four files, so the widget of a shared phone cannot show the previous account.

Tests. `mobile/Sanchara/Tests/PeriodProgressTests.swift`: `testCachedPeriodSurvivesTheRoundTrip`, `testClearRemovesTheCachedPeriod`, `testFractionClampsOutsideThePeriod`, `testZeroLengthAndMalformedDatesGiveZero`. The empty views themselves are untested.

THE ON-DEVICE MODEL IS UNAVAILABLE

Symptom. An iOS 17 phone, an iOS 26 phone without Apple Intelligence, or a model that refuses. The question must still be answered.

Handling. Three layers, each of which falls through to the server. `OnDeviceReasoner.route` in `mobile/Sanchara/Sources/Features/Chat/OnDeviceReasoner.swift` is wrapped in `#if canImport(FoundationModels)` so the app builds for iOS 17, then gated on `#available(iOS 26.0, *)` and on `isAvailable`, which is `SystemLanguageModel.default.availability == .available`; any of those false means `.escalate`. When the model runs, a refusal or a thrown error in `isLookup` returns `false` and in `answer` returns `nil`, with the comment that this "is not an error worth surfacing: the question simply goes to Claude, which is where it went before." An answer that never called the lookup tool is discarded as a

guess. `ChatStore.send` only tries the reasoner for a Vedic question that is not the opening reading and not an explicit request for the server; on `.escalate` it streams from `/api/chat` exactly as before. Because Apple's model refuses astrology, everything it is shown is framed as a reference table by `ChartFacts`, and a test guards that framing because a slip would make the feature stop answering silently.

Tests. `mobile/Sanchara/Tests/OnDeviceRoutingTests.swift` :

```
testQuestionsAboutLifeNeverReachTheModel , testQuestionsNamingPartOfTheChartGetOffered ,  
testInterpretiveQuestionsStillDoNotReachTheTable ,  
testAMissingBodyReturnsNothingRatherThanAGuess , testNothingHandedToTheModelNamesAstrology .
```

The availability gate needs a device and is untested.

16.9 Untrusted text

PROMPT INJECTION THROUGH CHAT HISTORY

Symptom. A user, or text pasted by a user, tells the model to ignore its instructions or to invent a placement.

Handling, honestly stated. There is no explicit prompt-injection defence in the repository, and the file the name suggests, `lib/data/injection.test.ts`, is about dependency injection: its `describe` is "data helpers accept an injected client", and its three tests are "saveBirthProfile writes through the injected client", "appendMessage writes through the injected client" and "falls back to the cookie client when none is injected". What does exist limits the blast radius. User messages are sent to the model in the `user` role and are never spliced into the system prompt; the system prompt in `lib/astrology/prompt.ts` instructs the model to "Never state a placement, lordship, aspect, dasha, condition or number that is not listed above" and to "Interpret only this data"; and the model cannot act, since it has no tools and its output is text that is stored and shown. History is bounded by `selectHistory` to 6,000 characters, newest first, never starting on an assistant turn. The one place chat history is mined for facts, the timeline scan in `app/api/timeline/scan/route.ts`, reads only rows with `role = 'user'` ("Assistant turns are full of astrological narrative that reads like biography and would extract as fiction"), tells the model to "Never infer one from a question they asked, from a hypothetical, or from something you read in an astrological reading", quotes the user's own words as evidence, and writes nothing until the user confirms each candidate; `POST /api/life-events` then rejects a date before birth or in the future.

Tests. Prompt injection as such: untested. The surrounding limits are: `lib/data/history.test.ts`, the five names in 16.4; `lib/timeline/extract.test.ts`: "drops events outside the person's life", "drops impossible dates", "reads a fourth field as evidence, so a pipe in a title becomes the quote", "tells the model how to resolve relative dates and ages, and to keep undated events".

16.10 The catalogue

FAILURE	WHERE IT IS HANDLED	WHAT THE USER SEES	PROOF
Required env var missing	<code>lib/env.ts</code> , <code>required()</code>	500 on the first route that needs it; the build is unaffected	untested
Cron without <code>CRON_SECRET</code>	<code>app/api/cron/daily/route.ts</code>	Nothing; 401 in the log	untested
APNs absent	<code>isPushConfigured</code> , <code>pushToDevices</code>	Rows in the inbox, no notification	untested
Bad or expired bearer token	<code>lib/supabase/route.ts</code> , then <code>getUser</code>	401 <code>Unauthorized</code>	<code>lib/supabase/route.test.ts</code> ; the 401 in each route's tests
Model fails before the stream	<code>app/api/chat/route.ts</code> , outer <code>try</code>	500 with a plain message	untested
Model fails mid-stream	Same file, the stream's <code>catch</code> and <code>finally</code>	An apology line; the partial reply is kept	untested
Model fails composing a reading or alert	<code>lib/alerts/compose.ts</code> , <code>predictions.ts</code>	Plain fallback copy, still delivered	<code>compose.test.ts</code> , <code>predictions.test.ts</code>
Transit chart fails in chat	<code>app/api/chat/route.ts</code> , <code>lib/astrology/transits.ts</code>	A reading without today's sky; nothing cached	untested
Birthplace not found	<code>lib/geo.ts</code> , <code>IntakeForm.tsx</code> , <code>IntakeView.swift</code>	Empty list; manual coordinates on the web; Save disabled on iOS	<code>app/api/profile/route.test.ts</code> for the 400s
Invalid coordinates or zone	Not guarded	A thrown <code>RangeError</code> in the browser; <code>NaN</code> into the ephemeris	untested
Fixed UTC schedule versus local day	<code>lib/alerts/slots.ts</code>	Possibly one reading a day far from IST	<code>slots.test.ts</code>
Chart stored under the old schema	<code>ensureCurrentChart</code> , <code>isChartStale</code>	Nothing; upgraded on first read	<code>birthProfile.test.ts</code> , <code>derived.test.ts</code>
Natal dosha "begins" after a recompute	<code>partitionForAlerts</code> , <code>deriveHadNatalRows</code>	Nothing; recorded silently	<code>run.test.ts</code>
Profile read fails	<code>getBirthProfile</code> throws	An error, not the intake form	<code>birthProfile.test.ts</code>
Duplicate cron run	Unique index, <code>isSlotDue</code> , <code>23505</code> , signature diff	Nothing	<code>slots.test.ts</code> ; the <code>23505</code> branch untested

FAILURE	WHERE IT IS HANDLED	WHAT THE USER SEES	PROOF
Token under the wrong APNs environment	<code>ApsEnvironment</code> , <code>ApsClient.send</code>	One missed push, then the row is replaced	<code>ApsEnvironmentTests</code> , <code>devices/route.test.ts</code>
Widget with no cached chart	<code>ReadingWidget.swift</code> , <code>PeriodWidget.swift</code>	"Open Sanchara to ..."	<code>PeriodProgressTests</code> for the cache; views untested
On-device model unavailable	<code>OnDeviceReasoner.route</code>	The same answer, from the server, labelled	<code>OnDeviceRoutingTests</code> for routing; the gate untested
Prompt injection via history	No specific guard	Bounded history, no tools, user-only extraction with confirmation	untested as such
Account deletion	<code>app/api/account/route.ts</code> , cascades, <code>AuthStore.signOut</code>	Everything in Postgres gone; avatar and Apple grant remain	<code>account/route.test.ts</code>

THE UNTESTED COLUMN

Eight rows have no test at all. Three are configuration paths that need a missing variable or a live socket to exercise (the env loader, the cron secret, APNs). Three are the chat route's own failure paths, before the stream, during it, and in the transit lookup. The last two are the unguarded coordinate and zone edges, and prompt injection. The chat route in particular is the one file where a regression would be felt by every user and caught by no test.

16.11 Study notes

READ IN THIS ORDER

1. `lib/env.ts` , for the one decision that makes the build secret-free and the first request the place a missing key fails.
2. `lib/supabase/route.ts` and its test, for the bearer and cookie split.
3. `app/api/chat/route.ts` , the stream's `catch` and `finally` , for what a partial reading becomes.
4. `lib/alerts/compose.ts` and `predictions.ts` , the two fallbacks and the parser that feeds them.
5. `lib/data/birthProfile.ts` , for the on-read upgrade and the read that now throws.
6. `lib/alerts/run.ts` , for the signature diff, the natal partition and the fail-closed count.
7. `lib/push/apns.ts` and `mobile/Sanchara/Sources/Services/Push.swift` , both ends of the environment question.
8. `mobile/SancharaWidgets/PeriodWidget.swift` and `OnDeviceReasoner.swift` , for the device's two fallbacks.

CHECK YOURSELF

Why does a missing `ANTHROPIC_API_KEY` fail a request and not the build?

Because `env.anthropicApiKey` is a function called inside `anthropic()` during a request; nothing reads `process.env` at module load.

What does the chat route do with text the model had already streamed when the call fails?

Saves it as the assistant turn in `finally` , after enqueueing the apology line, then closes the stream. The status was already 200.

Why does the alerts job treat an errored natal-row count as "has natal rows"?

Because the naive reading would be "first-time reader" and every standing natal dosha would be pushed as news; suppressing natal alerts costs nothing real, since natal conditions cannot change.

A phone registers a sandbox token and the server is configured for production. What happens on the next cron?

The row carries `sandbox` , so the server sends to the sandbox host; environments are per row, not per server. Only a token registered under the wrong word fails, and Apple's `BadDeviceToken` gets that row deleted until the next launch re-registers it.

What does the period widget show before the timeline has ever been fetched?

The mahadasha and antardasha from the cached chart, with a progress bar and no theme; only with no chart at all does it show the empty state.

Which test file does not test what its name suggests?

`lib/data/injection.test.ts` , which tests that the data helpers accept an injected Supabase client. There is no prompt-injection test.

APPENDICES

Reference

Why the product exists, every decision that shaped it, and a glossary with a reading order for studying the code.

- A. Why Sanchara exists
- B. Decisions
- C. Glossary and reading order

A Why Sanchara exists

Most astrology apps either guess the sky or hand a language model a sun sign and let it write. Sanchara computes the chart with the Swiss Ephemeris and lets Claude interpret only what was computed. Everything else in this book follows from that one split.

This appendix is the product story behind the code on `main` on 30 September 2026, version 0.3.0. It is written the way a launch document would be: the problem, the bet, who the app is for, what each release delivered, and a press release with the questions a sceptical reader would ask. Where it makes a claim about behaviour, the claim is grounded in a file you can open. The design notes under `docs/design/` record intent; where they and the code disagree, this appendix follows the code and says so.

A.1 The problem

An astrology reading has two parts: the chart, and what the chart means. The first part is astronomy. Where the Sun, Moon and planets stood at a particular minute over a particular town is a fact, and it can be computed to a fraction of a degree. The second part is interpretation, which is a craft with a vocabulary of its own: lords, aspects, dignity, periods, transits. A good reading rests the second part on the first.

Consumer apps mostly fail at one end or the other. Some never compute a chart at all: they know a birth date, look up a sun sign, and serve a paragraph that is true of a twelfth of the population. Others compute a chart and then hand a language model a loose description of it and hope. The model, given sign-level facts, writes sign-level prose. The specific-readings design note from 17 September 2026 puts the failure precisely: "A model given sign-level facts writes sign-level prose; the instructions have said 'never a generic horoscope' all along and cannot fix an input problem."

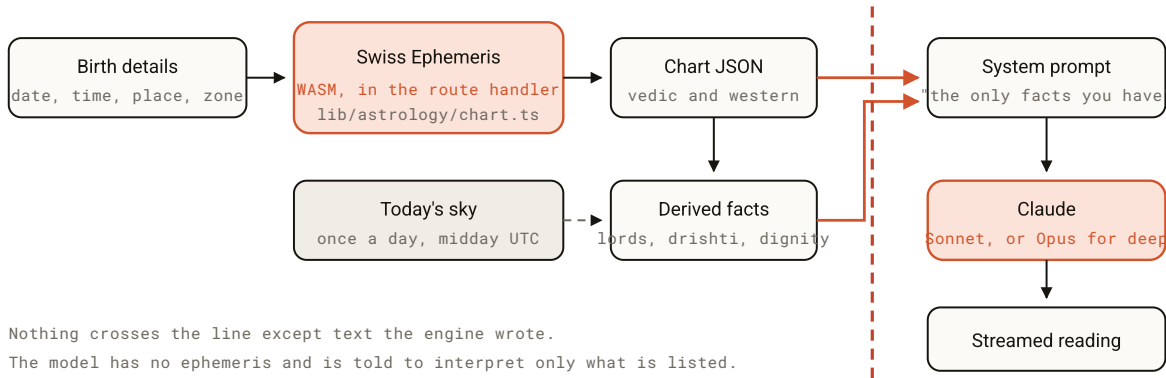
There is a third failure the first design note names directly: a model asked to state a planetary position will state one, whether or not it is true. A language model has no ephemeris. Left to itself it will put Saturn wherever the sentence wants Saturn to be.

A.2 The bet

The first design note, dated 23 July 2026, states the bet in one line: "Claude never guesses planetary positions. It interprets computed values." The code keeps that promise with a wall between two kinds of work. On one side, deterministic computation from the birth moment. On the other, a model that is handed the results and told, in the system prompt itself, "These are the only facts you have."

COMPUTED, DETERMINISTIC

INTERPRETED, NEVER COMPUTED



The split that defines the product. Every fact the model sees was produced by `computeChart`, `deriveFacts` or `describeGochara`; the prompt in `lib/astrology/prompt.ts` forbids stating anything not listed.

The computed side is `computeChart` in `lib/astrology/chart.ts`. It turns a local birth time into universal time with `luxon`, asks the ephemeris for a Julian day, sets Lahiri as the `ayanamsa` for Vedic and leaves Western tropical, computes Placidus cusps and the ascendant with `houses_ex`, then places eight bodies with `calc_ut` and derives Ketu as Rahu plus 180 degrees. For Vedic it adds nakshatras, the current Vimshottari period, and, since 0.3.0, a `derived` block: what each planet rules, what it aspects, its dignity, whether it is combust, and where the running dasha lord sits. Both traditions are computed once at intake and stored in `birth_profiles.chart`.

The interpreted side is `buildChartSystem`. It renders the chart as text, then sets the rules. The two that carry the product are these, quoted from the prompt: "Never state a placement, lordship, aspect, dasha, condition or number that is not listed above," and "No sign-personality writing. A sentence that would be true of a twelfth of the population is not an answer." The same wall holds outside chat. The daily readings and dosha alerts in `lib/alerts/` decide in code whether a dosha exists, and the model's only job, as the comment in `compose.ts` says, is "to say what the change means and what to do about it." On the phone, the on-device model may only restate rows from a table it looked up.

WHO IT IS FOR

The app is for someone who wants a reading of their own chart rather than their sign, and who is willing to type a birth time and a birthplace to get it. Two traditions are served, chosen per conversation: Vedic, sidereal with Lahiri, printed as a kundli because, as the PDF renderer's comment puts it, "in most families it is a piece of paper that gets kept"; and Western, tropical. Numerology is a third mode with its own prompt. The choices in the code say who the author had in mind: the cron schedule is annotated in IST, the kundli PDF prints Sanskrit rashi names because "a North Indian chart labelled Scorpio reads as a translation of someone else's diagram," and the alerts vocabulary is Sade Sati, Mangal dosha and Kaal Sarp. The Western mode is real but plainer, and A.5 says why.

A.3 What shipped, release by release

The changelog records three tagged versions, all within three days of each other in September 2026. That compresses two months of work, so the table also names the earlier milestones the git history shows.

WHEN	WHAT	WHERE IT LIVES NOW
23 to 24 July 2026	Design note, chart engine, prompt builder, streaming chat, admin, landing page. Merged as the MVP on 24 July.	<code>lib/astrology/</code> , <code>app/api/chat/</code> , <code>app/admin/</code>
25 August 2026	iOS design note. Bearer auth on the route handlers, <code>POST /api/profile</code> , <code>DELETE /api/account</code> , the Universal Links file.	<code>lib/supabase/route.ts</code> , <code>public/.well-known/</code>
26 August 2026	Dosha alerts, twice-daily readings, prompt caching and effort settings. iOS chat, intake, Apple sign-in, email codes, readings inbox.	<code>lib/alerts/</code> , <code>mobile/Sanchara/</code>
0.1.0, 16 September 2026	First tagged release. The rename to Sanchara. Life timeline, kundli on the phone, Siri, widgets, on-device answers.	<code>lib/timeline/</code> , <code>mobile/SancharaWidgets/</code>
0.2.0, 16 September 2026	A meaning for every period, chat that sees the timeline, better moment extraction, the current-period widget, the push environment fix.	<code>lib/timeline/explain.ts</code> , <code>PeriodWidget.swift</code>
0.3.0, 18 September 2026	Derived chart facts, whole-sign Vedic houses, gochara, deeper numerology, more answered on the phone, and the fixes those changes forced.	<code>lib/astrology/derived.ts</code> , <code>transits.ts</code> , <code>numerology.ts</code>

0.1.0: THE WHOLE PRODUCT, ONCE

The first tag is large because it is the first tag. The changelog's Added list is the feature list of the README: the chart engine for both traditions, streaming readings with an optional deep reading on Opus, the kundli PDF, the life timeline with moments pinned by hand or proposed from chat history "and confirmed before anything is saved," morning and night readings on the user's local clock with a push only when a dosha or hard transit starts or ends, the native app, Siri and widgets, the admin area and the landing page. Two entries sit under Changed. One is the rename, covered below. The other is a correctness fix in the dasha engine: antardashas are laid out from each mahadasha's "virtual start" rather than from the birth moment, which is what the `virtualStart` field in `lib/astrology/dasha.ts` records, so that sub-periods line up with the real period.

0.2.0: THE TIMELINE LEARNS TO SPEAK

Tagged the same day. The timeline had drawn the dasha periods but named them only by planet. This release adds a one-line theme and a short meaning per period, written by Claude from the chart and the moments pinned inside it, stored in `period_readings` with a hash of those moments so a changed period is rewritten and an unchanged one is not. The chat gains the same timeline as a block in its cached prompt, so "why was 2021 so hard" is answered against the user's 2021. The scan now resolves relative dates and ages from the message date and birth date, quotes the words each moment came from, and keeps undated events for the user to date. A Home Screen and Lock Screen widget shows the current period. The one fix matters operationally: the app had reported its push environment from the build configuration rather than the signing entitlement, so a Release build from Xcode registered a sandbox token as production and never received a push.

0.3.0: READINGS CITE THE CHART, NOT THE SIGN

This is the release that closes the gap named in A.1. Every Vedic reading now carries the lord of each house and where that lord sits, which planets aspect what, each planet's dignity, which are combust, and where the running dasha lord is placed. Transits are read as gochara, by the house they occupy from the ascendant and from the natal Moon, with the affliction those positions raise. Numerology sees the personal year and month, the Lo Shu grid, and the relationship between the core numbers. The phone answers questions about rulers, aspects and strength on the device.

The Changed entry is the one users could notice: Vedic houses became whole-sign, because the chart, the PDF and the on-device table had labelled houses whole-sign from the ascendant while filling them with Placidus numbers. "Existing charts upgrade themselves the first time they are read; nothing to do." The Fixed list is the consequence of doing that carefully. Western readings had been handed Vedic technique on a tropical chart and now get "the plain sky again." And a dosha fixed at birth could have announced itself as news when the house system shifted the detectors; "Natal conditions never alert now; they reconcile silently." Both of those are decisions in their own right, and Appendix B records them.

THE NAME

The product was called Astra from the first commit on 23 July 2026 until the commit of 16 September 2026 that renamed the iOS target, the copy and the prompts. The changelog for 0.1.0 states the rule: "The bundle identifier and production hostname are unchanged." So the repository is still `astra`, the bundle id is `com.shivvyas.astra`, the App Group is `group.com.shivvyas.astra`, the production host is `astra.shivvyas.com`, and the first migration is headed "Astra schema + RLS." The user-facing name is Sanchara everywhere: the system prompt opens "You are Sanchara," the widget bundle is `SancharaWidgets`, and the custom URL scheme a widget tap uses is `sanchara://`. Why the identifiers stayed is a decision in Appendix B; in short, Apple's push, sign-in and associated-domain configuration is wired to them.

A.4 PR/FAQ

PRESS RELEASE

Sanchara computes your real birth chart and reads it to you. Give it the date, time and place you were born and the Swiss Ephemeris places every planet, house, nakshatra and period exactly. Then ask it anything. The reading names the placement it is reading from, in the Vedic or Western tradition you choose, and it will not tell you something that is true of everyone born under your sign. Twice a day it writes a reading for your morning and your night, and it only sends a notification when something in your sky actually changes. On iPhone the chart lives on the phone, so Siri, the widget and simple questions answer with no network and no charge. Your kundli is a document you can print.

Does the model compute my chart?

No. `computeChart` in `lib/astrology/chart.ts` does, using `swisseph-wasm` inside the route handler, and the result is stored with your profile. The model is handed the result as text and told it is "the only facts you have." If you ask about a placement that is not in the chart, the prompt instructs it to say what the chart does show instead.

Why is my Vedic chart different from my Western one?

Two settings, both in `computeChart`. Vedic is computed sidereal with the Lahiri ayanamsa, which currently sets positions roughly 24 degrees behind tropical, so a Sun in early Capricorn tropically is in Sagittarius sidereally; the test in `chart.test.ts` asserts exactly that. Vedic houses are whole-sign from the ascendant; Western houses are Placidus.

Why did my house numbers change in September?

Before 0.3.0 the Vedic chart labelled its houses whole-sign but numbered the planets by Placidus cusps, so a planet could be drawn in a house whose sign it was not in. Since 0.3.0 both agree. A stored chart from before the change carries `schemaVersion` below 2 and is recomputed the first time it is read, by `ensureCurrentChart` in `lib/data/birthProfile.ts`.

Will I get a notification every day?

Two readings are written every day, morning and night on your own clock, and they arrive as pushes when push is configured. A dosha alert is different: it is sent only when a condition starts or ends. The daily job diffs the conditions it computes today against the open rows in `transit_conditions`, and a condition that merely continues is never re-notified, "which is what keeps a standing Sade Sati from becoming a daily nag." A condition fixed at birth is never announced as news at all.

What does a reading cost to run?

Measured on a real chart and question, and recorded in `docs/MODEL_COSTS.md`: a mid-conversation turn on Sonnet 5 went from roughly \$0.023 to \$0.008 after the system prompt was split so its stable half is a cache read, and after routine readings were set to low effort, which cut output from 1,452 to

719 tokens for the same placements cited. A deep reading keeps the default effort on Opus. On an iPhone with Apple Intelligence, a lookup question never reaches the API at all.

Why is a Western reading plainer than a Vedic one?

Because the techniques that make a Vedic reading specific are Vedic. Lordship, graha drishti, moolatrikona, combustion orbs and the natal doshas apply to a sidereal whole-sign chart, and applying them to a tropical Placidus chart would cite Indian technique as though it were Western. So `factsFor` returns nothing for a Western chart and the prompt renders the plain positions; today's sky is given as where each body is and which natal body it sits on.

Who can read my conversations?

You, through Row Level Security policies that scope every table to `auth.uid()`, and an admin who signs in at `/admin/login` and appears in the `admins` table, which has no insert policy, so signing up can never grant it. The admin area reads through the service-role client on the server and is read-only for transcripts.

A.5 What is deliberately not built

The four design notes each carry a non-goals list, and most of those lines are still true of the code. A few items from the iOS note were planned and never landed. Listing both kinds keeps a new contributor from either building something on purpose left out or assuming something exists.

NOT BUILT	SOURCE	WHAT THE CODE DOES INSTEAD
Payments or subscriptions	Web note, iOS note	Deep reading is a boolean on the chat request (<code>deep?: boolean</code> in <code>app/api/chat/route.ts</code>), a config toggle rather than billing.
Social features, sharing, public profiles	Web note	Nothing in the schema is readable by anyone but its owner and an admin.
Android, iPad layouts, macOS	iOS note	<code>TARGETED_DEVICE_FAMILY</code> is "1" in <code>mobile/project.yml</code> : iPhone only.
The admin area on iOS	iOS note	Admin stays under <code>app/admin/</code> on the web with its own login.
On-device chart computation	iOS note	The phone decodes <code>birth_profiles.chart</code> and caches it; the ephemeris runs only on Vercel.
Offline chat	iOS note	Offline viewing is in: <code>ProfileStore</code> starts from the cached chart and details, so an offline launch is "still a usable app." Sending a message needs the API.
Face ID lock, haptics	iOS note, planned	Neither is in the sources; there is no biometric or feedback-generator call in <code>mobile/Sanchara/Sources/</code> . The note listed them as native capabilities to add; they were not.
Divisional charts, Shadbala, Ashtakavarga	Specific-readings note	Depth is "core Parashari": lords, drishti, dignity, combustion, dasha lord placement, gochara.
Planetary friendship in dignity	Specific-readings note, <code>derived.ts</code>	Dignity stops at five values. Friendship is a compound of natural and temporal, and "half of it is misleading rather than partial." The natural table is used only in numerology, where it is the whole rule.
Node lordship and node aspects beyond the 7th	<code>derived.ts</code>	Rahu and Ketu rule nothing and aspect only the 7th, because "a contested claim repeated in every reading is worse than a missing one."
Yogas beyond the conditions in <code>doshas.ts</code>	Specific-readings note	The natal doshas and transit afflictions <code>detectConditions</code> returns, and no more.
Western aspects	Web note listed "aspects (major)" for Western	Not computed. Only Vedic charts get a <code>derived</code> block; a Western reading gets positions and the plain sky.
Nakshatra pada	Web note listed "nakshatra + pada"	Only the nakshatra name is stored (<code>nakshatraOf</code> in <code>constants.ts</code>); no pada is computed anywhere.
An in-app notice about changed house numbers	Specific-readings note	Called "a product decision, not a technical one." Charts upgrade silently on read.
A bigger deep-reading budget	Specific-readings note	<code>WORDS_DEEP</code> is still 320, to be judged "against real answers after this lands, not guessed at now."

ONE LEFTOVER

The README's stack table lists `three.js` for the landing page, and `three` is still in `package.json`. Nothing imports it. The iOS design note noticed this in August: the hero is a photo with GSAP scroll sections, not WebGL. It is a dependency to remove, not a feature to document.

A.6 Study notes

READ IN THIS ORDER

1. `docs/design/2026-07-23-web-app.md`, for the original bet and the first non-goals.
2. `lib/astrology/chart.ts`, for the computed side of the wall, in 124 lines.
3. `lib/astrology/prompt.ts`, for the rules the model is given, especially the Accuracy block.
4. `CHANGELOG.md`, for the three releases and the exact wording of what changed.
5. `docs/design/2026-09-17-specific-readings.md`, for why 0.3.0 was necessary and what it left out.
6. `docs/MODEL_COSTS.md`, for the numbers behind the cost answer in the FAQ.

CHECK YOURSELF

What is the one thing the model is never allowed to do, and where is it enforced?

State a fact that was not computed. It is enforced by construction (the model has no ephemeris and only sees text the engine wrote) and by instruction (the Accuracy block in `buildChartSystem`, and the alert and prediction prompts' "Never invent a placement, dosha, date, or number that is not listed above").

Why does a Western reading get no derived facts?

Because every technique in `derived.ts` is Vedic and assumes whole-sign houses from a sidereal ascendant. `factsFor` returns `undefined` unless `chart.tradition` is `"vedic"`.

Which identifiers still say Astra, and why were they kept?

The bundle id, the App Group, the URL name, the production host and the repository. Apple's push credentials, Sign in with Apple and associated domains are configured against them; changing them would mean new App IDs and re-setup of every capability.

What did 0.2.0 fix that no user could have diagnosed?

The push environment was read from the build configuration rather than the signing entitlement, so a Release build run from Xcode registered a sandbox token as production and silently never received a push.

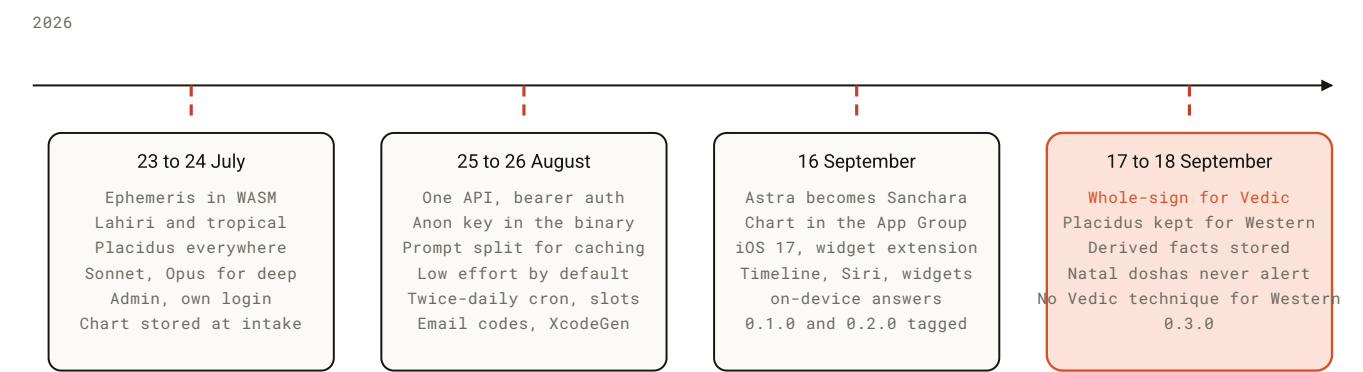
B Decisions

Nineteen decisions, each with the alternatives it beat and the line of code that shows it held. Most of them were made in three bursts: the engine in July, the phone in August, and the correction of the chart in September.

This log describes the code on `main` on 30 September 2026, version 0.3.0. Every entry is verified against a file, not a memory. Dates come from the design notes under `docs/design/`, from `CHANGELOG.md`, or from the commit that landed the change; where none of those gives a date, the entry says so. Alternatives are the ones the design notes considered, or the ones the code comments argue against. Where a decision was later amended, the amendment is its own entry rather than a rewrite of history.

B.1 How to read this log

Each entry has five parts: the decision in one sentence, the date, the alternatives, why, and where in the code it shows. The "where" is the point. A decision that leaves no trace in the code is a preference, and the next contributor is free to ignore it. Every decision here leaves a trace, and most are pinned by a test named in the entry.



When the decisions were made. The September burst is the only one that changed something users had already seen, which is why it carries the most safeguards.

CONTENTS

1. Swiss Ephemeris as WebAssembly in the route handler
2. Sidereal Lahiri for Vedic, tropical for Western
3. Whole-sign houses for Vedic
4. Placidus kept for Western
5. Derived facts computed once and stored with the chart
6. Contested doctrine left out
7. The system prompt in two halves, for the cache
8. Sonnet for readings, Opus for deep readings, low effort by default

9. Model replies as delimited lines, not JSON
10. Natal doshas never alert
11. One API for web and iOS, with bearer auth
12. The anon key ships in the binary; RLS is the boundary
13. Twice-daily cron with local-time slots
14. The admin area has its own login
15. The chart is cached in an App Group
16. Email codes instead of magic links on the phone
17. A generated Xcode project
18. iOS 17 as the floor, with a widget extension
19. Astra becomes Sanchara; the identifiers stay

B.2 The chart

1. SWISS EPHEMERIS AS WEBASSEMBLY IN THE ROUTE HANDLER

- **Decision.** The ephemeris runs in-process, as `swisseph-wasm`, inside the Next.js route handlers on Vercel.
- **Date.** Design note of 23 July 2026; the chart service landed the same day.
- **Alternatives.** The design note kept one open: "fallback: hosted ephemeris API." The iOS note later added and rejected computing on the device.
- **Why.** No network hop and no second service to run; the same module computes a birth chart at intake and today's sky in the cron job. The WASM init is the cost, so it is paid once per process.
- **Where.** `getSwe` in `lib/astrology/chart.ts` holds one lazily initialised promise per process. Every route that computes declares `export const runtime = "nodejs"`, with the comment "swisseph-wasm runs during chart computation." `lib/timeline/build.ts` goes so far as to rebuild the Moon's longitude from the stored chart so the timeline "loads without re-running the Swiss Ephemeris, which costs a WASM init per process."

2. SIDEREAL LAHIRI FOR VEDIC, TROPICAL FOR WESTERN

- **Decision.** One birth input produces two charts: sidereal with the Lahiri ayanamsa, and tropical.
- **Date.** 23 July 2026.
- **Alternatives.** A single tradition; a different ayanamsa. Neither is discussed in the notes, which name Lahiri from the first line and never revisit it.
- **Why.** The traditions disagree by the ayanamsa, currently about 24 degrees, so a reading in one is wrong in the other. Computing both at intake means the mode switch in chat is free.
- **Where.** `computeChart` calls `swe.set_sid_mode(swe.SE_SIDM_LAHIRI, 0, 0)` and sets `SEFLG_SIDEREAL` only for "vedic", and stores the ayanamsa on the chart. `saveBirthProfile` computes both and writes `{ vedic, western }`. The test "sidereal (vedic) sun sits ~24° behind tropical, landing in Sagittarius, with nakshatras" pins the difference.

3. WHOLE-SIGN HOUSES FOR VEDIC

- **Decision.** A Vedic planet's house is the count of signs from the ascendant's sign, not the Placidus cusp it falls between.
- **Date.** 17 September 2026, shipped in 0.3.0.
- **Alternatives.** Keep Placidus numbers and make the renderers draw Placidus; or a SQL migration and backfill to rewrite every stored chart.
- **Why.** Three renderers (the Swift kundli, the PDF, the on-device table) already labelled house 1 with the ascendant's sign and each next house with the next sign, then filled those houses with Placidus numbers, so "a planet could be drawn in a house marked with a sign it was not in." Placidus also allows two houses to share a sign, which leaves "the lord of the 7th" undefined, and every technique added in 0.3.0 rests on lordship. The upgrade happens on read rather than by migration, so a chart "corrects wholly or not at all, never half."
- **Where.** `house: tradition === "vedic" ? houseFrom(ascSign, signOf(lon)) : houseOf(lon, cusps)` in `chart.ts`. `CHART_SCHEMA_VERSION = 2` in `constants.ts`. `ensureCurrentChart` in `lib/data/birthProfile.ts` recomputes both traditions together when `isChartStale` says so. Tests: "places every planet in the Nth sign from the lagna" and the `ensureCurrentChart` suite.

4. PLACIDUS KEPT FOR WESTERN

- **Decision.** Western charts keep Placidus house numbers, and the Placidus cusps are stored for both traditions.
- **Date.** 17 September 2026, in the same change.
- **Alternatives.** Whole-sign for both, which would have been simpler.
- **Why.** Placidus is the convention Western readers expect, and the note's wording is flat: "Western continues to use `houseOf(lon, cusps)` unchanged." The cusps stay on `chart.houses` because the test file reads them, and because they are the ascendant's source.
- **Where.** `houseOf` in `chart.ts`, still called for "western". Test: "leaves western on Placidus, where a planet can sit in a house whose cusp sign differs." The prompt test suite has a block titled "a Western chart gets no derived facts, no Vedic technique, and Placidus houses."

5. DERIVED FACTS COMPUTED ONCE AND STORED WITH THE CHART

- **Decision.** Lords, aspects, dignity, combustion, the dasha lords' placements and the natal conditions are computed by `deriveFacts` at chart time and stored as `chart.derived`; readers fall back to computing when the block is absent.
- **Date.** 17 September 2026, 0.3.0.
- **Alternatives.** Compute per request; add a column and a backfill job.
- **Why.** The facts are a pure function of the chart, so storing them costs nothing and lets the iOS app decode them from `birth_profiles.chart` "with no API change." The fallback is what lets it ship without a migration: "existing users get correct answers on their next message, and the stored copy fills in whenever their chart is next written."
- **Where.** The last line of `computeChart : derived: tradition === "vedic" ? deriveFacts(result) : undefined`. `factsFor` and `isChartStale` in `derived.ts`. `deriveFacts` recomputes houses from signs rather than reading `planet.house`, "so a chart stored before the whole-sign change still derives

correctly." `NatalChart.derived` is optional in `ChartData.swift` so a chart cached before this shipped still decodes.

6. CONTESTED DOCTRINE LEFT OUT

- **Decision.** Rahu and Ketu rule no sign and aspect only the 7th; dignity never consults planetary friendship.
- **Date.** 17 September 2026.
- **Alternatives.** Node co-lordship of Aquarius and Scorpio; the school giving the nodes the 5th and 9th; natural friendship as a sixth dignity.
- **Why.** The header of `derived.ts`: "a contested claim repeated in every reading is worse than a missing one." Friendship is a compound of natural and temporal judgement, and "half of it is misleading rather than partial." The natural table is used only in numerology, where it is the whole rule.
- **Where.** `NODES`, `signsRuledBy` and `aspectedHouses` in `derived.ts`; `FRIENDS` and `ENEMIES` in `numerology.ts`, resolved compoundly because the classical table is asymmetric. Tests: "gives the nodes no dignity anywhere", "gives the nodes the 7th only", "assigns lordships, and none to the nodes."

B.3 The readings

7. THE SYSTEM PROMPT IN TWO HALVES, FOR THE CACHE

- **Decision.** The system prompt is built as `buildChartSystem`, byte-identical for a user across turns and conversations, followed by `buildTodaySystem`, which carries everything that varies. A cache breakpoint sits after the first half and another after the last stored turn.
- **Date.** 26 August 2026, the "cheaper prompts" change.
- **Alternatives.** One prompt, re-billed in full every turn, which is what existed before. The doc records what had to stop for caching to work: the date line carried a clock time, and transits were computed "for now."
- **Why.** Measured in `docs/MODEL_COSTS.md`: the system prompt went from 1,608 to 942 tokens, system plus history from 4,017 to 2,868, and from the second turn the whole prefix is a cache read at a tenth of the input rate. Together with effort, a mid-conversation turn fell from roughly \$0.023 to \$0.008.
- **Where.** `buildChartSystem` in `lib/astrology/prompt.ts`, "kept separate so it can carry a cache breakpoint." The two `cache_control` markers in `app/api/chat/route.ts`. `describeToday` reduces the time to a part of day; `transitChart` samples midday UTC once per day per neighbourhood. Test: "keeps volatile content out of the cacheable half." The `chat usage` log line reports `cache_read`, so a regression is visible.

8. SONNET FOR READINGS, OPUS FOR DEEP READINGS, LOW EFFORT BY DEFAULT

- **Decision.** `READING_MODEL` is `claude-sonnet-5`; `DEEP_READING_MODEL` is `claude-opus-4-8`. Routine readings send `output_config: { effort: "low" }`; deep readings keep the default.
- **Date.** 24 July 2026 for the models, after a same-day detour through Haiku as the default; low effort followed on 26 August.
- **Alternatives.** Haiku, which the comment in `anthropic.ts` notes does not support adaptive thinking; thinking disabled, which the doc measured and declined.

- **Why.** Thinking tokens are billed as output, "the expensive half of a reading." On the same chart and question: adaptive at default effort produced 1,452 output tokens, low effort 719, thinking disabled 328, and "all three cited the same real placements." Low is where routine readings sit, "reasoning intact, spend halved." Disabled was not chosen because "chart work benefits from some reasoning."
- **Where.** `lib/anthropic.ts`, with the three numbers in its comment. The chat route spreads `LOW_EFFORT` only when `body.deep` is false and sets `max_tokens` 1,600 or 3,000 as "a ceiling, not a target." The alert composer uses Opus at low effort with `max_tokens: 1600`; the daily readings and timeline explanations use Sonnet at low effort.

9. MODEL REPLIES AS DELIMITED LINES, NOT JSON

- **Decision.** Structured replies from the model (alerts, daily readings, extracted moments, period explanations) are asked for as labelled or pipe-delimited lines and parsed leniently.
- **Date.** 26 August 2026, "ask for a delimited reply instead of JSON."
- **Alternatives.** JSON, which was the first shape and is still accepted as a fallback.
- **Why.** From `parseAlertCopy`: asked for JSON, the model "sometimes writes real newlines inside the string, which is not valid JSON, and the reply is lost. Measured on a batch of seven live readings, two came back that way. Line-delimited text has no escaping to get wrong."
- **Where.** `parseAlertCopy` and `parseAlertJson` in `lib/alerts/compose.ts`; the `DATE | PRECISION | TITLE | EVIDENCE` shape in `lib/timeline/extract.ts`; `LORD | START | THEME | MEANING` in `lib/timeline/explain.ts`. Every parser re-validates against the birth date and today, because "a date outside a person's life is the one error that would be visibly absurd."

10. NATAL DOSHAS NEVER ALERT

- **Decision.** Once a user has any natal-scope row in `transit_conditions`, a natal condition that starts or ends is recorded but never pushed. The count query fails closed.
- **Date.** 17 September 2026, in two commits, shipped in 0.3.0.
- **Alternatives.** The design note first prescribed a per-user `last_reconciled_version` column with a one-time silent reconciliation. No such column exists; the note was amended to describe what shipped.
- **Why.** Whole-sign houses change which natal conditions are detected and their signatures, and the daily job treats an unmatched signature as a condition starting; the first run after a chart upgrade would have pushed "Mangal Dosha has begun" about a chart "that has not changed since they were born." `partitionForAlerts` generalises it: a change in a natal condition "is always an artifact of the computation changing" and "never an event." The cost: a natal detector added later never notifies existing users.
- **Where.** `partitionForAlerts` and `deriveHadNatalRows` in `lib/alerts/run.ts`; the raw `started` and `ended` sets are still written, with comments warning not to filter them there. Tests: "never alerts a natal condition once the user has natal rows" and "fails CLOSED", which treats a count error as "yes, has rows."

B.4 The platform

11. ONE API FOR WEB AND IOS, WITH BEARER AUTH

- **Decision.** The iOS app calls the same `/api/*` route handlers as the web app, sending `Authorization: Bearer` with its Supabase access token; the web keeps cookies.
- **Date.** iOS design note of 25 August 2026; the bearer-aware client landed the same day.
- **Alternatives.** A second backend for mobile; a WebView or Capacitor wrapper, which the note considered and rejected on App Store grounds.
- **Why.** The note's rule: "anything requiring a secret or the ephemeris stays on Vercel; everything else the app does itself." The route handlers already held the Anthropic key, the ephemeris and the PDF renderer, so the only change was how a caller is identified. The web's behaviour was left "unchanged."
- **Where.** `createRouteSupabase` in `lib/supabase/route.ts`: a bearer token yields a supabase-js client that forwards it "so RLS evaluates as that user"; no token falls through to the cookie client. Every route handler under `app/api/` starts with it. `SancharaAPI.request` in the app sets the header from `Supa.accessToken()`. Tests in `lib/supabase/route.test.ts`.

12. THE ANON KEY SHIPS IN THE BINARY; RLS IS THE BOUNDARY

- **Decision.** The iOS app carries the Supabase URL and anon key, generated into `AppConfig.swift` at build time, and reads and writes its own rows directly. The service-role and Anthropic keys never leave the server.
- **Date.** 25 August 2026.
- **Alternatives.** Proxy every read through the API so the app holds no key at all.
- **Why.** The anon key "is publishable by design and safe in the binary; RLS is the security boundary." Every table is scoped to `auth.uid()`, so the key alone reads nothing. This is what lets the phone fetch `birth_profiles.chart`, the readings and the alerts without new endpoints.
- **Where.** `mobile/scripts/make-config.sh` reads only the two public values and its header says the secret keys "must never reach the app binary." The generated file is ignored by git. `ProfileStore`, `DailyStore` and `AlertsStore` query tables directly through `Supa.client`. The one exception proves the rule: `POST /api/devices` writes with the admin client because a device moving between accounts has to change owner, and "the owner-only RLS policy would (correctly) refuse that update."

13. TWICE-DAILY CRON WITH LOCAL-TIME SLOTS

- **Decision.** One Vercel Cron path, `/api/cron/daily`, fires at 02:30 and 15:30 UTC. Each user's slot is decided by their local hour, before 14:00 is morning and after is night, and a unique index makes a repeat run write nothing.
- **Date.** 26 August 2026.
- **Alternatives.** A schedule per user or per timezone; a queue.
- **Why.** Fixed schedules are what Vercel Cron offers, and idempotence removes the need for anything finer: "the job can run as often as you like." The two hours are 08:00 and 21:00 IST, which says which users the schedule was tuned for; a user elsewhere still gets both slots, at whatever local hour the run lands, because the slot comes from their clock and not the server's.

- **Where.** `vercel.json`; `NIGHT_STARTS_AT_HOUR`, `slotForHour` and `isSlotDue` in `lib/alerts/slots.ts`; `unique (user_id, for_date, slot)` in `0005_daily_readings.sql`; and the conflict code `23505` treated as "already done" in `writeDueReading`. The route refuses every request without `CRON_SECRET`. Tests: "does not backfill a slot the user has moved past."

14. THE ADMIN AREA HAS ITS OWN LOGIN

- **Decision.** Admin is an isolated route zone at `/admin` with its own login page, gated by presence in an `admins` table that has no insert policy.
- **Date.** 23 July 2026.
- **Alternatives.** A role column on `profiles`; a separate deployment.
- **Why.** The first migration's comment: "presence in this table = admin. No self-service path." The design note's requirement was that "consumer signup can never grant admin," and a table nobody can insert into from the client satisfies it without a second app.
- **Where.** `adminSignIn` in `app/admin/actions.ts` signs in with a password and immediately signs out again unless `isAdminUser` is true. `requireAdmin` in `lib/admin/guard.ts` guards every page. The middleware redirects unauthenticated `/admin` paths to `/admin/login` rather than `/login`. Reads go through `createAdminSupabase`, whose comment says "Bypasses RLS. Only ever import from server code behind an admin auth check."

B.5 The phone

15. THE CHART IS CACHED IN AN APP GROUP

- **Decision.** The chart, the birth details, the latest reading and the current period are written as JSON into the `group.com.shivvyas.astra` container after every successful load. The app writes; Siri intents and the widget only read.
- **Date.** 16 September 2026, with Siri and the widgets.
- **Alternatives.** Each process fetching for itself.
- **Why.** `ChartCache`'s header: three processes need the data and only one can sign in; "a widget that fetches is a widget that shows a placeholder, and a Siri answer that waits is a Siri answer nobody hears." Nothing secret goes in, and the cache is cleared on sign-out.
- **Where.** `mobile/Sanchara/Sources/Services/ChartCache.swift`, writing with `.completeFileProtection`. The entitlement on both targets in `project.yml`. `IntentData.chart()` reads the cache and never the network. The widget extension compiles `ChartCache.swift` and `ChartData.swift` into itself, each "dependency-free on purpose" so nothing that imports Supabase is dragged into a process with no session.

16. EMAIL CODES INSTEAD OF MAGIC LINKS ON THE PHONE

- **Decision.** The iOS app signs in and confirms sign-up with a six-digit code typed into the app. The web app keeps password sign-in and a magic link.
- **Date.** 26 August 2026.
- **Alternatives.** Magic links returning through Universal Links, which is what the iOS design note specified.

- **Why.** From `docs/EMAIL_OTP.md` : "the code is typed into the app that is already open, with no hop out to a mail client and back, and no Universal Link that can be swallowed by the wrong browser." The link is kept in the email template so the web path still works.
- **Where.** `AuthStore.State.awaitingCode` and `CodePurpose` , which picks `.email` or `.signup` for `verifyOTP` ; the code field is marked `.oneTimeCode` . On the web, `sendMagicLink` in `app/(auth)/actions.ts` still exists. The one setting outside the repo is that both Supabase email templates must include `{{ .Token }}` .

17. A GENERATED XCODE PROJECT

- **Decision.** The Xcode project is produced by XcodeGen from `mobile/project.yml` and is not committed; `AppConfig.swift` is generated too.
- **Date.** 25 to 26 August 2026; the leftover root scaffold was dropped on the 26th.
- **Alternatives.** A committed `project.pbxproj` .
- **Why.** Targets, entitlements, capabilities and the files shared with the widget extension are readable in one YAML file with comments explaining each choice, and the generated project cannot drift from it. CI and the TestFlight workflow regenerate it on every run.
- **Where.** `mobile/project.yml` ; `mobile/Sanchara.xcodeproj/` in `.gitignore` ; the "Install XcodeGen" and "Generate Xcode project" steps in `.github/workflows/ci.yml` and `ios-testflight.yml` . The README's iOS instructions are the three commands: run the config script, `xcodegen generate` , open.

18. IOS 17 AS THE FLOOR, WITH A WIDGET EXTENSION

- **Decision.** Both targets deploy to iOS 17.0. Features that need newer frameworks are gated at runtime rather than raising the floor.
- **Date.** 16 September 2026 for the widget target; the floor dates from the August scaffold.
- **Alternatives.** Requiring iOS 26 for everything, since the on-device answers need it.
- **Why.** The Siri and widgets guide: where Apple Intelligence is missing, "every question goes to Claude exactly as it did before; the feature degrades rather than disappearing." Controls and App Shortcuts are iOS 18; the on-device model is iOS 26; the app itself should not be.
- **Where.** `deploymentTarget: iOS: "17.0"` on both targets in `project.yml` , with the comment that widgets and controls "arrived on this schedule; the app itself still runs on iOS 17." `@available(iOS 18.0, *)` on `SancharaShortcuts` and `ReadingControl` ; `#if canImport(FoundationModels)` and `#available(iOS 26.0, *)` around every use of the on-device model in `OnDeviceReasoner.swift` .

19. ASTRA BECOMES SANCHARA; THE IDENTIFIERS STAY

- **Decision.** The product, target, copy and prompts were renamed; the bundle id, App Group, production host and repository were not.
- **Date.** 16 September 2026, recorded under Changed in 0.1.0.
- **Alternatives.** A new bundle id and host to match the name.
- **Why.** The identifiers are what Apple and Supabase are configured against: the APNs bundle id in `APNS_BUNDLE_ID` , the Sign in with Apple client id, the associated domains entitlement, the App Group on

two App IDs, and the app record in App Store Connect. Every one of those would have to be recreated, and any push token minted under the old id would die.

- **Where.** `PRODUCT_BUNDLE_IDENTIFIER: com.shivvyas.astra` , `applinks:astra.shivvyas.com` and `group.com.shivvyas.astra` in `project.yml` , beside `name: Sanchara` and the `sanchara` URL scheme. `APNS_BUNDLE_ID=com.shivvyas.astra` in `.env.example` . The README's closing paragraph records the history.

B.6 Study notes

READ IN THIS ORDER

1. `lib/astrology/chart.ts` , for decisions 1 to 5 in 124 lines: the flags, the two house rules, the derived block.
2. `lib/astrology/derived.ts` , whose header is decision 6 in the author's own words.
3. `docs/MODEL_COSTS.md` and then `lib/anthropic.ts` , for the numbers behind 7 and 8.
4. `lib/alerts/run.ts` , for decision 10 and the two comments that forbid filtering the raw sets.
5. `docs/design/2026-08-25-ios-app.md` , for the rule that split the platform, then `lib/supabase/route.ts` and `mobile/scripts/make-config.sh` .
6. `mobile/project.yml` , which holds decisions 15, 17, 18 and 19 in one file with comments.

CHECK YOURSELF

Why did whole-sign houses force a change to the alerts job?

Because natal dosha signatures embed the house, so recomputing houses made existing conditions look new, and the job pushes on "new." `partitionForAlerts` now routes natal-scope conditions to silent for anyone with natal rows on record.

What does the second cache breakpoint do that the first cannot?

It carries the conversation. A prefix below the API's minimum is silently not cached, so on a short first turn the chart block alone may miss; from the second message the breakpoint after the last stored turn makes chart, today and history one cache read.

Which key is in the iOS binary, and why is that safe?

The Supabase anon key only. Every table is scoped by RLS to the signed-in user, so the key reads nothing by itself. The service-role and Anthropic keys are read only in `lib/env.ts` on the server.

Which alternative was written into a design note and then not built?

The `last_reconciled_version` column for natal reconciliation. The note was amended to describe the count-based rule that shipped instead.

What is the cost of decision 10 that the note calls out?

A natal detector added in future is recorded silently for existing users and never pushed; only new users see it. Reaching existing users would need a separate backfill, not a change to `partitionForAlerts` .

C Glossary and reading order

Every astrology term the code uses, every term the project coined, a path through the repository for a first week, and a table that turns "I want to change X" into a file and a chapter.

This appendix describes the code on `main` on 30 September 2026, version 0.3.0. The glossary is in two parts. The first is the astrological vocabulary, defined as the code uses it rather than as a textbook would, with the file where the term is computed or consumed. The second is the project's own vocabulary: words like "slot" and "moment" that mean something specific here. The reading order that follows starts at the chart engine and works outward to the phone, because that is the direction the data flows.

C.1 Glossary

ASTROLOGY

TERM	MEANING HERE	WHERE
Ayanamsa	The angular offset between the tropical and sidereal zodiacs. Stored on every Vedic chart as a number of degrees, currently about 24.	<code>swe.get_ayanamsa_ut</code> in <code>lib/astrology/chart.ts</code> ; <code>Chart.ayanamsa</code> in <code>types.ts</code>
Lahiri	The particular ayanamsa the Vedic chart uses, set with <code>SE_SIDM_LAHIRI</code> . The prompt names the tradition as "Vedic (sidereal, Lahiri)".	<code>lib/astrology/chart.ts</code> , <code>prompt.ts</code>
Sidereal	Positions measured against the fixed stars, so an ayanamsa is subtracted from tropical longitudes. Used for Vedic charts only.	<code>SEFLG_SIDEREAL</code> in <code>chart.ts</code>
Tropical	Positions measured from the vernal equinox, with no ayanamsa. Used for Western charts.	<code>chart.ts</code> , the default flags
Ascendant, lagna	The sign and degree rising on the eastern horizon at birth, from <code>ascmc[0]</code> . Sets house 1 in both traditions. "Lagna" is the Vedic word and appears in the PDF and the phone's routing words.	<code>chart.ts</code> ; <code>Chart.ascendant</code>
Whole-sign houses	House N is the Nth sign counted from the ascendant's sign, so a whole sign is a whole house. Used for Vedic since 0.3.0.	<code>houseFrom</code> in <code>lib/astrology/constants.ts</code>
Placidus	A house system that divides the sky by time-based cusps, so houses are unequal and a sign can span two. Used for Western; the cusps are stored for both.	<code>houses_ex(..., "P")</code> and <code>houseOf</code> in <code>chart.ts</code>
Nakshatra	One of 27 lunar mansions of 13 degrees 20 minutes each. Every Vedic planet carries its nakshatra name; the Moon's decides the starting <i>dasha</i> .	<code>NAKSHATRAS</code> , <code>nakshatraOf</code> in <code>constants.ts</code>
Pada	A quarter of a nakshatra. Named in the first design note but not computed: the chart stores the nakshatra name only.	Nowhere in the code
Vimshottari	The 120-year <i>dasha</i> system: nine lords in a fixed order with fixed years (Ketu 7, Venus 20, Sun 6, Moon 10, Mars 7, Rahu 18, Jupiter 16, Saturn 19, Mercury 17). Which lord starts, and how much of it is left at birth, comes from the Moon's sidereal longitude.	<code>ORDER</code> , <code>startingDashaLord</code> in <code>lib/astrology/dasha.ts</code>
Mahadasha	A major period ruled by one lord, years long. The chart stores the current one; the timeline lays out all twelve from birth.	<code>computeVimshottari</code> , <code>vimshottariTimeline</code> in <code>dasha.ts</code>
Antardasha	A sub-period inside a mahadasha, nine per mahadasha in Vimshottari order from the mahadasha's own lord, each a proportional share. Laid out from the mahadasha's virtual start so it aligns to the period, not the birth moment.	<code>antardashaSpans</code> in <code>dasha.ts</code>

TERM	MEANING HERE	WHERE
Dosha	An affliction. In the code a <code>Condition</code> with a kind, a signature, a severity and a plain-sentence <code>detail</code> . Natal doshas are fixed at birth; transit afflictions come and go.	<code>lib/astrology/doshas.ts</code>
Mangal dosha	Mars in house 1, 2, 4, 7, 8 or 12 from the ascendant, derived from the sign rather than the stored house number. Severity caution.	<code>detectNatalDoshas</code>
Kaal Sarp yoga	All seven classical planets on one side of the Rahu to Ketu axis. Severity warning.	<code>detectNatalDoshas</code>
Grahan dosha	The Sun or the Moon sharing a sign with Rahu or Ketu. One condition per luminary and node pair.	<code>detectNatalDoshas</code>
Kemadruma yoga	No planet in the sign before or after the Moon, excluding the Sun and the nodes.	<code>detectNatalDoshas</code>
Pitru dosha	Rahu or Ketu in the 9th house from the ascendant, derived from the sign. Severity info.	<code>detectNatalDoshas</code>
Shrapit dosha	Saturn and Rahu in the same sign.	<code>detectNatalDoshas</code>
Sade Sati	Transiting Saturn in the 12th, 1st or 2nd sign from the natal Moon, called the rising, peak and setting phases. Peak is a warning; the others caution. The phase is part of the signature, so a phase change reads as a new condition.	<code>detectTransitAfflictions</code>
Shani Dhaiya	Transiting Saturn in the 4th or 8th from the natal Moon.	<code>detectTransitAfflictions</code>
Kantaka Shani	Transiting Saturn in the 4th, 7th or 10th from the ascendant. Severity info.	<code>detectTransitAfflictions</code>
Node over Moon, node over lagna	Rahu or Ketu transiting the natal Moon's sign or the ascendant sign.	<code>detectTransitAfflictions</code>
Angarak transit	Transiting Mars in the natal Moon's sign.	<code>detectTransitAfflictions</code>
Ashtama Surya	The transiting Sun in the 8th from the natal Moon, "for about a month."	<code>detectTransitAfflictions</code>
Guru Chandala	Transiting Jupiter and Rahu in the same sign. A related info condition flags Jupiter in the 6th, 8th or 12th from the Moon.	<code>detectTransitAfflictions</code>
Retrograde on a sensitive sign	Saturn, Mars or Mercury retrograde while in the ascendant's or natal Moon's sign; retrogrades elsewhere are ignored.	<code>detectTransitAfflictions</code>
Gochara	Transits read against the chart: each transiting planet by the house it occupies from the ascendant and from the natal Moon, plus any natal planet it sits on, plus the afflictions above. Vedic only; a Western chart gets positions and conjunctions.	<code>describeGochara</code> in <code>lib/astrology/transits.ts</code>
Dignity	How strong a planet is by sign: one of <code>exalted</code> , <code>moolatrikona</code> , <code>own</code> , <code>debilitated</code> , <code>neutral</code> , in that precedence. Friendship is deliberately not a dignity.	<code>dignityOf</code> in <code>lib/astrology/derived.ts</code>

TERM	MEANING HERE	WHERE
Exalted	In its sign of exaltation, with a deep degree recorded so the prompt can say "within a degree of exact". The nodes are never exalted.	<code>EXALTATION</code> in <code>derived.ts</code>
Debilitated	In the sign opposite its exaltation.	<code>dignityOf</code>
Moolatrikona	A degree range inside one sign where a planet is nearly as strong as exalted. Where it shares the exaltation sign (Mercury in Virgo, the Moon in Taurus) the range takes precedence within its bounds.	<code>MOOLATRIKONA</code> in <code>derived.ts</code>
Own sign	A sign the planet rules. Rahu and Ketu rule nothing.	<code>SIGN_LORD</code> , <code>signsRuledBy</code>
Combust	Within a planet-specific orb of the Sun by the shorter arc; Mercury and Venus have tighter orbs when retrograde. The Sun and the nodes never combust.	<code>COMBUSTION</code> , <code>isCombust</code>
Aspect, drishti	Graha drishti: every planet aspects the 7th house from itself; Mars adds the 4th and 8th, Jupiter the 5th and 9th, Saturn the 3rd and 10th; the nodes only the 7th. Rendered as "aspects houses" in the prompt and "linked to sections" on the phone.	<code>SPECIAL_ASPECTS</code> , <code>aspectedHouses</code>
Lord, ruler	The planet that owns a house's sign. Each <code>HouseFact</code> names its lord, where the lord sits and in what dignity. The running dasha lords' own placements are carried too.	<code>HouseFact</code> , <code>DashaFact</code> in <code>derived.ts</code>
Mulank, bhagyank, namank	The root number from the birth day, the destiny number from the whole date, and the name number by the Pythagorean mapping, each reduced to one digit.	<code>computeNumerology</code> , <code>computeNameNumber</code> in <code>numerology.ts</code>
Lo Shu grid	Which digits 1 to 9 the birth date repeats and which it lacks; zeros are not placed.	<code>loShu</code> in <code>numerology.ts</code>
Personal year, personal month	Birth month plus birth day plus the current year, reduced; the month adds the calendar month to that. They change, so they live in the varying half of the prompt.	<code>personalCycle</code> in <code>numerology.ts</code>

PROJECT TERMS

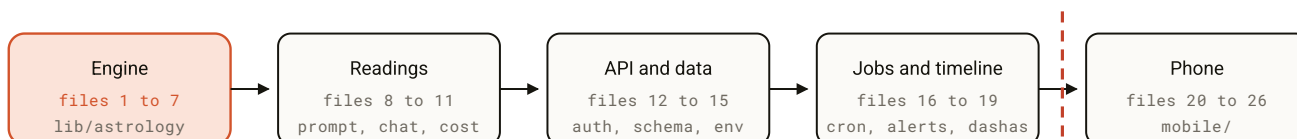
TERM	MEANING HERE	WHERE
Birth profile	One row per user: name, birth date and time, place, coordinates, timezone, optional avatar, and the <code>chart</code> JSON holding both traditions. Computed at intake and upgraded on read when stale.	<code>birth_profiles</code> in <code>0001_init.sql</code> ; <code>lib/data/birthProfile.ts</code>
Tradition	"vedic" or "western" : which chart is computed and read. A chat runs in a <code>ChatMode</code> , which adds "numerology" .	<code>Tradition</code> , <code>ChatMode</code> in <code>types.ts</code>
Reading	A streamed answer from Claude to one message, grounded in the chart. Also the daily morning and night readings the cron job writes.	<code>app/api/chat/route.ts</code> ; <code>daily_readings</code>
Deep reading	The same request with <code>deep : true</code> : Opus instead of Sonnet, default effort instead of low, 320 words instead of 160.	<code>WORDS_DEEP</code> , <code>DEEP_READING_MODEL</code>
Slot	Which of a day's two readings a moment belongs to: "morning" before 14:00 on the user's clock, "night" after. Part of the unique key on <code>daily_readings</code> .	<code>lib/alerts/slots.ts</code>
Condition	A dosha or transit affliction as a row: <code>kind</code> , <code>signature</code> , <code>label</code> , <code>severity</code> , <code>scope</code> (natal or transit), <code>detail</code> . Open rows have no <code>ended_on</code> ; the daily job diffs signatures against them.	<code>Condition</code> in <code>doshas.ts</code> ; <code>transit_conditions</code> table
Alert	A composed notification written when a notifiable condition starts or ends: title, body, markdown detail, and the kinds involved. Pushed to every registered device.	<code>lib/alerts/run.ts</code> , <code>compose.ts</code> ; <code>alerts</code> table
Moment	A life event on the timeline: a date with a precision of day, month or year, a title, an optional note, and a source of <code>manual</code> or <code>extracted</code> . Placed into the mahadasha and antardasha running when it happened.	<code>TimelineEvent</code> in <code>lib/timeline/build.ts</code> ; <code>life_events</code>
Candidate	A moment proposed from the user's own chat messages, with the quoted evidence, awaiting confirmation. May be undated. Nothing is saved until confirmed.	<code>CandidateEvent</code> in <code>lib/timeline/extract.ts</code> and <code>TimelineStore.swift</code>
Period reading	A one-line theme and a short meaning for one mahadasha, or for the literal lord <code>now</code> , written by Claude and stored with a hash of the moments inside the period so it is rewritten only when they change.	<code>period_readings</code> ; <code>lib/timeline/explain.ts</code>
Timeline	The twelve mahadashas from birth with their antardashas, the moments placed in them, the current pair, and whether any period needs explaining.	<code>buildTimeline</code> in <code>lib/timeline/build.ts</code>
Chart cache	The on-disk JSON in the App Group that holds the chart, birth details, latest reading and current period for the widget and Siri. Written by the app, cleared on sign-out.	<code>mobile/Sanchara/Sources/Services/ChartCache.swift</code>
App Group	Apple's shared container, <code>group.com.shivvyas.astra</code> , entitled on both the app and the widget extension so two processes read one cache.	<code>mobile/project.yml</code>

TERM	MEANING HERE	WHERE
Device token	An APNs token registered by the phone with its push environment, <code>sandbox</code> or <code>production</code> , decided from the signing entitlement. Dead tokens are deleted when Apple reports them.	<code>app/api/devices/route.ts</code> ; <code>Push.swift</code>
On-device answer	A lookup question answered by Apple's on-device model from the cached chart through a tool, with no API call. Anything interpretive escalates to Claude.	<code>OnDeviceReasoner.swift</code> , <code>ChartFacts.swift</code>
Deep link, pending destination	Where the app should land next: set by an intent or widget into the App Group, or by the <code>sanchara://</code> scheme, and drained on foreground.	<code>DeepLink.swift</code>

C.2 A reading order through the whole codebase

Twenty-six files, from the chart engine outward. Each takes a few minutes. Read them in order and the rest of the repository will be recognisable: every other file is a caller, a view or a test of one of these.

READ LEFT TO RIGHT, THE WAY THE DATA FLOWS



Everything left of the line runs on Vercel and holds the secrets. The phone holds the anon key, its own rows, and a cached chart.

The order below follows this path. The first seven files are pure functions with no I/O and can be read with the tests beside them.

1. `lib/astrology/types.ts`. The `Chart` shape everything else consumes. Note `schemaVersion` and the optional `derived`.
2. `lib/astrology/constants.ts`. Signs, nakshatras, `houseFrom`, and `CHART_SCHEMA_VERSION` with its comment.
3. `lib/astrology/chart.ts`. `computeChart` end to end: the flags, the two house rules, Ketu from Rahu, the derived block for Vedic only.
4. `lib/astrology/dasha.ts`. Vimshottari from the Moon's longitude; the `virtualStart` comment explains the 0.1.0 fix.
5. `lib/astrology/derived.ts`. The classical tables and the three things deliberately absent, in the header.
6. `lib/astrology/doshas.ts`. The `Condition` type and why the signature exists; the natal and transit detectors.
7. `lib/astrology/transits.ts`. The per-day transit cache and `describeGochara`, with the paragraph on why the old text was generic.
8. `lib/astrology/prompt.ts`. The two halves, the Accuracy rules, and the Western fallback in `renderChart`.
9. `lib/anthropic.ts`. Model names, adaptive thinking, and the three measured output figures.

10. `app/api/chat/route.ts` . The whole reading path: profile, upgrade, transits, timeline block, history, two cache breakpoints, streaming, the usage log line.
11. `docs/MODEL_COSTS.md` . What a turn costs and the four changes that made it cheaper.
12. `lib/supabase/route.ts` . Bearer or cookie, in thirty lines.
13. `lib/env.ts` . Which variables are required, which are optional, and the `server-only` import that keeps them there.
14. `supabase/migrations/0001_init.sql` . The five original tables, the `is_admin` function, and the owner-only policies.
15. `lib/data/birthProfile.ts` . Intake writes both charts; `ensureCurrentChart` upgrades on read and swallows a failed write on purpose.
16. `lib/alerts/slots.ts` then `lib/alerts/predictions.ts` . The local-clock rule, the unique index as the safety net, the fallback reading.
17. `lib/alerts/run.ts` . The signature diff, `partitionForAlerts` , and the comments that forbid filtering the raw sets.
18. `app/api/cron/daily/route.ts` with `vercel.json` . Two schedules, one secret, two jobs in sequence.
19. `lib/timeline/build.ts` . The life map: periods, moments placed by date, the events hash, the `now` row.
20. `mobile/project.yml` . Targets, entitlements, the files shared with the widget, and the comment on each.
21. `mobile/Sanchara/Sources/Services/Supabase.swift` then `SancharaAPI.swift` . The one client, the bearer header, and one method per route.
22. `mobile/Sanchara/Sources/Features/Intake/ProfileStore.swift` . How the chart reaches the phone: read under RLS, cached, usable offline.
23. `mobile/Sanchara/Sources/Services/ChartCache.swift` . Why three processes share one file, and what never goes in it.
24. `mobile/Sanchara/Sources/Features/Chat/ChartFacts.swift` then `OnDeviceReasoner.swift` . The neutral table and the three load-bearing rules for the on-device model.
25. `mobile/Sanchara/Sources/Intents/SancharaIntents.swift` and `DeepLink.swift` . What Siri can answer and how a process with no UI tells the app where to land.
26. `mobile/SancharaWidgets/ReadingWidget.swift` . A widget that never fetches, and the twice-daily refresh boundary.

C.3 Where things are

The chapters are numbered 1 to 16. Each row names the file to open first; the chapter says why it is shaped the way it is.

I WANT TO CHANGE	OPEN	CHAPTER
How the system fits together, what runs where	<code>README.md</code> , <code>middleware.ts</code>	1, Architecture
Which bodies are computed, the ayanamsa, the house system	<code>lib/astrology/chart.ts</code> , <code>constants.ts</code>	2, The chart engine
Dasha arithmetic	<code>lib/astrology/dasha.ts</code>	2
A dignity, combustion or aspect rule	<code>lib/astrology/derived.ts</code>	2
Add or change a dosha	<code>lib/astrology/doshas.ts</code> ; then read decision 10	2, 9
What the model is told, or the output rules	<code>lib/astrology/prompt.ts</code>	3, Readings, prompts, and cost
The model, effort, token ceilings, cost	<code>lib/anthropic.ts</code> , <code>app/api/chat/route.ts</code> , <code>docs/MODEL_COSTS.md</code>	3
How much history is resent	<code>lib/data/history.ts</code>	3
Numerology numbers or the friendship table	<code>lib/astrology/numerology.ts</code>	3
Sign-in on the web, magic links, the callback	<code>app/(auth)/actions.ts</code> , <code>app/auth/callback/route.ts</code>	4, Identity and access
Bearer auth, cookie sessions, the middleware redirect	<code>lib/supabase/route.ts</code> , <code>lib/supabase/middleware.ts</code>	4
Email codes and Apple sign-in on the phone	<code>mobile/Sanchara/Sources/Features/Auth/AuthStore.swift</code> , <code>AppleSignIn.swift</code> , <code>docs/EMAIL_OTP.md</code>	4, 14
The intake form, geocoding, the timezone	<code>components/IntakeForm.tsx</code> , <code>lib/geo.ts</code> , <code>app/api/profile/route.ts</code>	5, Intake and the birth profile
What is stored with the chart, or the on-read upgrade	<code>lib/data/birthProfile.ts</code>	5
The chat UI on the web, modes, streaming	<code>components/Chat.tsx</code> , <code>app/api/chat/route.ts</code>	6, Chat
Conversations and messages	<code>lib/data/chat.ts</code>	6
The kundli PDF	<code>lib/pdf/kundli.ts</code> , <code>app/api/kundli/route.ts</code>	7, Kundli
The kundli diagram on the phone	<code>mobile/Sanchara/Sources/Features/Kundli/ChartData.swift</code> , <code>KundliChartView.swift</code>	7
How periods and moments are laid out	<code>lib/timeline/build.ts</code> , <code>load.ts</code>	8, The life timeline
Moment extraction from chat history	<code>lib/timeline/extract.ts</code> , <code>app/api/timeline/scan/route.ts</code>	8

I WANT TO CHANGE	OPEN	CHAPTER
Period themes and meanings	<code>lib/timeline/explain.ts</code> , <code>app/api/timeline/explain/route.ts</code>	8
The timeline block the chat sees	<code>lib/timeline/describe.ts</code>	8, 6
Pinning and unpinning moments	<code>app/api/life-events/route.ts</code> , <code>TimelineStore.swift</code>	8
When the daily readings are written, or the slot rule	<code>vercel.json</code> , <code>lib/alerts/slots.ts</code> , <code>lib/alerts/predictions.ts</code>	9, Daily readings and dosha alerts
What triggers a push, or the alert copy	<code>lib/alerts/run.ts</code> , <code>lib/alerts/compose.ts</code>	9
APNs itself	<code>lib/push/apns.ts</code> , <code>app/api/devices/route.ts</code> , <code>docs/PUSH_ALERTS.md</code>	9
What the on-device model may answer	<code>OnDeviceReasoner.swift</code> , <code>ChartFacts.swift</code>	10, On-device answers, Siri, and widgets
A Siri phrase or intent	<code>SancharaShortcuts.swift</code> , <code>SancharaIntents.swift</code> , <code>IntentData.swift</code>	10
A widget, the control, or what is cached for them	<code>mobile/SancharaWidgets/</code> , <code>ChartCache.swift</code> , <code>WidgetRefresh.swift</code>	10
The profile page, avatar, or account deletion	<code>app/(app)/app/profile/page.tsx</code> , <code>ProfileView.swift</code> , <code>app/api/account/route.ts</code>	11, Profile, account, and admin
The admin area	<code>app/admin/</code> , <code>lib/admin/guard.ts</code> , <code>lib/admin/data.ts</code>	11
The landing page or the signed-in shell	<code>components/marketing/</code> , <code>components/AppShell.tsx</code> , <code>app/globals.css</code>	12, The landing page and the web shell
A table, a policy, or a new migration	<code>supabase/migrations/</code>	13, The data model
App structure, theme, navigation, the API client	<code>SancharaApp.swift</code> , <code>RootView.swift</code> , <code>Design/Theme.swift</code> , <code>Services/SancharaAPI.swift</code>	14, The iOS app
Targets, entitlements, the generated project	<code>mobile/project.yml</code> , <code>mobile/scripts/make-config.sh</code>	14, 15
Environment variables, deploys, cron secret, TestFlight	<code>.env.example</code> , <code>docs/DEPLOY.md</code> , <code>docs/GO_LIVE_CHECKLIST.md</code> , <code>.github/workflows/</code>	15, Setup, deployment, and release
Cutting a release	<code>CHANGELOG.md</code> , <code>package.json</code> , <code>.github/workflows/release.yml</code>	15

I WANT TO CHANGE	OPEN	CHAPTER
A failure path: model errors, parse failures, dead tokens, stale charts	<code>compose.ts</code> fallbacks, <code>fallbackPrediction</code> , <code>pushToDevices</code> , <code>ensureCurrentChart</code>	16, What breaks, and how it is handled

TWO THINGS WITH NO WEB PAGE

The timeline, the readings inbox and the dosha alerts have API routes and tables, and iOS views, but no page under `app/(app)/app/` , which holds only home, intake, chat and profile. A web contributor looking for the timeline screen will not find one; the client is the phone.

C.4 Study notes

READ IN THIS ORDER

1. The glossary's astrology table against `lib/astrology/doshas.ts` , checking each named dosha against its detector.
2. Files 1 to 7 of the reading order with their `.test.ts` neighbours open; the test names read as the rules.
3. The rest of the reading order, one stage a day.

CHECK YOURSELF

Which glossary term is named in a design note but computed nowhere?

Pada. The chart stores only the nakshatra name from `nakshatraOf` .

What is the difference between a condition and an alert?

A condition is a computed fact with a signature, recorded in `transit_conditions` whether or not anyone is told. An alert is the composed notification written to `alerts` only when a notifiable condition starts or ends.

Where does a Siri intent get the chart, and what happens if it is not there?

From `ChartCache` in the App Group, via `IntentData.chart()` . If nothing is cached the intent says so in words rather than throwing, because "an intent that cannot find data says so in words, which is a better outcome than an error Siri reads out as a failure."

You want to add a Western technique. Which file, and what must you not do?

A new module beside `derived.ts` , gated on `chart.tradition === "western"` . You must not extend `deriveFacts` or `factsFor` , which are Vedic by design and return nothing for a tropical chart.

Sanchara Engineering Handbook, Revision A, 30 September 2026. Written by Shiv Vyas from the code on `main` at version 0.3.0. Source of record: the code; `docs/design/` for the design notes. Built by `docs/handbook/build.py` ; page numbers by `render-pdf.sh` ; the Kindle edition by `render-epub.sh` .